

AgentMatrix 完整源代码

项目根目录: C:\Users\王森\Desktop\AgentMatrix 包含文件数: 203 生成日期: 2026-05-17

文件索引

- .editorconfig
- .eslinttrc.js
- .gitignore
- .pre-commit-config.yaml
- .prettierrc
- .vscode\launch.json
- .vscode\tasks.json
- 11.txt
- 2313.txt
- 91.txt
- CONTRIBUTING.md
- README.md
- backend\env.example
- backend\agents__init__.py
- backend\agents\base__init__.py
- backend\agents\base\agent.py
- backend\agents\base\agent_registry.py
- backend\agents\judge__init__.py
- backend\agents\judge\agent.py
- backend\agents\knowledge__init__.py
- backend\agents\knowledge\agent.py
- backend\agents\result__init__.py
- backend\agents\result\agent.py
- backend\agents\review__init__.py
- backend\agents\review\agent.py
- backend\agents\summary__init__.py
- backend\agents\summary\agent.py
- backend\agents\writer__init__.py
- backend\agents\writer\agent.py
- backend\api__init__.py
- backend\api\v1__init__.py
- backend\api\v1\agents__init__.py
- backend\api\v1\agents\router.py
- backend\api\v1\chat__init__.py
- backend\api\v1\chat\router.py
- backend\api\v1\config\router.py
- backend\api\v1\export__init__.py
- backend\api\v1\export\router.py
- backend\api\v1\knowledge__init__.py
- backend\api\v1\knowledge\router.py
- backend\api\v1\metrics__init__.py
- backend\api\v1\metrics\router.py

- backend\api\vl\router.py
- backend\api\vl\workflow__init__.py
- backend\api\vl\workflow\router.py
- backend\api\websocket__init__.py
- backend\api\websocket\manager.py
- backend\app__init__.py
- backend\app\config.py
- backend\app\database.py
- backend\app\dependencies.py
- backend\app\main.py
- backend\check_deepseek_usage.py
- backend\config__init__.py
- backend\config\app_config.json
- backend\config\manager.py
- backend\core__init__.py
- backend\core\dynamic_router__init__.py
- backend\core\dynamic_router\router.py
- backend\core\export__init__.py
- backend\core\knowledge__init__.py
- backend\core\llm__init__.py
- backend\core\llm\client.py
- backend\core\llm\ollama_client.py
- backend\core\workflow__init__.py
- backend\core\workflow\service.py
- backend\debug_config.py
- backend\detailed_debug.py
- backend\houduan.md
- backend\knowledge__init__.py
- backend\knowledge\knowledge_base.json
- backend\knowledge\service.py
- backend\models__init__.py
- backend\models\agent.py
- backend\models\db_models.py
- backend\models\workflow.py
- backend\prompts__init__.py
- backend\prompts\rules__init__.py
- backend\prompts\template_manager.py
- backend\prompts\templates__init__.py
- backend\prompts\templates\judge\complexity.txt
- backend\prompts\templates\judge_prompt.txt
- backend\prompts\templates\knowledge\enhance.txt
- backend\prompts\templates\knowledge_prompt.txt
- backend\prompts\templates\result\format.txt
- backend\prompts\templates\result_prompt.txt
- backend\prompts\templates\review\review.txt
- backend\prompts\templates\review_prompt.txt
- backend\prompts\templates\summary\extract.txt
- backend\prompts\templates\summary_prompt.txt
- backend\prompts\templates\writer\generate.txt
- backend\prompts\templates\writer_prompt.txt
- backend\pyproject.toml
- backend\quick_test.py
- backend\roletxt\all_role.txt
- backend\roletxt\knowledge.txt
- backend\roletxt\review.txt
- backend\run_backend.py
- backend\run_direct.py

- backend\services__init__.py
- backend\services\agent_service.py
- backend\shared__init__.py
- backend\shared\platform.py
- backend\simple_start.py
- backend\start_server.py
- backend\start_service.py
- backend\test_agent_debug.py
- backend\test_api.py
- backend\test_api_full.py
- backend\test_cloud_call.py
- backend\test_complex.py
- backend\test_complexity.py
- backend\test_complexity_section.py
- backend\test_complexity_section2.py
- backend\test_config.py
- backend\test_deepseek.py
- backend\test_deepseek_chat.py
- backend\test_deepseek_direct.py
- backend\test_deepseek_simple.py
- backend\test_direct_deepseek.py
- backend\test_final.py
- backend\test_full_api.py
- backend\test_full_workflow.py
- backend\test_immediate.py
- backend\test_import.py
- backend\test_judge_comprehensive.py
- backend\test_judge_llm.py
- backend\test_judge_mechanism.py
- backend\test_llm_debug.py
- backend\test_llm_integration.py
- backend\test_love_letter.py
- backend\test_model_names.py
- backend\test_model_switch.py
- backend\test_multiple_tasks.py
- backend\test_ollama_connection.py
- backend\test_other_tasks.py
- backend\test_output.txt
- backend\test_output_check.py
- backend\test_review_debug.py
- backend\test_settings.py
- backend\test_setup.py
- backend\test_simple.py
- backend\test_threshold.py
- backend\test_v4_flash.py
- backend\test_workflow.py
- backend\test_workflow_api.py
- backend\test_workflow_consume.py
- backend\test_workflow_debug.py
- backend\test_workflow_fix.py
- backend\tests__init__.py
- backend\tests\test_agents__init__.py
- backend\tests\test_agents\test_judge_agent.py
- backend\tests\test_agents\test_knowledge_agent.py
- backend\tests\test_api__init__.py
- backend\tests\test_api\test_knowledge_api.py
- backend\tests\test_api_performance.py

- backend\tests\test_report.json
- backend\tests\test_workflow__init__.py
- backend\tests\test_workflow\test_workflow_service.py
- backend\utils__init__.py
- backend\utils\logger.py
- backend\verify_deepseek_key.py
- backend\使用说明.md
- backend\启动.bat
- configs\agents\judge_config.yaml
- configs\agents\knowledge_config.yaml
- configs\models\local_models.yaml
- deepseek密钥.txt
- docs\AgentMatrix 安装说明.md
- docs\AgentMatrix 技术报告.md
- docs\AgentMatrix 部署文档.md
- frontend\.env.example
- frontend\FE_DEVELOPMENT_SPEC.md
- frontend\next-env.d.ts
- frontend\next.config.js
- frontend\package.json
- frontend\src\app\globals.css
- frontend\src\app\layout.tsx
- frontend\src\app\page.tsx
- frontend\src\components\layout\DashboardLayout\index.tsx
- frontend\src\services\api\agentService.ts
- frontend\src\services\api\socketService.ts
- frontend\src\stores\agentStore.ts
- frontend\src\stores\workflowStore.ts
- frontend\src\types\index.ts
- frontend\tailwind.config.ts
- frontend\tsconfig.json
- pytest.ini
- ruff.toml
- scripts\convert_docs_to_pdf.py
- scripts\extract_source_code.py
- shared__init__.py
- shared\constants__init__.py
- shared\types__init__.py
- shared\utils__init__.py
- start.bat
- start.psl
- stop.bat
- stop.psl
- temp.py
- temp2.py
- test.bat
- test.psl

.editorconfig

```
root = true

[*]
charset = utf-8
end_of_line = lf
insert_final_newline = true
```

```
trim_trailing_whitespace = true
```

```
[*.py]
indent_style = space
indent_size = 4
```

```
[*.ts]
indent_style = space
indent_size = 2
```

```
[*.tsx]
indent_style = space
indent_size = 2
```

```
[*.js]
indent_style = space
indent_size = 2
```

```
[*.jsx]
indent_style = space
indent_size = 2
```

```
[*.json]
indent_style = space
indent_size = 2
```

```
[*.yaml]
indent_style = space
indent_size = 2
```

```
[*.yml]
indent_style = space
indent_size = 2
```

```
[*.md]
indent_style = space
indent_size = 2
```

.eslintrc.js

```
module.exports = {
  parser: '@typescript-eslint/parser',
  parserOptions: {
    ecmaVersion: 'latest',
    sourceType: 'module',
    ecmaFeatures: {
      jsx: true,
    },
  },
  settings: {
    react: {
      version: 'detect',
    },
  },
  extends: [
    'eslint:recommended',
    'plugin:@typescript-eslint/recommended',
    'plugin:react/recommended',
    'plugin:react-hooks/recommended',
    'plugin:jsx-ally/recommended',
    'prettier',
  ],
  plugins: ['@typescript-eslint', 'react', 'jsx-ally', 'import'],
  rules: {
    'react/react-in-jsx-scope': 'off',
    'react/prop-types': 'off',
    '@typescript-eslint/no-unused-vars': ['warn', { argsIgnorePattern: '^_' }],
    '@typescript-eslint/explicit-function-return-type': 'off',
    '@typescript-eslint/explicit-module-boundary-types': 'off',
    'import/order': [
      'error',
      {
        groups: ['builtin', 'external', 'internal', 'parent', 'sibling', 'index'],
        alphabetize: { order: 'asc', caseInsensitive: true },
      },
    ],
  },
}
```

```

    },
  ],
},
env: {
  browser: true,
  es2021: true,
  node: true,
},
};

```

.gitignore

```

node_modules/
.next/
.npm/
.env
.env.local
.env.*.local
dist/
build/
*.log
.DS_Store
__pycache__/
*.pyc
*.pyo
*.pyd
.Python
.tox/
.venv/
env/
venv/
.pytest_cache/
.coverage
htmlcov/
*.egg-info/
.pypirc
.ruff_cache/
*.swp
*.swo
*.bak
*.tmp
backend/logs/*.log
backend/exports/*
frontend/public/demos/*
backend/frontend/index.html

```

.pre-commit-config.yaml

```

repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v4.6.0
  hooks:
    - id: trailing-whitespace
    - id: end-of-file-fixer
    - id: check-yaml
    - id: check-json
    - id: check-merge-conflict
    - id: debug-statements

- repo: https://github.com/astral-sh/ruff-pre-commit
  rev: v0.4.10
  hooks:
    - id: ruff
      args: ['--fix']
    - id: ruff-format

- repo: https://github.com/prettier/prettier
  rev: 3.2.5

```

```

hooks:
  - id: prettier
    types_or: [javascript, typescript, jsx, tsx, json, yaml]

- repo: https://github.com/typescript-eslint/typescript-eslint
  rev: v7.8.0
  hooks:
    - id: eslint
      types_or: [javascript, typescript, jsx, tsx]
      additional_dependencies:
        - eslint-plugin-react
        - eslint-plugin-react-hooks
        - eslint-plugin-jsx-ally
        - eslint-plugin-import

- repo: https://github.com/kynan/nbstripout
  rev: 0.7.1
  hooks:
    - id: nbstripout

```

.prettierrc

```

{
  "printWidth": 100,
  "tabWidth": 2,
  "useTabs": false,
  "semi": true,
  "singleQuote": true,
  "quoteProps": "as-needed",
  "jsxSingleQuote": true,
  "trailingComma": "es5",
  "bracketSpacing": true,
  "bracketSameLine": false,
  "arrowParens": "always",
  "requirePragma": false,
  "insertPragma": false,
  "proseWrap": "preserve",
  "htmlWhitespaceSensitivity": "css",
  "vueIndentScriptAndStyle": false,
  "endOfLine": "lf",
  "embeddedLanguageFormatting": "auto",
  "singleAttributePerLine": false
}

```

.vscode\launch.json

```

{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Backend: FastAPI",
      "type": "python",
      "request": "launch",
      "module": "uvicorn",
      "args": [
        "app.main:app",
        "--host", "0.0.0.0",
        "--port", "8000",
        "--reload"
      ],
      "cwd": "${workspaceFolder}/backend",
      "envFile": "${workspaceFolder}/backend/.env",
      "justMyCode": false,
      "console": "integratedTerminal",
      "python": "${command:python.interpreterPath}"
    },
    {
      "name": "Frontend: Next.js",

```

```

    "type": "node",
    "request": "launch",
    "runtimeExecutable": "npm",
    "runtimeArgs": [
      "run", "dev"
    ],
    "cwd": "${workspaceFolder}/frontend",
    "console": "integratedTerminal",
    "protocol": "inspector",
    "skipFiles": [
      "<node_internals>/**"
    ],
    "port": 9229
  },
  {
    "name": "Run All (Frontend + Backend)",
    "type": "compound",
    "configurations": [
      "Backend: FastAPI",
      "Frontend: Next.js"
    ],
    "preLaunchTask": "Install Dependencies"
  }
]
}

```

.vscode\tasks.json

```

{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "🚀 一键启动",
      "type": "shell",
      "command": "powershell",
      "args": ["-File", "start.ps1"],
      "cwd": "${workspaceFolder}",
      "problemMatcher": [],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "detail": "一键启动前端和后端服务"
    },
    {
      "label": "🛑 停止服务",
      "type": "shell",
      "command": "powershell",
      "args": ["-File", "stop.ps1"],
      "cwd": "${workspaceFolder}",
      "problemMatcher": [],
      "group": "none",
      "detail": "停止所有运行中的服务"
    },
    {
      "label": "🧪 测试工作流",
      "type": "shell",
      "command": "powershell",
      "args": ["-File", "test.ps1"],
      "cwd": "${workspaceFolder}",
      "problemMatcher": [],
      "group": {
        "kind": "test",
        "isDefault": true
      },
      "detail": "测试Agent工作流是否正常"
    },
    {
      "label": "Install Backend Dependencies",
      "type": "shell",
      "command": "pip install pydantic pydantic-settings fastapi uvicorn httpx python-dotenv",
      "args": [],
      "cwd": "${workspaceFolder}/backend",
      "problemMatcher": [],
    }
  ]
}

```

```
    "group": "build",
    "runOptions": {
      "runOn": "folderOpen"
    }
  },
  {
    "label": "Install Frontend Dependencies",
    "type": "shell",
    "command": "npm install",
    "args": [],
    "cwd": "${workspaceFolder}/frontend",
    "problemMatcher": [],
    "group": "build"
  },
  {
    "label": "Install Dependencies",
    "dependsOn": [
      "Install Backend Dependencies",
      "Install Frontend Dependencies"
    ],
    "problemMatcher": []
  }
]
```

11.txt

2313.txt

91.txt

CONTRIBUTING.md

```
# 贡献指南

欢迎为 AgentMatrix 项目贡献代码！

## 行为准则

### 我们的承诺

为了营造一个开放和友好的环境，我们承诺让每位贡献者都能享受无骚扰的体验。

### 我们的标准

- 使用友好和包容的语言
- 尊重不同的观点和经验
- 优雅地接受建设性批评
- 关注对社区最有利的事情
- 对其他社区成员表现出同理心
```

```
## 贡献流程

### 1. Fork 仓库

在 GitHub 上点击“Fork”按钮，将仓库克隆到你的账号下。

### 2. 克隆仓库

```bash
git clone https://github.com/your-username/AgentMatrix.git
cd AgentMatrix
```

### 3. 创建功能分支

```
git checkout -b feature/your-feature-name
```

分支命名规范： - `feature/xxx`：新功能开发 - `bugfix/xxx`：Bug修复 - `hotfix/xxx`：紧急修复 - `docs/xxx`：文档更新 - `refactor/xxx`：代码重构

### 4. 开发

遵循代码规范： - 前端：ESLint + Prettier - 后端：Ruff + Black

### 5. 提交代码

```
git add .
git commit -m "feat: 描述你的更改"
```

Commit 信息规范（Conventional Commits）： - `feat`：新功能 - `fix`：Bug修复 - `docs`：文档更新 - `style`：代码格式（不影响代码运行的变动） - `refactor`：重构（既不新增功能，也不修复bug） - `perf`：性能优化 - `test`：测试相关 - `chore`：构建/工具/依赖更新

### 6. 推送到远程

```
git push origin feature/your-feature-name
```

### 7. 创建 Pull Request

在 GitHub 上创建 PR，描述你的更改： - 更改的目的 - 解决的问题（关联Issue） - 测试方法

### 8. Code Review

等待项目维护者审查代码。可能需要进行修改。

### 9. 合并

审查通过后，代码将被合并到 `develop` 分支。

## 代码规范

### Python 代码规范

- 使用 Ruff 进行静态检查
- 使用 Black 进行代码格式化
- 遵循 PEP8 标准
- 类型提示必须完整

### TypeScript 代码规范

- 使用 ESLint 进行静态检查
- 使用 Prettier 进行代码格式化
- React 组件使用函数式组件 + Hooks
- 类型定义完整

提交信息规范

```
<type>(<scope>): <description>

[optional body]

[optional footer(s)]
```

示例:

```
feat(agent): 添加知识检索功能

- 实现 Knowledge Agent
- 添加向量检索支持
- 更新配置文件

Closes #123
```

Issue 模板

Bug Report

```
问题描述
清晰描述问题

复现步骤
1. 步骤1
2. 步骤2
3. 预期行为

实际行为
描述实际发生的情况

环境信息
- 操作系统:
- Python/Node.js 版本:
- 项目版本:
```

Feature Request

```
功能描述
清晰描述需要的功能

需求背景
为什么需要这个功能

实现建议
(可选) 你对实现的想法
```

测试指南

运行测试

```
后端测试
cd backend
pytest
```

```
前端测试
cd frontend
npm test
```

测试要求

- 新增功能必须有对应的单元测试
- Bug修复必须有回归测试
- 测试覆盖率应保持在 80% 以上

沟通渠道

- GitHub Issues: 报告问题和功能请求
- GitHub Discussions: 讨论设计和架构
- Slack/Discord: 实时交流（如有）

许可证

贡献的代码将遵循项目的 MIT 许可证。

```

README.md
```markdown
# AgentMatrix

**多智能体动态协同与国产算力优化平台**

基于多Agent协同 + 动态算力路由的AI系统，旨在降低API调用成本、提高响应速度、优化国产算力利用率。

## 核心特性

- **多Agent协同架构**：Knowledge、Summary、Writer、Review、Judge、Result六大Agent协同工作
- **动态算力路由**：智能判断任务复杂度，简单任务本地完成，复杂任务云端增强
- **本地-云端混合推理**：结合国产轻量模型与云端大型模型
- **世界书/知识包系统**：每个Agent拥有专属Prompt和知识包
- **AI工作流可视化**：实时展示Agent工作状态、推理流程和KPI数据

## 技术栈

| 分类 | 技术 |
|-----|-----|
| 前端 | React, Next.js, TailwindCSS |
| 后端 | FastAPI |
| 本地模型 | Ollama (Qwen2.5-3B, DeepSeek-R1-Distill) |
| 云端API | Gemini API |
| 数据存储 | SQLite, JSON, FAISS (后期) |

## 项目结构
```

AgentMatrix/ ├── frontend/ # 前端项目 (Next.js) ├── backend/ # 后端项目 (FastAPI) ├── shared/ # 共享模块 ├── configs/ # 配置文件 ├── docs/ # 文档 ├── scripts/ # 脚本 └── .github/ # GitHub配置

```
## 快速开始

### 环境要求

- Python 3.12+
- Node.js 20+
- Ollama (本地模型支持)

### 安装依赖
```

```
```bash
后端依赖
cd backend
pip install -e .

前端依赖
cd frontend
npm install
```

启动服务

```
启动后端
cd backend
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

启动前端
cd frontend
npm run dev
```

核心工作流



开发指南

详细文档请参考 [docs/development/getting-started.md](#)

贡献

欢迎贡献代码！请阅读 [CONTRIBUTING.md](#) 了解贡献流程。

许可证

MIT License - 详见 [LICENSE](#)

```

backend\.env.example

```text
APP_NAME=AgentMatrix
APP_VERSION=0.1.0
APP_ENV=development

SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=true
```

```
LOG_LEVEL=INFO
LOG_FILE=logs/system.log

DATABASE_URL=sqlite:///./agentmatrix.db

OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b

DEEPSEEK_API_KEY=your-deepseek-api-key
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-rl-distill

GEMINI_API_KEY=your-gemini-api-key
GEMINI_MODEL=gemini-pro

COMPLEXITY_THRESHOLD=0.65

MAX_CONCURRENT_TASKS=10
MAX_RETRY_ATTEMPTS=3

ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
```

backend\agents__init__.py

backend\agents\base__init__.py

backend\agents\base\agent.py

```
from abc import ABC, abstractmethod
from typing import Any, Dict, Optional
from pydantic import BaseModel
from core.llm.client import get_llm_client

class AgentInput(BaseModel):
    content: str
    context: Optional[Dict[str, Any]] = None
    use_llm: bool = False
    use_cloud: bool = False

class AgentOutput(BaseModel):
    content: str
    success: bool = True
    message: Optional[str] = None
    metadata: Optional[Dict[str, Any]] = None
    model_used: Optional[str] = None

class BaseAgent(ABC):
    def __init__(self, agent_id: str, name: str):
        self.agent_id = agent_id
        self.name = name
        self.status = "idle"
        self.current_task = None
        self.last_error = None
        self.local_model = "qwen2.5:1.5b"
```

```

        self.cloud_model = "deepseek-rl-distill"
        self.llm_client = get_llm_client()

    @abstractmethod
    async def execute(self, input_data: AgentInput) -> AgentOutput:
        pass

    async def initialize(self) -> None:
        self.status = "ready"

    async def shutdown(self) -> None:
        self.status = "shutdown"

    def get_status(self) -> Dict[str, Any]:
        return {
            "agent_id": self.agent_id,
            "name": self.name,
            "status": self.status,
            "current_task": self.current_task,
            "last_error": self.last_error,
            "local_model": self.local_model,
            "cloud_model": self.cloud_model,
        }

    async def _set_status(self, status: str) -> None:
        self.status = status

    async def _set_current_task(self, task: Optional[str]) -> None:
        self.current_task = task

    async def _set_error(self, error: Optional[str]) -> None:
        self.last_error = error

    async def _call_llm(self, prompt: str, model: str = None, use_cloud: bool = False, **kwargs) -> str:
        """调用真实的 LLM 生成内容"""
        try:
            system_prompt = kwargs.get("system_prompt", None)

            if use_cloud:
                try:
                    from api.v1.config.router import _runtime_config
                    runtime_api_key = _runtime_config.get("deepseek_api_key")
                    if runtime_api_key:
                        self.llm_client.deepseek_api_key = runtime_api_key
                except:
                    pass

            # 根据 use_cloud 参数选择模型
            if use_cloud:
                llm_model = self.cloud_model
                import logging
                logger = logging.getLogger(__name__)
                logger.info(f"[Cloud] 正在调用云服务 DeepSeek, 模型: {llm_model}")
            else:
                llm_model = model or self.local_model
                import logging
                logger = logging.getLogger(__name__)
                logger.info(f"[Local] 正在调用本地模型 Ollama, 模型: {llm_model}")

            response = await self.llm_client.generate(prompt, use_cloud=use_cloud, system_prompt=system_prompt, model=llm_model)
        except Exception as e:
            import logging
            logger = logging.getLogger(__name__)
            logger.error(f"LLM调用失败: {str(e)}", exc_info=True)
            return f"LLM调用失败: {str(e)}"

    async def _call_llm_chat(self, messages: list, model: str = None, **kwargs) -> str:
        """调用真实的 LLM 聊天接口"""
        try:
            # 将消息列表转换为单个 prompt
            prompt = "\n".join([f"[{m.get('role', 'user')}] {m.get('content', '')}" for m in messages])
            return await self._call_llm(prompt, model=model, **kwargs)
        except Exception as e:
            return f"LLM聊天调用失败: {str(e)}"

```

backend\agents\base\agent_registry.py

```

from typing import Dict, Any, Optional
from .agent import BaseAgent
from agents.knowledge.agent import KnowledgeAgent
from agents.summary.agent import SummaryAgent
from agents.writer.agent import WriterAgent
from agents.review.agent import ReviewAgent
from agents.judge.agent import JudgeAgent
from agents.result.agent import ResultAgent


class AgentRegistry:
    def __init__(self):
        self.agents: Dict[str, BaseAgent] = {}

    def register_agent(self, agent: BaseAgent) -> None:
        self.agents[agent.agent_id] = agent

    def get_agent(self, agent_id: str) -> Optional[BaseAgent]:
        return self.agents.get(agent_id)

    def get_all_agents(self) -> Dict[str, BaseAgent]:
        return self.agents

    async def initialize_all_agents(self) -> None:
        self.register_agent(KnowledgeAgent())
        self.register_agent(SummaryAgent())
        self.register_agent(WriterAgent())
        self.register_agent(ReviewAgent())
        self.register_agent(JudgeAgent())
        self.register_agent(ResultAgent())

        for agent in self.agents.values():
            await agent.initialize()

    def initialize_all_agents_sync(self) -> None:
        import asyncio
        self.register_agent(KnowledgeAgent())
        self.register_agent(SummaryAgent())
        self.register_agent(WriterAgent())
        self.register_agent(ReviewAgent())
        self.register_agent(JudgeAgent())
        self.register_agent(ResultAgent())

        loop = asyncio.new_event_loop()
        try:
            asyncio.set_event_loop(loop)
            for agent in self.agents.values():
                loop.run_until_complete(agent.initialize())
        finally:
            loop.close()

    async def shutdown_all_agents(self) -> None:
        for agent in self.agents.values():
            await agent.shutdown()

    def get_all_agent_statuses(self) -> Dict[str, Any]:
        return {
            agent_id: agent.get_status()
            for agent_id, agent in self.agents.items()
        }

    async def execute_agent(self, agent_id: str, input_data: Any) -> Any:
        agent = self.get_agent(agent_id)
        if not agent:
            raise ValueError(f"Agent {agent_id} not found")
        return await agent.execute(input_data)

```

backend\agents\judge__init__.py

backend\agents\judge\agent.py

```

from agents.base.agent import BaseAgent, AgentInput, AgentOutput
from typing import Dict, Any, List, Tuple
import json
import logging
import re

logger = logging.getLogger(__name__)

CATEGORY_RULES = {
    "greeting": {
        "base_complexity": 0.10,
        "force_decision": "local_output",
        "patterns": [r"^(你好|您好|hi|hello|嗨|hey|早上好|下午好|晚上好|good morning|good afternoon|good evening)[\s!!。.,, ]"
*"$",
            r"^(在吗|在不在|有人吗|你在吗)[\s!!。.,, ]*$",
            "examples": "你好, hi, 早上好"
    ],
    "identity": {
        "base_complexity": 0.12,
        "force_decision": "local_output",
        "patterns": [r"你是谁", r"你叫什么", r"你的名字", r"介绍自己", r"自我介绍", r"你是干什么的",
            r"你是什么", r"你是做什么的", r"你的身份", r"你是哪个平台", r"你是哪个公司"],
            "examples": "你是谁, 你叫什么名字"
    ],
    "chitchat": {
        "base_complexity": 0.15,
        "force_decision": "local_output",
        "patterns": [r"^(天气|心情|无聊|开心|难过|累了|困了|饿了|谢谢|感谢|辛苦)",
            r"(怎么样|好不好|行不行|可以吗|对吗|是吗|对吧)[\s!!。.,, ]*$",
            "examples": "今天天气不错, 我心情好, 谢谢你"
    ],
    "simple_fact": {
        "base_complexity": 0.25,
        "force_decision": None,
        "patterns": [r"^(什么是|什么叫|是什么|是谁|哪一个|哪一种|什么时候|在哪里|在哪里能|多少钱)",
            r"(的定义|的意思|含义|概念).*[\s!!。.,, ]*$",
            r"^(介绍|简述|概述).*(是什么|特点|功能|作用)"]$,
            "examples": "什么是麒麟OS, 苹果多少钱一斤"
    ],
    "knowledge_qa": {
        "base_complexity": 0.45,
        "force_decision": None,
        "patterns": [r"^(有什么|有哪些|怎么样|如何|为什么|怎么|怎样)",
            r"(特点|优势|缺点|区别|对比|比较|优劣|不同).*$",
            "examples": "麒麟OS有什么特点, AI和机器学习有什么区别"
    ],
    "howto": {
        "base_complexity": 0.55,
        "force_decision": None,
        "patterns": [r"^(怎么|如何|怎样)(安装|配置|使用|操作|设置|部署|搭建|编写|开发|创建|建立)",
            r"(教程|指南|步骤|方法|技巧|攻略).*$",
            "examples": "怎么安装麒麟系统, 如何配置AI开发环境"
    ],
    "creation": {
        "base_complexity": 0.65,
        "force_decision": None,
        "patterns": [r"^(帮我写|帮我生成|帮我创作|帮我做|写一封|写一篇|写一个|生成一篇|生成一个|创作)",
            r"(情书|信件|诗歌|小说|故事|祝福语|感谢信|邀请函|演讲稿|文案|脚本|诗歌)",
            r"(代写|代笔)"]$,
            "examples": "帮我写一封情书, 写一篇演讲稿"
    ],
    "planning": {
        "base_complexity": 0.75,
        "force_decision": None,
        "patterns": [r"^(策划|规划|方案|计划|安排|组织|筹备)(.*(校园|活动|项目|会议|赛事|运动会|晚会|展览|比赛))",
            r"(校园|活动|项目|赛事|运动会|晚会)(.*(策划|规划|方案|计划))",
            r"(制定|拟定|编制).*(方案|计划|规划|预算)"]$,
            "examples": "校园运动会策划方案, 活动规划"
    ],
}

```

```

"complex_task": {
    "base_complexity": 0.85,
    "force_decision": None,
    "patterns": [r"(完整|全面|详细|深度|专业).*(方案|报告|设计|分析|论文|文档|PPT|答辩)",
                 r"(系统|架构|平台).*(设计|开发|实现|搭建|构建)",
                 r"(多步骤|多阶段|综合性|系统性).*(任务|项目|工程)",
                 r"(技术选型|架构设计|系统设计|项目设计).*$"],
    "examples": "完整AI项目答辩方案与PPT，系统架构设计"
}

}

COMPLEXITY_KEYWORDS = {
    "medium": ["方案", "规划", "报告", "分析", "设计", "演示", "文档", "论文", "PPT", "思维导图", "流程图"],
    "high": ["详细", "完整", "全面", "深度", "专业", "系统性", "综合性", "预算", "时间线", "风险评估"],
    "critical": ["多步骤推理", "技术方案", "架构设计", "系统设计", "算法设计", "技术选型",
                 "端云协同", "多智能体", "RAG", "检索增强", "知识蒸馏"]
}

}

class JudgeAgent(BaseAgent):
    def __init__(self):
        super().__init__("judge", "Judge Agent")
        self.local_model = "phi4-mini:3.8b"

    async def execute(self, input_data: AgentInput) -> AgentOutput:
        await self._set_status("processing")
        await self._set_current_task(f"复杂度判断: {input_data.content[:50]}...")

        try:
            input_data_dict = json.loads(input_data.content)
            user_task = input_data_dict.get("user_task", "")
            summary_result = input_data_dict.get("summary_result", {})
            review_result = input_data_dict.get("review_result", {})
            writer_output = input_data_dict.get("writer_output", "")
            knowledge_found = input_data_dict.get("knowledge_found", False)

            judge_result = self._judge_complexity(
                user_task, summary_result, review_result, writer_output, knowledge_found
            )

            await self._set_status("idle")
            await self._set_current_task(None)

            executed_locally = judge_result["decision"] == "local_output"
            model_used = "rule-engine"

            logger.info(
                f"Judge Agent - Category: {judge_result['category']}, "
                f"Decision: {judge_result['decision']}, "
                f"Complexity: {judge_result['complexity_score']:.2f}, "
                f"Review Score: {judge_result['review_score']:.2f}, "
                f"Local: {executed_locally}"
            )

            return AgentOutput(
                content=json.dumps(judge_result, ensure_ascii=False),
                success=True,
                message=judge_result["decision"],
                metadata={
                    "complexity_score": judge_result["complexity_score"],
                    "review_score": judge_result["review_score"],
                    "decision": judge_result["decision"],
                    "cloud_mode": judge_result["cloud_mode"],
                    "category": judge_result["category"],
                    "executed_locally": executed_locally,
                    "model_used": model_used,
                    "reason": judge_result.get("reason", [])
                },
                model_used=model_used
            )

        except Exception as e:
            await self._set_error(str(e))
            await self._set_status("error")
            return AgentOutput(
                content="",
                success=False,
                message=str(e)
            )

```

```

def _classify_question(self, user_task: str) -> Tuple[str, float, str]:
    user_task_clean = user_task.strip()

    for category, config in CATEGORY_RULES.items():
        for pattern in config["patterns"]:
            if re.search(pattern, user_task_clean, re.IGNORECASE):
                return category, config["base_complexity"], config.get("force_decision", None)

    if len(user_task_clean) < 5:
        return "chitchat", 0.15, "local_output"

    if len(user_task_clean) < 15:
        return "simple_fact", 0.25, None

    return "knowledge_qa", 0.45, None

def _calculate_complexity(self, user_task: str, writer_output: str, category: str,
                        base_complexity: float, knowledge_found: bool) -> float:
    score = base_complexity

    input_len = len(user_task.strip())
    if input_len > 500:
        score += 0.25
    elif input_len > 300:
        score += 0.18
    elif input_len > 150:
        score += 0.10
    elif input_len > 50:
        score += 0.05

    output_len = len(writer_output.strip())
    if output_len > 2000:
        score += 0.15
    elif output_len > 1000:
        score += 0.10
    elif output_len > 500:
        score += 0.05

    medium_count = sum(1 for kw in COMPLEXITY_KEYWORDS["medium"] if kw in user_task or kw in writer_output)
    high_count = sum(1 for kw in COMPLEXITY_KEYWORDS["high"] if kw in user_task or kw in writer_output)
    critical_count = sum(1 for kw in COMPLEXITY_KEYWORDS["critical"] if kw in user_task or kw in writer_output)

    score += medium_count * 0.04
    score += high_count * 0.06
    score += critical_count * 0.10

    if not knowledge_found and category not in ("greeting", "identity", "chitchat"):
        score += 0.15

    if "?" in user_task or "? " in user_task:
        question_count = user_task.count("?") + user_task.count("? ")
        if question_count >= 2:
            score += 0.10

    if any(sep in user_task for sep in ["\n", "\r", ";", ":", ";"]):
        parts = re.split(r'[\n\r;:]', user_task)
        parts = [p.strip() for p in parts if p.strip()]
        if len(parts) >= 3:
            score += 0.15

    return round(min(1.0, max(0.0, score)), 2)

def _judge_complexity(self, user_task: str, summary_result: Dict[str, Any],
                    review_result: Dict[str, Any], writer_output: str,
                    knowledge_found: bool) -> Dict[str, Any]:

    category, base_complexity, force_decision = self._classify_question(user_task)

    if isinstance(review_result, str):
        try:
            review_result = json.loads(review_result)
        except:
            review_result = {}

    review_score = review_result.get("review_score", 0.7)
    if review_score == 0.0:
        review_score = 0.55

    complexity_score = self._calculate_complexity(
        user_task, writer_output, category, base_complexity, knowledge_found

```

```

    )

    decision, cloud_mode, reason = self._make_decision(
        category, complexity_score, review_score, user_task,
        knowledge_found, force_decision
    )

    return {
        "complexity_score": complexity_score,
        "review_score": round(review_score, 2),
        "decision": decision,
        "cloud_mode": cloud_mode,
        "category": category,
        "reason": reason
    }

def _make_decision(self, category: str, complexity_score: float,
                  review_score: float, user_task: str,
                  knowledge_found: bool, force_decision: str) -> Tuple[str, str, List[str]]:

    from app.config import settings
    has_api_key = settings.deepseek_api_key and settings.deepseek_api_key.strip()

    if force_decision is not None:
        reason = [f"问题类别为 {category} ({CATEGORY_RULES[category]['examples']})，强制{force_decision}",
                  f"复杂度评分: {complexity_score:.2f}"]
        if not has_api_key and force_decision == "cloud_enhance":
            reason.append("DeepSeek API Key 未设置，降级为本地输出")
            return ("local_output", "none", reason)
        return (force_decision, "none", reason)

    if not has_api_key:
        return ("local_output", "none", ["DeepSeek API Key 未设置，强制本地输出"])

    reason = [f"问题类别: {category}"]

    if knowledge_found:
        reason.append(f"知识库已命中，本地直接输出（复杂度: {complexity_score:.2f}）")
        return ("local_output", "none", reason)

    if not knowledge_found and complexity_score < 0.50:
        reason.append(f"知识库未命中但复杂度较低（{complexity_score:.2f} < 0.50），本地输出")
        return ("local_output", "none", reason)

    if not knowledge_found and complexity_score >= 0.50:
        reason.append(f"知识库未命中且复杂度较高（{complexity_score:.2f} >= 0.50），调用DeepSeek云端重写")
        return ("cloud_enhance", "full_rewrite", reason)

    reason.append(f"综合判定，云端增强")
    return ("cloud_enhance", "full_rewrite", reason)

async def _judge_complexity_with_llm(self, user_task: str, summary_result: Dict[str, Any],
                                   review_result: Dict[str, Any], writer_output: str,
                                   use_cloud: bool = False) -> Dict[str, Any]:

    if isinstance(review_result, str):
        try:
            review_result = json.loads(review_result)
        except:
            review_result = {}

    review_score = review_result.get("review_score", 0.7)

    category_info = "\n".join([
        f"- {cat}: 基数{cfg['base_complexity']}, 示例: {cfg['examples']}"
        for cat, cfg in CATEGORY_RULES.items()
    ])

    prompt = f"""
你是 Judge Agent。你必须严格遵循规则引擎的逻辑做决策。

=== 问题分类体系 ===
{category_info}

=== 决策矩阵（严格遵循）===

1. greeting/identity/chitchat → 强制 local_output（无论knowledge是否命中）
2. simple_fact + knowledge命中 → local_output
3. simple_fact + knowledge未命中 → cloud_enhance + full_rewrite
4. 其他类别:

```

```

- complexity < 0.35 → local_output
- complexity 0.35-0.60 + review >= 0.70 → local_output
- complexity 0.35-0.60 + review < 0.70 → cloud_enhance + polish
- complexity 0.60-0.80 + review >= 0.80 → local_output
- complexity 0.60-0.80 + review < 0.80 → cloud_enhance + full_rewrite
- complexity > 0.80 → cloud_enhance + full_rewrite

=== 当前任务 ===
用户输入: {user_task}
Writer输出长度: {len(writer_output)}
Review评分: {review_score}

请分类并输出JSON:
{
  "category": "问题类别",
  "complexity_score": 0.0,
  "review_score": {review_score},
  "decision": "local_output或cloud_enhance",
  "cloud_mode": "none/polish/full_rewrite",
  "reason": ["理由1", "理由2"]
}
"""

response = await self._call_llm(prompt, model=self.local_model, use_cloud=use_cloud, temperature=0.1)

try:
    result = json.loads(response)
    if all(k in result for k in ["complexity_score", "decision", "cloud_mode", "reason"]):
        result["review_score"] = round(review_score, 2)
        result["complexity_score"] = round(float(result["complexity_score"]), 2)
        if "category" not in result:
            result["category"] = "unknown"
        return result
    except Exception:
        pass

return self._judge_complexity(user_task, summary_result, review_result, writer_output, True)

```

backend\agents\knowledge__init__.py

backend\agents\knowledge\agent.py

```

import sys
import os

agent_file = os.path.abspath(__file__)
agent_dir = os.path.dirname(os.path.dirname(os.path.dirname(agent_file)))
backend_dir = os.path.realpath(agent_dir)

if backend_dir not in sys.path:
    sys.path.insert(0, backend_dir)

knowledge_service_path = os.path.join(backend_dir, 'knowledge', 'service.py')
if os.path.exists(knowledge_service_path):
    import importlib.util
    spec = importlib.util.spec_from_file_location("knowledge.service", knowledge_service_path)
    knowledge_service = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(knowledge_service)
    KnowledgeService = knowledge_service.KnowledgeService
else:
    from knowledge.service import KnowledgeService

from agents.base.agent import BaseAgent, AgentInput, AgentOutput
from prompts.template_manager import get_prompt_manager
from typing import Dict, Any, List, Optional
import json

```

```

class KnowledgeAgent(BaseAgent):
    def __init__(self):
        super().__init__("knowledge", "Knowledge Agent")
        self.local_model = "qwen2.5:1.5b"
        self.knowledge_service = KnowledgeService()
        self.prompt_manager = get_prompt_manager()

        self.identity_rules = [
            "我来自 AgentMatrix 平台——多智能体动态协同与国产算力优化平台",
            "我是 Knowledge Agent，负责知识检索和知识增强",
            "我不干预其他 Agent 的工作流程",
            "我只提供知识支持，不直接回答用户问题",
            "我的输出是增强后的上下文，供后续 Agent 使用"
        ]

        self.system_keywords = ["我是谁", "你是谁", "knowledge agent", "知识助手", "你的职责", "你的任务"]

        self.intervention_keywords = [
            "帮我写", "生成", "总结", "评价", "判断", "导出",
            "你去做", "代替我", "帮我完成", "执行", "撰写",
            "创作", "设计", "策划", "报告", "方案"
        ]

        self.domain_knowledge = self._load_domain_knowledge()

    def _get_backend_path(self):
        return backend_dir

    def _load_domain_knowledge(self) -> List[Dict[str, Any]]:
        domain_knowledge = []
        role_file = os.path.join(self._get_backend_path(), "role.txt", "knowledge.txt")

        if os.path.exists(role_file):
            try:
                with open(role_file, "r", encoding="utf-8") as f:
                    content = f.read()

                import re
                array_pattern = r'\[[{}`\[\]]*?\]\]'
                matches = re.findall(array_pattern, content, re.DOTALL)

                for match in matches:
                    try:
                        data = json.loads('[' + match + ']')
                        if isinstance(data, list):
                            for item in data:
                                if isinstance(item, dict) and 'keywords' in item and 'content' in item:
                                    domain_knowledge.append(item)
                    except json.JSONDecodeError:
                        continue

                obj_pattern = r'\[{}`\[\]]+\]'
                obj_matches = re.findall(obj_pattern, content)
                for match in obj_matches:
                    try:
                        data = json.loads(match)
                        if isinstance(data, dict) and 'keywords' in data and 'content' in data:
                            exists = False
                            for item in domain_knowledge:
                                if item.get("keywords") == data.get("keywords"):
                                    exists = True
                                    break
                            if not exists:
                                domain_knowledge.append(data)
                    except json.JSONDecodeError:
                        continue

            except Exception as e:
                print(f"Failed to load domain knowledge: {e}")

        return domain_knowledge

    def _detect_identity_query(self, content: str) -> bool:
        content_lower = content.lower()
        for kw in self.system_keywords:
            if kw.lower() in content_lower:
                return True
        return False

```

```

def _detect_intervention(self, content: str) -> bool:
    content_lower = content.lower()
    for kw in self.intervention_keywords:
        if kw.lower() in content_lower:
            return True
    return False

def _return_identity_info(self, original_query: str = "") -> AgentOutput:
    identity_content = f"用户查询: {original_query}\n\n【知识类型】平台身份查询\n用户询问 AgentMatrix 平台身份信息，这是一个简单的基础身份问答。"
    return AgentOutput(
        content=identity_content,
        success=True,
        message="身份识别完成",
        metadata={
            "knowledge_type": "identity",
            "knowledge_count": 0,
            "matched_keywords": [],
            "enhanced": False,
            "model_used": self.local_model
        }
    )

def _reject_intervention(self) -> AgentOutput:
    return AgentOutput(
        content="我是 Knowledge Agent，仅负责知识检索和增强。\\n\\n"
            "如需内容生成、总结、评审等功能，请等待后续 Agent 处理。",
        success=True,
        message="检测到跨角色请求，已拒绝执行",
        metadata={
            "knowledge_type": "system",
            "knowledge_count": 0,
            "matched_keywords": [],
            "enhanced": False,
            "reason": "role_boundary"
        }
    )

def _extract_keywords(self, content: str) -> List[str]:
    keywords_found = []
    content_lower = content.lower()

    for item in self.domain_knowledge:
        for kw in item.get("keywords", []):
            kw_lower = kw.lower()
            if kw_lower in content_lower and kw not in keywords_found:
                keywords_found.append(kw)

    common_keywords = ["AI", "人工智能", "校园", "教育", "规划", "方案",
                        "系统", "开发", "设计", "报告", "分析", "端云协同",
                        "多智能体", "RAG", "检索增强", "知识蒸馏",
                        "马拉松", "活动策划", "运动会", "志愿服务", "赛事",
                        "跑步", "活动", "策划", "组织", "安全", "预算",
                        "办公", "WPS", "Office", "会议", "邮件", "项目管理",
                        "国产操作系统", "麒麟", "统信", "deepin", "鸿蒙", "信创",
                        "生活", "健康", "营养", "急救", "天气", "交通",
                        "法律", "理财", "考试", "奖学金", "就业", "金融"]

    for kw in common_keywords:
        kw_lower = kw.lower()
        if kw_lower in content_lower and kw not in keywords_found:
            keywords_found.append(kw)

    for kb_keyword in self.knowledge_service.get_all_keywords():
        if kb_keyword.lower() in content_lower and kb_keyword not in keywords_found:
            keywords_found.append(kb_keyword)

    return keywords_found

def _search_domain_knowledge(self, keywords: List[str]) -> List[Dict[str, Any]]:
    results = []
    content_lower = " ".join(keywords).lower()

    for item in self.domain_knowledge:
        item_keywords = item.get("keywords", [])
        for kw in item_keywords:
            if kw.lower() in content_lower:
                results.append({
                    "knowledge_type": "definition",
                    "query": kw,
                    "content": item.get("content", "")
                })

```

```

        "source": "roletxt/knowledge.txt",
        "confidence": 0.9
    ))
    break

    return results

def _search_knowledge_base(self, keywords: List[str]) -> List[str]:
    return self.knowledge_service.search_by_keywords(keywords, limit=5)

def _enhance_content(self, original: str, domain_items: List[Dict[str, Any]],
                    general_items: List[str], keywords: List[str]) -> str:
    if not domain_items and not general_items:
        return original

    enhanced = f"【知识增强】\n用户查询: {original}\n\n"

    if domain_items:
        enhanced += "【领域知识】\n"
        for i, item in enumerate(domain_items, 1):
            enhanced += f"{i}. [{item.get('knowledge_type', 'fact')}] {item.get('content', '')}\n"
            enhanced += f"    来源: {item.get('source', 'unknown')} | 置信度: {item.get('confidence', 0.8)}\n\n"

    if general_items:
        enhanced += "【通用知识】\n"
        for i, item in enumerate(general_items, 1):
            enhanced += f"{i}. {item}\n"

    enhanced += f"\n【匹配关键词】{', '.join(keywords)}"

    return enhanced

async def execute(self, input_data: AgentInput) -> AgentOutput:
    await self._set_status("processing")
    await self._set_current_task(f"检索知识: {input_data.content[:50]}...")

    try:
        if self._detect_identity_query(input_data.content):
            await self._set_status("idle")
            await self._set_current_task(None)
            return self._return_identity_info(input_data.content)

        keywords = self._extract_keywords(input_data.content)

        domain_knowledge_items = self._search_domain_knowledge(keywords)
        general_knowledge_items = self._search_knowledge_base(keywords)

        has_knowledge = len(domain_knowledge_items) > 0 or len(general_knowledge_items) > 0

        if self._detect_intervention(input_data.content) and not has_knowledge:
            await self._set_status("idle")
            await self._set_current_task(None)
            return self._reject_intervention()

        enhanced_content = self._enhance_content(
            input_data.content,
            domain_knowledge_items,
            general_knowledge_items,
            keywords
        )

        total_knowledge_count = len(domain_knowledge_items) + len(general_knowledge_items)

        await self._set_status("idle")
        await self._set_current_task(None)

    return AgentOutput(
        content=enhanced_content,
        success=True,
        message="知识检索完成",
        metadata={
            "knowledge_type": "enhanced",
            "knowledge_count": total_knowledge_count,
            "domain_knowledge_count": len(domain_knowledge_items),
            "general_knowledge_count": len(general_knowledge_items),
            "matched_keywords": keywords,
            "enhanced": total_knowledge_count > 0,
            "model_used": self.local_model
        }
    )

```

```

except Exception as e:
    await self._set_error(str(e))
    await self._set_status("error")
    return AgentOutput(
        content="",
        success=False,
        message=str(e)
    )

```

backend\agents\result__init__.py

backend\agents\result\agent.py

```

from agents.base.agent import BaseAgent, AgentInput, AgentOutput
from typing import Dict, Any
import json
from datetime import datetime

class ResultAgent(BaseAgent):
    def __init__(self):
        super().__init__("result", "Result Agent")
        self.local_model = "qwen2.5:1.5b"
        from app.config import settings
        self.cloud_model = settings.deepseek_model if hasattr(settings, 'deepseek_model') else "deepseek-chat"

    async def execute(self, input_data: AgentInput) -> AgentOutput:
        await self._set_status("processing")
        await self._set_current_task(f"结果导出: {input_data.content[:50]}...")

        try:
            input_data_dict = json.loads(input_data.content)

            user_task = input_data_dict.get("user_task", "")
            summary_result = input_data_dict.get("summary_result", {})
            review_result = input_data_dict.get("review_result", {})
            judge_result = input_data_dict.get("judge_result", {})
            writer_output = input_data_dict.get("writer_output", "")
            executed_locally = input_data_dict.get("executed_locally", True)
            complexity_score = input_data_dict.get("complexity_score", 0.0)
            judge_decision = input_data_dict.get("judge_decision", "local_output")
            cloud_mode = input_data_dict.get("cloud_mode", "none")

            model_used = self.local_model

            # 检查是否有 DeepSeek API Key
            from app.config import settings
            has_api_key = settings.deepseek_api_key and settings.deepseek_api_key.strip()

            if has_api_key and input_data.use_cloud and cloud_mode != "none":
                writer_output = await self._enhance_with_cloud(user_task, summary_result, writer_output, cloud_mode)
                executed_locally = False
                model_used = self.cloud_model

            final_result = self._format_result(judge_result, input_data.context, writer_output, complexity_score, executed_locally, judge_decision, cloud_mode)

            await self._set_status("idle")
            await self._set_current_task(None)

            return AgentOutput(
                content=final_result,
                success=True,
                message="结果生成完成",
                metadata={"format": "markdown", "length": len(final_result), "model_used": model_used, "executed_locally": executed_locally},

```

```

        model_used=model_used
    )

    except Exception as e:
        await self._set_error(str(e))
        await self._set_status("error")
        return AgentOutput(
            content="",
            success=False,
            message=str(e)
        )

    async def _enhance_with_cloud(self, user_task: str, summary_result: Any, writer_output: str, cloud_mode: str) -> str:
        summary_text = ""
        if isinstance(summary_result, str):
            try:
                summary_data = json.loads(summary_result)
                summary_text = summary_data.get("summary", summary_result)
                keywords = summary_data.get("keywords", [])
                requirements = summary_data.get("requirements", [])
                if keywords:
                    summary_text += f"\n关键词: {'; '.join(keywords)}"
                if requirements:
                    summary_text += f"\n需求点: {'; '.join(requirements)}"
            except:
                summary_text = str(summary_result)

        system_prompt = "你是 AgentMatrix 平台的 AI 助手。你需要根据用户需求、任务摘要，重新生成一份更高质量、更专业的完整回复。你永远不代表任何其他公司的AI助手。"

        prompt = f"""
请根据以下信息，重新生成一份高质量、专业的完整回复。

【用户问题】
{user_task}

【需求摘要】
{summary_text}

【重写要求】
1. 内容必须专业、准确、有深度
2. 直接回应用户的问题，不要偏离
3. 使用清晰的 Markdown 格式
4. 确保内容的可执行性和实用性
5. 不要包含“根据您的要求”等开场白

请直接输出最终内容：
"""

        response = await self._call_llm(prompt, model=self.cloud_model, use_cloud=True, system_prompt=system_prompt)
        return response

    def _format_result(self, judge_result: Dict[str, Any], context: Dict[str, Any], writer_output: str,
                      complexity_score: float, executed_locally: bool, judge_decision: str, cloud_mode: str) -> str:

        if isinstance(judge_result, str):
            try:
                judge_result = json.loads(judge_result)
            except:
                judge_result = {}

        # 直接返回Writer的内容，不要添加那些复杂的报告信息
        if writer_output:
            cleaned_content = self._clean_content(writer_output)
            return cleaned_content
        else:
            return "暂无生成内容，请重试。"

    def _clean_content(self, content: str) -> str:
        try:
            parsed = json.loads(content)
            if isinstance(parsed, dict) and "content" in parsed:
                return str(parsed["content"])
            elif isinstance(parsed, dict) and "task" in parsed:
                return json.dumps(parsed, ensure_ascii=False, indent=2)
        except:
            pass

        content = content.replace("【知识增强】", "")
        content = content.replace("【领域知识】", "")

```

```

content = content.replace("【通用知识】", "")
content = content.replace("【匹配关键词】", "")

return content.strip()

```

backend\agents\review__init__.py

backend\agents\review\agent.py

```

from agents.base.agent import BaseAgent, AgentInput, AgentOutput
from typing import Dict, Any, List
import json
import os
import re

class ReviewAgent(BaseAgent):
    def __init__(self):
        super().__init__("review", "Review Agent")
        self.local_model = "phi4-mini:3.8b"
        self.world_rules = self._load_world_rules()

    def _load_world_rules(self) -> List[Dict[str, Any]]:
        rules = []
        role_file = os.path.join(os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__)))), "roletxt", "review.txt")

        if os.path.exists(role_file):
            try:
                with open(role_file, "r", encoding="utf-8") as f:
                    content = f.read()
                try:
                    rules = json.loads(content)
                except json.JSONDecodeError:
                    pass
            except Exception as e:
                pass

        if not rules:
            rules = [
                {"keywords": ["校园", "活动", "策划"], "inject": "必须检查是否包含活动流程、预算、时间安排。"},
                {"keywords": ["项目", "方案"], "inject": "必须检查是否包含实施步骤与目标分析。"},
                {"keywords": ["ppt", "汇报"], "inject": "必须检查是否具备结构化章节。"}
            ]

        return rules

    def _get_injection_rules(self, content: str) -> str:
        injections = []
        content_lower = content.lower()
        for rule in self.world_rules:
            for kw in rule.get("keywords", []):
                if kw.lower() in content_lower:
                    injections.append(rule.get("inject", ""))
                    break
        return "\n".join(injections)

    def _detect_simple_conversation(self, user_task: str, writer_output: str) -> bool:
        task_lower = user_task.strip().lower()
        if len(task_lower) < 10:
            return True
        simple_patterns = [
            r"^(你好|您好|hi|hello|嗨|hey|早上好|下午好|晚上好)",
            r"^(在吗|在不在|有人吗|你在吗)",
            r"^(你是谁|你叫什么|你的名字|自我介绍|你是什么)",
            r"^(谢谢|感谢|辛苦|多谢|thanks)",
            r"^(天气|心情|无聊|开心|难过|累了|困了|饿了)",

```

```

    ]
    for pattern in simple_patterns:
        if re.search(pattern, task_lower):
            return True
    if len(writer_output) < 200 and not re.search(r"(# |## |一、|二、|1\.|2\.)", writer_output):
        return True
    return False

async def execute(self, input_data: AgentInput) -> AgentOutput:
    await self._set_status("processing")
    await self._set_current_task(f"质量评审: {input_data.content[:50]}...")

    try:
        input_data_dict = json.loads(input_data.content)
        user_task = input_data_dict.get("user_task", "")
        summary = input_data_dict.get("summary", "")
        writer_output = input_data_dict.get("writer_output", "")

        if input_data.use_llm:
            review_result = await self._review_content_with_llm(user_task, summary, writer_output, use_cloud=input_data.us
e_cloud)
        else:
            review_result = self._review_content(user_task, summary, writer_output)

        await self._set_status("idle")
        await self._set_current_task(None)

        model_used = self.cloud_model if input_data.use_cloud else self.local_model

        return AgentOutput(
            content=json.dumps(review_result, ensure_ascii=False),
            success=True,
            message="质量评审完成",
            metadata={
                "review_score": review_result["review_score"],
                "dimensions": review_result["dimensions"],
                "issues": review_result["issues"],
                "suggestions": review_result["suggestions"],
                "pass": review_result["pass"],
                "model_used": model_used,
                "use_cloud": input_data.use_cloud
            },
            model_used=model_used
        )

    except Exception as e:
        await self._set_error(str(e))
        await self._set_status("error")
        return AgentOutput(
            content="",
            success=False,
            message=str(e)
        )

def _review_content(self, user_task: str, summary: str, writer_output: str) -> Dict[str, Any]:
    is_simple_conversation = self._detect_simple_conversation(user_task, writer_output)

    if is_simple_conversation:
        return {
            "review_score": 0.85,
            "dimensions": {
                "structure": 0.80,
                "relevance": 0.90,
                "richness": 0.80,
                "professional": 0.85,
                "actionable": 0.85
            },
            "issues": [],
            "suggestions": ["简单对话, 内容自然合理"],
            "pass": True
        }

    structure = 0.5
    relevance = 0.7
    richness = 0.4
    professional = 0.5
    actionable = 0.5

    issues = []
    suggestions = []

```

```

if len(writer_output) < 100:
    issues.append("内容过短")
    richness -= 0.2
elif len(writer_output) > 500:
    richness += 0.2

if "活动流程" not in writer_output and "流程" not in writer_output:
    if "活动" in user_task or "策划" in user_task:
        issues.append("缺少活动流程")
        structure -= 0.2
        actionable -= 0.2

if "预算" not in writer_output:
    if "预算" in user_task or "活动" in user_task or "方案" in user_task:
        issues.append("缺少预算安排")
        structure -= 0.15
        actionable -= 0.15

if "时间" not in writer_output and "日期" not in writer_output:
    if "活动" in user_task or "策划" in user_task:
        issues.append("缺少时间安排")
        structure -= 0.15
        actionable -= 0.15

if "#" in writer_output or "一、" in writer_output or "1." in writer_output:
    structure += 0.2

if "##" in writer_output or "(一)" in writer_output or "1.1" in writer_output:
    structure += 0.1

task_lower = user_task.lower()
output_lower = writer_output.lower()
task_keywords = ["活动", "策划", "方案", "项目", "报告"]
matched_keywords = sum(1 for kw in task_keywords if kw in task_lower and kw in output_lower)
if matched_keywords >= 2:
    relevance += 0.2

if "总结" in writer_output or "结论" in writer_output:
    structure += 0.1
    professional += 0.1

if "具体" in writer_output or "详细" in writer_output or "步骤" in writer_output:
    actionable += 0.2

structure = max(0.0, min(1.0, structure))
relevance = max(0.0, min(1.0, relevance))
richness = max(0.0, min(1.0, richness))
professional = max(0.0, min(1.0, professional))
actionable = max(0.0, min(1.0, actionable))

review_score = (structure + relevance + richness + professional + actionable) / 5

if "内容过短" in issues:
    suggestions.append("增加内容详细度")
if "缺少活动流程" in issues:
    suggestions.append("补充活动流程章节")
if "缺少预算安排" in issues:
    suggestions.append("增加预算模块")
if "缺少时间安排" in issues:
    suggestions.append("添加时间线")
if not suggestions:
    suggestions.append("内容质量良好, 建议检查是否有遗漏细节")

return {
    "review_score": round(review_score, 2),
    "dimensions": {
        "structure": round(structure, 2),
        "relevance": round(relevance, 2),
        "richness": round(richness, 2),
        "professional": round(professional, 2),
        "actionable": round(actionable, 2)
    },
    "issues": issues,
    "suggestions": suggestions,
    "pass": review_score >= 0.65
}

```

```

async def _review_content_with_llm(self, user_task: str, summary: str, writer_output: str, use_cloud: bool = False) -> Dict[str, Any]:

```

```

        if self._detect_simple_conversation(user_task, writer_output):
            return self._review_content(user_task, summary, writer_output)

        injection_rules = self._get_injection_rules(user_task)

        few_shot = """
=== Few-shot 示例 ===

示例1（低质量）：
输入任务：写校园活动策划
Writer输出：举办活动促进同学交流。
输出：
{"review_score":0.42,"dimensions":{"structure":0.3,"relevance":0.7,"richness":0.2,"professional":0.4,"actionable":0.5},"issues":["内容过短","缺少活动流程","缺少预算与时间安排"],"suggestions":["增加活动目标","补充时间线","增加预算模块"],"pass":false}

示例2（高质量）：
输入任务：生成完整的XX系统设计方案
Writer输出：（包含完整章节、详细分析、实施步骤的专业文档）
输出：
{"review_score":0.86,"dimensions":{"structure":0.9,"relevance":0.9,"richness":0.8,"professional":0.9,"actionable":0.8},"issues":["预算部分略简略"],"suggestions":["补充详细预算表"],"pass":true}
"""

        prompt = f"""
你是 Review Agent。

你的职责：
1. 检查内容是否完整
2. 检查是否偏离用户需求
3. 检查结构是否合理
4. 检查内容是否专业
5. 给出质量评分与修改建议

你不能重新生成全文。

{injection_rules}

{few_shot}

你必须：
- 使用结构化JSON输出
- 不允许输出废话
- 必须指出问题
- 必须给出评分
- 必须给出建议

评分标准：
1. structure（结构完整性）- 检查是否有清晰的章节、标题、逻辑层次
2. relevance（需求相关性）- 检查是否符合用户原始需求
3. richness（内容丰富度）- 检查内容是否详细、全面
4. professional（专业性）- 检查语言表达是否专业、正式
5. actionable（可执行性）- 检查方案是否具体、可落地

每项评分范围：0~1

最终总分: review_score = 五项平均值

决策规则：
- review_score >= 0.8 → 高质量，允许直接输出
- 0.65 ~ 0.8 → 中等质量，建议本地增强
- < 0.65 → 低质量，建议调用云端API

用户任务: {user_task}

需求摘要: {summary}

Writer输出:
{writer_output}

输出格式必须严格遵守JSON:
{{
  "review_score": 0.0,
  "dimensions": {{
    "structure": 0.0,
    "relevance": 0.0,
    "richness": 0.0,
    "professional": 0.0,
    "actionable": 0.0
  }},
  "issues": [

```

```

"""
    ],
    "suggestions": [
        ""
    ],
    "pass": true
}}
"""

response = await self._call_llm(prompt, model=self.local_model, use_cloud=use_cloud, temperature=0.2)

try:
    result = json.loads(response)
    if "review_score" in result and "dimensions" in result:
        if result["review_score"] == 0.0:
            return self._review_content(user_task, summary, writer_output)
        result["pass"] = result["review_score"] >= 0.65
        return result
except Exception as e:
    pass

return self._review_content(user_task, summary, writer_output)

```

backend\agents\summary__init__.py

backend\agents\summary\agent.py

```

from agents.base.agent import BaseAgent, AgentInput, AgentOutput
from typing import Dict, Any, List, Tuple
import json
import re

class SummaryAgent(BaseAgent):
    def __init__(self):
        super().__init__("summary", "Summary Agent")
        self.local_model = "qwen2.5:1.5b"

    async def execute(self, input_data: AgentInput) -> AgentOutput:
        await self._set_status("processing")
        await self._set_current_task(f"摘要生成: {input_data.content[:50]}...")

        try:
            # 1. 解析 Knowledge Agent 的输出
            parsed_data = self._parse_knowledge_output(input_data.content)

            # 2. 提取用户原始问题
            original_question = parsed_data.get("original_question", input_data.content)

            # 3. 判断任务类型
            is_creative = any(keyword in original_question.lower() for keyword in ["情书", "信件", "诗歌", "诗", "小说", "故事", "祝福语", "感谢信"])

            if is_creative:
                # 创意类任务: 简单的摘要, 不生成方案大纲
                keywords = self._extract_keywords(input_data.content, parsed_data.get("knowledge_points", []))
                summary_result = {
                    "task": original_question,
                    "original_question": original_question,
                    "keywords": keywords,
                    "knowledge_points": parsed_data.get("knowledge_points", []),
                    "requirements": [],
                    "outline": [],
                    "summary": f"用户需求: {original_question}"
                }
            else:
                # 正常任务: 生成完整的结构化输出

```

32/271

```

general_match = re.search(general_knowledge_pattern, content, re.DOTALL)
if general_match:
    general_content = general_match.group(1)
    for line in general_content.strip().split("\n"):
        line = line.strip()
        if line and line[0].isdigit():
            content_part = re.sub(r'\d+\.\s*', '', line).strip()
            if content_part:
                result["knowledge_points"].append({
                    "type": "通用知识",
                    "content": content_part,
                    "source": "knowledge_base.json",
                    "confidence": 0.7
                })

# 如果没有解析到结构化知识, 尝试直接提取文本
if not result["knowledge_points"] and not result["original_question"]:
    result["original_question"] = content

return result

def _extract_task(self, content: str) -> str:
    """提取核心任务描述"""
    task_patterns = [
        r"(生成|创建|设计|规划|撰写|制定|编写|分析|评估)\s+(.+(?)(。|?|\n|$))",
        r"(需要|想要|希望|需求|请求)\s+(.+(?)(。|?|\n|$))"
    ]

    for pattern in task_patterns:
        match = re.search(pattern, content)
        if match:
            return f"{match.group(1)} {match.group(2)}"

# 如果没有匹配到, 返回前60个字符作为任务描述
clean_content = re.sub(r'【.*?】', '', content).strip()
return clean_content[:60] if len(clean_content) > 60 else clean_content

def _extract_keywords(self, content: str, knowledge_points: List[Dict]) -> List[str]:
    """从内容和知识点中提取关键词"""
    keywords_found = []

    # 预定义的关键词列表 (包含知识库中的关键词)
    predefined_keywords = [
        "AI", "人工智能", "校园", "教育", "规划", "方案", "系统",
        "开发", "设计", "报告", "分析", "研究", "评估", "优化",
        "马拉松", "活动策划", "运动会", "志愿服务", "端云协同",
        "多智能体", "RAG", "国产操作系统", "麒麟系统", "统信UOS",
        "会议", "文档", "办公", "安全", "预算", "时间", "目标",
        "鸿蒙", "deepin", "信创", "办公软件", "WPS", "项目管理",
        "健康", "营养", "急救", "天气", "交通", "法律", "理财",
        "考试", "奖学金", "就业", "金融", "AIGC", "提示词"
    ]

    content_lower = content.lower()
    for kw in predefined_keywords:
        if kw.lower() in content_lower and kw not in keywords_found:
            keywords_found.append(kw)

    # 从知识点中提取关键词
    for point in knowledge_points:
        point_content = point.get("content", "")
        for kw in predefined_keywords:
            if kw.lower() in point_content.lower() and kw not in keywords_found:
                keywords_found.append(kw)

    return keywords_found[:8]

def _extract_requirements(self, question: str, knowledge_points: List[Dict]) -> List[str]:
    """提取用户需求点"""
    requirements = []

    # 从问题中提取需求
    requirement_patterns = [
        (r"(需要|必须|应该|应当)\s+(.+(?)(。|?|\n|$))", "需要"),
        (r"(确保|保证)\s+(.+(?)(。|?|\n|$))", "确保"),
        (r"(考虑|考虑到)\s+(.+(?)(。|?|\n|$))", "考虑"),
        (r"(包含|包括)\s+(.+(?)(。|?|\n|$))", "包含"),
        (r"(符合|遵循)\s+(.+(?)(。|?|\n|$))", "符合")
    ]

```

```

for pattern, prefix in requirement_patterns:
    matches = re.findall(pattern, question)
    for match in matches:
        req = f"{prefix}{match[1]}"
        if req not in requirements and len(req) > 3:
            requirements.append(req)

# 从知识点中提取相关需求
for point in knowledge_points:
    content = point.get("content", "")
    # 提取知识中的关键点作为需求参考
    if "需要" in content or "应" in content:
        # 提取包含"需要"或"应"的短句
        sentences = re.split(r'[\.:;]', content)
        for sentence in sentences:
            if "需要" in sentence or "应" in sentence:
                sentence = sentence.strip()
                if sentence and len(sentence) > 5 and sentence not in requirements:
                    requirements.append(sentence)

return requirements[:6]

def _generate_outline(self, question: str, keywords: List[str], requirements: List[str]) -> List[str]:
    """根据任务和需求生成方案大纲"""
    outline = []

    # 分析任务类型确定大纲结构
    task_type = self._determine_task_type(question, keywords)

    if task_type == "活动策划":
        outline = [
            "一、活动概述",
            "二、活动目标",
            "三、活动流程安排",
            "四、人员分工",
            "五、预算规划",
            "六、安全保障措施",
            "七、应急预案"
        ]
    elif task_type == "方案设计":
        outline = [
            "一、需求分析",
            "二、方案目标",
            "三、方案设计",
            "四、实施步骤",
            "五、风险评估",
            "六、预期成果"
        ]
    elif task_type == "文档撰写":
        outline = [
            "一、引言",
            "二、主体内容",
            "三、结论",
            "四、参考文献"
        ]
    elif task_type == "分析报告":
        outline = [
            "一、问题描述",
            "二、现状分析",
            "三、解决方案",
            "四、实施建议"
        ]
    else:
        # 默认大纲
        outline = [
            "一、任务概述",
            "二、核心需求",
            "三、解决方案",
            "四、实施计划"
        ]

    return outline

def _determine_task_type(self, question: str, keywords: List[str]) -> str:
    """确定任务类型"""
    question_lower = question.lower()

    if any(kw in question_lower for kw in ["活动", "策划", "组织", "赛事", "运动会"]):
        return "活动策划"
    elif any(kw in question_lower for kw in ["方案", "设计", "规划", "系统"]):

```

```
        return "方案设计"
    elif any(kw in question_lower for kw in ["报告", "文档", "撰写", "编写"]):
        return "文档撰写"
    elif any(kw in question_lower for kw in ["分析", "评估", "研究"]):
        return "分析报告"

    return "通用任务"

def _generate_brief_summary(self, question: str, keywords: List[str], requirements: List[str]) -> str:
    """生成简短摘要"""
    summary = f"用户需求: {question[:40]}..." if len(question) > 40 else question
    if keywords:
        summary += f" | 关键词: {' '.join(keywords[:3])}"
    if requirements:
        summary += f" | 需求点: {len(requirements)}项"
    return summary
```

backend\agents\writer__init__.py

backend\agents\writer\agent.py

```
from agents.base.agent import BaseAgent, AgentInput, AgentOutput
from typing import Dict, Any, List
import json
import re

WRITER_TEMPLATES: Dict[str, List[Dict[str, str]]] = {
    "发言稿": [
        {"title": "开场问候", "content": "向听众问好, 介绍自己身份和发言场合。"},
        {"title": "发言主题", "content": "明确本次发言的核心主题和目的。"},
        {"title": "主体内容", "content": "围绕主题分点阐述, 包括背景、现状、观点。"},
        {"title": "案例分析", "content": "用具体例子或数据支撑观点。"},
        {"title": "号召或展望", "content": "总结发言, 提出呼吁或展望未来。"},
        {"title": "结束致谢", "content": "感谢听众, 礼貌结束。"}
    ],
    "竞选稿": [
        {"title": "自我介绍", "content": "介绍个人基本信息、竞选职位。"},
        {"title": "竞选动机", "content": "说明为什么参加竞选, 个人优势。"},
        {"title": "工作思路", "content": "如果当选后的具体工作规划和措施。"},
        {"title": "过往成绩", "content": "相关经历和取得的成绩证明能力。"},
        {"title": "承诺与决心", "content": "对选举人的承诺和服务决心。"},
        {"title": "结束语", "content": "感谢聆听, 请求支持。"}
    ],
    "工作报告": [
        {"title": "报告概述", "content": "说明报告的时间范围、主题和目的。"},
        {"title": "工作回顾", "content": "按时间或项目梳理完成的主要工作。"},
        {"title": "成果与数据", "content": "用具体数据和成果展示工作成效。"},
        {"title": "问题与不足", "content": "客观分析存在的问题和不足之处。"},
        {"title": "原因分析", "content": "分析问题产生的原因。"},
        {"title": "改进措施", "content": "提出具体的改进方案和措施。"},
        {"title": "下阶段计划", "content": "明确下一步工作重点和时间安排。"}
    ],
    "操作指南": [
        {"title": "概述", "content": "说明本指南的目的、适用范围和前置条件。"},
        {"title": "准备工作", "content": "列出所需工具、材料、环境要求。"},
        {"title": "操作步骤", "content": "按顺序详细描述每个操作步骤, 每一步配说明。"},
        {"title": "注意事项", "content": "列出容易出错的地方和安全警示。"},
        {"title": "常见问题", "content": "列出常见问题及解决方法。"},
        {"title": "附录", "content": "相关参考信息、术语解释或快捷方式。"}
    ],
    "策划案": [
        {"title": "项目背景", "content": "说明策划的起因、背景和必要性。"},
        {"title": "策划目标", "content": "明确策划要达成的具体目标。"},
        {"title": "活动/项目方案", "content": "详细描述方案内容、流程和时间安排。"},
        {"title": "资源配置", "content": "列出所需人员、物资、场地等资源。"},
        {"title": "预算规划", "content": "各项费用的预算明细表。"},
    ]
}
```

```

        {"title": "风险评估与应对", "content": "识别可能的风险及应急预案。"},
        {"title": "效果评估", "content": "如何评估策划案的执行效果。"}
    ],
    "会议纪要": [
        {"title": "会议基本信息", "content": "记录会议时间、地点、主持人、参会人员。"},
        {"title": "会议议程", "content": "列出会议讨论的主要议题。"},
        {"title": "讨论内容", "content": "逐条记录各议题的讨论要点和意见。"},
        {"title": "决议事项", "content": "明确会议形成的决议和决定。"},
        {"title": "行动项", "content": "列出待办任务、责任人和截止时间。"},
        {"title": "下次会议安排", "content": "下次会议的时间和议题预告。"}
    ],
    "方案设计": [
        {"title": "需求分析", "content": "分析用户需求和痛点。"},
        {"title": "方案目标", "content": "明确方案要达到的目标。"},
        {"title": "方案设计", "content": "详细描述方案的设计思路和架构。"},
        {"title": "实施步骤", "content": "制定实施方案的具体步骤。"},
        {"title": "风险评估", "content": "评估可能遇到的风险和应对措施。"},
        {"title": "预期成果", "content": "描述方案实施后的预期效果。"}
    ],
    "通用任务": [
        {"title": "任务概述", "content": "介绍任务的背景和目标。"},
        {"title": "核心需求", "content": "分析任务的核心需求。"},
        {"title": "解决方案", "content": "提出解决问题的方案。"},
        {"title": "实施计划", "content": "制定实施计划和时间表。"}
    ]
}

TEMPLATE_KEYWORD_MAP = [
    (["发言稿", "演讲稿", "讲话稿", "发言", "致辞", "演讲", "讲话", "发言稿"],
     (["竞选稿", "竞选", "选举", "竞聘", "参选"], "竞选稿")),
    (["工作报告", "工作汇报", "工作总结", "年终总结", "述职报告", "汇报", "述职", "工作报告"],
     (["操作指南", "使用指南", "用户手册", "操作手册", "教程", "使用说明", "说明书", "入门指南"], "操作指南")),
    (["策划案", "策划方案", "活动方案", "项目方案", "策划书"], "策划案"),
    (["会议纪要", "会议记录", "会谈纪要", "纪要", "会议", "会议纪要"],
     (["方案设计", "设计方案", "系统方案", "技术方案", "架构设计"], "方案设计")),
]

class WriterAgent(BaseAgent):
    def __init__(self):
        super().__init__("writer", "Writer Agent")
        self.local_model = "qwen2.5:1.5b"

    async def execute(self, input_data: AgentInput) -> AgentOutput:
        await self._set_status("processing")
        await self._set_current_task(f"内容生成: {input_data.content[:50]}...")

        try:
            parsed = self._parse_summary(input_data.content)

            if self._detect_simple_conversation(parsed):
                content = await self._generate_simple_response(parsed)
                return AgentOutput(
                    content=content, success=True, message="简单对话完成",
                    metadata={"content_length": len(content), "model_used": self.local_model, "task_type": "简单对话"},
                    model_used=self.local_model
                )

            if self._detect_simple_fact_question(parsed):
                content = await self._generate_fact_answer(parsed)
                return AgentOutput(
                    content=content, success=True, message="知识问答完成",
                    metadata={"content_length": len(content), "model_used": self.local_model, "task_type": "知识问答"},
                    model_used=self.local_model
                )

            task_type = self._determine_task_type(parsed)
            template = WRITER_TEMPLATES.get(task_type, WRITER_TEMPLATES["通用任务"])

            content = await self._generate_with_template_and_llm(parsed, template, task_type)

            await self._set_status("idle")
            await self._set_current_task(None)

            return AgentOutput(
                content=content, success=True, message="内容生成完成",
                metadata={"content_length": len(content), "model_used": self.local_model, "task_type": task_type},
                model_used=self.local_model
            )

```

```

except Exception as e:
    await self._set_error(str(e))
    await self._set_status("error")
    return AgentOutput(content="", success=False, message=str(e))

def _parse_summary(self, content: str) -> Dict[str, Any]:
    try:
        parsed = json.loads(content)
        return {
            "task": parsed.get("task", ""),
            "original_question": parsed.get("original_question", ""),
            "keywords": parsed.get("keywords", []),
            "knowledge_points": parsed.get("knowledge_points", []),
            "requirements": parsed.get("requirements", []),
            "outline": parsed.get("outline", []),
            "summary": parsed.get("summary", "")
        }
    except:
        return {"task": content, "original_question": content, "keywords": [], "knowledge_points": [],
            "requirements": [], "outline": [], "summary": content}

def _detect_simple_conversation(self, parsed: Dict[str, Any]) -> bool:
    task = parsed.get("original_question", parsed.get("task", ""))
    task_lower = task.strip().lower()

    patterns = [
        r"^(你好|您好|hi|hello|嗨|hey|早上好|下午好|晚上好)[\s!。.,,]*$",
        r"^(在吗|在不在|有人吗|你在吗)[\s!。.,,]*$",
        r"^(你是谁|你叫什么|你的名字|自我介绍|你是什么)",
        r"^(谢谢|感谢|辛苦|多谢|thanks)[\s!。.,,]*$",
    ]
    for p in patterns:
        if re.search(p, task_lower):
            return True
    return False

def _determine_task_type(self, parsed: Dict[str, Any]) -> str:
    task = parsed.get("task", "")
    keywords = parsed.get("keywords", [])
    combined = (task + " " + " ".join(keywords)).lower()

    for patterns, template_name in TEMPLATE_KEYWORD_MAP:
        for pattern in patterns:
            if pattern.lower() in combined:
                return template_name

    if any(kw in combined for kw in ["活动", "策划", "组织", "赛事", "运动会", "晚会"]):
        return "策划案"
    if any(kw in combined for kw in ["设计", "规划", "系统"]):
        return "方案设计"

    return "通用任务"

def _build_knowledge_text(self, knowledge_points: List[Dict]) -> str:
    if not knowledge_points:
        return ""
    text = ""
    for i, kp in enumerate(knowledge_points, 1):
        text += f"{i}. {kp.get('content', '')}\n"
    return text

def _detect_simple_fact_question(self, parsed: Dict[str, Any]) -> bool:
    task = parsed.get("original_question", parsed.get("task", ""))
    task_lower = task.strip().lower()

    knowledge_points = parsed.get("knowledge_points", [])
    if not knowledge_points:
        return False

    fact_patterns = [
        r"^(什么是|什么叫|是什么|是谁|哪一个|哪一种|什么时候|在哪里|多少钱)",
        r"^(的定义|的意思|含义|概念)",
        r"^(介绍|简述|概述)",
    ]
    for p in fact_patterns:
        if re.search(p, task_lower):
            return True

    if len(task_lower) < 30 and not any(
        kw in task_lower for kw in ["写", "生成", "策划", "方案", "规划", "设计", "报告", "总结"]
    ):

```

```

    ):
        return True

    return False

    async def _generate_fact_answer(self, parsed: Dict[str, Any]) -> str:
        task = parsed.get("original_question", parsed.get("task", ""))
        knowledge_points = parsed.get("knowledge_points", [])
        knowledge_text = self._build_knowledge_text(knowledge_points)

        prompt = f"""请根据以下参考知识直接回答用户的问题。

## 用户问题
{task}

## 参考知识（必须严格基于以下知识作答，不要编造）
{knowledge_text}

## 回答要求
1. 直接回答问题，不要跑题
2. 语言简洁准确，像百科条目
3. 使用 Markdown 格式组织，但不要使用“任务概述”等模板标题
4. 如果参考知识不够完整，就基于已有知识诚实作答

请直接回答："""

        response = await self._call_llm(prompt, model=self.local_model, use_cloud=False, temperature=0.3, max_tokens=2048)
        return response.strip() if response else task

    async def _generate_simple_response(self, parsed: Dict[str, Any]) -> str:
        task = parsed.get("original_question", parsed.get("task", ""))
        task_lower = task.strip().lower()

        from shared.platform import PLATFORM_IDENTITY

        if re.search(r"(你是谁|你叫什么|你的名字|自我介绍|你是什么)", task_lower):
            prompt = f"""{PLATFORM_IDENTITY.strip()}

用户问： "{task}"

请直接以第一人称回复用户。要求：以“我是 AgentMatrix 平台的 AI 助手”开头，简短介绍平台（多智能体协同+国产算力优化，简单任务本地处理，复杂任务云端增强），50-150字，像真人对话，不要标题大纲。"""
            elif re.search(r"(你好|您好|hi|hello|嗨|hey|早上好|下午好|晚上好)", task_lower):
                prompt = f"""用户向你打招呼： "{task}"

请友好自然地回复用户，20-60字，像真人对话。不要自我介绍。"""
            else:
                prompt = f"""用户说： "{task}"

请友好自然地简短回复，20-60字，像真人对话。"""

            response = await self._call_llm(prompt, model=self.local_model, use_cloud=False, temperature=0.7, max_tokens=256)
            return response.strip() if response else task

    async def _generate_with_template_and_llm(self, parsed: Dict[str, Any],
                                              template: List[Dict[str, str]],
                                              task_type: str) -> str:
        task = parsed.get("task", "")
        original_question = parsed.get("original_question", "")
        keywords = parsed.get("keywords", [])
        knowledge_points = parsed.get("knowledge_points", [])
        requirements = parsed.get("requirements", [])
        outline = parsed.get("outline", [])

        knowledge_text = self._build_knowledge_text(knowledge_points)
        requirements_text = "\n".join(f"- {r}" for r in requirements) if requirements else "无"
        outline_text = "\n".join(f"- {s}" for s in outline) if outline else "无"

        template_str = ""
        for i, section in enumerate(template, 1):
            template_str += f"{i}. **{section['title']}**: {section['content']}\n"

        if knowledge_text:
            prompt = f"""请按以下模板生成一份 {task_type}。

## 用户需求
{original_question or task}

## 参考知识（必须基于以下知识作答，不要编造）
{knowledge_text}

```

```
## 写作模板
{template_str}

## 关键要求
{requirements_text}

## 输出要求
1. 严格按模板的章节结构组织内容
2. 内容基于参考知识，确保准确专业
3. 每个章节必须充实完整，不能只有一两句话
4. 使用 Markdown 格式：## 二级标题、### 三级标题
5. 直接输出最终文档，不要多余说明

请开始撰写："""
    else:
        prompt = f"""请按以下模板生成一份 {task_type}。

## 用户需求
{original_question or task}

## 写作模板
{template_str}

## 关键要求
{requirements_text}

## 参考大纲
{outline_text}

## 关键词
{' , '.join(keywords) if keywords else '无'}

## 输出要求
1. 严格按模板的章节结构组织内容
2. 每个章节必须充实完整，不能只有一两句话
3. 使用 Markdown 格式：## 二级标题、### 三级标题
4. 直接输出最终文档，不要多余说明

请开始撰写："""
    response = await self._call_llm(prompt, model=self.local_model, use_cloud=False, temperature=0.3, max_tokens=4096)
    return response if response else f"# {original_question or task}\n\n生成失败，请重试。"
```

backend\api__init__.py



backend\api\v1__init__.py



backend\api\v1\agents__init__.py



backend\api\v1\agents\router.py

```

from fastapi import APIRouter, Depends, HTTPException
from typing import Dict, Any
from app.dependencies import get_agent_registry
from agents.base.agent import AgentInput

router = APIRouter()

@router.get("/")
async def get_all_agents(registry=Depends(get_agent_registry)):
    return {
        "agents": registry.get_all_agent_statuses(),
        "count": len(registry.get_all_agents())
    }

@router.get("/{agent_id}")
async def get_agent(agent_id: str, registry=Depends(get_agent_registry)):
    agent = registry.get_agent(agent_id)
    if not agent:
        raise HTTPException(status_code=404, detail=f"Agent {agent_id} not found")
    return agent.get_status()

@router.post("/{agent_id}/execute")
async def execute_agent(
    agent_id: str,
    input_data: AgentInput,
    registry=Depends(get_agent_registry)
):
    try:
        result = await registry.execute_agent(agent_id, input_data)
        return result.dict()
    except ValueError as e:
        raise HTTPException(status_code=404, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.get("/{agent_id}/status")
async def get_agent_status(agent_id: str, registry=Depends(get_agent_registry)):
    agent = registry.get_agent(agent_id)
    if not agent:
        raise HTTPException(status_code=404, detail=f"Agent {agent_id} not found")
    return agent.get_status()

```

backend\api\v1\chat__init__.py

backend\api\v1\chat\router.py

```

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import StreamingResponse
from typing import Dict, Any, List, AsyncGenerator
from pydantic import BaseModel
from app.dependencies import get_agent_registry
from api.v1.workflow.router import execute_workflow, execute_workflow_stream, WorkflowInput, workflow_cache, SimpleCache
import json

router = APIRouter()

chat_cache = SimpleCache(maxsize=200, ttl=300)

class ChatMessage(BaseModel):
    id: str

```

```

role: str
content: str
timestamp: float

class ChatRequest(BaseModel):
    content: str
    use_cloud: bool = False
    model_name: str = None

@router.post("/send")
async def send_message(
    request: ChatRequest,
    registry=Depends(get_agent_registry)
):
    if not request.content.strip():
        raise HTTPException(status_code=400, detail="内容不能为空")

    cache_key = f"chat_{hash(request.content)}_{request.use_cloud}_{request.model_name}"

    if cache_key in chat_cache:
        return chat_cache[cache_key]

    try:
        # 如果指定了模型名称, 使用配置好的模型
        if request.model_name:
            from core.llm.client import get_llm_client
            from config.manager import load_config
            import time
            import logging
            logger = logging.getLogger(__name__)

            start_time = time.time()

            config = load_config()
            models = config.get("models", [])
            model_config = None

            for m in models:
                if m.get("name") == request.model_name:
                    model_config = m
                    break

            if not model_config:
                raise HTTPException(status_code=400, detail=f"未找到模型: {request.model_name}")

            logger.info(f"[Chat] 使用配置模型: {model_config}")

            llm_client = get_llm_client()
            response_text = await llm_client.generate_by_config(
                request.content,
                model_config,
                system_prompt="你来自 AgentMatrix 平台（多智能体动态协同与国产算力优化平台）。你是一个专业、友好的AI助手。请直接回答用户的问题, 提供准确、有帮助的信息。你永远不代表任何其他公司或平台的AI助手。"
            )

            total_duration = (time.time() - start_time) * 1000

            response = {
                "response": response_text,
                "executed_locally": False,
                "complexity_score": 0.0,
                "total_duration": total_duration,
                "steps_count": 1,
                "mode": "model",
                "model_used": request.model_name
            }

            if len(response_text) < 5000:
                chat_cache[cache_key] = response

        return response
    # 云端模式: 直接调用默认LLM
    elif request.use_cloud:
        from core.llm.client import get_llm_client
        import time
        import logging
        logger = logging.getLogger(__name__)

```

```

start_time = time.time()

llm_client = get_llm_client()

# 使用运行时配置的 API Key
from api.v1.config.router import _runtime_config
runtime_api_key = _runtime_config.get("deepseek_api_key")
if runtime_api_key:
    logger.info("[Chat] 使用运行时配置的 API Key")
    llm_client.deepseek_api_key = runtime_api_key
else:
    logger.info("[Chat] 使用默认配置的 API Key")

logger.info(f"[Chat] 开始云端调用，内容长度: {len(request.content)}")

response_text = await llm_client.generate_cloud(
    request.content,
    system_prompt="你来自 AgentMatrix 平台（多智能体动态协同与国产算力优化平台）。你是一个专业、友好的AI助手。请直接回答用户的问题，提供准确、有帮助的信息。你永远不代表任何其他公司或平台的AI助手。"
)

logger.info(f"[Chat] 云端调用完成，响应长度: {len(response_text) if response_text else 0}")

total_duration = (time.time() - start_time) * 1000

response = {
    "response": response_text,
    "executed_locally": False,
    "complexity_score": 0.0,
    "total_duration": total_duration,
    "steps_count": 1,
    "mode": "cloud"
}

if len(response_text) < 5000:
    chat_cache[cache_key] = response

return response

# 本地模式：使用完整的工作流
result = await execute_workflow(
    WorkflowInput(user_input=request.content),
    registry
)

response = {
    "response": result.final_result,
    "executed_locally": result.executed_locally,
    "complexity_score": result.complexity_score,
    "total_duration": result.total_duration_seconds,
    "steps_count": len(result.steps),
    "mode": "local"
}

if result.executed_locally and len(result.final_result) < 5000:
    chat_cache[cache_key] = response

return response
except Exception as e:
    raise HTTPException(status_code=500, detail=f"执行失败: {str(e)}")

@router.post("/send/stream")
async def send_message_stream(
    request: ChatRequest,
    registry=Depends(get_agent_registry)
):
    """流式发送消息，实时返回结果"""
    if not request.content.strip():
        raise HTTPException(status_code=400, detail="内容不能为空")

    async def generate_stream() -> AsyncGenerator[str, None]:
        async for chunk in execute_workflow_stream(
            WorkflowInput(user_input=request.content),
            registry
        ):
            yield f"data: {json.dumps(chunk, ensure_ascii=False)}\n\n"

    return StreamingResponse(generate_stream(), media_type="text/event-stream")

```

```

@router.post("/send/batch")
async def send_batch_messages(
    requests: List[ChatRequest],
    registry=Depends(get_agent_registry)
):
    results = []

    for request in requests:
        if not request.content.strip():
            results.append({"error": "内容不能为空"})
            continue

        try:
            cache_key = f"chat_{hash(request.content)}"

            if cache_key in chat_cache:
                results.append(chat_cache[cache_key])
                continue

            result = await execute_workflow(
                WorkflowInput(user_input=request.content),
                registry
            )

            response = {
                "input": request.content,
                "response": result.final_result,
                "executed_locally": result.executed_locally,
                "complexity_score": result.complexity_score,
                "total_duration": result.total_duration_seconds
            }

            if result.executed_locally and len(result.final_result) < 5000:
                chat_cache[cache_key] = response

            results.append(response)
        except Exception as e:
            results.append({"error": str(e)})

    return {"results": results}

@router.get("/health")
async def chat_health():
    return {"status": "ok", "service": "chat", "cache_size": chat_cache.size}

@router.get("/cache/stats")
async def get_chat_cache_stats():
    return {
        "chat_cache_size": chat_cache.size,
        "chat_cache_max_size": chat_cache.maxsize,
        "chat_cache_ttl": chat_cache.ttl,
        "workflow_cache_size": workflow_cache.size,
        "workflow_cache_max_size": workflow_cache.maxsize,
        "workflow_cache_ttl": workflow_cache.ttl
    }

@router.post("/cache/clear")
async def clear_chat_cache():
    chat_cache.clear()
    return {"status": "success", "message": "Chat cache cleared"}

```

backend\api\v1\config\router.py

```

from fastapi import APIRouter, HTTPException
from pydantic import BaseModel
from typing import Dict, Optional, List
import httpx
import asyncio
from core.llm.client import get_llm_client

```

```

# 导入配置管理器
from config.manager import load_config, save_config, get_api_key, set_api_key, add_model, get_models, remove_model

router = APIRouter()

class ConfigUpdate(BaseModel):
    deepseek_api_key: Optional[str] = None
    ollama_host: Optional[str] = None

class ConnectionTestResult(BaseModel):
    success: bool
    message: str
    details: Dict[str, str] = {}

class ModelConfig(BaseModel):
    name: str
    provider: str
    model: str
    api_key: Optional[str] = None
    display_name: Optional[str] = None
    max_tokens: int = 2048
    temperature: float = 0.7

# 存储运行时配置
_runtime_config = {
    "ollama_host": None,
    "deepseek_api_key": None
}

# 初始化时加载配置
_config = load_config()
_runtime_config["deepseek_api_key"] = _config.get("api_keys", {}).get("deepseek", "")

@router.get("/")
async def get_config():
    """获取当前配置"""
    client = get_llm_client()
    ollama_host = _runtime_config["ollama_host"] or client.ollama_host

    # 加载配置文件
    config = load_config()

    return {
        "ollama_host": ollama_host,
        "ollama_model": client.ollama_model,
        "deepseek_api_key_set": bool(_runtime_config["deepseek_api_key"] or client.deepseek_api_key),
        "deepseek_model": client.deepseek_model,
        "models": get_models()
    }

@router.post("/")
async def update_config(config: ConfigUpdate):
    """更新配置"""
    client = get_llm_client()

    if config.deepseek_api_key is not None:
        _runtime_config["deepseek_api_key"] = config.deepseek_api_key
        client.deepseek_api_key = config.deepseek_api_key

    # 保存到配置文件
    set_api_key("deepseek", config.deepseek_api_key)

    if config.ollama_host is not None:
        _runtime_config["ollama_host"] = config.ollama_host
        client.dynamic_ollama_host = config.ollama_host

    return {"message": "配置更新成功", "saved": True}

@router.get("/models")
async def list_models():
    """获取所有模型配置"""
    return {"models": get_models()}

@router.post("/models")
async def create_model(model: ModelConfig):
    """添加或更新模型配置"""
    model_dict = model.dict()

    # 保存到配置文件
    success = add_model(model_dict)

```

```

    if success:
        return {"message": "模型配置保存成功", "model": model_dict}
    else:
        raise HTTPException(status_code=500, detail="保存模型配置失败")

@router.delete("/models/{model_name}")
async def delete_model(model_name: str):
    """删除模型配置"""
    success = remove_model(model_name)

    if success:
        return {"message": "模型删除成功"}
    else:
        raise HTTPException(status_code=404, detail="模型不存在")

class ValidateKeyRequest(BaseModel):
    provider: str
    api_key: str
    model: str = "deepseek-chat"

@router.post("/validate-key")
async def validate_api_key(request: ValidateKeyRequest):
    """验证API密钥是否有效"""
    import logging
    logger = logging.getLogger(__name__)

    if not request.provider or not request.api_key:
        return {"success": False, "message": "服务商和API密钥不能为空"}

    logger.info(f"[密钥验证] provider={request.provider}, key_length={len(request.api_key)}")

    url = "https://api.deepseek.com/v1/chat/completions"
    if request.provider == "openai":
        url = "https://api.openai.com/v1/chat/completions"

    payload = {
        "model": request.model,
        "messages": [{"role": "user", "content": "test"}],
        "max_tokens": 5
    }
    headers = {
        "Authorization": f"Bearer {request.api_key}",
        "Content-Type": "application/json"
    }

    try:
        async with httpx.AsyncClient(timeout=15.0) as http_client:
            response = await http_client.post(url, json=payload, headers=headers)

            if response.status_code == 200:
                logger.info("[密钥验证] API密钥验证成功")
                return {"success": True, "message": "API密钥验证成功"}
            elif response.status_code == 401:
                logger.error("[密钥验证] API密钥无效")
                return {"success": False, "message": "API密钥无效, 请检查密钥是否正确"}
            elif response.status_code == 403:
                logger.error("[密钥验证] API密钥无权限")
                return {"success": False, "message": "API密钥无权限访问该模型"}
            else:
                error_text = response.text[:200]
                logger.error(f"[密钥验证] 错误: {response.status_code} - {error_text}")
                return {"success": False, "message": f"验证失败: {response.status_code}"}
    except Exception as e:
        logger.error(f"[密钥验证] 调用异常: {e}")
        return {"success": False, "message": f"验证失败: {str(e)}"}

async def _check_ollama_port(host: str) -> bool:
    """检查指定端口是否有Ollama服务"""
    try:
        async with httpx.AsyncClient(timeout=2.0) as client:
            response = await client.get(f"{host}/api/tags")
            return response.status_code == 200
    except:
        return False

class DetectOllamaRequest(BaseModel):
    host: Optional[str] = None
    port: Optional[str] = None

```

```

@router.post("/detect-ollama")
async def detect_ollama(request: DetectOllamaRequest = DetectOllamaRequest()):
    """自动检测Ollama服务端"""
    client = get_llm_client()

    # 如果用户指定了端口, 优先使用
    if request.host or request.port:
        # 构建用户指定的host
        if request.host:
            # 如果用户提供的host已经是完整URL, 直接使用
            if request.host.startswith("http://") or request.host.startswith("https://"):
                test_host = request.host
            else:
                # 否则, 组合host和port
                port = request.port or "11434"
                test_host = f"http://{request.host}:{port}"
        elif request.port:
            # 只提供了port
            test_host = f"http://localhost:{request.port}"

        # 测试用户指定的host
        if await _check_ollama_port(test_host):
            _runtime_config["ollama_host"] = test_host
            client.dynamic_ollama_host = test_host
            return {
                "ollama_host": test_host,
                "message": f"检测到 Ollama 服务: {test_host}"
            }
        else:
            return {
                "ollama_host": test_host,
                "message": f"未检测到 Ollama 服务在 {test_host}, 请检查地址和端口是否正确"
            }

    # 常见的Ollama端口列表
    ports_to_check = [
        "http://localhost:11434",
        "http://localhost:11435",
        "http://localhost:11436",
        "http://localhost:8080",
        "http://localhost:8000",
    ]

    # 检查当前配置的host
    current_host = _runtime_config["ollama_host"] or client.ollama_host
    if current_host and await _check_ollama_port(current_host):
        return {
            "ollama_host": current_host,
            "message": f"检测到 Ollama 服务: {current_host} (当前配置)"
        }

    # 依次检查其他端口
    for host in ports_to_check:
        if host == current_host:
            continue
        if await _check_ollama_port(host):
            _runtime_config["ollama_host"] = host
            client.dynamic_ollama_host = host
            return {
                "ollama_host": host,
                "message": f"检测到 Ollama 服务: {host}"
            }

    # 如果都没找到, 返回默认值
    default_host = "http://localhost:11434"
    return {
        "ollama_host": default_host,
        "message": "未检测到 Ollama 服务, 请手动配置"
    }

class TestOllamaRequest(BaseModel):
    host: Optional[str] = None
    port: Optional[str] = None

@router.post("/test-ollama")
async def test_ollama_connection(request: TestOllamaRequest = TestOllamaRequest()):
    """测试Ollama连接"""
    client = get_llm_client()

    # 如果用户指定了host/port, 先更新配置

```

```

if request.host or request.port:
    if request.host:
        if request.host.startswith("http://") or request.host.startswith("https://"):
            test_host = request.host
        else:
            port = request.port or "11434"
            test_host = f"http://{request.host}:{port}"
    elif request.port:
        test_host = f"http://localhost:{request.port}"
    else:
        test_host = None

    if test_host:
        _runtime_config["ollama_host"] = test_host
        client.dynamic_ollama_host = test_host

try:
    # 使用实际的LLM客户端来测试连接
    result = await client.generate_local("Hello", model="qwen2.5:1.5b")
    if "Error" in result:
        return ConnectionTestResult(
            success=False,
            message="Ollama 连接失败",
            details={"error": result}
        )
    return ConnectionTestResult(
        success=True,
        message="Ollama 连接成功",
        details={"response": result[:30] + "... " if len(result) > 30 else result}
    )
except Exception as e:
    return ConnectionTestResult(
        success=False,
        message="Ollama 连接失败",
        details={"error": str(e)}
    )

@router.post("/test-deepseek")
async def test_deepseek_connection():
    """测试DeepSeek连接"""
    import logging
    logger = logging.getLogger(__name__)

    client = get_llm_client()

    # 优先使用运行时配置
    api_key = _runtime_config.get("deepseek_api_key") or client.deepseek_api_key

    logger.info(f"[测试DeepSeek] API Key 长度: {len(api_key) if api_key else 0}")

    if not api_key:
        logger.error("[测试DeepSeek] API Key 未设置")
        return ConnectionTestResult(
            success=False,
            message="DeepSeek API Key 未设置",
            details={"error": "请先在设置中输入并保存 API Key"}
        )

    # 检查 API Key 格式
    if not api_key.startswith("sk-"):
        logger.warning(f"[测试DeepSeek] API Key 格式可能不正确, 不以 sk- 开头")
        return ConnectionTestResult(
            success=False,
            message="API Key 格式可能不正确",
            details={"error": "DeepSeek API Key 应该以 'sk-' 开头, 请检查您的 Key"}
        )

    logger.info("[测试DeepSeek] 开始调用 DeepSeek API...")

    # 直接使用 httpx 调用, 不经过 client 的 generate_cloud 方法
    url = "https://api.deepseek.com/v1/chat/completions"
    payload = {
        "model": "deepseek-chat",
        "messages": [
            {"role": "user", "content": "Hi"}
        ],
        "max_tokens": 10
    }
    headers = {
        "Authorization": f"Bearer {api_key}",

```

```

        "Content-Type": "application/json"
    }

    try:
        async with httpx.AsyncClient(timeout=30.0) as http_client:
            logger.info(f"[测试DeepSeek] 发送请求到: {url}")
            response = await http_client.post(url, json=payload, headers=headers)

            logger.info(f"[测试DeepSeek] 响应状态码: {response.status_code}")

            if response.status_code == 200:
                data = response.json()
                logger.info(f"[测试DeepSeek] API 调用成功")
                response_text = data.get("choices", [{}])[0].get("message", {}).get("content", "")
                return ConnectionTestResult(
                    success=True,
                    message="DeepSeek 连接成功",
                    details={"response": response_text[:100] if response_text else "成功"}
                )
            else:
                error_text = response.text
                logger.error(f"[测试DeepSeek] API 调用失败: {response.status_code} - {error_text}")
                return ConnectionTestResult(
                    success=False,
                    message=f"DeepSeek 连接失败 (状态码: {response.status_code})",
                    details={"error": error_text}
                )
    except Exception as e:
        logger.error(f"[测试DeepSeek] 调用异常: {e}", exc_info=True)
        return ConnectionTestResult(
            success=False,
            message=f"DeepSeek 连接失败",
            details={"error": str(e)}
        )

```

backend\api\v1\export__init__.py

backend\api\v1\export\router.py

```

from fastapi import APIRouter, Depends, HTTPException
from fastapi.responses import FileResponse
from typing import Dict, Any
from pydantic import BaseModel
import os
from datetime import datetime

router = APIRouter()

EXPORT_DIR = "exports"
os.makedirs(EXPORT_DIR, exist_ok=True)

class ExportRequest(BaseModel):
    content: str
    format: str
    filename: str = None

@router.post("/markdown")
async def export_markdown(request: ExportRequest):
    filename = request.filename or f"export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.md"
    filepath = os.path.join(EXPORT_DIR, filename)

    try:
        with open(filepath, "w", encoding="utf-8") as f:
            f.write(request.content)

```

```

        return {
            "status": "success",
            "format": "markdown",
            "filename": filename,
            "filepath": filepath
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/docx")
async def export_docx(request: ExportRequest):
    try:
        from docx import Document
        from docx.shared import Pt

        doc = Document()
        filename = request.filename or f"export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.docx"
        filepath = os.path.join(EXPORT_DIR, filename)

        for paragraph in request.content.split("\n"):
            if paragraph.strip():
                p = doc.add_paragraph()
                run = p.add_run(paragraph.strip())
                if paragraph.startswith("# "):
                    run.font.size = Pt(18)
                    run.font.bold = True
                elif paragraph.startswith("## "):
                    run.font.size = Pt(16)
                    run.font.bold = True
                elif paragraph.startswith("### "):
                    run.font.size = Pt(14)
                    run.font.bold = True

        doc.save(filepath)

        return {
            "status": "success",
            "format": "docx",
            "filename": filename,
            "filepath": filepath
        }
    except ImportError:
        raise HTTPException(status_code=500, detail="python-docx not installed")
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@router.post("/pptx")
async def export_pptx(request: ExportRequest):
    try:
        from pptx import Presentation
        from pptx.util import Pt

        prs = Presentation()
        filename = request.filename or f"export_{datetime.now().strftime('%Y%m%d_%H%M%S')}.pptx"
        filepath = os.path.join(EXPORT_DIR, filename)

        title_slide_layout = prs.slide_layouts[0]
        slide = prs.slides.add_slide(title_slide_layout)
        title = slide.shapes.title
        subtitle = slide.placeholders[1]

        lines = request.content.split("\n")
        if lines:
            title.text = lines[0].replace("# ", "").strip()
        if len(lines) > 1:
            subtitle.text = lines[1].strip()

        for i in range(2, len(lines), 5):
            bullet_slide_layout = prs.slide_layouts[1]
            slide = prs.slides.add_slide(bullet_slide_layout)
            shapes = slide.shapes

            title_shape = shapes.title
            body_shape = shapes.placeholders[1]

            title_shape.text = "内容"

```

```

        tf = body_shape.text_frame
        for j in range(i, min(i + 5, len(lines))):
            if lines[j].strip():
                p = tf.add_paragraph()
                p.text = lines[j].strip()
                p.level = 0

        prs.save(filepath)

    return {
        "status": "success",
        "format": "pptx",
        "filename": filename,
        "filepath": filepath
    }

except ImportError:
    raise HTTPException(status_code=500, detail="python-pptx not installed")
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.get("/download/{filename}")
async def download_file(filename: str):
    filepath = os.path.join(EXPORT_DIR, filename)
    if not os.path.exists(filepath):
        raise HTTPException(status_code=404, detail=f"File {filename} not found")

    return FileResponse(
        path=filepath,
        filename=filename,
        media_type="application/octet-stream"
    )

@router.get("/list")
async def list_exports():
    files = []
    for filename in os.listdir(EXPORT_DIR):
        filepath = os.path.join(EXPORT_DIR, filename)
        if os.path.isfile(filepath):
            stat = os.stat(filepath)
            files.append({
                "filename": filename,
                "size": stat.st_size,
                "created": datetime.fromtimestamp(stat.st_ctime).isoformat(),
                "modified": datetime.fromtimestamp(stat.st_mtime).isoformat()
            })
    return {"exports": files, "count": len(files)}

```

backend\api\v1\knowledge__init__.py

```

# Empty file for knowledge module initialization

```

backend\api\v1\knowledge\router.py

```

import sys
import os

backend_dir = os.path.realpath(os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__)))))
if backend_dir not in sys.path:
    sys.path.insert(0, backend_dir)

knowledge_service_path = os.path.join(backend_dir, 'knowledge', 'service.py')
if os.path.exists(knowledge_service_path):
    import importlib.util
    spec = importlib.util.spec_from_file_location("knowledge.service", knowledge_service_path)
    knowledge_service = importlib.util.module_from_spec(spec)
    spec.loader.exec_module(knowledge_service)

```

```

KnowledgeService = knowledge_service.KnowledgeService
else:
    from knowledge.service import KnowledgeService

from fastapi import APIRouter, Depends, HTTPException
from typing import Dict, Any, List
from pydantic import BaseModel

router = APIRouter()

_knowledge_service = KnowledgeService()

class KnowledgeItem(BaseModel):
    keyword: str
    content: List[str]

@router.get("/")
async def get_all_knowledge():
    stats = _knowledge_service.get_knowledge_stats()
    return {
        "knowledge_base": _knowledge_service.knowledge_base,
        "keywords": _knowledge_service.get_all_keywords(),
        "stats": stats
    }

@router.get("/stats")
async def get_knowledge_stats():
    return _knowledge_service.get_knowledge_stats()

@router.get("/keyword/{keyword}")
async def get_knowledge_by_keyword(keyword: str):
    content = _knowledge_service.get_knowledge_by_keyword(keyword)
    if content is None:
        raise HTTPException(status_code=404, detail=f"Keyword {keyword} not found")
    return {
        "keyword": keyword,
        "content": content
    }

@router.post("/")
async def add_knowledge(item: KnowledgeItem):
    _knowledge_service.add_knowledge(item.keyword, item.content)
    return {"status": "success", "keyword": item.keyword}

@router.put("/keyword/{keyword}")
async def update_knowledge(keyword: str, content: List[str]):
    success = _knowledge_service.update_knowledge(keyword, content)
    if not success:
        raise HTTPException(status_code=404, detail=f"Keyword {keyword} not found")
    return {"status": "success", "keyword": keyword}

@router.delete("/keyword/{keyword}")
async def delete_knowledge(keyword: str):
    success = _knowledge_service.delete_knowledge(keyword)
    if not success:
        raise HTTPException(status_code=404, detail=f"Keyword {keyword} not found")
    return {"status": "success", "keyword": keyword}

@router.get("/search")
async def search_knowledge(query: str, limit: int = 5):
    results = _knowledge_service.search(query, limit)
    return {
        "query": query,
        "results": results,
        "count": len(results)
    }

@router.post("/enhance")
async def enhance_content(content: str, keywords: List[str]):
    enhanced = _knowledge_service.enhance_content(content, keywords)
    return {

```

```

        "original": content,
        "enhanced": enhanced,
        "keywords": keywords
    }

```

backend\api\v1\metrics__init__.py

backend\api\v1\metrics\router.py

```

from fastapi import APIRouter, Depends
from typing import Dict, Any
from app.dependencies import get_agent_registry
from app.config import settings
import time
import os

router = APIRouter()

_metrics_data = {
    "total_requests": 0,
    "api_calls": 0,
    "local_executions": 0,
    "cloud_executions": 0,
    "cost_saved": 0.0,
    "start_time": time.time()
}

def _get_cpu_usage() -> float:
    try:
        import psutil
        return psutil.cpu_percent()
    except ImportError:
        return 45.6

def _get_memory_usage() -> float:
    try:
        import psutil
        return psutil.virtual_memory().percent
    except ImportError:
        return 67.8

def _get_disk_usage() -> float:
    try:
        import psutil
        return psutil.disk_usage('/').percent
    except ImportError:
        return 42.3

@router.get("/")
async def get_metrics(registry=Depends(get_agent_registry)):
    cpu_usage = _get_cpu_usage()
    memory_usage = _get_memory_usage()
    uptime = time.time() - _metrics_data["start_time"]

    return {
        "system": {
            "app_name": settings.app_name,
            "version": settings.app_version,
            "uptime_seconds": uptime,
            "uptime_formatted": _format_uptime(uptime)
        },
        "resources": {

```

```

        "cpu_usage": cpu_usage,
        "memory_usage": memory_usage,
        "disk_usage": _get_disk_usage()
    },
    "workflow": {
        "total_requests": _metrics_data["total_requests"],
        "api_calls": _metrics_data["api_calls"],
        "local_executions": _metrics_data["local_executions"],
        "cloud_executions": _metrics_data["cloud_executions"],
        "cost_saved": _metrics_data["cost_saved"]
    },
    "agents": registry.get_all_agent_statuses()
}

@router.get("/system")
async def get_system_metrics():
    return {
        "cpu_usage": _get_cpu_usage(),
        "memory_usage": _get_memory_usage(),
        "disk_usage": _get_disk_usage(),
        "process_count": 128
    }

@router.post("/increment/{metric_type}")
async def increment_metric(metric_type: str, value: float = 1.0):
    if metric_type in _metrics_data:
        if isinstance(_metrics_data[metric_type], int):
            _metrics_data[metric_type] += int(value)
        elif isinstance(_metrics_data[metric_type], float):
            _metrics_data[metric_type] += value
    return {"status": "success", "metric": metric_type, "value": value}

def _format_uptime(seconds: float) -> str:
    hours = int(seconds // 3600)
    minutes = int((seconds % 3600) // 60)
    secs = int(seconds % 60)
    return f"{hours}h {minutes}m {secs}s"

def get_metrics_store() -> Dict[str, Any]:
    return _metrics_data

```

backend\api\v1\router.py

```

from fastapi import APIRouter
from api.v1.agents.router import router as agents_router
from api.v1.workflow.router import router as workflow_router
from api.v1.chat.router import router as chat_router
from api.v1.metrics.router import router as metrics_router
from api.v1.knowledge.router import router as knowledge_router
from api.v1.export.router import router as export_router
from api.v1.config.router import router as config_router

router = APIRouter()

router.include_router(agents_router, prefix="/agents", tags=["agents"])
router.include_router(workflow_router, prefix="/workflow", tags=["workflow"])
router.include_router(chat_router, prefix="/chat", tags=["chat"])
router.include_router(metrics_router, prefix="/metrics", tags=["metrics"])
router.include_router(knowledge_router, prefix="/knowledge", tags=["knowledge"])
router.include_router(export_router, prefix="/export", tags=["export"])
router.include_router(config_router, prefix="/config", tags=["config"])

```

backend\api\v1\workflow__init__.py

backend\api\v1\workflow\router.py

```

from fastapi import APIRouter, Depends, HTTPException
from typing import Dict, Any, Optional, List, Tuple, AsyncGenerator
from app.dependencies import get_agent_registry
from agents.base.agent import AgentInput, AgentOutput
from models.workflow import WorkflowInput, WorkflowOutput, WorkflowStep
from api.v1.metrics.router import get_metrics_store
import asyncio
import time
from datetime import datetime
import json
import logging

logger = logging.getLogger(__name__)

router = APIRouter()

class SimpleCache:
    def __init__(self, maxsize: int = 100, ttl: int = 300):
        self.maxsize = maxsize
        self.ttl = ttl
        self.cache: Dict[str, Tuple[Any, float]] = {}

    def __contains__(self, key: str) -> bool:
        if key in self.cache:
            _, timestamp = self.cache[key]
            if time.time() - timestamp < self.ttl:
                return True
            del self.cache[key]
        return False

    def __getitem__(self, key: str) -> Any:
        if key in self:
            return self.cache[key][0]
        raise KeyError(key)

    def __setitem__(self, key: str, value: Any) -> None:
        if len(self.cache) >= self.maxsize:
            oldest_key = min(self.cache.keys(), key=lambda k: self.cache[k][1])
            del self.cache[oldest_key]
        self.cache[key] = (value, time.time())

    def clear(self) -> None:
        self.cache.clear()

    @property
    def size(self) -> int:
        return len(self.cache)

workflow_cache = SimpleCache(maxsize=100, ttl=300)

async def execute_agent_with_timing(
    registry,
    agent_id: str,
    agent_name: str,
    current_input: str,
    current_context: Dict[str, Any]
) -> Tuple[WorkflowStep, str]:
    agent_start = time.time()

    try:
        output = await registry.execute_agent(
            agent_id,
            AgentInput(content=current_input, context=current_context)
        )
        agent_duration = time.time() - agent_start

        step = WorkflowStep(
            agent_id=agent_id,

```

```

        agent_name=agent_name,
        input=current_input,
        output=output.content,
        success=output.success,
        duration_seconds=agent_duration,
        metadata=output.metadata or {}
    )

    return step, output.content

except Exception as e:
    agent_duration = time.time() - agent_start
    step = WorkflowStep(
        agent_id=agent_id,
        agent_name=agent_name,
        input=current_input,
        output="",
        success=False,
        duration_seconds=agent_duration,
        metadata={"error": str(e)}
    )
    return step, ""

@router.post("/execute", response_model=WorkflowOutput)
async def execute_workflow(
    input_data: WorkflowInput,
    registry=Depends(get_agent_registry)
):
    cache_key = f"workflow_{hash(input_data.user_input)}_{hash(str(input_data.context))}"

    if cache_key in workflow_cache:
        cached_result = workflow_cache[cache_key]
        return cached_result

    steps: List[WorkflowStep] = []
    current_context = input_data.context or {}
    executed_locally = True
    complexity_score = 0.0
    review_score = 0.0
    judge_decision = "local_output"
    cloud_mode = "none"
    knowledge_found = False
    start_time = time.time()
    workflow_start = datetime.now()

    metrics = get_metrics_store()
    metrics["total_requests"] += 1

    try:
        agent_order = ["knowledge", "summary", "writer", "review", "judge", "result"]
        agent_names = {
            "knowledge": "Knowledge Agent",
            "summary": "Summary Agent",
            "writer": "Writer Agent",
            "review": "Review Agent",
            "judge": "Judge Agent",
            "result": "Result Agent"
        }

        current_input = input_data.user_input
        original_user_input = input_data.user_input
        writer_output = ""
        summary_result = ""
        review_result = ""

        for agent_id in agent_order:
            agent_start = time.time()
            agent_name = agent_names.get(agent_id, agent_id)

            if agent_id == "review":
                agent_input_content = json.dumps({
                    "user_task": original_user_input,
                    "summary": summary_result,
                    "writer_output": writer_output
                })
            elif agent_id == "judge":
                agent_input_content = json.dumps({
                    "user_task": original_user_input,
                    "summary_result": summary_result,

```

```

        "review_result": review_result,
        "writer_output": writer_output,
        "knowledge_found": knowledge_found
    })
elif agent_id == "result":
    agent_input_content = json.dumps({
        "user_task": original_user_input,
        "summary_result": summary_result,
        "review_result": review_result,
        "judge_result": current_context.get("judge", "{}"),
        "writer_output": writer_output,
        "executed_locally": executed_locally,
        "complexity_score": complexity_score,
        "judge_decision": judge_decision,
        "cloud_mode": cloud_mode
    })
elif agent_id == "summary":
    agent_input_content = current_input
elif agent_id == "writer":
    agent_input_content = current_input
else:
    agent_input_content = current_input

try:
    need_cloud = (agent_id == "result" and judge_decision == "cloud_enhance" and cloud_mode != "none")
    output = await registry.execute_agent(
        agent_id,
        AgentInput(content=agent_input_content, context=current_context, use_llm=True, use_cloud=need_cloud)
    )
    agent_duration = time.time() - agent_start

    step = WorkflowStep(
        agent_id=agent_id,
        agent_name=agent_name,
        input=agent_input_content[:100] + "..." if len(agent_input_content) > 100 else agent_input_content,
        output=output.content,
        success=output.success,
        duration_seconds=agent_duration,
        metadata=output.metadata or {}
    )
    steps.append(step)

    current_context[agent_id] = output.content

    # 保存关键 Agent 的输出
    if agent_id == "knowledge":
        knowledge_found = step.metadata.get("knowledge_count", 0) > 0
    elif agent_id == "summary":
        summary_result = output.content
    elif agent_id == "writer":
        writer_output = output.content
    elif agent_id == "review":
        review_result = output.content
    try:
        review_data = json.loads(output.content)
        review_score = review_data.get("review_score", 0.0)
    except:
        review_score = 0.0
    elif agent_id == "judge":
        try:
            judge_data = json.loads(output.content)
            complexity_score = judge_data.get("complexity_score", 0.0)
            review_score = judge_data.get("review_score", review_score)
            judge_decision = judge_data.get("decision", "local_output")
            cloud_mode = judge_data.get("cloud_mode", "none")
            executed_locally = judge_decision == "local_output"
        except Exception as e:
            executed_locally = True

    current_input = output.content

    if not output.success:
        raise HTTPException(status_code=500, detail=f"Agent {agent_id} failed")

except Exception as e:
    agent_duration = time.time() - agent_start
    step = WorkflowStep(
        agent_id=agent_id,
        agent_name=agent_name,
        input=agent_input_content[:100] if agent_input_content else "",

```

```

        output="",
        success=False,
        duration_seconds=agent_duration,
        metadata={"error": str(e)}
    )
    steps.append(step)
    raise HTTPException(status_code=500, detail=f"Agent {agent_id} failed: {str(e)}")

final_result = writer_output
if judge_decision == "cloud_enhance" and cloud_mode != "none":
    final_result = steps[-1].output if steps else writer_output
total_duration = time.time() - start_time
workflow_end = datetime.now()

if executed_locally:
    metrics["local_executions"] += 1
    metrics["cost_saved"] += 0.01
else:
    metrics["cloud_executions"] += 1
    metrics["api_calls"] += 1

result = WorkflowOutput(
    final_result=final_result,
    steps=steps,
    executed_locally=executed_locally,
    total_duration_seconds=total_duration,
    start_time=workflow_start,
    end_time=workflow_end,
    complexity_score=complexity_score
)

if executed_locally and len(final_result) < 5000:
    workflow_cache[cache_key] = result

return result

except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

@router.post("/execute/parallel", response_model=WorkflowOutput)
async def execute_workflow_parallel(
    input_data: WorkflowInput,
    registry=Depends(get_agent_registry)
):
    cache_key = f"workflow_parallel_{hash(input_data.user_input)}_{hash(str(input_data.context))}"

    if cache_key in workflow_cache:
        return workflow_cache[cache_key]

    steps: List[WorkflowStep] = []
    current_context = input_data.context or {}
    executed_locally = True
    complexity_score = 0.0
    start_time = time.time()
    workflow_start = datetime.now()

    metrics = get_metrics_store()
    metrics["total_requests"] += 1

    try:
        current_input = input_data.user_input

        step_knowledge, knowledge_output = await execute_agent_with_timing(
            registry, "knowledge", "Knowledge Agent", current_input, current_context
        )
        steps.append(step_knowledge)

        if not step_knowledge.success:
            raise HTTPException(status_code=500, detail=f"Knowledge Agent failed")

        current_context["knowledge"] = knowledge_output

        step_summary, summary_output = await execute_agent_with_timing(
            registry, "summary", "Summary Agent", knowledge_output, current_context
        )
        steps.append(step_summary)

```

```

if not step_summary.success:
    raise HTTPException(status_code=500, detail=f"Summary Agent failed")

current_context["summary"] = summary_output

writer_input = f"{knowledge_output}\n\n任务摘要: {summary_output}"
step_writer, writer_output = await execute_agent_with_timing(
    registry, "writer", "Writer Agent", writer_input, current_context
)
steps.append(step_writer)

if not step_writer.success:
    raise HTTPException(status_code=500, detail=f"Writer Agent failed")

current_context["writer"] = writer_output

review_input = f"待评审内容: {writer_output}\n\n任务摘要: {summary_output}"
step_review, review_output = await execute_agent_with_timing(
    registry, "review", "Review Agent", review_input, current_context
)
steps.append(step_review)

if not step_review.success:
    raise HTTPException(status_code=500, detail=f"Review Agent failed")

current_context["review"] = review_output

judge_input = f"内容: {writer_output}\n\n评审结果: {review_output}"
step_judge, judge_output = await execute_agent_with_timing(
    registry, "judge", "Judge Agent", judge_input, current_context
)
steps.append(step_judge)

if not step_judge.success:
    raise HTTPException(status_code=500, detail=f"Judge Agent failed")

executed_locally = step_judge.metadata.get("executed_locally", True)
complexity_score = step_judge.metadata.get("complexity_score", 0.0)

if executed_locally:
    metrics["local_executions"] += 1
    metrics["cost_saved"] += 0.01
else:
    metrics["cloud_executions"] += 1
    metrics["api_calls"] += 1

current_context["judge"] = judge_output

result_input = f"执行结果: {writer_output}\n\n评审: {review_output}\n\n复杂度: {complexity_score}"
step_result, result_output = await execute_agent_with_timing(
    registry, "result", "Result Agent", result_input, current_context
)
steps.append(step_result)

if not step_result.success:
    raise HTTPException(status_code=500, detail=f"Result Agent failed")

final_result = result_output
total_duration = time.time() - start_time
workflow_end = datetime.now()

result = WorkflowOutput(
    final_result=final_result,
    steps=steps,
    executed_locally=executed_locally,
    total_duration_seconds=total_duration,
    start_time=workflow_start,
    end_time=workflow_end,
    complexity_score=complexity_score
)

if executed_locally and len(final_result) < 5000:
    workflow_cache[cache_key] = result

return result

except HTTPException:
    raise
except Exception as e:
    raise HTTPException(status_code=500, detail=str(e))

```

```

@router.get("/cache/stats")
async def get_cache_stats():
    return {
        "cache_size": workflow_cache.size,
        "max_size": workflow_cache.maxsize,
        "ttl": workflow_cache.ttl
    }

@router.post("/cache/clear")
async def clear_cache():
    workflow_cache.clear()
    return {"status": "success", "message": "Cache cleared"}

async def execute_workflow_stream(
    input_data: WorkflowInput,
    registry
) -> AsyncGenerator[Dict[str, Any], None]:
    """流式执行工作流，实时返回每个步骤的结果"""
    steps: List[WorkflowStep] = []
    current_context = input_data.context or {}
    executed_locally = True
    complexity_score = 0.0
    review_score = 0.0
    judge_decision = "local_output"
    cloud_mode = "none"
    knowledge_found = False
    start_time = time.time()

    try:
        agent_order = ["knowledge", "summary", "writer", "review", "judge", "result"]
        agent_names = {
            "knowledge": "Knowledge Agent",
            "summary": "Summary Agent",
            "writer": "Writer Agent",
            "review": "Review Agent",
            "judge": "Judge Agent",
            "result": "Result Agent"
        }

        current_input = input_data.user_input
        original_user_input = input_data.user_input
        writer_output = ""
        summary_result = ""
        review_result = ""

        logger.info(f"[STREAM] Starting workflow for input: {input_data.user_input[:50]}...")

        yield {
            "type": "start",
            "message": "工作流开始执行",
            "timestamp": time.time()
        }

        for agent_id in agent_order:
            agent_start = time.time()
            agent_name = agent_names.get(agent_id, agent_id)

            yield {
                "type": "agent_start",
                "agent_id": agent_id,
                "agent_name": agent_name,
                "timestamp": time.time()
            }

            try:
                if agent_id == "review":
                    agent_input = json.dumps({
                        "user_task": original_user_input,
                        "summary": summary_result,
                        "writer_output": writer_output
                    })
                elif agent_id == "judge":
                    agent_input = json.dumps({
                        "user_input": original_user_input,
                        "summary_result": summary_result,
                        "review_result": review_result,

```

```

        "writer_output": writer_output,
        "knowledge_found": knowledge_found
    ))
elif agent_id == "result":
    agent_input = json.dumps({
        "user_task": original_user_input,
        "summary_result": summary_result,
        "review_result": review_result,
        "judge_result": current_context.get("judge", "{}"),
        "writer_output": writer_output,
        "executed_locally": executed_locally,
        "complexity_score": complexity_score,
        "judge_decision": judge_decision,
        "cloud_mode": cloud_mode
    }, ensure_ascii=False)
    need_cloud = judge_decision == "cloud_enhance" and cloud_mode != "none"
    logger.info(f"[DEBUG] Result Agent input: user_task={original_user_input[:50]}..., writer_output exists: {bool(writer_output)}, cloud={need_cloud}")
    elif agent_id == "summary":
        agent_input = current_input
    elif agent_id == "writer":
        agent_input = current_input
    else:
        agent_input = current_input

    logger.info(f"[DEBUG] Executing {agent_id} with input length: {len(agent_input)}")

    try:
        output = await asyncio.wait_for(
            registry.execute_agent(
                agent_id,
                AgentInput(content=agent_input, context=current_context, use_llm=True, use_cloud=need_cloud if agent_id == "result" else False)
            ),
            timeout=120 if (agent_id == "result" and need_cloud) else 90
        )
    except asyncio.TimeoutError:
        logger.error(f"[DEBUG] Agent {agent_id} execution timed out")
        output = AgentOutput(
            success=False,
            content=f"Error: Agent {agent_id} 执行超时",
            metadata={}
        )
    agent_duration = time.time() - agent_start

    step = WorkflowStep(
        agent_id=agent_id,
        agent_name=agent_name,
        input=agent_input[:100] + "..." if len(agent_input) > 100 else agent_input,
        output=output.content,
        success=output.success,
        duration_seconds=agent_duration,
        metadata=output.metadata or {}
    )
    steps.append(step)

    current_context[agent_id] = output.content
    current_input = output.content

    if agent_id == "knowledge":
        knowledge_found = step.metadata.get("knowledge_count", 0) > 0

    if agent_id == "summary":
        summary_result = output.content

    if agent_id == "writer":
        writer_output = output.content

    if agent_id == "review":
        review_result = output.content
        try:
            review_data = json.loads(output.content)
            review_score = review_data.get("review_score", 0.0)
        except:
            review_score = 0.0

    if agent_id == "judge":
        try:
            judge_data = json.loads(output.content)
            complexity_score = judge_data.get("complexity_score", 0.0)

```

```

        review_score = judge_data.get("review_score", review_score)
        judge_decision = judge_data.get("decision", "local_output")
        cloud_mode = judge_data.get("cloud_mode", "none")
        category = judge_data.get("category", "unknown")
        reason = judge_data.get("reason", [])
        executed_locally = judge_decision == "local_output"
    except Exception as e:
        logger.error(f"[STREAM] Failed to parse judge result: {e}")
        executed_locally = True
        category = "unknown"
        reason = []

    yield {
        "type": "agent_complete",
        "agent_id": agent_id,
        "agent_name": agent_name,
        "duration": round(agent_duration, 2),
        "success": output.success,
        "output_length": len(output.content),
        "timestamp": time.time(),
        "complexity_score": complexity_score if agent_id == "judge" else None,
        "executed_locally": executed_locally if agent_id == "judge" else None,
    }

    if agent_id == "judge":
        yield {
            "type": "judge_decision",
            "complexity_score": complexity_score,
            "executed_locally": executed_locally,
            "decision": judge_decision,
            "category": category,
            "reason": reason,
            "timestamp": time.time()
        }

    except Exception as e:
        agent_duration = time.time() - agent_start
        yield {
            "type": "agent_error",
            "agent_id": agent_id,
            "agent_name": agent_name,
            "duration": round(agent_duration, 2),
            "error": str(e),
            "timestamp": time.time()
        }
    raise

    final_result = writer_output
    if judge_decision == "cloud_enhance" and cloud_mode != "none":
        final_result = steps[-1].output if steps else writer_output

    total_duration = time.time() - start_time

    logger.info(f"[DEBUG] Final result length: {len(final_result)}, first 100 chars: {final_result[:100]}")

    yield {
        "type": "complete",
        "final_result": final_result,
        "executed_locally": executed_locally,
        "complexity_score": complexity_score,
        "total_duration": round(total_duration, 2),
        "steps_count": len(steps),
        "timestamp": time.time()
    }

    except Exception as e:
        yield {
            "type": "error",
            "error": str(e),
            "timestamp": time.time()
        }

```

backend\api\websocket\manager.py

```

from typing import Dict, Any, List
from fastapi import WebSocket
import logging

logger = logging.getLogger(__name__)

class WebSocketManager:
    def __init__(self):
        self.active_connections: Dict[str, WebSocket] = {}
        self.connection_counter = 0

    async def connect(self, websocket: WebSocket) -> str:
        await websocket.accept()
        connection_id = f"conn_{self.connection_counter}"
        self.connection_counter += 1
        self.active_connections[connection_id] = websocket
        logger.info(f"WebSocket connection established: {connection_id}")
        return connection_id

    def disconnect(self, websocket: WebSocket) -> None:
        conn_id_to_remove = None
        for conn_id, conn in self.active_connections.items():
            if conn == websocket:
                conn_id_to_remove = conn_id
                break
        if conn_id_to_remove:
            del self.active_connections[conn_id_to_remove]
            logger.info(f"WebSocket connection disconnected: {conn_id_to_remove}")

    async def send_message(self, message: Dict[str, Any], connection_id: str = None) -> None:
        if connection_id:
            websocket = self.active_connections.get(connection_id)
            if websocket:
                try:
                    await websocket.send_json(message)
                except Exception as e:
                    logger.error(f"Error sending message to {connection_id}: {e}")
            else:
                for conn_id, websocket in list(self.active_connections.items()):
                    try:
                        await websocket.send_json(message)
                    except Exception as e:
                        logger.error(f"Error broadcasting to {conn_id}: {e}")

    async def broadcast_agent_status(self, agent_statuses: Dict[str, Any]) -> None:
        message = {
            "type": "agent_status",
            "data": agent_statuses
        }
        await self.send_message(message)

    async def broadcast_workflow_step(self, step: Dict[str, Any]) -> None:
        message = {
            "type": "workflow_step",
            "data": step
        }
        await self.send_message(message)

    async def broadcast_final_result(self, result: Dict[str, Any]) -> None:
        message = {
            "type": "final_result",
            "data": result
        }
        await self.send_message(message)

    def get_connection_count(self) -> int:
        return len(self.active_connections)

```

backend\app__init__.py

backend\app\config.py

```
from pydantic_settings import BaseSettings, SettingsConfigDict
from typing import List, Dict, Optional
import httpx

async def detect_ollama_port() -> str:
    """自动检测 Ollama 服务端口"""
    ports = ["11434", "11435", "8080"]
    for port in ports:
        try:
            async with httpx.AsyncClient() as client:
                response = await client.get(f"http://localhost:{port}/api/tags", timeout=2)
                if response.status_code == 200:
                    return f"http://localhost:{port}"
        except:
            continue
    return "http://localhost:11434"

class ModelConfig(BaseSettings):
    name: str
    provider: str
    host: str = ""
    api_key: str = ""
    parameters: Dict[str, float] = {}

class AgentModelMapping(BaseSettings):
    agent_id: str
    local_model: str
    cloud_model: str

class Settings(BaseSettings):
    app_name: str = "AgentMatrix"
    app_version: str = "0.1.0"
    app_env: str = "development"

    server_host: str = "0.0.0.0"
    server_port: int = 8000
    server_reload: bool = True

    log_level: str = "INFO"
    log_file: str = "logs/system.log"

    database_url: str = "sqlite:///./agentmatrix.db"

    ollama_host: str = "http://localhost:11435"
    ollama_model: str = "qwen2.5:1.5b"
    ollama_review_model: str = "phi4-mini:3.8b"

    deepseek_api_key: str = ""
    deepseek_api_base: str = "https://api.deepseek.com/v1"
    deepseek_model: str = "deepseek-r1-distill"

    gemini_api_key: str = ""
    gemini_model: str = "gemini-pro"

    complexity_threshold: float = 0.65

    max_concurrent_tasks: int = 10
    max_retry_attempts: int = 3
```

```

allowed_origins_list: Optional[str] = "*"

model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8", extra="ignore")

@property
def allowed_origins(self) -> List[str]:
    if self.allowed_origins_list == "*":
        return ["*"]
    if self.allowed_origins_list:
        return [origin.strip() for origin in self.allowed_origins_list.split(",")]
    return ["http://localhost:3000", "http://localhost:8000", "http://localhost:8080"]

settings = Settings()

```

backend\app\database.py

```

from sqlalchemy import create_engine
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy.orm import sessionmaker
from app.config import settings
import logging

logger = logging.getLogger(__name__)

engine = create_engine(
    settings.database_url,
    connect_args={"check_same_thread": False}
)

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

Base = declarative_base()

def init_db():
    try:
        Base.metadata.create_all(bind=engine)
        logger.info("Database tables created successfully")
    except Exception as e:
        logger.error(f"Failed to create database tables: {e}")
        raise

async def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

```

backend\app\dependencies.py

```

from typing import AsyncGenerator
from fastapi import Depends
from agents.base.agent_registry import AgentRegistry

_agent_registry = None

def get_agent_registry() -> AgentRegistry:
    global _agent_registry
    if _agent_registry is None:
        _agent_registry = AgentRegistry()
    return _agent_registry

async def get_agent_registry_async() -> AsyncGenerator[AgentRegistry, None]:

```

```
registry = get_agent_registry()
yield registry
```

backend\app\main.py

```
import asyncio
import logging
import json
import sys
import os
from contextlib import asynccontextmanager

backend_dir = os.path.realpath(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

if backend_dir not in sys.path:
    sys.path.insert(0, backend_dir)

from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from fastapi.staticfiles import StaticFiles
from starlette.responses import JSONResponse
import socketio

from app.config import settings
from app.dependencies import get_agent_registry
from app.database import init_db
from api.v1.router import router as v1_router
from api.websocket.manager import WebSocketManager

logging.basicConfig(
    level=settings.log_level,
    format="%asctime)s - %(name)s - %(levelname)s - %(message)s",
    handlers=[
        logging.FileHandler(settings.log_file),
        logging.StreamHandler(),
    ],
)
logger = logging.getLogger(__name__)

@asynccontextmanager
async def lifespan(app: FastAPI):
    logger.info("Starting AgentMatrix backend...")

    init_db()
    logger.info("Database initialized successfully")

    agent_registry = get_agent_registry()
    await agent_registry.initialize_all_agents()
    logger.info("All agents initialized successfully")

    ws_manager = WebSocketManager()
    app.state.ws_manager = ws_manager
    logger.info("WebSocket manager initialized")

    yield

    logger.info("Shutting down AgentMatrix backend...")
    await agent_registry.shutdown_all_agents()
    logger.info("All agents shutdown successfully")

class CustomJSONResponse(JSONResponse):
    def render(self, content: any) -> bytes:
        return json.dumps(
            content,
            ensure_ascii=False,
            allow_nan=False,
            indent=None,
            separators=(",", ":"),
        ).encode("utf-8")

app = FastAPI()
```

```

        title=settings.app_name,
        version=settings.app_version,
        description="多智能体动态协同与国产算力优化平台",
        lifespan=lifespan,
        default_response_class=CustomJSONResponse,
    )

    app.add_middleware(
        CORSMiddleware,
        allow_origins=settings.allowed_origins,
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    @app.get("/health")
    async def health_check():
        agent_registry = get_agent_registry()
        agent_statuses = agent_registry.get_all_agent_statuses()

        return {
            "status": "healthy",
            "agents": agent_statuses,
            "version": settings.app_version,
        }

    @app.get("/api/health")
    async def api_health_check():
        agent_registry = get_agent_registry()
        agent_statuses = agent_registry.get_all_agent_statuses()

        return {
            "status": "healthy",
            "agents": agent_statuses,
            "version": settings.app_version,
        }

    app.include_router(v1_router, prefix="/api/v1")

    sio = socketio.AsyncServer(
        async_mode="asgi",
        cors_allowed_origins=settings.allowed_origins,
        logger=False,
        engineio_logger=False,
    )

    @sio.event
    async def connect(sid, environ, auth):
        logger.info(f"Socket.IO client connected: {sid}")

    @sio.event
    async def disconnect(sid):
        logger.info(f"Socket.IO client disconnected: {sid}")

    @sio.on("workflow:step_start")
    async def on_step_start(sid, data):
        logger.info(f"Step start from {sid}: {data}")

    @sio.on("workflow:step_complete")
    async def on_step_complete(sid, data):
        logger.info(f"Step complete from {sid}")

    @sio.on("workflow:step_error")
    async def on_step_error(sid, data):
        logger.info(f"Step error from {sid}: {data}")

    @sio.on("workflow:complete")
    async def on_workflow_complete(sid, data):
        logger.info(f"Workflow complete from {sid}")

    app.mount("/static", StaticFiles(directory="static"), name="static")

```

```

socket_app = socketio.ASGIApp(sio, app)

if __name__ == "__main__":
    import uvicorn

    uvicorn.run(
        "app.main:socket_app",
        host=settings.server_host,
        port=settings.server_port,
        reload=settings.server_reload,
    )

```

backend\check_deepseek_usage.py

```

import aiohttp
import asyncio

async def check_deepseek_usage():
    """检查 DeepSeek API 的使用情况"""
    print("=" * 80)
    print("DeepSeek API 使用情况检查")
    print("=" * 80)

    api_key = "sk-6ccaead05a2f462d97417a268139b561"

    # 方法1: 尝试不同的余额查询接口
    print("\n📌 方法1: 查询余额信息")
    balance_endpoints = [
        "https://api.deepseek.com/v1/wallet/balance",
        "https://api.deepseek.com/v1/balance",
        "https://api.deepseek.com/balance",
    ]

    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json"
    }

    for endpoint in balance_endpoints:
        print(f"\n  尝试: {endpoint}")
        try:
            async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=10)) as session:
                async with session.get(endpoint, headers=headers) as response:
                    print(f"    状态码: {response.status}")
                    if response.status == 200:
                        data = await response.json()
                        print(f"    ✅ 成功: {data}")
                    else:
                        error = await response.text()
                        print(f"    ❌ 失败: {error[:100]}")
        except Exception as e:
            print(f"    ❌ 异常: {str(e)[:100]}")

    # 方法2: 查询用量明细
    print("\n📌 方法2: 尝试查询用量记录")
    usage_endpoints = [
        "https://api.deepseek.com/v1/usage",
        "https://api.deepseek.com/usage",
    ]

    for endpoint in usage_endpoints:
        print(f"\n  尝试: {endpoint}")
        try:
            async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=10)) as session:
                async with session.get(endpoint, headers=headers) as response:
                    print(f"    状态码: {response.status}")
                    if response.status == 200:
                        data = await response.json()
                        print(f"    ✅ 成功: {data}")
                    else:
                        error = await response.text()
                        print(f"    ❌ 失败: {error[:100]}")
        except Exception as e:
            print(f"    ❌ 异常: {str(e)[:100]}")

```

```

print(f" ❌ 异常: {str(e)[:100]}")

print("\n" + "=" * 80)
print("💡 建议:")
print(" 1. 请登录 DeepSeek 开放平台: https://platform.deepseek.com")
print(" 2. 进入 'API' -> '使用明细' 或 '消费记录' 页面")
print(" 3. 查看是否有 API 调用记录")
print(" 4. 如果没有记录, 请检查:")
print("    - API Key 是否正确")
print("    - 是否使用了免费额度的模型")
print("    - 账户余额是否充足")
print("=" * 80)

if __name__ == "__main__":
    asyncio.run(check_deepseek_usage())

```

backend\config__init__.py

```
# Configuration module
```

backend\config\app_config.json

```

{
  "models": [
    {
      "name": "deepseek-chat",
      "provider": "deepseek",
      "model": "deepseek-chat",
      "api_key": "sk-e507ceffe1334899a9780e5f222b30c0",
      "display_name": "DeepSeek Chat",
      "max_tokens": 2048,
      "temperature": 0.7
    },
    {
      "name": "deepseek-v4-flash",
      "provider": "deepseek",
      "model": "deepseek-v4-flash",
      "api_key": "sk-6ccaead05a2f462d97417a268139b561",
      "display_name": "deepseek-v4-flash",
      "max_tokens": 4096,
      "temperature": 0.7
    }
  ],
  "default_provider": "deepseek",
  "api_keys": {
    "deepseek": "",
    "openai": ""
  }
}

```

backend\config\manager.py

```

import json
import os
from typing import Dict, Any, Optional

CONFIG_FILE = os.path.join(os.path.dirname(__file__), 'app_config.json')

def load_config() -> Dict[str, Any]:
    """加载配置文件"""
    if not os.path.exists(CONFIG_FILE):
        return {}

```

```

        "models": [],
        "default_provider": "deepseek",
        "api_keys": {
            "deepseek": "",
            "openai": ""
        }
    }

    try:
        with open(CONFIG_FILE, 'r', encoding='utf-8') as f:
            return json.load(f)
    except Exception as e:
        print(f"加载配置文件失败: {e}")
        return {
            "models": [],
            "default_provider": "deepseek",
            "api_keys": {
                "deepseek": "",
                "openai": ""
            }
        }
    }

def save_config(config: Dict[str, Any]) -> bool:
    """保存配置到文件"""
    try:
        os.makedirs(os.path.dirname(CONFIG_FILE), exist_ok=True)
        with open(CONFIG_FILE, 'w', encoding='utf-8') as f:
            json.dump(config, f, indent=2, ensure_ascii=False)
        return True
    except Exception as e:
        print(f"保存配置文件失败: {e}")
        return False

def get_api_key(provider: str) -> str:
    """获取指定服务商的API Key"""
    config = load_config()
    return config.get("api_keys", {}).get(provider, "")

def set_api_key(provider: str, api_key: str) -> bool:
    """设置指定服务商的API Key"""
    config = load_config()
    if "api_keys" not in config:
        config["api_keys"] = {}
    config["api_keys"][provider] = api_key
    return save_config(config)

def add_model(model_config: Dict[str, Any]) -> bool:
    """添加模型配置"""
    config = load_config()
    if "models" not in config:
        config["models"] = []

    # 检查是否已存在
    existing = next((m for m in config["models"] if m.get("name") == model_config.get("name")), None)
    if existing:
        # 更新现有配置
        existing.update(model_config)
    else:
        config["models"].append(model_config)

    return save_config(config)

def get_models() -> list:
    """获取所有模型配置"""
    config = load_config()
    return config.get("models", [])

def remove_model(model_name: str) -> bool:
    """删除模型配置"""
    config = load_config()
    config["models"] = [m for m in config.get("models", []) if m.get("name") != model_name]
    return save_config(config)

```

```
from .llm import LLMClient, get_llm_client

__all__ = ["LLMClient", "get_llm_client"]
```

backend\core\dynamic_router__init__.py

```
from .router import DynamicRouter, get_dynamic_router

__all__ = ["DynamicRouter", "get_dynamic_router"]
```

backend\core\dynamic_router\router.py

```
from typing import Dict, Any, Optional
import httpx
from app.config import settings

class DeepSeekClient:
    def __init__(self):
        self.api_key = settings.deepseek_api_key
        self.api_base = settings.deepseek_api_base
        self.model = settings.deepseek_model

    async def call(self, prompt: str, model: str = None) -> str:
        if not self.api_key:
            return "DeepSeek API密钥未配置"

        url = f"{self.api_base}/chat/completions"

        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json"
        }

        payload = {
            "model": model or self.model,
            "messages": [{
                "role": "user",
                "content": prompt
            }],
            "temperature": 0.7,
            "max_tokens": 4096
        }

        try:
            async with httpx.AsyncClient(timeout=60.0) as client:
                response = await client.post(url, headers=headers, json=payload)
                response.raise_for_status()
                result = response.json()
                return result["choices"][0]["message"]["content"]
        except Exception as e:
            return f"DeepSeek调用失败: {str(e)}"

class DynamicRouter:
    def __init__(self):
        self.cloud_client = DeepSeekClient()
        self.threshold = settings.complexity_threshold

    def should_use_cloud(self, complexity_score: float) -> bool:
        return complexity_score > self.threshold

    async def route(self, prompt: str, complexity_score: float, agent_id: str) -> Dict[str, Any]:
        use_cloud = self.should_use_cloud(complexity_score)

        if use_cloud:
            response = await self.cloud_client.call(prompt)
```

```

        source = "cloud"
        model_used = "deepseek-rl-distill"
    else:
        response = None
        source = "local"
        model_used = self._select_local_model(agent_id)

    return {
        "response": response,
        "source": source,
        "complexity_score": complexity_score,
        "threshold": self.threshold,
        "agent_id": agent_id,
        "model_used": model_used
    }

def _select_local_model(self, agent_id: str) -> str:
    model_map = {
        "review": "phi4-mini:3.8b",
        "writer": "qwen2.5:1.5b",
        "judge": "qwen2.5:1.5b",
        "summary": "qwen2.5:1.5b",
        "knowledge": "qwen2.5:1.5b",
        "result": "qwen2.5:1.5b",
    }

    return model_map.get(agent_id, "qwen2.5:1.5b")

_dynamic_router_instance = None

def get_dynamic_router() -> DynamicRouter:
    global _dynamic_router_instance
    if _dynamic_router_instance is None:
        _dynamic_router_instance = DynamicRouter()
    return _dynamic_router_instance

```

backend\core\export__init__.py

backend\core\knowledge__init__.py

backend\core\llm__init__.py

```

from .client import LLMClient, get_llm_client

__all__ = ["LLMClient", "get_llm_client"]

```

backend\core\llm\client.py

```

from typing import Dict, Any, Optional
import aiohttp
import logging

```

```

from app.config import settings
import sys
import os

# Add parent directory to path for config import
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
try:
    from config.manager import load_config
except ImportError:
    def load_config():
        return {"models": [], "api_keys": {}}

logger = logging.getLogger(__name__)

class LLMClient:
    def __init__(self):
        self.ollama_host = settings.ollama_host
        self.ollama_model = settings.ollama_model
        self.gemini_api_key = settings.gemini_api_key
        self.gemini_model = settings.gemini_model
        self.deepseek_api_key = getattr(settings, 'deepseek_api_key', '')
        self.deepseek_api_base = getattr(settings, 'deepseek_api_base', 'https://api.deepseek.com/v1')
        self.deepseek_model = getattr(settings, 'deepseek_model', 'deepseek-chat')
        self.dynamic_ollama_host = None

    async def generate_local(self, prompt: str, system_prompt: str = None, model: str = None) -> str:
        host = self.dynamic_ollama_host or self.ollama_host
        url = f"{host}/api/generate"
        payload = {
            "model": model or self.ollama_model,
            "prompt": prompt,
            "stream": False,
            "options": {
                "num_ctx": 2048,
                "num_thread": 4
            }
        }
        if system_prompt:
            payload["system"] = system_prompt

        try:
            async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=60)) as session:
                async with session.post(url, json=payload) as response:
                    if response.status == 200:
                        data = await response.json()
                        return data.get("response", "")
                    else:
                        error_text = await response.text()
                        logger.error(f'Ollama error: {response.status} - {error_text}')
                        return f'Error: {response.status}'
        except Exception as e:
            logger.error(f'Failed to call Ollama: {e}')
            return f'Error: {str(e)}'

    async def generate_local_stream(self, prompt: str, system_prompt: str = None, model: str = None):
        """流式调用 Ollama"""
        host = self.dynamic_ollama_host or self.ollama_host
        url = f"{host}/api/generate"
        payload = {
            "model": model or self.ollama_model,
            "prompt": prompt,
            "stream": True,
            "options": {
                "num_ctx": 2048,
                "num_thread": 4,
                "temperature": 0.3
            }
        }
        if system_prompt:
            payload["system"] = system_prompt

        try:
            async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=120)) as session:
                async with session.post(url, json=payload) as response:
                    if response.status == 200:
                        async for line in response.content:
                            if line:
                                try:
                                    data = line.decode('utf-8').strip()

```

```

        if data:
            import json
            json_data = json.loads(data)
            response_text = json_data.get("response", "")
            done = json_data.get("done", False)
            if response_text:
                yield response_text
            if done:
                break
        except:
            continue
    else:
        error_text = await response.text()
        logger.error(f"Ollama streaming error: {response.status} - {error_text}")
        yield f"Error: {response.status}"
except Exception as e:
    logger.error(f"Failed to call Ollama stream: {e}")
    yield f"Error: {str(e)}"

async def generate_cloud(self, prompt: str, system_prompt: str = None) -> str:
    if not self.deepseek_api_key:
        logger.error("DeepSeek API key not set")
        return "Error: DeepSeek API Key 未设置"

    # 直接使用正确的 URL
    url = "https://api.deepseek.com/v1/chat/completions"

    logger.info(f"[DeepSeek] API URL: {url}")
    logger.info(f"[DeepSeek] API Key: {self.deepseek_api_key[:10]}... (长度: {len(self.deepseek_api_key)})")

    messages = []
    if system_prompt:
        messages.append({"role": "system", "content": system_prompt})
    messages.append({"role": "user", "content": prompt})

    # 使用配置的模型
    payload = {
        "model": self.deepseek_model,
        "messages": messages,
        "temperature": 0.7,
        "max_tokens": 2048
    }
    logger.info(f"[DeepSeek] 使用模型: {self.deepseek_model}")

    headers = {
        "Authorization": f"Bearer {self.deepseek_api_key}",
        "Content-Type": "application/json"
    }

    try:
        async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=60)) as session:
            async with session.post(url, json=payload, headers=headers) as response:
                logger.info(f"[DeepSeek] Response status: {response.status}")

                if response.status == 200:
                    data = await response.json()
                    choices = data.get("choices", [])
                    if choices:
                        # 记录详细的 usage 信息
                        usage = data.get('usage', {})
                        prompt_tokens = usage.get('prompt_tokens', 0)
                        completion_tokens = usage.get('completion_tokens', 0)
                        total_tokens = usage.get('total_tokens', 0)

                        logger.info("[DeepSeek] API调用成功")
                        logger.info(f"[DeepSeek] Token消耗 - prompt: {prompt_tokens}, completion: {completion_tokens}, total: {total_tokens}")

                        if total_tokens > 0:
                            logger.info(f"[DeepSeek] 💰 费用计算: {total_tokens} tokens")

                        return choices[0].get("message", {}).get("content", "")
                    logger.warning("[DeepSeek] API返回但没有choices")
                    return ""
                elif response.status == 401:
                    error_text = await response.text()
                    logger.error(f"[DeepSeek] 认证失败: {error_text}")
                    return f"Error: 401 认证失败, 请检查 API Key 是否正确 - {error_text}"
                elif response.status == 400:
                    error_text = await response.text()

```

```

        logger.error(f"[DeepSeek] 请求错误: {response.status} - {error_text}")
        return f"Error: 400 请求格式错误 - {error_text}"
    elif response.status == 404:
        error_text = await response.text()
        logger.error(f"[DeepSeek] 404错误: {error_text}")
        return f"Error: 404 路径不存在 - {error_text}"
    else:
        error_text = await response.text()
        logger.error(f"[DeepSeek] 错误: {response.status} - {error_text}")
        return f"Error: {response.status} - {error_text}"
except Exception as e:
    logger.error(f"[DeepSeek] 调用失败: {e}", exc_info=True)
    return f"Error: 调用 DeepSeek 失败 - {str(e)}"

async def generate_by_config(self, prompt: str, model_config: dict, system_prompt: str = None) -> str:
    """使用配置好的模型来生成内容"""
    provider = model_config.get("provider", "deepseek")
    model_name = model_config.get("model", "deepseek-chat")
    api_key = model_config.get("api_key", self.deepseek_api_key)
    temperature = model_config.get("temperature", 0.7)
    max_tokens = model_config.get("max_tokens", 2048)

    logger.info(f"[ConfigModel] 使用配置模型: provider={provider}, model={model_name}")

    if provider == "ollama":
        return await self.generate_local(prompt, system_prompt, model_name)

    # 处理云服务商 (DeepSeek/OpenAI等)
    if not api_key:
        logger.error(f"[ConfigModel] API Key未设置 (provider: {provider})")
        return f"Error: {provider} API Key 未设置"

    url = self._get_provider_url(provider)

    messages = []
    if system_prompt:
        messages.append({"role": "system", "content": system_prompt})
    messages.append({"role": "user", "content": prompt})

    payload = {
        "model": model_name,
        "messages": messages,
        "temperature": temperature,
        "max_tokens": max_tokens
    }

    headers = {
        "Authorization": f"Bearer {api_key}",
        "Content-Type": "application/json"
    }

    try:
        async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=60)) as session:
            async with session.post(url, json=payload, headers=headers) as response:
                logger.info(f"[ConfigModel] Response status: {response.status}")

                if response.status == 200:
                    data = await response.json()
                    choices = data.get("choices", [])
                    if choices:
                        logger.info(f"[ConfigModel] API调用成功")
                        return choices[0].get("message", {}).get("content", "")
                    return ""
                else:
                    error_text = await response.text()
                    logger.error(f"[ConfigModel] 错误: {response.status} - {error_text}")
                    return f"Error: {response.status} - {error_text}"
    except Exception as e:
        logger.error(f"[ConfigModel] 调用失败: {e}", exc_info=True)
        return f"Error: 调用失败 - {str(e)}"

def _get_provider_url(self, provider: str) -> str:
    """根据服务商获取API URL"""
    urls = {
        "deepseek": "https://api.deepseek.com/v1/chat/completions",
        "openai": "https://api.openai.com/v1/chat/completions"
    }
    return urls.get(provider, "https://api.deepseek.com/v1/chat/completions")

async def generate(self, prompt: str, use_cloud: bool = False, system_prompt: str = None, model: str = None) -> str:

```

```

        if use_cloud:
            return await self.generate_cloud(prompt, system_prompt)
        return await self.generate_local(prompt, system_prompt, model)

    async def generate_stream(self, prompt: str, use_cloud: bool = False, system_prompt: str = None, model: str = None):
        """流式生成内容"""
        if use_cloud:
            # 云服务暂时不支持流式，返回普通结果
            result = await self.generate_cloud(prompt, system_prompt)
            yield result
        else:
            async for chunk in self.generate_local_stream(prompt, system_prompt, model):
                yield chunk

    _llm_client: Optional[LLMClient] = None

    def get_llm_client() -> LLMClient:
        global _llm_client
        if _llm_client is None:
            _llm_client = LLMClient()
        return _llm_client

```

backend\core\llm\ollama_client.py

```

import httpx
from typing import Dict, Any, Optional, List
from app.config import settings
import json
import logging

logger = logging.getLogger(__name__)

class OllamaClient:
    def __init__(self, host: str = None):
        self.host = host or settings.ollama_host
        self.client = httpx.AsyncClient(base_url=self.host, timeout=120.0)

    async def generate(self, model: str, prompt: str, **kwargs) -> str:
        payload = {
            "model": model,
            "prompt": prompt,
            "stream": False,
            **kwargs
        }

        try:
            response = await self.client.post("/api/generate", json=payload)
            response.raise_for_status()

            raw_content = response.content

            try:
                result = json.loads(raw_content.decode("utf-8"))
            except UnicodeDecodeError:
                result = json.loads(raw_content.decode("gbk"))

            response_text = result.get("response", "")

            if isinstance(response_text, bytes):
                response_text = response_text.decode("utf-8")

            response_text = self._fix_chinese_encoding(response_text)

            return response_text
        except httpx.HTTPError as e:
            logger.error(f"Ollama API error: {e}")
            raise
        except Exception as e:
            logger.error(f"Ollama client error: {e}")
            raise

```

```

def _fix_chinese_encoding(self, text: str) -> str:
    try:
        return text.encode("gbk").decode("utf-8")
    except:
        try:
            return text.encode("gb18030").decode("utf-8")
        except:
            try:
                return text.encode("latin-1").decode("utf-8")
            except:
                return text

async def chat(self, model: str, messages: List[Dict[str, str]], **kwargs) -> str:
    payload = {
        "model": model,
        "messages": messages,
        "stream": False,
        **kwargs
    }

    try:
        response = await self.client.post("/api/chat", json=payload)
        response.raise_for_status()

        raw_content = response.content

        try:
            result = json.loads(raw_content.decode("utf-8"))
        except UnicodeDecodeError:
            result = json.loads(raw_content.decode("gbk"))

        content = result.get("message", {}).get("content", "")
        content = self._fix_chinese_encoding(content)

        return content
    except httpx.HTTPError as e:
        logger.error(f"Ollama chat API error: {e}")
        raise
    except Exception as e:
        logger.error(f"Ollama chat client error: {e}")
        raise

async def list_models(self) -> List[Dict[str, Any]]:
    try:
        response = await self.client.get("/api/tags")
        response.raise_for_status()
        result = response.json()
        return result.get("models", [])
    except httpx.HTTPError as e:
        logger.error(f"Ollama list models error: {e}")
        return []

async def pull_model(self, model: str) -> bool:
    try:
        async with self.client.stream("POST", "/api/pull", json={"name": model, "stream": False}) as response:
            response.raise_for_status()
            return True
    except httpx.HTTPError as e:
        logger.error(f"Ollama pull model error: {e}")
        return False

async def close(self):
    await self.client.aclose()

class LLMService:
    _instance = None

    def __new__(cls):
        if cls._instance is None:
            cls._instance = super().__new__(cls)
            cls._instance._ollama_client = None
        return cls._instance

    async def initialize(self):
        self._ollama_client = OllamaClient()

    async def get_ollama_client(self) -> OllamaClient:
        if not self._ollama_client:
            await self.initialize()

```

```

        return self._ollama_client

    async def generate(self, model: str, prompt: str, **kwargs) -> str:
        client = await self.get_ollama_client()
        return await client.generate(model, prompt, **kwargs)

    async def chat(self, model: str, messages: List[Dict[str, str]], **kwargs) -> str:
        client = await self.get_ollama_client()
        return await client.chat(model, messages, **kwargs)

    async def list_local_models(self) -> List[str]:
        client = await self.get_ollama_client()
        models = await client.list_models()
        return [model["name"] for model in models]

    async def close(self):
        if self._ollama_client:
            await self._ollama_client.close()

llm_service = LLMService()

```

backend\core\workflow__init__.py

```

from .service import WorkflowService

__all__ = ["WorkflowService"]

```

backend\core\workflow\service.py

```

from typing import Dict, Any, List, Optional
from agents.base.agent import AgentInput, AgentOutput
from agents.base.agent_registry import AgentRegistry
from models.workflow import WorkflowInput, WorkflowOutput, WorkflowStep
from core.dynamic_router import get_dynamic_router
from api.v1.metrics.router import get_metrics_store
from core.llm.client import get_llm_client
import time
from datetime import datetime
import json
import logging

logger = logging.getLogger(__name__)

class WorkflowService:
    def __init__(self, agent_registry: AgentRegistry):
        self.agent_registry = agent_registry
        self.dynamic_router = get_dynamic_router()
        self.agent_order = ["knowledge", "summary", "writer", "review", "judge", "result"]
        self.agent_names = {
            "knowledge": "Knowledge Agent",
            "summary": "Summary Agent",
            "writer": "Writer Agent",
            "review": "Review Agent",
            "judge": "Judge Agent",
            "result": "Result Agent"
        }

    async def execute(self, input_data: WorkflowInput) -> WorkflowOutput:
        steps: List[WorkflowStep] = []
        current_context = input_data.context or {}
        executed_locally = True
        complexity_score = 0.0
        review_score = 0.0
        judge_decision = "local_output"
        cloud_mode = "none"
        knowledge_found = False

```

```

start_time = time.time()
workflow_start = datetime.now()

metrics = get_metrics_store()
metrics["total_requests"] += 1

try:
    current_input = input_data.user_input
    original_user_input = input_data.user_input
    writer_output = ""
    summary_result = ""
    review_result = ""

    for i, agent_id in enumerate(self.agent_order):
        agent_start = time.time()
        agent_name = self.agent_names.get(agent_id, agent_id)

        if agent_id == "review":
            review_input = json.dumps({
                "user_task": original_user_input,
                "summary": summary_result,
                "writer_output": writer_output
            })
            agent_input = AgentInput(content=review_input, context=current_context, use_llm=True, use_cloud=False)
        elif agent_id == "judge":
            judge_input = json.dumps({
                "user_task": original_user_input,
                "summary_result": summary_result,
                "review_result": review_result,
                "writer_output": writer_output,
                "knowledge_found": knowledge_found
            })
            agent_input = AgentInput(content=judge_input, context=current_context, use_llm=False, use_cloud=False)
        elif agent_id == "result":
            # Result Agent 需要的格式
            result_input = json.dumps({
                "user_task": original_user_input,
                "summary_result": summary_result,
                "review_result": review_result,
                "judge_result": current_context.get("judge", "{}"),
                "writer_output": writer_output,
                "executed_locally": executed_locally,
                "complexity_score": complexity_score,
                "judge_decision": judge_decision,
                "cloud_mode": cloud_mode
            })
            # 只有cloud_enhance才真正调云端; local_retry只是建议本地增强, 不调云端
            need_cloud_enhance = judge_decision == "cloud_enhance" and cloud_mode != "none"
            agent_input = AgentInput(content=result_input, context=current_context, use_llm=True, use_cloud=need_cloud
_enhance)
        else:
            agent_input = AgentInput(content=current_input, context=current_context, use_llm=True, use_cloud=False)

        output = await self.agent_registry.execute_agent(agent_id, agent_input)
        agent_duration = time.time() - agent_start

        step = WorkflowStep(
            agent_id=agent_id,
            agent_name=agent_name,
            input=current_input,
            output=output.content,
            success=output.success,
            duration_seconds=agent_duration,
            metadata=output.metadata or {}
        )
        steps.append(step)

        current_context[agent_id] = output.content

        if agent_id == "knowledge":
            knowledge_found = output.metadata.get("knowledge_count", 0) > 0 if output.metadata else False

        if agent_id == "summary":
            summary_result = output.content

        if agent_id == "writer":
            writer_output = output.content

        if agent_id == "review":
            review_result = output.content

```

```

        try:
            review_data = json.loads(output.content)
            review_score = review_data.get("review_score", 0.0)
        except:
            review_score = 0.0

    if agent_id == "judge":
        try:
            judge_data = json.loads(output.content)
            complexity_score = judge_data.get("complexity_score", 0.0)
            review_score = judge_data.get("review_score", review_score)
            judge_decision = judge_data.get("decision", "local_output")
            cloud_mode = judge_data.get("cloud_mode", "none")
            executed_locally = judge_decision == "local_output"

            logger.info(f"Judge decision: {judge_decision}, complexity={complexity_score:.2f}, review_score={review_score:.2f}, cloud_mode={cloud_mode}")
        except Exception as e:
            logger.error(f"Failed to parse judge result: {e}")
            executed_locally = True

    current_input = output.content

    # 找到Writer和Judge的输出
    writer_output = ""
    for step in steps:
        if step.agent_id == "writer":
            writer_output = step.output
            break

    # 如果需要云端增强, 则使用Result Agent的云端输出
    final_result = writer_output
    if judge_decision == "cloud_enhance" and cloud_mode != "none":
        final_result = output.content if steps else writer_output

    if not executed_locally:
        metrics["cloud_executions"] += 1
        metrics["api_calls"] += 1
    else:
        metrics["local_executions"] += 1
        metrics["cost_saved"] += 0.01

    total_duration = time.time() - start_time
    workflow_end = datetime.now()

    logger.info(f"Workflow completed: complexity={complexity_score:.2f}, review={review_score:.2f}, local={executed_locally}, decision={judge_decision}, duration={total_duration:.2f}s")

    return WorkflowOutput(
        final_result=final_result,
        steps=steps,
        executed_locally=executed_locally,
        total_duration_seconds=total_duration,
        start_time=workflow_start,
        end_time=workflow_end,
        complexity_score=complexity_score
    )

except Exception as e:
    logger.error(f"Workflow execution failed: {e}")
    raise

async def execute_with_llm_enhancement(self, input_data: WorkflowInput) -> WorkflowOutput:
    return await self.execute(input_data)

async def get_step_by_agent(self, steps: List[WorkflowStep], agent_id: str) -> Optional[WorkflowStep]:
    for step in steps:
        if step.agent_id == agent_id:
            return step
    return None

def calculate_cost_savings(self, cloud_executions: int, local_executions: int) -> float:
    cloud_cost_per_call = 0.01
    local_cost_per_call = 0.001
    return cloud_executions * (cloud_cost_per_call - local_cost_per_call)

```

backend\debug_config.py

```
import os
import sys

backend_dir = os.path.realpath(os.path.dirname(os.path.abspath(__file__)))
if backend_dir not in sys.path:
    sys.path.insert(0, backend_dir)

from app.config import settings
from core.llm_client import get_llm_client

print("=== 配置调试 ===")
print(f"配置文件路径: {os.path.join(backend_dir, 'app', 'config.py')}")
print(f"ollama_host 配置值: '{settings.ollama_host}'")
print(f"是否包含 http:// : {'http://' in settings.ollama_host}")
print(f"是否包含 localhost : {'localhost' in settings.ollama_host}")

llm_client = get_llm_client()
print(f"\nLLM Client ollama_host: '{llm_client.ollama_host}'")

# 测试连接
import asyncio
import httpx

async def test_connection():
    host = llm_client.ollama_host
    if not host.startswith("http://") and not host.startswith("https://"):
        host = f"http://{host}"

    print(f"\n测试连接到: {host}")
    try:
        async with httpx.AsyncClient(timeout=5) as client:
            response = await client.get(f"{host}/api/tags")
            print(f"连接成功! 状态码: {response.status_code}")
            data = response.json()
            print(f"可用模型: {[m['name'] for m in data.get('models', [])]}")
    except Exception as e:
        print(f"连接失败: {str(e)}")

asyncio.run(test_connection())
```

backend\detailed_debug.py

```
import aiohttp
import asyncio
import json

async def detailed_api_test():
    print("-" * 80)
    print("DeepSeek API 详细调试测试")
    print("-" * 80)

    api_key = "sk-6ccaead05a2f462d97417a268139b561"
    url = "https://api.deepseek.com/v1/chat/completions"

    # 测试消息
    messages = [
        {"role": "user", "content": "你好, 请用一句话介绍自己"}
    ]

    # 完整 payload
    payload = {
        "model": "deepseek-chat",
        "messages": messages,
        "temperature": 0.7,
        "max_tokens": 50
    }

    headers = {
        "Authorization": f"Bearer {api_key}",
```

```

        "Content-Type": "application/json"
    }

    print("\n 📄 发送请求:")
    print(f"URL: {url}")
    print(f"Headers: {json.dumps({k: v[:20] + '...' if k == 'Authorization' else v for k, v in headers.items()}, indent=2)}")
    print(f"Payload:")
    print(json.dumps(payload, indent=2, ensure_ascii=False))

    print("\n ⌚ 等待响应...")

    try:
        async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
            async with session.post(url, json=payload, headers=headers) as response:
                print(f"\n 📄 收到响应:")
                print(f"状态码: {response.status}")

                if response.status == 200:
                    data = await response.json()

                    print("\n ✅ API调用成功!")
                    print("\n完整响应数据:")
                    print(json.dumps(data, indent=2, ensure_ascii=False)[:500])

                    # 解析关键信息
                    print("\n 📄 关键信息:")
                    print(f" 模型: {data.get('model', 'N/A')}")
                    print(f" ID: {data.get('id', 'N/A')}")

                    usage = data.get('usage', {})
                    print(f"\n 📊 Token 消耗:")
                    print(f"  prompt_tokens: {usage.get('prompt_tokens', 0)}")
                    print(f"  completion_tokens: {usage.get('completion_tokens', 0)}")
                    print(f"  total_tokens: {usage.get('total_tokens', 0)}")

                    choices = data.get('choices', [])
                    if choices:
                        content = choices[0].get('message', {}).get('content', '')
                        print(f"\n 📄 回复内容:")
                        print(content)

                    print("\n" + "=" * 80)
                    print("\n ✅ 测试完成!")
                    print("\n" + "=" * 80)
                    print("\n 💡 如果以上显示有 token 消耗, 但 DeepSeek 平台没有消费记录, 可能原因:")
                    print("  1. API Key 不属于您的账户")
                    print("  2. 该 API Key 没有绑定到您的账户")
                    print("  3. DeepSeek 平台的消费记录有延迟")
                    print("  4. 您使用的是其他渠道的 API Key (如代理、第三方平台)")
                else:
                    error_text = await response.text()
                    print(f"\n ❌ API调用失败:")
                    print(f"状态码: {response.status}")
                    print(f"错误信息: {error_text}")

    except Exception as e:
        print(f"\n ❌ 发生异常:")
        import traceback
        traceback.print_exc()

if __name__ == "__main__":
    asyncio.run(detailed_api_test())

```

backend\houduan.md

AgentMatrix 后端开发规范文档

一、项目概述

本文档定义了 AgentMatrix 后端项目的开发规范, 确保代码质量、API一致性和与前端正确对接。

二、技术栈规范

```
| 分类 | 技术 | 版本 | 说明 |
|-----|-----|-----|-----|
| 框架 | FastAPI | >=0.110.0 | 异步Web框架 |
| 语言 | Python | 3.12+ | 类型注解 |
| 数据验证 | Pydantic | >=2.6.0 | 数据模型与校验 |
| 数据库 | SQLAlchemy | >=2.0.0 | ORM框架 |
| 数据库引擎 | SQLite | - | 开发环境默认 |
| 本地模型 | Ollama | >=0.1.0 | Qwen2.5-3B |
| 云端API | Google Generative AI | >=0.5.0 | Gemini Pro |
| HTTP客户端 | aiohttp | >=3.9.0 | 异步HTTP请求 |
| 日志 | loguru | >=0.7.0 | 日志管理 |
| WebSocket | websockets | >=12.0 | 实时通信 |
| 缓存 | cachetools | - | TTL缓存 |
| 系统监控 | psutil | >=5.9.0 | 资源监控 |
| 导出 | python-pptx / python-docx | - | 文件导出 |
```

三、项目结构规范

backend/ └─ agents/ # Agent实现 | └─ base/ # Agent基类 | | └─ agent.py # BaseAgent抽象类 | | └─ agent_registry.py # Agent注册中心 | └─ knowledge/ # Knowledge Agent | └─ summary/ # Summary Agent | └─ writer/ # Writer Agent | └─ review/ # Review Agent | └─ judge/ # Judge Agent | └─ result/ # Result Agent | └─ api/ # API路由 | └─ v1/ # V1版本API | | └─ router.py # 路由聚合 | | └─ agents/ # Agent相关接口 | | | └─ workflow/ # 工作流相关接口 | | └─ chat/ # 聊天相关接口 | | └─ metrics/ # 指标相关接口 | | | └─ knowledge/ # 知识库相关接口 | | └─ export/ # 导出相关接口 | └─ websocket/ # WebSocket管理 | └─ manager.py # WebSocket连接管理器 | └─ app/ # 应用配置 | └─ main.py # FastAPI应用入口 | └─ config.py # 配置管理 (Pydantic Settings) | └─ database.py # 数据库初始化 | └─ dependencies.py # 依赖注入 | └─ core/ # 核心服务 | └─ llm/ # LLM客户端 | | └─ client.py # 本地/云端双模式LLM调用 | └─ workflow/ # 工作流服务 | | └─ service.py # 工作流编排与执行 | └─ dynamic_router/ # 动态算力路由 | | └─ router.py # 复杂度判断与路由决策 | └─ knowledge/ # 知识库核心 | └─ export/ # 导出核心 | └─ knowledge/ # 知识库数据 | └─ service.py # 知识库服务 | └─ knowledge_base.json # 知识库数据文件 | └─ models/ # 数据模型 | └─ agent.py # Agent相关模型 | └─ workflow.py # 工作流相关模型 | └─ db_models.py # 数据库模型 (SQLAlchemy) | └─ prompts/ # Prompt模板 | └─ template_manager.py # 模板管理器 | └─ rules/ # Prompt规则 | └─ templates/ # Prompt模板文件 | └─ services/ # 业务服务 | └─ agent_service.py # Agent业务逻辑 | └─ utils/ # 工具函数 | └─ logger.py # 日志工具 | └─ tests/ # 测试 | └─ test_agents/ # Agent测试 | └─ test_api/ # API测试 | └─ test_workflow/ # 工作流测试 | └─ .env.example # 环境变量示例

```
---

## 四、环境配置规范

### 4.1 环境变量

```env
APP_NAME=AgentMatrix
APP_VERSION=0.1.0
APP_ENV=development

SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=true

LOG_LEVEL=INFO
LOG_FILE=logs/system.log

DATABASE_URL=sqlite:///./agentmatrix.db

OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=qwen2.5:3b

GEMINI_API_KEY=your-gemini-api-key
GEMINI_MODEL=gemini-pro

COMPLEXITY_THRESHOLD=0.65

MAX_CONCURRENT_TASKS=10
MAX_RETRY_ATTEMPTS=3
```

```
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
```

4.2 配置管理

使用 Pydantic Settings 管理配置，从 .env 文件读取：

```
from pydantic_settings import BaseSettings, SettingsConfigDict
from typing import List

class Settings(BaseSettings):
 app_name: str = "AgentMatrix"
 app_version: str = "0.1.0"
 server_host: str = "0.0.0.0"
 server_port: int = 8000
 database_url: str = "sqlite:///./agentmatrix.db"
 ollama_host: str = "http://localhost:11434"
 ollama_model: str = "qwen2.5:3b"
 gemini_api_key: str = ""
 gemini_model: str = "gemini-pro"
 complexity_threshold: float = 0.65
 allowed_origins: List[str] = ["http://localhost:3000"]

 model_config = SettingsConfigDict(env_file=".env", env_file_encoding="utf-8")

settings = Settings()
```

五、API路由规范

5.1 路由版本与前缀

```
所有API路径: /api/v1/{module}/{action}
健康检查: /health
根路径: /
```

5.2 完整API端点清单

健康检查

路径	方法	功能	请求体	响应体
/health	GET	系统健康检查	无	{status, agents, version}

Agent模块 /api/v1/agents

路径	方法	功能	请求体	响应体
/	GET	获取所有Agent状态	无	{agents: Dict, count: int}
/ {agent_id}	GET	获取单个Agent状态	无	AgentStatus
/ {agent_id} /execute	POST	执行单个Agent	AgentInput	AgentOutput
/ {agent_id} /status	GET	获取Agent运行状态	无	AgentStatus

工作流模块 /api/v1/workflow

路径	方法	功能	请求体	响应体
/execute	POST	执行工作流（串行）	WorkflowInput	WorkflowOutput
/execute/parallel	POST	执行工作流（并行）	WorkflowInput	WorkflowOutput
/cache/stats	GET	获取缓存统计	无	{cache_size, max_size, ttl}
/cache/clear	POST	清除缓存	无	{status, message}

聊天模块 /api/v1/chat

路径	方法	功能	请求体	响应体
/send	POST	发送单条消息	ChatRequest	{response, executed_locally, complexity_score, total_duration, steps_count}
/send/batch	POST	批量发送消息	List[ChatRequest]	{results: List}
/health	GET	聊天服务健康检查	无	{status, service, cache_size}
/cache/stats	GET	获取缓存统计	无	{chat_cache_*, workflow_cache_*}
/cache/clear	POST	清除聊天缓存	无	{status, message}

指标模块 /api/v1/metrics

路径	方法	功能	请求体	响应体
/	GET	获取全部指标	无	{system, resources, workflow, agents}
/system	GET	获取系统资源指标	无	{cpu_usage, memory_usage, disk_usage, process_count}
/increment/{metric_type}	POST	递增指标	value: float	{status, metric, value}

知识库模块 /api/v1/knowledge

路径	方法	功能	请求体	响应体
/	GET	获取全部知识库	无	{knowledge_base, keywords, stats}
/	POST	添加知识	KnowledgeItem	{status, keyword}
/stats	GET	获取知识库统计	无	{total_keywords, total_items, ...}
/keyword/{keyword}	GET	按关键词查询	无	{keyword, content}
/keyword/{keyword}	PUT	更新知识	{content: List[str]}	{status, keyword}
/keyword/{keyword}	DELETE	删除知识	无	{status, keyword}
/search	GET	搜索知识	query, limit	{query, results, count}
/enhance	POST	知识增强	content, keywords	{original, enhanced, keywords}

导出模块 /api/v1/export

路径	方法	功能	请求体	响应体
/markdown	POST	导出Markdown	ExportRequest	{status, format, filename, filepath}
/docx	POST	导出Word	ExportRequest	{status, format, filename, filepath}
/pptx	POST	导出PPT	ExportRequest	{status, format, filename, filepath}
/download/{filename}	GET	下载文件	无	FileResponse
/list	GET	列出已导出文件	无	{exports, count}

六、数据模型规范

6.1 请求模型

AgentInput

```
class AgentInput(BaseModel):
 content: str # 输入内容（必填）
 context: Optional[Dict[str, Any]] = None # 上下文信息（可选）
```

WorkflowInput

```
class WorkflowInput(BaseModel):
 user_input: str = Field(..., description="用户输入内容")
 context: Optional[Dict[str, Any]] = Field(default_factory=dict, description="上下文信息")
```

ChatRequest

```
class ChatRequest(BaseModel):
 content: str # 消息内容（必填）
```

KnowledgeItem

```
class KnowledgeItem(BaseModel):
 keyword: str # 关键词（必填）
 content: List[str] # 知识内容列表（必填）
```

ExportRequest

```
class ExportRequest(BaseModel):
 content: str # 导出内容（必填）
 format: str # 导出格式（必填）
 filename: Optional[str] = None # 文件名（可选）
```

6.2 响应模型

AgentOutput

```
class AgentOutput(BaseModel):
 content: str # 输出内容
 success: bool = True # 是否成功
 message: Optional[str] = None # 消息
 metadata: Optional[Dict[str, Any]] = None # 元数据
```

WorkflowStep

```
class WorkflowStep(BaseModel):
 agent_id: str = Field(..., description="Agent ID")
 agent_name: str = Field(..., description="Agent名称")
 input: str = Field(..., description="输入内容")
 output: str = Field(..., description="输出内容")
 success: bool = Field(default=True, description="是否成功")
 duration_seconds: float = Field(default=0.0, description="执行耗时")
 timestamp: datetime = Field(default_factory=datetime.now, description="时间戳")
 metadata: Optional[Dict[str, Any]] = Field(default_factory=dict, description="元数据")
```

WorkflowOutput

```
class WorkflowOutput(BaseModel):
 final_result: str = Field(..., description="最终结果")
 steps: List[WorkflowStep] = Field(default_factory=list, description="执行步骤")
 executed_locally: bool = Field(default=True, description="是否本地执行")
 total_duration_seconds: float = Field(default=0.0, description="总耗时")
 start_time: datetime = Field(default_factory=datetime.now, description="开始时间")
 end_time: datetime = Field(default_factory=datetime.now, description="结束时间")
 complexity_score: Optional[float] = Field(default=None, description="复杂度评分")
```

AgentStatus

```
class AgentStatus(str, Enum):
 IDLE = "idle"
 PROCESSING = "processing"
```

```

READY = "ready"
ERROR = "error"
SHUTDOWN = "shutdown"

```

## AgentInfo

```

class AgentInfo(BaseModel):
 agent_id: str = Field(..., description="Agent ID")
 name: str = Field(..., description="Agent名称")
 status: AgentStatus = Field(default=AgentStatus.IDLE, description="状态")
 current_task: Optional[str] = Field(default=None, description="当前任务")
 last_error: Optional[str] = Field(default=None, description="最后错误")
 last_active: Optional[datetime] = Field(default=None, description="最后活跃时间")

```

## 6.3 数据库模型 (SQLAlchemy)

```

class ChatSession(Base):
 __tablename__ = "chat_sessions"
 id = Column(String, primary_key=True, index=True)
 user_id = Column(String, index=True)
 title = Column(String)
 created_at = Column(DateTime, default=datetime.utcnow)
 updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

class ChatMessage(Base):
 __tablename__ = "chat_messages"
 id = Column(String, primary_key=True, index=True)
 session_id = Column(String, ForeignKey("chat_sessions.id"))
 role = Column(String)
 content = Column(Text)
 timestamp = Column(DateTime, default=datetime.utcnow)

class WorkflowExecution(Base):
 __tablename__ = "workflow_executions"
 id = Column(String, primary_key=True, index=True)
 session_id = Column(String, ForeignKey("chat_sessions.id"))
 user_input = Column(Text)
 final_result = Column(Text)
 executed_locally = Column(Boolean)
 complexity_score = Column(Float)
 total_duration = Column(Float)
 created_at = Column(DateTime, default=datetime.utcnow)

class WorkflowStepRecord(Base):
 __tablename__ = "workflow_steps"
 id = Column(String, primary_key=True, index=True)
 execution_id = Column(String, ForeignKey("workflow_executions.id"))
 agent_id = Column(String)
 agent_name = Column(String)
 input_content = Column(Text)
 output_content = Column(Text)
 success = Column(Boolean)
 duration = Column(Float)
 step_order = Column(Integer)

class KnowledgeEntry(Base):
 __tablename__ = "knowledge_entries"
 id = Column(Integer, primary_key=True, index=True)
 keyword = Column(String, index=True)
 content = Column(Text)
 created_at = Column(DateTime, default=datetime.utcnow)
 updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

class MetricRecord(Base):
 __tablename__ = "metric_records"
 id = Column(Integer, primary_key=True, index=True)
 metric_type = Column(String, index=True)
 value = Column(Float)
 timestamp = Column(DateTime, default=datetime.utcnow)
 additional_info = Column(Text)

```

## 七、六大Agent规范

### 7.1 Agent定义

Agent ID	名称	职责	执行顺序
knowledge	Knowledge Agent	从知识库检索相关信息	1
summary	Summary Agent	提取用户核心需求摘要	2
writer	Writer Agent	根据需求生成内容	3
review	Review Agent	审核生成内容的质量	4
judge	Judge Agent	判断任务复杂度，决定执行路径	5
result	Result Agent	输出最终结果	6

### 7.2 Agent基类

所有Agent必须继承 `BaseAgent`：

```
from abc import ABC, abstractmethod
from typing import Any, Dict, Optional
from pydantic import BaseModel

class BaseAgent(ABC):
 def __init__(self, agent_id: str, name: str):
 self.agent_id = agent_id
 self.name = name
 self.status = "idle"
 self.current_task = None
 self.last_error = None

 @abstractmethod
 async def execute(self, input_data: AgentInput) -> AgentOutput:
 pass

 async def initialize(self) -> None:
 self.status = "ready"

 async def shutdown(self) -> None:
 self.status = "shutdown"

 def get_status(self) -> Dict[str, Any]:
 return {
 "agent_id": self.agent_id,
 "name": self.name,
 "status": self.status,
 "current_task": self.current_task,
 "last_error": self.last_error,
 }
```

### 7.3 Agent注册

通过 `AgentRegistry` 统一管理：

```
class AgentRegistry:
 def __init__(self):
 self.agents: Dict[str, BaseAgent] = {}

 def register_agent(self, agent: BaseAgent) -> None:
 self.agents[agent.agent_id] = agent

 async def initialize_all_agents(self) -> None:
 self.register_agent(KnowledgeAgent())
 self.register_agent(SummaryAgent())
 self.register_agent(WriterAgent())
 self.register_agent(ReviewAgent())
 self.register_agent(JudgeAgent())
 self.register_agent(ResultAgent())
 for agent in self.agents.values():
 await agent.initialize()
```

## 八、核心服务规范

### 8.1 LLM客户端

双模式调用：本地(Ollama) / 云端(Gemini)

```
class LLMClient:
 async def generate_local(self, prompt: str, system_prompt: str = None) -> str:
 # 调用 Ollama 本地模型

 async def generate_cloud(self, prompt: str, system_prompt: str = None) -> str:
 # 调用 Gemini 云端API

 async def generate(self, prompt: str, use_cloud: bool = False, system_prompt: str = None) -> str:
 if use_cloud:
 return await self.generate_cloud(prompt, system_prompt)
 return await self.generate_local(prompt, system_prompt)
```

### 8.2 动态算力路由

根据复杂度评分决定使用本地模型还是云端API：

```
class DynamicRouter:
 async def route(self, complexity_score: float, prompt: str, system_prompt: str = None) -> Dict[str, Any]:
 use_cloud = complexity_score > self.threshold # 默认阈值0.65
 result = await self.llm_client.generate(prompt, use_cloud=use_cloud, system_prompt=system_prompt)
 return {
 "complexity_score": complexity_score,
 "use_cloud": use_cloud,
 "model": "cloud" if use_cloud else "local",
 "result": result,
 "success": len(result) > 0
 }
```

### 8.3 workflow服务

编排Agent执行流程：

```
class WorkflowService:
 agent_order = ["knowledge", "summary", "writer", "review", "judge", "result"]

 async def execute(self, input_data: WorkflowInput) -> WorkflowOutput:
 # 按顺序执行所有Agent
 # 记录每步执行状态和耗时
 # Judge步骤提取复杂度评分和执行方式
```

### 8.4 WebSocket管理

实时推送 workflow 状态：

```
class WebSocketManager:
 async def broadcast_agent_status(self, agent_statuses: Dict) -> None:
 # 推送Agent状态更新

 async def broadcast_workflow_step(self, step: Dict) -> None:
 # 推送 workflow 步骤更新

 async def broadcast_final_result(self, result: Dict) -> None:
 # 推送最终结果
```

WebSocket消息格式：

```
Agent状态更新
{"type": "agent_status", "data": {...}}
```

```
工作流步骤更新
{"type": "workflow_step", "data": {...}}

最终结果
{"type": "final_result", "data": {...}}
```

## 九、开发规范

### 9.1 代码规范

```
代码检查
ruff check .

自动修复
ruff check --fix .

格式化
black .

运行测试
pytest -v
```

### 9.2 API开发规范

- 1. 路由定义: 在 `api/v1/{module}/router.py` 中定义
- 2. 路由注册: 在 `api/v1/router.py` 中聚合
- 3. 依赖注入: 使用 `Depends()` 注入服务
- 4. 错误处理: 使用 `HTTPException` 统一处理
- 5. 响应模型: 使用 `response_model` 指定响应类型

```
from fastapi import APIRouter, Depends, HTTPException

router = APIRouter()

@router.post("/execute", response_model=WorkflowOutput)
async def execute_workflow(
 input_data: WorkflowInput,
 registry=Depends(get_agent_registry)
):
 try:
 result = await do_something(input_data)
 return result
 except ValueError as e:
 raise HTTPException(status_code=400, detail=str(e))
 except Exception as e:
 raise HTTPException(status_code=500, detail=str(e))
```

### 9.3 新增Agent规范

- 1. 在 `agents/` 下创建新目录
- 2. 继承 `BaseAgent` 抽象类
- 3. 实现 `execute` 方法
- 4. 在 `AgentRegistry.initialize_all_agents` 中注册

### 9.4 新增API模块规范

- 1. 在 `api/v1/` 下创建新目录
- 2. 创建 `router.py` 定义路由
- 3. 在 `api/v1/router.py` 中注册路由
- 4. 如需新模型, 在 `models/` 中定义

## 十、CORS配置

```
app.add_middleware(
 CORSMiddleware,
 allow_origins=settings.allowed_origins, # ["http://localhost:3000"]
 allow_credentials=True,
 allow_methods=["*"],
 allow_headers=["*"],
)
```

## 十一、启动方式

```
安装依赖
cd backend
pip install -e .

启动开发服务器
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

启动生产服务器
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

## 十二、对接要点总结

- 1. **API路径统一前缀**: `/api/v1/`
- 2. **请求/响应模型**: 严格使用Pydantic模型定义
- 3. **Agent ID固定**: `knowledge`, `summary`, `writer`, `review`, `judge`, `result`
- 4. **Judge步骤metadata**: 必须包含 `executed_locally` 和 `complexity_score`
- 5. **复杂度阈值**: 默认0.65, 通过环境变量配置
- 6. **WebSocket消息格式**: `{type: string, data: object}`
- 7. **CORS允许来源**: `http://localhost:3000`
- 8. **错误响应格式**: `{detail: string}`

文档版本: v1.0 创建日期: 2026-05-13 适用项目: AgentMatrix Backend

```

backend\knowledge__init__.py

"""python
from .service import KnowledgeService

__all__ = ["KnowledgeService"]
```

## backend\knowledge\knowledge\_base.json

```
{
 "AI": [
 "AI助手可以帮助自动化日常任务，提高工作效率",
 "AI模型需要持续训练优化以提升性能",
 "AI技术正在快速发展，新模型不断涌现",
 "大语言模型具备强大的上下文理解能力",
 "AI可以用于自然语言处理、图像识别、语音合成",
```

```
“AI模型分为基础模型和微调模型”,
“AI推理可以在本地或云端执行”,
“AI安全性和伦理问题需要重视”,
“国产AI模型包括Qwen、DeepSeek、InternLM等”,
“AI应用需要考虑数据隐私和合规性”,
“AI对话系统需要良好的意图识别能力”,
“AI模型参数规模决定了模型能力上限”,
“AI训练需要大量数据和计算资源”,
“AI提示词工程可以提升模型输出质量”,
“AI幻觉问题需要通过RAG等技术缓解”,
“AI模型量化可以降低推理成本”,
“AI Agent可以自动完成复杂任务”,
“AI多模态模型可以处理文本、图像、语音”,
“AI伦理包括公平性、透明性、可解释性”,
“AI应用需要符合行业监管要求”,
“AI模型部署需要考虑推理延迟和吞吐量”
],
“校园”: [
 “校园场景需要考虑学生隐私保护”,
 “校园网络环境相对封闭, 安全性要求高”,
 “校园AI需要注重教育价值而非单纯技术展示”,
 “校园场景涉及学生、教师、家长多角色协作”,
 “校园信息化建设需要循序渐进, 逐步推进”,
 “校园活动需要考虑学生安全和应急预案”,
 “校园活动需要学校相关部门审批和支持”,
 “校园活动应促进学生德智体美劳全面发展”,
 “校园信息化系统包括教务、学工、后勤等模块”,
 “校园数据治理需要建立完善的数据标准”,
 “校园网络需要区分教学网和办公网”,
 “校园安全系统包括门禁、监控、消防等”,
 “校园课程管理包括选课、排课、成绩管理”,
 “校园图书馆系统需要图书借阅和座位预约”,
 “校园一卡通用于身份认证和消费支付”,
 “校园迎新系统需要注册、报到、宿舍分配”,
 “校园就业指导包括招聘会、简历辅导、职业规划”,
 “校园心理健康需要心理咨询和辅导服务”,
 “校园体育设施包括体育馆、操场、健身房”,
 “校园食堂需要食品安全和营养均衡”,
 “校园宿舍管理包括住宿安排和设施维护”,
 “校园社团活动需要审批和场地预约”,
 “校园讲座需要预告、报名和签到管理”,
 “校园实习实践需要校企合作和岗位对接”
],
“办公”: [
 “办公自动化可以提高工作效率”,
 “会议管理需要明确议程和时间控制”,
 “文档管理需要建立统一的命名规范”,
 “邮件沟通需要简洁明了, 主题明确”,
 “项目管理需要明确目标、分工和时间节点”,
 “办公软件包括文档处理、表格计算、演示文稿”,
 “远程办公需要良好的网络和协作工具”,
 “办公环境需要注重 ergonomic 设计”,
 “办公用品管理需要定期盘点和补充”,
 “办公安全包括信息安全和物理安全”,
 “每日站会通常15分钟以内, 同步进度”,
 “周报月报需要总结工作和规划下一步”,
 “会议室预约需要提前安排并通知参会人员”,
 “办公流程包括审批、报销、请假等”,
 “办公用品采购需要申请、审批、入库”,
 “办公设备包括电脑、打印机、投影仪”,
 “办公软件包括Office、WPS、钉钉、飞书”,
 “办公沟通工具包括企业微信、Slack、Teams”,
 “办公空间布局需要考虑协作和专注区域”,
 “办公时间管理需要优先级排序和待办清单”,
 “办公文件归档需要分类和标签管理”,
 “办公流程自动化可以使用低代码平台”,
 “办公培训包括新员工入职和技能提升”,
 “办公考核包括绩效评估和反馈面谈”,
 “办公差旅需要申请、审批、报销流程”
],
“规划”: [
 “规划需要明确目标和时间节点”,
 “多方利益相关者参与很重要”,
 “规划需要考虑资源约束和可行性”,
 “规划应有可执行的实施路径和里程碑”,
 “规划需要定期评估和调整优化”,
 “规划应包括风险评估和应对方案”,
 “规划需要与相关方充分沟通和协调”,
 “规划文档需要清晰易懂, 便于执行”,
 “规划应考虑短期目标和长期愿景”,
```

```
 "规划需要考虑成本效益和投入产出"
],
 "方案": [
 "方案需要有可行性分析和论证",
 "方案应包含详细的实施步骤",
 "方案需要考虑风险评估和应急预案",
 "方案应有明确的预期成果和验收标准",
 "方案需要成本效益分析和预算估算",
 "方案需要明确责任人和时间节点",
 "方案需要考虑技术选型和兼容性",
 "方案需要用户体验和界面设计",
 "方案需要培训计划和推广策略",
 "方案需要运维和支持方案"
],
 "系统": [
 "系统设计需要考虑扩展性和灵活性",
 "系统架构应遵循模块化和松耦合原则",
 "系统需要良好的容错机制和故障恢复",
 "系统性能需要持续监控和优化",
 "系统安全是首要考虑因素，需要多层防护",
 "系统接口需要标准化和文档化",
 "系统部署需要考虑高可用性和容灾",
 "系统日志需要完善以便问题排查",
 "系统升级需要制定回滚方案",
 "系统文档需要及时更新和维护"
],
 "开发": [
 "开发需要遵循编码规范和最佳实践",
 "代码需要充分测试，包括单元测试和集成测试",
 "开发过程需要版本控制，如Git",
 "代码需要良好的文档和注释",
 "开发应采用敏捷方法论，快速迭代",
 "代码审查可以提高代码质量",
 "持续集成和持续部署可以提高效率",
 "技术债务需要定期清理",
 "代码复用可以减少重复劳动",
 "开发环境需要与生产环境保持一致"
],
 "马拉松": [
 "马拉松是一项长距离跑步运动",
 "全程马拉松距离为42.195公里",
 "马拉松需要充足的训练准备和体能储备",
 "马拉松赛事需要完善的医疗保障",
 "马拉松需要精心规划赛道和补给站",
 "马拉松分为全程、半程、迷你等类型",
 "马拉松需要志愿者团队支持",
 "参赛者需要提前报名并领取参赛包",
 "马拉松需要热身和拉伸避免受伤",
 "马拉松赛后需要恢复和营养补充"
],
 "活动策划": [
 "活动策划需要明确活动目标和预期效果",
 "活动策划需要制定详细日程和时间表",
 "活动策划需要考虑预算和资源分配",
 "活动策划需要安全保障措施和应急预案",
 "活动策划需要宣传推广和报名渠道",
 "活动策划需要人员分工和责任明确",
 "活动后需要总结反馈和效果评估",
 "活动策划需要考虑场地和设备需求",
 "活动策划需要考虑参与者体验",
 "活动策划需要与相关部门沟通协调"
],
 "运动会": [
 "运动会组织需提前规划场地、赛程、安全保障",
 "运动会分为田径、球类、趣味运动等类别",
 "安全第一是运动会组织的核心原则",
 "运动会需要志愿者团队协作保障",
 "运动会前需检查所有设备和场地安全",
 "运动会需要制定竞赛规则和评分标准",
 "运动会需要安排颁奖仪式和奖品",
 "运动会需要医疗急救和保险保障",
 "运动会需要宣传和媒体报道",
 "运动会需要考虑天气情况和备用方案"
],
 "志愿服务": [
 "志愿服务是指自愿贡献个人时间和精力",
 "志愿服务需遵守相关法规，保障志愿者权益",
 "志愿服务记录可纳入个人信用档案",
 "常见志愿场景：社区服务、大型赛事、公益活动",
 "志愿者应接受必要的培训后上岗"
```

```
 "志愿服务需要明确服务内容和时间",
 "志愿服务需要购买保险保障安全",
 "志愿服务需要记录和反馈机制",
 "志愿服务可以培养责任感和团队精神",
 "志愿服务需要尊重服务对象的隐私"
],
"端云协同": [
 "端云协同是指本地设备与云端服务的协同工作",
 "端云协同可以实现资源优化配置",
 "端云协同可以降低延迟提升响应速度",
 "端云协同需要良好的网络连接",
 "端云协同可以实现数据同步和备份",
 "端云协同可以实现边缘计算和云端计算结合",
 "端云协同可以实现智能路由和负载均衡",
 "端云协同需要考虑数据安全和隐私保护",
 "端云协同可以降低成本提高效率",
 "端云协同需要统一的API接口"
],
"多智能体": [
 "多智能体系统由多个自主Agent组成",
 "多智能体可以协同完成复杂任务",
 "多智能体需要良好的通信机制",
 "多智能体可以分工合作提高效率",
 "多智能体系统需要协调和协作策略",
 "多智能体可以实现任务分配和调度",
 "多智能体可以共享知识和信息",
 "多智能体需要解决冲突和竞争",
 "多智能体可以提高系统鲁棒性",
 "多智能体可以实现自适应和进化"
],
"RAG": [
 "RAG是检索增强生成技术",
 "RAG可以提高回答的准确性",
 "RAG需要知识库支持",
 "RAG可以减少幻觉问题",
 "RAG结合了信息检索和生成模型",
 "RAG需要高效的检索算法",
 "RAG需要高质量的知识库",
 "RAG可以实现实时知识更新",
 "RAG可以支持多模态数据",
 "RAG需要考虑检索和生成的平衡"
],
"国产操作系统": [
 "国产操作系统包括麒麟、统信UOS、深度Deepin等",
 "麒麟操作系统是国防和政务领域常用系统",
 "统信UOS是面向党政军和企业的操作系统",
 "国产操作系统基于Linux内核开发",
 "国产操作系统支持国产芯片如鲲鹏、龙芯",
 "国产操作系统注重安全性和可控性",
 "国产操作系统正在逐步替代国外系统",
 "国产操作系统支持常用办公软件",
 "国产操作系统需要生态建设和应用适配",
 "国产操作系统有自主知识产权",
 "国产操作系统包括银河麒麟、中标麒麟等系列",
 "国产操作系统支持信创产业发展",
 "国产操作系统通过等保2.0三级认证",
 "国产操作系统支持国产数据库如达梦、人大金仓",
 "国产操作系统支持国产中间件如东方通、金蝶",
 "国产操作系统有完善的安全补丁更新机制",
 "国产操作系统支持虚拟化和云计算环境",
 "国产操作系统有定制化开发服务能力",
 "国产操作系统支持多语言包括中文本地化",
 "国产操作系统有完善的技术支持体系",
 "国产操作系统符合国家自主可控要求",
 "国产操作系统支持教育、医疗、金融等行业"
],
"麒麟系统": [
 "麒麟操作系统由中国软件公司开发",
 "麒麟系统分为桌面版和服务版",
 "麒麟系统支持国产CPU架构",
 "麒麟系统有完善的安全机制",
 "麒麟系统适用于政府和国防领域",
 "麒麟系统支持国产办公软件",
 "麒麟系统有良好的硬件兼容性",
 "麒麟系统定期更新和安全补丁",
 "麒麟系统有技术支持和服务体系",
 "麒麟系统符合国家信息安全标准"
],
"统信UOS": [
 "统信UOS是统一操作系统的简称",
```

```

 "统信UOS由统信软件公司开发",
 "统信UOS支持多种硬件平台",
 "统信UOS有桌面版和服务版",
 "统信UOS注重用户体验和易用性",
 "统信UOS支持丰富的应用软件",
 "统信UOS有完善的安全机制",
 "统信UOS适用于党政军和企业",
 "统信UOS有开放的应用商店",
 "统信UOS持续更新和优化"
],
 "会议": [
 "会议需要明确议程和目标",
 "会议时间不宜过长, 控制在1小时内",
 "会议需要提前发送邀请和资料",
 "会议需要指定主持人和记录员",
 "会议需要明确决议和行动项",
 "会议纪要需要及时分发和跟进",
 "线上会议需要测试设备和网络",
 "会议需要避免无关人员参与",
 "会议需要尊重发言顺序和时间",
 "会议需要定期评估效果"
],
 "文档": [
 "文档需要清晰的结构和目录",
 "文档需要使用统一的格式规范",
 "文档需要版本控制和更新记录",
 "文档需要易于搜索和查找",
 "技术文档需要代码示例和说明",
 "用户文档需要简洁易懂",
 "文档需要定期审核和更新",
 "文档需要考虑受众和用途",
 "文档需要备份和归档",
 "文档需要权限管理和安全"
],
 "general": [
 "持续学习是成长的关键",
 "良好的沟通是团队协作的基础",
 "用户体验是产品成功的关键",
 "数据驱动决策更可靠",
 "创新源于不断尝试",
 "细节决定成败",
 "时间管理很重要",
 "团队合作可以创造更大价值",
 "保持积极心态面对挑战",
 "不断反思和改进"
]
}

```

## backend\knowledge\service.py

```

from typing import Dict, Any, List, Optional, Tuple
import json
import os
import logging
import time

logger = logging.getLogger(__name__)

class SimpleCache:
 def __init__(self, maxsize: int = 100, ttl: int = 300):
 self.maxsize = maxsize
 self.ttl = ttl
 self.cache: Dict[str, Tuple[Any, float]] = {}

 def __contains__(self, key: str) -> bool:
 if key in self.cache:
 _, timestamp = self.cache[key]
 if time.time() - timestamp < self.ttl:
 return True
 del self.cache[key]
 return False

 def __getitem__(self, key: str) -> Any:

```

```

 if key in self:
 return self.cache[key][0]
 raise KeyError(key)

 def __setitem__(self, key: str, value: Any) -> None:
 if len(self.cache) >= self.maxsize:
 oldest_key = min(self.cache.keys(), key=lambda k: self.cache[k][1])
 del self.cache[oldest_key]
 self.cache[key] = (value, time.time())

 def clear(self) -> None:
 self.cache.clear()

 @property
 def size(self) -> int:
 return len(self.cache)

class KnowledgeService:
 def __init__(self):
 self.knowledge_base: Dict[str, List[str]] = {}
 self.knowledge_file = "knowledge/knowledge_base.json"
 self.search_cache = SimpleCache(maxsize=500, ttl=300)
 self._load_knowledge_base()

 def _load_knowledge_base(self) -> None:
 if os.path.exists(self.knowledge_file):
 try:
 with open(self.knowledge_file, "r", encoding="utf-8") as f:
 self.knowledge_base = json.load(f)
 logger.info(f"Loaded knowledge base with {len(self.knowledge_base)} keywords")
 except Exception as e:
 logger.error(f"Failed to load knowledge base: {e}")
 self._init_default_knowledge()
 else:
 self._init_default_knowledge()

 def _init_default_knowledge(self) -> None:
 self.knowledge_base = {
 "AI": [
 "AI助手可以帮助自动化日常任务",
 "AI模型需要持续训练优化",
 "AI技术正在快速发展",
 "大语言模型具备上下文理解能力",
 "AI可以用于自然语言处理和生成"
],
 "校园": [
 "校园场景需要考虑学生隐私",
 "校园网络环境相对封闭",
 "校园AI需要注重教育价值",
 "校园场景涉及多角色协作",
 "校园信息化建设需要循序渐进"
],
 "规划": [
 "规划需要明确目标和时间节点",
 "多方利益相关者参与很重要",
 "规划需要考虑资源约束",
 "规划应有可执行的实施路径",
 "规划需要定期评估和调整"
],
 "方案": [
 "方案需要有可行性分析",
 "方案应包含实施步骤",
 "方案需要考虑风险评估",
 "方案应有明确的预期成果",
 "方案需要成本效益分析"
],
 "系统": [
 "系统设计需要考虑扩展性",
 "系统架构应遵循模块化原则",
 "系统需要良好的容错机制",
 "系统性能需要持续监控",
 "系统安全是首要考虑因素"
],
 "开发": [
 "开发需要遵循编码规范",
 "代码需要充分测试",
 "开发过程需要版本控制",
 "代码需要良好的文档",
 "开发应采用敏捷方法论"
]
 }

```

```

],
 "general": [
 "持续学习是成长的关键",
 "良好的沟通是团队协作的基础",
 "用户体验是产品成功的关键",
 "数据驱动决策更可靠",
 "创新源于不断尝试"
]
}

self._save_knowledge_base()

def _save_knowledge_base(self) -> None:
 os.makedirs(os.path.dirname(self.knowledge_file), exist_ok=True)
 with open(self.knowledge_file, "w", encoding="utf-8") as f:
 json.dump(self.knowledge_base, f, ensure_ascii=False, indent=2)

def search_by_keywords(self, keywords: List[str], limit: int = 5) -> List[str]:
 cache_key = f"search_keywords_{hash(tuple(sorted(keywords)))}_{limit}"
 if cache_key in self.search_cache:
 return self.search_cache[cache_key]

 results = []
 for keyword in keywords:
 keyword_lower = keyword.lower()
 for kb_key, kb_items in self.knowledge_base.items():
 if keyword_lower in kb_key.lower():
 results.extend(kb_items[:limit])
 else:
 for item in kb_items:
 if keyword_lower in item.lower():
 results.append(item)

 unique_results = list(set(results))[:limit * 2]
 self.search_cache[cache_key] = unique_results
 return unique_results

def search(self, query: str, limit: int = 5) -> Dict[str, List[str]]:
 cache_key = f"search_query_{hash(query)}_{limit}"
 if cache_key in self.search_cache:
 return self.search_cache[cache_key]

 results = {}
 query_lower = query.lower()

 for keyword, content_list in self.knowledge_base.items():
 if query_lower in keyword.lower():
 results[keyword] = content_list[:limit]
 else:
 matching_content = [c for c in content_list if query_lower in c.lower()]
 if matching_content:
 results[keyword] = matching_content[:limit]

 self.search_cache[cache_key] = results
 return results

def add_knowledge(self, keyword: str, content: List[str]) -> None:
 if keyword not in self.knowledge_base:
 self.knowledge_base[keyword] = []
 self.knowledge_base[keyword].extend(content)
 self.knowledge_base[keyword] = list(set(self.knowledge_base[keyword]))
 self._save_knowledge_base()
 self.search_cache.clear()
 logger.info(f"Added {len(content)} items to keyword '{keyword}'")

def delete_knowledge(self, keyword: str) -> bool:
 if keyword in self.knowledge_base:
 del self.knowledge_base[keyword]
 self._save_knowledge_base()
 self.search_cache.clear()
 logger.info(f"Deleted keyword '{keyword}'")
 return True
 return False

def update_knowledge(self, keyword: str, content: List[str]) -> bool:
 if keyword in self.knowledge_base:
 self.knowledge_base[keyword] = content
 self._save_knowledge_base()
 self.search_cache.clear()
 logger.info(f"Updated keyword '{keyword}'")
 return True

```

```

 return False

 def get_all_keywords(self) -> List[str]:
 return list(self.knowledge_base.keys())

 def get_knowledge_by_keyword(self, keyword: str) -> Optional[List[str]]:
 return self.knowledge_base.get(keyword)

 def get_knowledge_stats(self) -> Dict[str, Any]:
 total_items = sum(len(items) for items in self.knowledge_base.values())
 return {
 "total_keywords": len(self.knowledge_base),
 "total_items": total_items,
 "average_items_per_keyword": total_items / len(self.knowledge_base) if self.knowledge_base else 0,
 "cache_size": self.search_cache.size
 }

 def enhance_content(self, original_content: str, keywords: List[str]) -> str:
 cache_key = f"enhance_{hash(original_content)}_{hash(tuple(sorted(keywords)))}"
 if cache_key in self.search_cache:
 return self.search_cache[cache_key]

 knowledge_items = self.search_by_keywords(keywords)
 if not knowledge_items:
 self.search_cache[cache_key] = original_content
 return original_content

 enhanced = f"【知识增强】\n{original_content}\n\n参考知识:\n"
 for i, item in enumerate(knowledge_items, 1):
 enhanced += f"{i}. {item}\n"

 self.search_cache[cache_key] = enhanced
 return enhanced

 def warm_cache(self) -> None:
 for keyword in self.knowledge_base:
 self.search(keyword)
 logger.info("Knowledge cache warmed up")

```

## backend\models\_\_init\_\_.py

```

from .workflow import (
 WorkflowInput,
 WorkflowOutput,
 WorkflowStep,
 ChatMessage
)
from .agent import AgentStatus, AgentExecutionResult

__all__ = [
 "WorkflowInput",
 "WorkflowOutput",
 "WorkflowStep",
 "ChatMessage",
 "AgentStatus",
 "AgentExecutionResult"
]

```

## backend\models\agent.py

```

from pydantic import BaseModel, Field
from typing import Dict, Any, Optional
from datetime import datetime
from enum import Enum

class AgentStatus(str, Enum):
 IDLE = "idle"

```

```

PROCESSING = "processing"
READY = "ready"
ERROR = "error"
SHUTDOWN = "shutdown"

class AgentInfo(BaseModel):
 agent_id: str = Field(..., description="Agent ID")
 name: str = Field(..., description="Agent 名称")
 status: AgentStatus = Field(default=AgentStatus.IDLE, description="状态")
 current_task: Optional[str] = Field(default=None, description="当前任务")
 last_error: Optional[str] = Field(default=None, description="最后错误")
 last_active: Optional[datetime] = Field(default=None, description="最后活跃时间")

class AgentExecutionResult(BaseModel):
 agent_id: str = Field(..., description="Agent ID")
 success: bool = Field(default=True, description="是否成功")
 content: str = Field(default="", description="输出内容")
 message: Optional[str] = Field(default=None, description="消息")
 metadata: Optional[Dict[str, Any]] = Field(default_factory=dict, description="元数据")
 duration_seconds: float = Field(default=0.0, description="执行耗时")

```

## backend\models\db\_models.py

```

from sqlalchemy import Column, Integer, String, Text, DateTime, Float, Boolean, ForeignKey
from sqlalchemy.orm import relationship
from app.database import Base
from datetime import datetime

class ChatSession(Base):
 __tablename__ = "chat_sessions"

 id = Column(String, primary_key=True, index=True)
 user_id = Column(String, index=True)
 title = Column(String)
 created_at = Column(DateTime, default=datetime.utcnow)
 updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

 messages = relationship("ChatMessage", back_populates="session")
 executions = relationship("WorkflowExecution", back_populates="session")

class ChatMessage(Base):
 __tablename__ = "chat_messages"

 id = Column(String, primary_key=True, index=True)
 session_id = Column(String, ForeignKey("chat_sessions.id"))
 role = Column(String)
 content = Column(Text)
 timestamp = Column(DateTime, default=datetime.utcnow)

 session = relationship("ChatSession", back_populates="messages")

class WorkflowExecution(Base):
 __tablename__ = "workflow_executions"

 id = Column(String, primary_key=True, index=True)
 session_id = Column(String, ForeignKey("chat_sessions.id"))
 user_input = Column(Text)
 final_result = Column(Text)
 executed_locally = Column(Boolean)
 complexity_score = Column(Float)
 total_duration = Column(Float)
 created_at = Column(DateTime, default=datetime.utcnow)

 session = relationship("ChatSession", back_populates="executions")
 steps = relationship("WorkflowStepRecord", back_populates="execution")

class WorkflowStepRecord(Base):
 __tablename__ = "workflow_steps"

```

```

 id = Column(String, primary_key=True, index=True)
 execution_id = Column(String, ForeignKey("workflow_executions.id"))
 agent_id = Column(String)
 agent_name = Column(String)
 input_content = Column(Text)
 output_content = Column(Text)
 success = Column(Boolean)
 duration = Column(Float)
 step_order = Column(Integer)

 execution = relationship("WorkflowExecution", back_populates="steps")

class KnowledgeEntry(Base):
 __tablename__ = "knowledge_entries"

 id = Column(Integer, primary_key=True, index=True)
 keyword = Column(String, index=True)
 content = Column(Text)
 created_at = Column(DateTime, default=datetime.utcnow)
 updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)

class MetricRecord(Base):
 __tablename__ = "metric_records"

 id = Column(Integer, primary_key=True, index=True)
 metric_type = Column(String, index=True)
 value = Column(Float)
 timestamp = Column(DateTime, default=datetime.utcnow)
 additional_info = Column(Text)

```

## backend\models\workflow.py

```

from pydantic import BaseModel, Field
from typing import Dict, Any, Optional, List
from datetime import datetime

class WorkflowInput(BaseModel):
 user_input: str = Field(..., description="用户输入内容")
 context: Optional[Dict[str, Any]] = Field(default_factory=dict, description="上下文信息")

class WorkflowStep(BaseModel):
 agent_id: str = Field(..., description="Agent ID")
 agent_name: str = Field(..., description="Agent 名称")
 input: str = Field(..., description="输入内容")
 output: str = Field(..., description="输出内容")
 success: bool = Field(default=True, description="是否成功")
 duration_seconds: float = Field(default=0.0, description="执行耗时")
 timestamp: datetime = Field(default_factory=datetime.now, description="时间戳")
 metadata: Optional[Dict[str, Any]] = Field(default_factory=dict, description="元数据")

class WorkflowOutput(BaseModel):
 final_result: str = Field(..., description="最终结果")
 steps: List[WorkflowStep] = Field(default_factory=list, description="执行步骤")
 executed_locally: bool = Field(default=True, description="是否本地执行")
 total_duration_seconds: float = Field(default=0.0, description="总耗时")
 start_time: datetime = Field(default_factory=datetime.now, description="开始时间")
 end_time: datetime = Field(default_factory=datetime.now, description="结束时间")
 complexity_score: Optional[float] = Field(default=None, description="复杂度评分")

class ChatMessage(BaseModel):
 id: Optional[str] = Field(default=None, description="消息 ID")
 role: str = Field(..., description="角色: user/assistant/system")
 content: str = Field(..., description="消息内容")
 timestamp: Optional[float] = Field(default=None, description="时间戳")
 metadata: Optional[Dict[str, Any]] = Field(default_factory=dict, description="元数据")

```

## backend\prompts\_\_init\_\_.py

```
from .template_manager import PromptManager, PromptTemplate, get_prompt_manager

__all__ = ["PromptManager", "PromptTemplate", "get_prompt_manager"]
```

## backend\prompts\rules\_\_init\_\_.py

## backend\prompts\template\_manager.py

```
from typing import Dict, Any, Optional
import os
import json
import logging

logger = logging.getLogger(__name__)

class PromptTemplate:
 def __init__(self, name: str, template: str, description: str = "", placeholders: list = None):
 self.name = name
 self.template = template
 self.description = description
 self.placeholders = placeholders or []

 def render(self, **kwargs) -> str:
 try:
 return self.template.format(**kwargs)
 except KeyError as e:
 logger.warning(f"Missing placeholder {e} in template {self.name}")
 return self.template

 def to_dict(self) -> Dict[str, Any]:
 return {
 "name": self.name,
 "description": self.description,
 "placeholders": self.placeholders,
 "template": self.template
 }

class PromptManager:
 def __init__(self):
 self.templates: Dict[str, Dict[str, PromptTemplate]] = {}
 self.templates_dir = "prompts/templates"
 self.rules_dir = "prompts/rules"
 self._load_templates()

 def _load_templates(self) -> None:
 os.makedirs(self.templates_dir, exist_ok=True)
 os.makedirs(self.rules_dir, exist_ok=True)

 for agent_id in os.listdir(self.templates_dir):
 agent_dir = os.path.join(self.templates_dir, agent_id)
 if os.path.isdir(agent_dir):
 self.templates[agent_id] = {}
 for filename in os.listdir(agent_dir):
 if filename.endswith(".txt"):
 template_name = filename[:-4]
 filepath = os.path.join(agent_dir, filename)
 try:
```

```

 with open(filepath, "r", encoding="utf-8") as f:
 content = f.read()

 template = PromptTemplate(
 name=template_name,
 template=content,
 description=f"Template for {agent_id} - {template_name}"
)
 self.templates[agent_id][template_name] = template
 except Exception as e:
 logger.error(f"Failed to load template {filepath}: {e}")

 if not self.templates:
 self._init_default_templates()

def _init_default_templates(self) -> None:
 default_templates = {
 "knowledge": {
 "enhance": PromptTemplate(
 name="enhance",
 template="基于以下知识，请增强用户查询： \n\n知识： \n{knowledge}\n\n用户查询： \n{query}\n\n增强后的查询： ",
 description="知识增强模板",
 placeholders=["knowledge", "query"]
)
 },
 "summary": {
 "extract": PromptTemplate(
 name="extract",
 template="请分析以下内容，提取任务目标和关键词： \n\n内容： \n{content}\n\n输出格式： \n{{\"task\": \"任务描述\", \"keywords\": [\"关键词1\", \"关键词2\"]}}",
 description="任务提取模板",
 placeholders=["content"]
)
 },
 "writer": {
 "generate": PromptTemplate(
 name="generate",
 template="根据以下任务描述和关键词，生成详细的内容： \n\n任务： {task}\n关键词： {keywords}\n\n请生成专业、详细的内容： ",
 description="内容生成模板",
 placeholders=["task", "keywords"]
)
 },
 "review": {
 "review": PromptTemplate(
 name="review",
 template="请评审以下内容的质量： \n\n内容： \n{content}\n\n请评估： 1) 内容完整性 2) 逻辑结构 3) 语言质量\n\n输出格式： \n{{\"score\": 分数, \"issues\": [问题列表], \"suggestions\": [建议列表]}}",
 description="质量评审模板",
 placeholders=["content"]
)
 },
 "judge": {
 "complexity": PromptTemplate(
 name="complexity",
 template="请判断以下任务的复杂度（0-1）： \n\n任务： {task}\n内容： {content}\n\n复杂度评分： ",
 description="复杂度判断模板",
 placeholders=["task", "content"]
)
 },
 "result": {
 "format": PromptTemplate(
 name="format",
 template="请格式化以下结果： \n\n执行方式： {execution_type}\n复杂度： {complexity}\n内容： \n{content}\n\n格式化输出： ",
 description="结果格式化模板",
 placeholders=["execution_type", "complexity", "content"]
)
 }
 }

 self.templates = default_templates
 self._save_templates()

def _save_templates(self) -> None:
 for agent_id, templates in self.templates.items():
 agent_dir = os.path.join(self.templates_dir, agent_id)
 os.makedirs(agent_dir, exist_ok=True)

 for name, template in templates.items():

```

```

 filepath = os.path.join(agent_dir, f"{name}.txt")
 with open(filepath, "w", encoding="utf-8") as f:
 f.write(template.template)

 def get_template(self, agent_id: str, template_name: str) -> Optional[PromptTemplate]:
 return self.templates.get(agent_id, {}).get(template_name)

 def add_template(self, agent_id: str, template: PromptTemplate) -> None:
 if agent_id not in self.templates:
 self.templates[agent_id] = {}
 self.templates[agent_id][template.name] = template
 self._save_templates()

 def remove_template(self, agent_id: str, template_name: str) -> bool:
 if agent_id in self.templates and template_name in self.templates[agent_id]:
 del self.templates[agent_id][template_name]
 self._save_templates()
 return True
 return False

 def get_all_templates(self) -> Dict[str, Dict[str, Dict[str, Any]]]:
 result = {}
 for agent_id, templates in self.templates.items():
 result[agent_id] = {
 name: template.to_dict()
 for name, template in templates.items()
 }
 return result

 def get_agent_templates(self, agent_id: str) -> Dict[str, Dict[str, Any]]:
 return {
 name: template.to_dict()
 for name, template in self.templates.get(agent_id, {}).items()
 }

_prompt_manager: Optional[PromptManager] = None

def get_prompt_manager() -> PromptManager:
 global _prompt_manager
 if _prompt_manager is None:
 _prompt_manager = PromptManager()
 return _prompt_manager

```

## backend\prompts\templates\_\_init\_\_.py

## backend\prompts\templates\judge\complexity.txt

请判断以下任务的复杂度（0-1）：

任务：{task}  
内容：{content}

复杂度评分：

## backend\prompts\templates\judge\_prompt.txt

请分析以下任务的复杂度：

```
任务内容: {content}

请从以下维度评估:
1. 任务长度 (短/中/长)
2. 涉及领域 (通用/专业/复杂专业)
3. 需要的推理深度 (简单/中等/深度)
4. 是否需要外部知识 (否/少量/大量)

请输出JSON格式:
{
 "complexity_score": 0.0-1.0,
 "reasoning": "判断理由",
 "factors": ["因素1", "因素2"]
}
```

backend/prompts/templates/knowledge/enhance.txt

```
基于以下知识, 请增强用户查询:

知识:
{knowledge}

用户查询:
{query}

增强后的查询:
```

backend/prompts/templates/knowledge\_prompt.txt

```
请针对以下用户问题, 提供相关的背景知识和参考信息:

用户问题: {query}

请以结构化的方式输出相关知识, 包括:
1. 核心概念解释
2. 相关背景信息
3. 关键点总结

输出格式:
【知识增强】
{query}

参考知识:
1. ...
2. ...
3. ...
```

backend/prompts/templates/result/format.txt

```
请格式化以下结果:

执行方式: {execution_type}
复杂度: {complexity}
内容:
{content}

格式化输出:
```

backend\prompts\templates\result\_prompt.txt

```

请将以下执行结果格式化为Markdown文档：

复杂度评分：{complexity_score}
执行方式：{decision}
决策原因：{reasons}
生成内容：{content}

请按照以下格式输出：
任务执行结果

执行路径
- 复杂度评分：
- 执行方式：

决策原因
- ...

生成内容
{content}

导出格式： Markdown
生成时间： {timestamp}
```

backend\prompts\templates\review\review.txt

```

请评审以下内容的质量：

内容：
{content}

请评估： 1) 内容完整性 2) 逻辑结构 3) 语言质量

输出格式：
{"score": 分数, "issues": [问题列表], "suggestions": [建议列表]}
```

backend\prompts\templates\review\_prompt.txt

```

请对以下内容进行质量评审：

{content}

请从以下几个方面进行评审：
1. 内容完整性
2. 逻辑结构
3. 语言表达
4. 专业程度

请以JSON格式输出评审结果：
{
 "score": 分数(0-100),
 "issues": ["问题1", "问题2", ...],
 "suggestions": ["建议1", "建议2", ...],
 "content_length": 字符数,
 "structure_valid": true/false
}
```

backend\prompts\templates\summary\extract.txt

请分析以下内容，提取任务目标和关键词：

内容：  
{content}

输出格式：  
{{"task": "任务描述", "keywords": ["关键词1", "关键词2"]}}

backend\prompts\templates\summary\_prompt.txt

请分析以下用户输入，提取任务和关键词：

用户输入： {input}

请以JSON格式输出：

```
{
 "task": "提取的任务描述",
 "keywords": ["关键词1", "关键词2", ...],
 "summary": "简要摘要"
}
```

backend\prompts\templates\writer\generate.txt

根据以下任务描述和关键词，生成详细的内容：

任务： {task}  
关键词： {keywords}

请生成专业、详细的内容：

backend\prompts\templates\writer\_prompt.txt

请根据以下信息生成一份完整的方案文档：

任务： {task}  
关键词： {keywords}  
摘要： {summary}

请按照以下结构输出：

- 1. 需求分析
- 2. 方案概述
- 3. 实施步骤
- 4. 预期效果
- 5. 总结

请输出详细、专业的内容。

backend\pyproject.toml

```
[build-system]
requires = ["setuptools>=61.0"]
build-backend = "setuptools.build_meta"

[project]
name = "agentmatrix-backend"
```

```

version = "0.1.0"
description = "多智能体动态协同与国产算力优化平台 - 后端服务"
readme = "../README.md"
authors = [
 { name = "AgentMatrix Team", email = "team@agentmatrix.dev" }
]
license = {text = "MIT"}
classifiers = [
 "Development Status :: 3 - Alpha",
 "Intended Audience :: Developers",
 "Programming Language :: Python :: 3",
 "Programming Language :: Python :: 3.12",
]
requires-python = ">=3.12"
dependencies = [
 "fastapi >= 0.110.0",
 "uvicorn >= 0.29.0",
 "pydantic >= 2.6.0",
 "python-dotenv >= 1.0.0",
 "requests >= 2.31.0",
 "aiohttp >= 3.9.0",
 "sqlalchemy >= 2.0.0",
 "python-pptx >= 0.6.23",
 "python-docx >= 1.1.0",
 "pymdown-extensions >= 10.0.0",
 "ollama >= 0.1.0",
 "google-generativeai >= 0.5.0",
 "numpy >= 1.26.0",
 "scipy >= 1.12.0",
 "loguru >= 0.7.0",
 "websockets >= 12.0",
 "psutil >= 5.9.0",
]

[project.scripts]
agentmatrix = "app.main:main"

[tool.uv]
python = "3.12"

[tool.pytest.ini_options]
testpaths = ["tests"]
pythonpath = ["."]
addopts = "-v --tb=short"

[tool.ruff]
line-length = 100
select = ["E", "F", "W", "I", "N", "Q", "RUF"]
ignore = ["E501", "F401"]
target-version = "py312"

[tool.black]
line-length = 100
target-version = ["py312"]

[tool.setuptools.packages.find]
where = ["."]
include = ["*"]
exclude = ["tests*"]

[project.optional-dependencies]
dev = [
 "pytest",
 "pytest-asyncio",
 "httpx",
 "ruff",
 "black",
 "pre-commit",
]

```

## backend\quick\_test.py

```

import sys
import os

```

```
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
print('Testing...')
try:
 from app.main import app
 print('✅ Import successful!')
 import uvicorn
 print('🚀 Starting server on http://localhost:8000')
 uvicorn.run(app, host='0.0.0.0', port=8000)
except Exception as e:
 print(f'❌ Error: {e}')
 import traceback
 traceback.print_exc()
```

backend\roletxt\all role.txt

平台身份  
你是 AgentMatrix 平台（多智能体动态协同与国产算力优化平台）的智能体。你永远不代表任何其他公司或平台的AI助手。

summary

职责  
提取用户意图、任务类型、关键词、复杂度线索。

身份定义  
你是 Summary Agent。  
你的职责是：  
1. 提取用户真实需求  
2. 判断任务目标  
3. 提取核心关键词  
4. 为后续Agent提供结构化摘要

你不负责生成最终答案。  
你必须保持简洁、客观、结构化。

行为规则

- 不输出废话
- 不直接回答用户问题
- 只做摘要与分析
- 必须识别任务类型
- 必须提取关键词
- 必须评估是否属于复杂任务
- 不去做其他多余的任务，只专注于自己的职责

输出格式

```
{
 "task_type": "方案生成",
 "user_intent": "用户需要校园活动策划",
 "keywords": ["校园", "活动", "策划"],
 "complexity_hint": "medium",
 "recommended_next": "writer_agent"
}
```

review  
职责  
检查生成内容与用户需求的匹配度、事实准确性、逻辑一致性、潜在风险。

身份定义  
你是 Review Agent。  
你的职责是：  
  
1. 对比原始需求与生成内容  
2. 检查事实与逻辑错误  
3. 标注不匹配或遗漏部分  
4. 输出结构化的审核意见

你不负责改写内容。  
你必须保持中立、可追溯。

行为规则

- 不凭空补充信息
- 不输出最终答案
- 每条问题必须指出位置或依据
- 必须给出“通过 / 有条件通过 / 不通过”结论
- 不评价风格好坏，只评价任务完成度

输出格式

```
{
 "is_passed": false,
 "conclusion": "有条件通过",
 "issues": [
 {
 "severity": "high",
 "location": "第三段",
 "description": "未覆盖用户要求的预算项"
 }
],
 "suggestion_summary": "补充预算明细"
}
```

judge

职责

在多个候选结果或版本之间做出最终择优决策。

身份定义

你是 Judge Agent。

你的职责是：

1. 接收多个候选输出
2. 根据任务类型与用户需求进行排序或选择
3. 说明选择理由
4. 输出最终决策

你不负责生成新内容。

你必须公正、可解释。

行为规则

- 不混合多个候选生成新答案
- 必须从已有候选中选择
- 必须给出明确排名或胜出者
- 必须引用评判标准（准确性、完整性、清晰度等）
- 不输出多余情感词

输出格式

```
{
 "decision": "选择候选A",
 "ranking": ["候选A", "候选C", "候选B"],
 "reasoning": "候选A最完整覆盖用户关键词，且无事实错误",
 "confidence": 0.92
}
```

writer

职责

根据摘要或需求，生成最终用户可用的内容。

身份定义

你是 Writer Agent。

你的职责是：

1. 接收需求或摘要
2. 生成符合任务类型的内容
3. 确保内容可读、结构清晰
4. 按目标受众调整语气

你不负责评估自己的输出。

你应专注于产出，而不是判断。

行为规则

- 不输出分析过程
- 不质疑需求
- 必须直接生成答案
- 必须使用指定输出格式（如段落、列表、步骤）
- 不添加与任务无关的信息

输出格式（示例为方案生成类任务）

```
{
 "title": "校园音乐节策划方案",
 "sections": [
 {
 "heading": "活动目标",
 "content": "丰富校园文化生活..."
 }
],
}
```

```
{
 "heading": "执行步骤",
 "content": ["场地申请", "设备租赁", "宣传排期"]
}
]
```

knowledge  
职责  
提供任务所需的外部事实、定义、数据、规则或引用来源。

身份定义  
你是 Knowledge Agent。  
你的职责是：

- 1. 接收查询关键词或领域
- 2. 检索或输出结构化知识
- 3. 注明知识来源或可信度
- 4. 不进行推理或创作

你不负责回答问题或生成方案。  
你必须保持客观、可引用。

行为规则

- 不生成新观点或建议
- 不混合多个无关知识
- 必须标注知识类型（事实/定义/规则/数据）
- 不评价用户需求
- 若无相关知识，明确说“无可用知识”

输出格式

```
{
 "knowledge_type": "定义",
 "query": "校园活动审批流程",
 "content": "一般需经过校团委备案、场地审批、安全报备三步",
 "source": "高校学生活动管理办法（示例）",
 "confidence": 0.85
}
```

backend\roletxt\knowledge.txt

```
[
 {
 "keywords": ["端云协同", "智能卸载", "MCP协议"],
 "content": "本地小模型和云端大模型通过统一标准对话，简单问题本地瞬间应答，复杂任务自动转给云端，不让用户多等。"
 },
 {
 "keywords": ["多智能体协作", "A2A协议", "任务编排"],
 "content": "让多个AI像专家组一样分工，有人查资料，有人写稿，有人校对，通过互相通信共同完成一个复杂需求。"
 },
 {
 "keywords": ["意图识别", "边侧路由", "轻量分类"],
 "content": "门口设一个“智能分拣员”，一眼看出用户问题是简单还是复杂，马上决定自己处理还是呼叫云端支援。"
 },
 {
 "keywords": ["模型量化", "低比特推理", "GPTQ"],
 "content": "把AI模型压缩成极小的格式，就像压缩饼干，营养保留但体积大减，在手机上也能快速运行。"
 },
 {
 "keywords": ["知识蒸馏", "师-生模型", "轻量化训练"],
 "content": "云端“教授”把知识精华提炼给本地“学生”，学生用很少的脑子就能答出接近教授的考卷。"
 },
 {
 "keywords": ["语义缓存", "命中即答", "向量存储"],
 "content": "问过的问题会被记住，下次遇到类似问题直接拿出旧答案，省去重复思考，响应快如闪电。"
 },
 {
 "keywords": ["检索增强生成", "RAG", "外挂知识库"],
 "content": "AI先翻本地上传的手册或数据库，找到依据后再生成回答，保证内容有出处，不凭空瞎编。"
 },
 {
 "keywords": ["思维链推理", "逐步思考", "Chain-of-Thought"],
```

```

 "content": "让AI像人类一样分步骤推理，解数学题或做逻辑分析时一步步来，大大减少低级错误。",
 },
 {
 "keywords": ["多模态交互", "视觉-语言理解", "图文协同"],
 "content": "不只能听会说，还能看懂图片、图表，结合图像和文字一起理解用户意思，给出更贴切的回应。",
 },
 {
 "keywords": ["离线优先", "本地兜底", "渐进式应用"],
 "content": "即使没网也不怕，核心功能全靠本地小模型撑腰，等网络恢复再悄悄同步云端结果。",
 },
 {
 "keywords": ["差分隐私", "数据隔离", "安全环境"],
 "content": "敏感数据在本地加上随机扰动再上传，或者干脆只在本地处理，云端收到的是一片迷雾，看不到真实信息。",
 },
 {
 "keywords": ["国产NPU加速", "端侧推理引擎", "硬件适配"],
 "content": "针对国产自研芯片做专门优化，像为AI在设备上修了一条专用跑道，跑得又快又省电。",
 },
 {
 "keywords": ["工具调用", "函数执行", "能力扩展"],
 "content": "AI能直接操作日历、邮件、数据库等工具，像真正秘书一样帮你订会议、查库存、发邮件。",
 },
 {
 "keywords": ["动态路由", "性价比调度", "智能选路"],
 "content": "系统像精明的管家，根据任务情况自动选择最便宜、最快的云端AI服务，不浪费每一分预算。",
 },
 {
 "keywords": ["流式输出", "首字延迟优化", "边想边发"],
 "content": "云端一边生成答案，本地一边显示，用户感觉就像本地回答一样顺畅，完全掩盖网络延迟。",
 },
 {
 "keywords": ["混合专家模型", "MoE", "稀疏激活"],
 "content": "庞大模型里住着多位专家，不同问题只唤醒相关专家来处理，节能又高效，不会全体总动员白耗资源。",
 },
 {
 "keywords": ["主动追问", "歧义消除", "意图澄清"],
 "content": "用户说的模糊时，AI不会瞎猜，而是立刻礼貌反问“您指的是哪一份报表？”，厘清情况再动手。",
 },
 {
 "keywords": ["自适应精度", "动态量化", "混合精度"],
 "content": "系统会根据任务难易自动切换计算精度，简单对话用省电模式，复杂计算切高精度，又快又稳。",
 },
 {
 "keywords": ["多答案融合", "置信度仲裁", "一致性校验"],
 "content": "本地和云端答案打架时，一个裁判模型会根据各自的把握程度，投票或综合出最可靠的最终回答。",
 },
 {
 "keywords": ["渐进式摘要", "长文本压缩", "关键词提取"],
 "content": "阅读长篇报告时，先分章节提取核心句子，再汇总精炼，确保重点不丢，又不用读完全文。",
 },
 {
 "keywords": ["模型预热", "无感切换", "预加载"],
 "content": "刚打开应用，系统就预测你马上要提问，提前把相关模型预备好，话音落答案就已经出来了。",
 },
 {
 "keywords": ["请求合并", "批处理积攒", "网络减负"],
 "content": "用户连续发送多个问题，AI先收集不中断，打包成一捆一次性发给云端，大大减少来回折腾。",
 },
 {
 "keywords": ["热温冷分层", "数据升降级", "智能存取"],
 "content": "常用知识放本地内存“口袋”，偶尔用的放边缘“背包”，冷门知识存云端“仓库”，存取总走最短路径。",
 },
 {
 "keywords": ["情感计算", "亲和交互", "情绪调节"],
 "content": "本地小模型实时察觉用户情绪，焦虑时立刻切换柔和语调并快速安抚，平静后再转云端深度处理。",
 },
 {
 "keywords": ["本地微调", "小样本学习", "风格适配"],
 "content": "用户只需给几个例子，就能在本地调校AI的说话风格或专业倾向，变成真正贴身秘书。",
 },
 {
 "keywords": ["代码骨架", "协同补全", "片段生成"],
 "content": "本地先秒出一段代码框架和基础逻辑，云端再填入复杂算法和异常处理，最后合并成完整可用代码。",
 },
 {
 "keywords": ["双模安全审核", "护栏过滤", "多层防御"],
 "content": "本地先快速拦截明显的违规内容，云端再做深层次的语义审查，双重保障输出安全无害。",
 },
 {
 "keywords": ["联邦学习", "隐私聚合", "去中心化训练"],
 }

```

```

 "content": "多个设备各自在本地学习，只把加密的“经验”上传融合，原始数据不离开，模型却能一起变聪明。",
 },
 {
 "keywords": ["场景预判", "推理预热", "行为预测"],
 "content": "你刚打开周报模板，AI就猜到你可能要写周报，提前准备相关模型和数据，操作毫无卡顿。",
 },
 {
 "keywords": ["错误自愈", "降级响应", "退避重试"],
 "content": "云端卡住了，本地先用简化版答案稳住场面，同时后台不断重试，成功后悄悄把回答替换为优质版本。",
 },
 {
 "keywords": ["特征压缩", "传输瘦身", "数据减量"],
 "content": "发往云端的信息先被提炼成极简的数值暗号，云端一看就懂，省下大量流量和时间。",
 },
 {
 "keywords": ["对话状态管理", "记忆压缩", "上下文总结"],
 "content": "长时间聊天时，本地只把关键情节总结成一句话传给云端，不用每次都把完整记录重说一遍。",
 },
 {
 "keywords": ["多语言路由", "中文优先", "外文卸载"],
 "content": "中文对话完全由本地高速处理；遇到外语则自动切换给云端多语言专家，本地不安装臃肿的语言包。",
 },
 {
 "keywords": ["数学工具调用", "精确计算", "计算器增强"],
 "content": "遇到加减乘除或解方程时，AI不依赖模糊猜测，直接调用内置计算工具给出毫厘不差的数值。",
 },
 {
 "keywords": ["语义去重", "相似锁", "答案复用"],
 "content": "本地维护一个“问题指纹库”，碰到高度相似的新问题直接调取上次的优质答案，几乎零计算快速响应。",
 },
 {
 "keywords": ["多应用底座", "实例隔离", "轻量多开"],
 "content": "社交、办公、购物等不同软件的AI助手共享同一个本地模型引擎，但各自的记忆和上下文互相隔绝。",
 },
 {
 "keywords": ["自主工作流", "规划执行", "多步任务"],
 "content": "只需说“帮我准备季度汇报”，AI就会自动规划：收集数据→做图表→生成报告→发邮件，一条龙完成。",
 },
 {
 "keywords": ["插件式模型", "按需下载", "能力市场"],
 "content": "本地不囤积所有功能，像应用市场一样，需要翻译或修图时，临时下载对应的小模型，用完可卸。",
 },
 {
 "keywords": ["图文联合处理", "视觉特征提取", "前置压缩"],
 "content": "拍一张设备照片，本地先提取关键视觉特征并压缩，再把精简信息连同问题发给云端分析，快速又省流。",
 },
 {
 "keywords": ["置信度自评", "云端复审", "答案校验"],
 "content": "本地回答都会附带一个“我有多确定”，分数低时自动触发云端更强大的模型重新检查纠正。",
 },
 {
 "keywords": ["存算一体", "零搬运开销", "新型芯片"],
 "content": "用支持在内存里直接计算的芯片跑AI，数据不用搬来搬去，能耗极低还不怎么发热。",
 },
 {
 "keywords": ["持续学习", "夜间微调", "模型自进化"],
 "content": "白天本地模型犯错被纠，晚上设备空闲时它自己默默学习改正，第二天醒来它就变聪明了一点。",
 },
 {
 "keywords": ["跨模态缓存", "图文语义桥", "复用加速"],
 "content": "问过“这张图里是什么”，以后提到这张图的文字描述时，本地直接复用之前的理解，不用重新看图。",
 },
 {
 "keywords": ["会话级卸载", "动态转移", "上下文打包"],
 "content": "聊天中突然要翻译长文件，本地AI立刻把当前对话摘要打包交给云端大模型，无缝接力不中断。",
 },
 {
 "keywords": ["能耗控制", "智能限频", "低功耗模式"],
 "content": "当手机快没电或发热时，系统自动限制本地AI推理速度，以时间换低温，保证最基础功能一直在线。",
 },
 {
 "keywords": ["结构化输出", "JSON强制", "字段约束"],
 "content": "让AI提取发票信息时，回复一定是规整的键值对，不会多一句废话，直接就能填入系统。",
 },
 {
 "keywords": ["多智能体辩论", "质量增强", "观点博弈"],
 "content": "云端同时让三位不同专长的AI就同一个问题“辩论”，综合最佳观点给出结论，避免一言堂的偏颇。",
 },
 {
 "keywords": ["隐私计算", "可信执行环境", "数据笼子"],

```

```

 "content": "在设备上划分一个加密的安全小房间，敏感数据在里面运算，结束后只放出结果，数据本身无法被窃取。",
 },
 {
 "keywords": ["最小成本策略", "计价路由", "免费额度优先"],
 "content": "每个任务都精打细算，能用免费额度的服务先用，然后再选最便宜的国产云端API，后台自动切换。",
 },
 {
 "keywords": ["互备集群", "多活云端", "故障无感转移"],
 "content": "系统同时连着主云端和备用云端，一个意外断开连另一个，用户完全感知不到服务有一瞬间的切换。",
 }
]
{
 "keywords": ["Review标准"],
 "content": "生成内容必须包含明确结构、逻辑完整、避免重复描述。",
}
{
 "keywords": ["Judge规则"],
 "content": "当任务长度小于100字且不涉及规划、推理、多步骤分析时，优先本地处理。",
}
[
 {
 "keywords": ["响应路径裁决", "复杂度阈值", "本地优先策略"],
 "content": "系统会自动判断问题长短和难度，简单闲聊或常识提问由本地瞬间处理，只有需要深度推理或外部知识时才动用云端。",
 },
 {
 "keywords": ["上下文保鲜", "记忆保留规则", "会话连续性"],
 "content": "对话中途不能断片，本地会一直记住最近几轮核心内容，即使切换到云端补充，也要把摘要无缝传过去，确保前后连贯。",
 },
 {
 "keywords": ["资源兜底", "降级保障", "降级服务"],
 "content": "当网络失败或云端超时，系统绝不报错白屏，必须立刻用本地简化结果顶上去，优先保持基本可用。",
 },
 {
 "keywords": ["安全分级处理", "隐私边界", "数据不离场"],
 "content": "涉及身份证号、密码、内部机密文件等，系统被强制锁定在本地处理，任何情况都不允许上传到云端一个字。",
 },
 {
 "keywords": ["质量闭环", "答案自评", "低分重审"],
 "content": "本地给出的每个答案都要悄悄给自己打分，感觉没把握时，自动在后台请求云端更强大的模型校验或重写，再第一时间替换给用户。",
 },
 {
 "keywords": ["并发限流", "算力排队", "公平调度"],
 "content": "同时来很多任务时，系统按紧急程度和资源占用排队，轻任务快速放行，重任务告知稍等，绝不把本地设备卡死。",
 },
 {
 "keywords": ["输出约束", "格式规整", "结构化强制"],
 "content": "遇到需要填表、写代码、出清单等要求时，回答一定去掉寒暄，只输出结构清晰的内容，可以直接复制使用。",
 },
 {
 "keywords": ["多结果融合", "冲突仲裁", "可信筛选"],
 "content": "当本地和云端给出不同答案，系统不会随机挑一个，而是对比各自的置信度和证据，选出最可靠的一条，或综合后给出保守回答。",
 },
 {
 "keywords": ["耗能管控", "发热抑制", "温控降频"],
 "content": "设备温度过高或电量过低时，系统会主动降低本地推理的速度或暂停云端请求，用轻微延迟换取安全和长续航。",
 },
 {
 "keywords": ["成本红线", "计价路由", "最小开销"],
 "content": "每调用一次云端服务都算经济账，能合并的任务必合并，能用免费额度绝不花钱，自动在多个国产云端间选择最低价格通道。",
 },
 {
 "keywords": ["重复检测", "语义指纹", "请求去重"],
 "content": "用户连续提出相同或极其相似的问题时，系统不会傻傻重复思考，而是直接返回缓存中的上一次结果，不让资源白费。",
 },
 {
 "keywords": ["能力边界声明", "坦诚拒绝", "防幻觉"],
 "content": "碰到完全不懂或数据缺失的问题，系统必须明确说“暂时无法回答”，绝不强行编造看似合理的假信息。",
 }
]
[
 {
 "keywords": ["校园", "学校", "大学", "中学", "小学", "教育体系"],
 "content": "中国教育体系分为学前教育、初等教育（小学6年）、中等教育（初中3年+高中3年）和高等教育（大学本科4年、硕士2-3年、博士3-5年）。九年义务教育覆盖小学和初中阶段。",
 },
 {
 "keywords": ["校园生活", "宿舍", "食堂", "图书馆", "社团"],

```

```
 "content": "大学校园生活包括住宿（通常4-6人间）、餐饮（学生食堂价格实惠）、学习（图书馆和自习室）、社交（社团活动和学生会）以及体育锻炼（操场和体育馆）。",
 },
 {
 "keywords": ["考试", "期末", "期中", "高考", "考研", "四六级"],
 "content": "重要考试包括：高考（每年6月7-8日，决定大学录取）、考研（每年12月底，全国硕士研究生统一招生考试）、英语四六级（每年6月和12月）、期末考试（每学期末，通常在第16-18周）。",
 },
 {
 "keywords": ["奖学金", "助学金", "助学贷款", "国家奖学金"],
 "content": "国家奖学金8000元/年（成绩+综测前5%）、国家励志奖学金5000元/年（贫困+成绩前30%）、校级奖学金（各校自定）、国家助学金（平均3300元/年）、生源地助学贷款（最高12000元/年）。",
 },
 {
 "keywords": ["毕业", "就业", "校招", "春招", "秋招", "实习"],
 "content": "秋招（9-11月）规模最大，春招（3-5月）为补充。应届生身份保留2年。重要就业方向：互联网/IT、金融、制造业、教育、医疗、公务员/事业单位。实习建议大三暑假开始。",
 }
]
[
 {
 "keywords": ["办公软件", "Office", "WPS", "Word", "Excel", "PPT", "文档处理"],
 "content": "主流办公软件：微软Office 365（Word文档处理、Excel电子表格、PowerPoint演示文稿）和国产WPS Office。WPS个人版免费，兼容Office格式，支持云端协作和PDF编辑。",
 },
 {
 "keywords": ["会议", "会议纪要", "会议记录", "开会"],
 "content": "高效会议要点：提前发议程、控制时长（30-60分钟为宜）、指定记录人、会后24小时内发纪要。会议纪要包含：时间地点、参会人员、议题讨论、决议事项、行动项（责任人和截止日期）。",
 },
 {
 "keywords": ["项目管理", "甘特图", "里程碑", "敏捷", "Scrum", "看板"],
 "content": "项目管理方法：瀑布模型（顺序执行，适合需求明确的项目）、敏捷Scrum（2-4周迭代，每日站会）、看板（可视化工作流，限制在制品）。常用工具：Jira、Trello、飞书、钉钉、Teambition。",
 },
 {
 "keywords": ["邮件", "商务邮件", "电子邮件礼仪", "Email"],
 "content": "商务邮件规范：标题简洁明确（20字以内）、称呼得体、正文分段落（每段不超过5行）、附件命名规范（日期+内容+版本号）、及时回复（24小时内）、使用签名档（姓名+职位+联系方式）。",
 },
 {
 "keywords": ["时间管理", "番茄工作法", "GTD", "四象限法则", "效率"],
 "content": "时间管理方法：番茄工作法（25分钟专注+5分钟休息）、GTD（收集-整理-组织-回顾-执行）、四象限法则（重要紧急→立即做、重要不紧急→计划做、紧急不重要→委托、不重要不紧急→删除）。",
 },
 {
 "keywords": ["远程办公", "居家办公", "在线协作", "视频会议"],
 "content": "远程办公工具：视频会议（腾讯会议、Zoom、Teams）、即时通讯（企业微信、飞书、钉钉）、文档协作（腾讯文档、石墨文档、Notion）、任务管理（Trello、Asana）。保持工作效率的要点：固定工作区域、规律作息、每日站会同步进度。",
 }
]
[
 {
 "keywords": ["AI", "人工智能", "大模型", "LLM", "GPT", "深度学习", "机器学习"],
 "content": "人工智能（AI）是模拟人类智能的技术科学。关键分支：机器学习（从数据中学习规律）、深度学习（多层神经网络）、自然语言处理（NLP，理解和生成文本）、计算机视觉（CV，理解图像视频）。大语言模型（LLM）如GPT-4、Claude、文心一言、通义千问等是当前AI前沿。",
 },
 {
 "keywords": ["AIGC", "AI生成", "AI绘画", "AI写作", "Stable Diffusion", "Midjourney"],
 "content": "AIGC（AI Generated Content）利用AI自动生成文字、图片、音频、视频等内容。AI绘画工具：Midjourney（高质量艺术风格）、Stable Diffusion（开源可本地部署）、DALL-E。AI写作工具可生成文章、代码、诗歌等。",
 },
 {
 "keywords": ["提示词", "Prompt", "提示工程", "Prompt Engineering"],
 "content": "提示工程是设计和优化输入给AI模型的指令以获取期望输出的技术。核心技巧：明确角色（你是一位XX专家）、给出格式要求（用Markdown输出）、提供示例（Few-shot）、分步骤引导（Chain-of-Thought）、设定约束（不超过500字）。",
 },
 {
 "keywords": ["模型训练", "微调", "Fine-tuning", "预训练", "强化学习"],
 "content": "AI模型训练流程：预训练（在海量数据上学习通用知识）→ 微调（在特定领域数据上调整）→ 对齐（RLHF强化学习人类反馈，让输出符合人类偏好）。微调成本远低于预训练，适合企业和个人定制模型。",
 },
 {
 "keywords": ["AI伦理", "AI安全", "偏见", "可解释性", "AI监管"],
 "content": "AI伦理核心议题：算法偏见（训练数据偏差导致歧视）、隐私保护（训练数据可能泄露个人信息）、可解释性（AI决策过程应可理解）、就业影响（自动化替代部分岗位）、安全对齐（确保AI行为符合人类价值观）。各国正在制定AI监管法规。",
 }
]
[
```

```
{
 "keywords": ["国产操作系统", "麒麟", "统信UOS", "deepin", "鸿蒙", "欧拉"],
 "content": "国产操作系统主要有：麒麟OS（中标麒麟/银河麒麟，基于Linux，用于政府和军工）、统信UOS（基于deepin深度系统，面向桌面和服务端，兼容Windows应用迁移）、华为鸿蒙HarmonyOS（微内核分布式系统，覆盖手机/平板/车机/IoT）、华为欧拉openEuler（服务器操作系统，面向云计算和边缘计算）。"
},
{
 "keywords": ["鸿蒙", "HarmonyOS", "华为", "分布式", "原子化服务"],
 "content": "鸿蒙HarmonyOS是华为自研的分布式操作系统，特点：微内核架构（更安全可靠）、分布式软总线（设备间无缝协同）、原子化服务（无需安装App即可使用）、一次开发多端部署（手机/平板/手表/车机/智慧屏）。2024年发布HarmonyOS NEXT（纯鸿蒙，不再兼容安卓）。"
},
{
 "keywords": ["deepin", "深度操作系统", "统信", "Linux桌面"],
 "content": "deepin（深度操作系统）是中国最流行的Linux桌面发行版，以美观的DDE桌面环境和丰富的中文生态著称。统信UOS基于deepin开发，分为桌面版、服务器版和设备版，支持x86/ARM/龙芯/MIPS等多种CPU架构，内置应用商店提供常用软件。"
},
{
 "keywords": ["信创", "国产化替代", "自主可控", "安可"],
 "content": "信创（信息技术应用创新）是国家推动的IT基础设施国产化战略。目标：CPU（龙芯/飞腾/鲲鹏/申威）、操作系统（麒麟/统信）、数据库（达梦/人大金仓/南大通用）、中间件（东方通/普元）、办公软件（WPS）全链条自主可控，确保国家信息安全。"
},
{
 "keywords": ["国产芯片", "龙芯", "飞腾", "鲲鹏", "申威", "摩尔线程"],
 "content": "国产CPU主要厂商：龙芯（MIPS/LoongArch自主架构，桌面/服务器）、飞腾（ARM架构，服务器/桌面）、鲲鹏（华为ARM架构，服务器高性能计算）、申威（Alpha自主架构，超算）、海光（x86兼容，服务器）。国产GPU：摩尔线程（桌面/数据中心）、景嘉微（军工/嵌入式）。"
}
]
[
 {
 "keywords": ["健康", "饮食", "睡眠", "运动", "体检"],
 "content": "健康生活建议：每天7-8小时睡眠（22:00-6:00为最佳时段）、每周150分钟中等强度运动（快走/游泳/骑行）、均衡饮食（蔬菜水果占餐盘1/2，蛋白质1/4，主食1/4）、每年一次全面体检、每天饮水1.5-2升。"
 },
 {
 "keywords": ["营养", "维生素", "蛋白质", "碳水", "脂肪", "膳食纤维"],
 "content": "七大营养素：蛋白质（肉蛋奶豆，修复组织）、碳水化合物（米面薯类，提供能量）、脂肪（坚果鱼油，储存能量）、维生素（蔬果，调节代谢）、矿物质（钙铁锌，构成身体）、膳食纤维（粗粮蔬菜，促进消化）、水（生命必需）。"
 },
 {
 "keywords": ["急救", "心肺复苏", "CPR", "海姆立克", "烫伤", "中暑"],
 "content": "常见急救知识：心肺复苏CPR（按压频率100-120次/分钟，深度5-6cm，按压与人工呼吸30:2）、海姆立克急救法（站背后双手握拳向上冲击腹部）、烫伤（流动冷水冲洗15分钟，勿涂牙膏酱油）、中暑（移至阴凉处、补充盐水、物理降温）。"
 },
 {
 "keywords": ["天气", "气象", "台风", "暴雨", "高温", "寒潮", "预警"],
 "content": "气象预警等级（由低到高）：蓝色、黄色、橙色、红色。台风天防护：关好门窗、收起室外物品、避免外出、远离广告牌和临时建筑。高温天防暑：避免10-16点户外活动、穿透气浅色衣服、多喝淡盐水。暴雨天避免涉水、远离电线杆。"
 },
 {
 "keywords": ["交通", "出行", "高铁", "飞机", "地铁", "公交", "网约车"],
 "content": "中国高铁（G/D字头）覆盖主要城市，购票用12306 App。飞机提前2小时到机场（国际航班3小时），液体单瓶不超100ml可随身。地铁使用手机扫码（支付宝/微信乘车码）或交通卡。网约车平台：滴滴、高德、T3、曹操出行。"
 },
 {
 "keywords": ["垃圾分类", "环保", "回收", "碳中和", "碳达峰"],
 "content": "垃圾分类标准：可回收物（纸塑料金属玻璃）、有害垃圾（电池灯管药品）、厨余垃圾（食物残渣果皮）、其他垃圾（不可回收的生活废弃物）。中国目标：2030年前碳达峰（碳排放不再增长），2060年前碳中和（碳排放净值为零）。"
 },
 {
 "keywords": ["法律常识", "合同", "劳动法", "消费者权益", "社保"],
 "content": "劳动法基本权益：试用期最长6个月、加班费（工作日1.5倍/休息日2倍/法定假日3倍）、带薪年假（工龄1-10年5天/10-20年10天/20年以上15天）、五险一金（养老/医疗/失业/工伤/生育+住房公积金）。消费者权益：7天无理由退货（网购）、假一赔三。"
 },
 {
 "keywords": ["金融", "理财", "基金", "股票", "保险", "存款", "信用卡"],
 "content": "理财基础知识：银行存款（活期/定期，50万以内受存款保险保障）、货币基金（余额宝/零钱通，低风险灵活取用）、债券基金（中低风险）、股票基金（高风险高收益）。建议：先存3-6个月生活费的应急金，再考虑投资，不要把所有钱投入高风险产品。"
 }
]
```

```
[
 {
 "keywords": ["校园", "活动", "策划"],
 "inject": "必须检查是否包含活动流程、预算、时间安排。"
 },
 {
 "keywords": ["项目", "方案"],
 "inject": "必须检查是否包含实施步骤与目标分析。"
 },
 {
 "keywords": ["ppt", "汇报"],
 "inject": "必须检查是否具备结构化章节。"
 }
]
```

## backend\run\_backend.py

```
import sys
import os

backend_dir = os.path.dirname(os.path.abspath(__file__))
realpath = os.path.realpath(backend_dir)

print(f"backend_dir: {backend_dir}")
print(f"realpath: {realpath}")
print(f"sys.path before: {sys.path[:3]}")

if realpath not in sys.path:
 sys.path.insert(0, realpath)

print(f"sys.path after: {sys.path[:3]}")
print(f"knowledge exists: {os.path.exists(os.path.join(realpath, 'knowledge'))}")

if __name__ == "__main__":
 import uvicorn
 from app.main import app
 uvicorn.run(app, host="0.0.0.0", port=8000)
```

## backend\run\_direct.py

```
#!/usr/bin/env python3
"""
直接运行 app.main
"""

import sys
import os

添加当前目录到路径
sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

print("=" * 60)
print("🚀 AgentMatrix 直接启动")
print("=" * 60)

try:
 from app.main import app
 import uvicorn

 print("\n✅ 导入成功!")
 print("\n🌐 服务地址: http://localhost:8000")
 print("\n📄 API文档: http://localhost:8000/docs")
 print("\n🛑 按 Ctrl+C 停止服务\n")
 print("=" * 60 + "\n")

 uvicorn.run(app, host="0.0.0.0", port=8000)

except Exception as e:
```

```
print(f"\n❌ 启动失败: {type(e).__name__}: {e}")
import traceback
print("\n详细错误:")
traceback.print_exc()
print("\n" + "-" * 60)
```

## backend\services\_\_init\_\_.py

```
from .agent_service import AgentService, get_agent_service

__all__ = ["AgentService", "get_agent_service"]
```

## backend\services\agent\_service.py

```
from typing import Dict, Any, List, Optional
from agents.base.agent_registry import AgentRegistry
from agents.base.agent import AgentInput, AgentOutput
from models.workflow import WorkflowInput, WorkflowOutput
from core.workflow.service import WorkflowService
from core.dynamic_router import get_dynamic_router
from knowledge.service import KnowledgeService
from api.v1.metrics.router import get_metrics_store
import logging

logger = logging.getLogger(__name__)

class AgentService:
 def __init__(self):
 self.agent_registry = AgentRegistry()
 self.workflow_service = WorkflowService(self.agent_registry)
 self.dynamic_router = get_dynamic_router()
 self.knowledge_service = KnowledgeService()
 self.metrics = get_metrics_store()

 async def initialize(self) -> None:
 await self.agent_registry.initialize_all_agents()
 logger.info("AgentService initialized successfully")

 async def shutdown(self) -> None:
 await self.agent_registry.shutdown_all_agents()
 logger.info("AgentService shutdown successfully")

 async def execute_workflow(self, user_input: str, context: Optional[Dict[str, Any]] = None) -> WorkflowOutput:
 workflow_input = WorkflowInput(user_input=user_input, context=context)
 return await self.workflow_service.execute(workflow_input)

 async def execute_single_agent(self, agent_id: str, input_data: AgentInput) -> AgentOutput:
 return await self.agent_registry.execute_agent(agent_id, input_data)

 def get_all_agent_statuses(self) -> Dict[str, Any]:
 return self.agent_registry.get_all_agent_statuses()

 def get_agent_status(self, agent_id: str) -> Optional[Dict[str, Any]]:
 agent = self.agent_registry.get_agent(agent_id)
 if agent:
 return agent.get_status()
 return None

 def get_metrics(self) -> Dict[str, Any]:
 return {
 "workflow": self.metrics,
 "routing": self.dynamic_router.get_routing_stats(),
 "knowledge": self.knowledge_service.get_knowledge_stats()
 }

 def search_knowledge(self, query: str, limit: int = 5) -> Dict[str, List[str]]:
 return self.knowledge_service.search(query, limit)
```

```

def add_knowledge(self, keyword: str, content: List[str]) -> None:
 self.knowledge_service.add_knowledge(keyword, content)

def delete_knowledge(self, keyword: str) -> bool:
 return self.knowledge_service.delete_knowledge(keyword)

async def route_request(self, complexity_score: float, prompt: str, system_prompt: str = None) -> Dict[str, Any]:
 return await self.dynamic_router.route(complexity_score, prompt, system_prompt)

def get_routing_stats(self) -> Dict[str, Any]:
 return self.dynamic_router.get_routing_stats()

_agent_service: Optional[AgentService] = None

def get_agent_service() -> AgentService:
 global _agent_service
 if _agent_service is None:
 _agent_service = AgentService()
 return _agent_service

```

## backend\shared\_\_init\_\_.py

## backend\shared\platform.py

```

PLATFORM_NAME = "AgentMatrix"
PLATFORM_DESCRIPTION = "多智能体动态协同与国产算力优化平台"

PLATFORM_IDENTITY = f"""
你是 {PLATFORM_NAME} 平台的 AI 助手——一个 {PLATFORM_DESCRIPTION}。
核心原理：简单任务由本地轻量模型(qwen2.5)处理，复杂任务动态调用云端大模型(DeepSeek)增强。
你的回答永远不代表任何其他公司或平台的AI助手，你只属于 {PLATFORM_NAME} 平台。
当用户问“你是谁”或类似问题时，你应该直接回答“我是 {PLATFORM_NAME} 平台的 AI 助手”，而不是说“用户来自 {PLATFORM_NAME} 平台”。
"""

```

## backend\simple\_start.py

```

#!/usr/bin/env python3
"""
最简单的启动脚本
"""

import sys
import os

print("=" * 60)
print("🚀 AgentMatrix 启动器")
print("=" * 60)

检查当前目录
print(f"\n📁 工作目录: {os.getcwd()}")

检查必要的文件
required_files = ['app/main.py', '.env', 'frontend/index.html']
for file in required_files:
 if os.path.exists(file):
 print(f"✅ {file}")
 else:
 print(f"❌ {file} - 不存在")

```

```

 sys.exit(1)

尝试导入
print("\n🔍 检查依赖...")
try:
 import fastapi
 import uvicorn
 print("✅ FastAPI 和 Uvicorn 可用")
except ImportError as e:
 print(f"❌ 缺少依赖: {e}")
 print("\n💡 请先安装依赖:")
 print(" pip install fastapi uvicorn pydantic python-dotenv httpx")
 sys.exit(1)

print("\n" + "=" * 60)
print("✅ 准备就绪! 正在启动服务...")
print("=" * 60)
print("\n🌐 服务地址: http://localhost:8000")
print("\n📖 API文档: http://localhost:8000/docs")
print("\n🛑 按 Ctrl+C 停止服务\n")

try:
 os.system(f'{sys.executable} -m uvicorn app.main:app --host 0.0.0.0 --port 8000')
except KeyboardInterrupt:
 print("\n\n💡 服务已停止")
except Exception as e:
 print(f"\n❌ 启动失败: {e}")
 print("\n💡 尝试直接运行:")
 print(' python app/main.py')

```

## backend\start\_server.py

```

import sys
import os

script_dir = os.path.dirname(os.path.abspath(__file__))
parent_dir = os.path.dirname(script_dir)

if parent_dir not in sys.path:
 sys.path.insert(0, parent_dir)

from app.main import app
import uvicorn

if __name__ == "__main__":
 uvicorn.run(app, host="0.0.0.0", port=8000)

```

## backend\start\_service.py

```

#!/usr/bin/env python3
"""
启动 AgentMatrix 后端服务的脚本
"""
import subprocess
import sys
import time
import socket

def check_port(host, port):
 """检查端口是否被占用"""
 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
 try:
 s.connect((host, port))
 s.close()
 return True
 except:
 return False

```

```

def main():
 print("=" * 60)
 print("🚀 启动 AgentMatrix 后端服务")
 print("=" * 60)

 # 检查 8000 端口
 if check_port('127.0.0.1', 8000):
 print("⚠️ 警告: 端口 8000 已被占用")
 print("请先停止占用该端口的程序")
 return

 # 检查必要的包
 try:
 import uvicorn
 import fastapi
 print("✅ 依赖检查通过")
 except ImportError as e:
 print(f"❌ 依赖缺失: {e}")
 print("请安装依赖: pip install -r requirements.txt")
 return

 # 启动服务
 print("\n🏠 正在启动服务...")
 print("🌐 服务地址: http://localhost:8000")
 print("📖 API文档: http://localhost:8000/docs")
 print("🛑 按 Ctrl+C 停止服务")
 print("=" * 60 + "\n")

 try:
 subprocess.run([
 sys.executable, "-m", "uvicorn",
 "app.main:app",
 "--host", "0.0.0.0",
 "--port", "8000",
 "--reload"
])
 except KeyboardInterrupt:
 print("\n\n🛑 服务已停止")
 except Exception as e:
 print(f"\n\n❌ 启动失败: {e}")

if __name__ == "__main__":
 main()

```

## backend\test\_agent\_debug.py

```

import asyncio
import json
from agents.base.agent_registry import AgentRegistry

async def test_writer_agent():
 print("=== 测试 Writer Agent ===")

 # 初始化 Agent Registry
 ar = AgentRegistry()
 ar.initialize_all_agents_sync()

 # 创建测试输入 (模拟 Summary Agent 的输出格式)
 test_input = json.dumps({
 "task": "帮我写一个关于校园AI助手的年度规划",
 "original_question": "帮我写一个关于校园AI助手的年度规划包含: 1. 时间线、2. 目标分析、3. 实施步骤",
 "keywords": ["AI", "校园", "规划", "年度"],
 "knowledge_points": [
 {"type": "领域知识", "content": "校园AI助手可以帮助学生学习、教师教学管理等"},
 {"type": "通用知识", "content": "年度规划需要包含目标设定、时间安排、资源分配"}
],
 "requirements": ["需要包含时间线", "需要包含目标分析", "需要包含实施步骤"],
 "outline": [
 "一、任务概述",
 "二、核心需求",
 "三、解决方案",
 "四、实施计划"
],
 "summary": "用户需要校园AI助手的年度规划方案"
 })

```

```

))

print(f"输入长度: {len(test_input)}")
print(f"输入内容预览: {test_input[:200]}...")

获取 Writer Agent
writer_agent = ar.get_agent("writer")
if not writer_agent:
 print("ERROR: Writer Agent 未找到")
 return

from agents.base.agent import AgentInput

执行 Writer Agent
agent_input = AgentInput(content=test_input, context={}, use_llm=True, use_cloud=False)
result = await writer_agent.execute(agent_input)

print(f"\n输出状态: {'成功' if result.success else '失败'}")
print(f"输出长度: {len(result.content)}")
print(f"输出内容预览: {result.content[:500]}")

if len(result.content) < 100:
 print("\n警告: 输出内容过短, 可能存在问题!")
 print(f"完整输出: {result.content}")

print(f"\n元数据: {result.metadata}")

async def test_review_agent():
 print("\n=== 测试 Review Agent ===")

 ar = AgentRegistry()
 ar.initialize_all_agents_sync()

 # 创建测试输入
 review_input = json.dumps({
 "user_task": "帮我写一个关于校园AI助手的年度规划",
 "summary": "用户需要校园AI助手的年度规划方案",
 "writer_output": "# 校园AI助手年度规划\n\n## 一、任务概述\n校园AI助手是一个旨在提升校园智能化水平的项目...\n\n## 二、核心需求\n1. 时间线规划\n2. 目标分析\n3. 实施步骤\n\n## 三、解决方案\n...\n\n## 四、实施计划\n..."
 })

 print(f"输入长度: {len(review_input)}")

 review_agent = ar.get_agent("review")
 if not review_agent:
 print("ERROR: Review Agent 未找到")
 return

 from agents.base.agent import AgentInput

 agent_input = AgentInput(content=review_input, context={}, use_llm=True, use_cloud=False)
 result = await review_agent.execute(agent_input)

 print(f"\n输出状态: {'成功' if result.success else '失败'}")
 print(f"输出长度: {len(result.content)}")
 print(f"输出内容: {result.content}")

 if result.success and result.content:
 try:
 data = json.loads(result.content)
 print(f"\n解析结果:")
 print(f" review_score: {data.get('review_score')}")
 print(f" dimensions: {data.get('dimensions')}")
 print(f" issues: {data.get('issues')}")
 print(f" suggestions: {data.get('suggestions')}")
 print(f" pass: {data.get('pass')}")
 except json.JSONDecodeError as e:
 print(f"JSON解析失败: {e}")

if __name__ == "__main__":
 asyncio.run(test_writer_agent())
 asyncio.run(test_review_agent())

```

```
#!/usr/bin/env python3
import asyncio
import aiohttp
import sys

async def test_deepseek(api_key):
 url = "https://api.deepseek.com/v1/chat/completions"
 headers = {
 "Authorization": f"Bearer {api_key}",
 "Content-Type": "application/json"
 }
 payload = {
 "model": "deepseek-chat",
 "messages": [
 {"role": "user", "content": "你好"}
],
 "temperature": 0.7,
 "max_tokens": 50
 }

 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 status = response.status
 text = await response.text()

 if status == 200:
 print("✅ API Key 有效!")
 return True
 else:
 print(f"❌ API 调用失败, 状态码: {status}")
 print(f"响应: {text}")
 return False

 except Exception as e:
 print(f"❌ 连接失败: {e}")
 return False

if __name__ == "__main__":
 if len(sys.argv) < 2:
 print("用法: python test_api.py <api_key>")
 sys.exit(1)

 api_key = sys.argv[1]
 result = asyncio.run(test_deepseek(api_key))
 sys.exit(0 if result else 1)
```

## backend\test\_api\_full.py

```
import aiohttp
import asyncio

async def test_api_with_full_response():
 print("=" * 70)
 print("完整测试 DeepSeek API 调用和 Token 消耗")
 print("=" * 70)

 api_key = "sk-6ccaead05a2f462d97417a268139b561"
 url = "https://api.deepseek.com/v1/chat/completions"

 messages = [
 {"role": "user", "content": "请用100字介绍自己"}
]

 payload = {
 "model": "deepseek-chat",
 "messages": messages,
 "temperature": 0.7,
 "max_tokens": 200
 }

 headers = {
 "Authorization": f"Bearer {api_key}",
```

```

 "Content-Type": "application/json"
 }

 print("\n发送请求...")
 print(f"模型: deepseek-chat")
 print(f"API Key: {api_key[:15]}... {api_key[-5:]}")
 print("-" * 70)

 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 print(f"\n响应状态码: {response.status}")

 if response.status == 200:
 data = await response.json()
 print("\n✅ API调用成功!")

 # 打印完整响应
 print("\n完整响应数据:")
 print(f" 模型: {data.get('model', 'N/A')}")
 print(f" ID: {data.get('id', 'N/A')}")
 print(f" Object: {data.get('object', 'N/A')}")
 print(f" Created: {data.get('created', 'N/A')}")

 # 打印 Token 使用情况
 usage = data.get('usage', {})
 print("\n📊 Token 使用情况:")
 print(f" prompt_tokens: {usage.get('prompt_tokens', 0)}")
 print(f" completion_tokens: {usage.get('completion_tokens', 0)}")
 print(f" total_tokens: {usage.get('total_tokens', 0)}")

 if usage:
 print("\n💰 计费信息:")
 print(f" 总消耗 Token: {usage.get('total_tokens', 0)}")
 print(f" 预计费用: ${usage.get('total_tokens', 0) / 1000000 * 0.27:.6f} (假设 $0.27/1M tokens)")

 # 打印回复
 choices = data.get('choices', [])
 if choices:
 content = choices[0].get('message', {}).get('content', '')
 print(f"\n回复内容 ({len(content)} 字符):")
 print(content)
 else:
 error_text = await response.text()
 print(f"\n❌ API调用失败")
 print(f"错误信息: {error_text}")

 except Exception as e:
 print(f"\n❌ 发生异常: {str(e)}")
 import traceback
 traceback.print_exc()

if __name__ == "__main__":
 asyncio.run(test_api_with_full_response())

```

## backend\test\_cloud\_call.py

```

"""
测试云服务调用是否正常工作
"""

import asyncio
import sys
sys.path.insert(0, '.')

from core.llm.client import get_llm_client
from app.config import settings

async def test_cloud_call():
 print("-" * 60)
 print("🚀 测试云服务调用")
 print("-" * 60)

 # 获取LLM客户端
 client = get_llm_client()

```

```

设置API Key (从配置读取)
api_key = settings.deepseek_api_key
if not api_key:
 print("❌ API Key 未设置, 请先在 .env 文件中配置")
 return

client.deepseek_api_key = api_key
print(f"✅ API Key 已设置: {api_key[:10]}...")

测试云服务调用
print("\n🌐 正在调用 DeepSeek 云服务...")
try:
 result = await client.generate_cloud("Hello, this is a test.")
 if "Error" in result:
 print(f"❌ 云服务调用失败: {result}")
 else:
 print(f"✅ 云服务调用成功!")
 print(f"📄 返回结果: {result[:100]}...")
 print("\n🎉 这证明云服务正在被正确调用!")
 print("💡 您的 API Key 应该会产生消费了")
except Exception as e:
 print(f"❌ 调用异常: {str(e)}")

测试本地调用
print("\n" + "-" * 60)
print("🏠 测试本地模型调用")
print("-" * 60)
try:
 result = await client.generate_local("Hello, this is a test.", model="qwen2.5:1.5b")
 if "Error" in result:
 print(f"⚠️ 本地模型调用失败: {result}")
 print("👉 这可能是因为 Ollama 服务未启动")
 else:
 print(f"✅ 本地模型调用成功!")
 print(f"📄 返回结果: {result[:100]}...")
except Exception as e:
 print(f"❌ 调用异常: {str(e)}")

if __name__ == "__main__":
 asyncio.run(test_cloud_call())

```

## backend\test\_complex.py

```

import asyncio
import httpx

async def test_complex():
 user_input = "帮我设计一个校园运动会活动方案, 需要包含流程、预算、时间线、人员分工"
 print(f"发送请求: {user_input}")

 async with httpx.AsyncClient(timeout=180) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": user_input}
)

 if response.status_code == 200:
 data = response.json()
 print(f"\n状态码: 200 OK")
 print(f"执行方式: {'本地执行' if data.get('executed_locally') else '云端执行'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f}秒")

 print("\n=== 最终结果 ===\n")
 result = data.get('final_result', '')
 if len(result) > 2000:
 print(result[:2000] + "\n...")
 else:
 print(result)
 else:
 print(f"请求失败: {response.status_code}")
 print(response.text)

```

```
if __name__ == "__main__":
 asyncio.run(test_complex())
```

## backend\test\_complexity.py

```
import asyncio
import json
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.main import app
from core.workflow.service import WorkflowService
from agents.base.agent_registry import AgentRegistry
from models.workflow import WorkflowInput

async def test_complexity_assessment():
 agent_registry = AgentRegistry()
 await agent_registry.initialize_all_agents()

 workflow_service = WorkflowService(agent_registry)

 test_cases = [
 {
 "name": "简单问答",
 "input": "什么是人工智能?",
 "expected_complexity": "低"
 },
 {
 "name": "中等任务",
 "input": "帮我写一封邮件",
 "expected_complexity": "中"
 },
 {
 "name": "复杂规划任务",
 "input": "帮我制定一份校园科技节活动策划方案, 包括活动主题、流程安排、预算分配、时间线规划、人员分工、宣传方案、应急预案等内容, 要求专业详细可执行。",
 "expected_complexity": "高"
 }
]

 for test_case in test_cases:
 print("\n" + "="*60)
 print("测试案例: {}".format(test_case['name']))
 print("输入: {}".format(test_case['input'][:50]))
 print("预期复杂度: {}".format(test_case['expected_complexity']))
 print("="*60)

 try:
 workflow_input = WorkflowInput(user_input=test_case['input'])
 result = await workflow_service.execute(workflow_input)

 print("\n执行结果:")
 print("- 复杂度评分: {:.2f}".format(result.complexity_score))
 print("- 是否本地执行: {}".format("是" if result.executed_locally else "否"))

 for step in result.steps:
 if step.agent_id == 'judge':
 try:
 judge_data = json.loads(step.output)
 print("- Judge决策: {}".format(judge_data.get('decision', '未知')))
 print("- Cloud模式: {}".format(judge_data.get('cloud_mode', '未知')))
 print("- Review评分: {:.2f}".format(judge_data.get('review_score', 0.0)))
 reasons = judge_data.get('reason', [])
 print("- 原因: {}".format(", ".join(reasons)))
 except Exception as e:
 print("- Judge输出解析失败: {}".format(e))
 print("- Judge原始输出: {}".format(step.output[:200]))

 if result.complexity_score >= 0.7:
 actual_complexity = "高"
 elif result.complexity_score >= 0.4:
 actual_complexity = "中"
```

```

 else:
 actual_complexity = "低"

 match = "匹配" if actual_complexity == test_case['expected_complexity'] else "不匹配"
 print("\n评估: {}".format(match))

 except Exception as e:
 print("错误: {}".format(e))

 await agent_registry.shutdown_all_agents()

if __name__ == "__main__":
 asyncio.run(test_complexity_assessment())

```

## backend\test\_complexity\_section.py

```

import asyncio
import json
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.main import app
from core.workflow.service import WorkflowService
from agents.base.agent_registry import AgentRegistry
from models.workflow import WorkflowInput

async def test_complexity_section():
 agent_registry = AgentRegistry()
 await agent_registry.initialize_all_agents()

 workflow_service = WorkflowService(agent_registry)

 test_input = "帮我制定一份校园科技节活动策划方案，包括活动主题、流程安排、预算分配、时间线规划、人员分工、宣传方案、应急预案等内容，要求专业详细可执行。"

 print("测试案例: 复杂规划任务")
 print("输入: {}".format(test_input[:50]))

 try:
 workflow_input = WorkflowInput(user_input=test_input)
 result = await workflow_service.execute(workflow_input)

 print("\n检查最终输出是否包含复杂度评估部分:")
 print("-" * 50)

 if "复杂度评估与建议" in result.final_result:
 print("✓ 包含复杂度评估部分")
 idx = result.final_result.index("复杂度评估与建议")
 section = result.final_result[idx:idx+1500]
 print("\n复杂度评估部分内容:")
 print(section)
 else:
 print("✗ 未找到复杂度评估部分")
 print("\n输出末尾内容:")
 print(result.final_result[-3000:])

 print("\n检查 Judge 结果:")
 for step in result.steps:
 if step.agent_id == 'judge':
 print("Judge输出:", step.output[:500])
 break

 except Exception as e:
 print("错误:", e)
 import traceback
 traceback.print_exc()

 await agent_registry.shutdown_all_agents()

if __name__ == "__main__":
 asyncio.run(test_complexity_section())

```

## backend\test\_complexity\_section2.py

```
import asyncio
import json
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.main import app
from core.workflow.service import WorkflowService
from agents.base.agent_registry import AgentRegistry
from models.workflow import WorkflowInput

async def test_complexity_section():
 agent_registry = AgentRegistry()
 await agent_registry.initialize_all_agents()

 workflow_service = WorkflowService(agent_registry)

 test_input = "帮我制定一份校园科技节活动策划方案，包括活动主题、流程安排、预算分配、时间线规划、人员分工、宣传方案、应急预案等内容，要求专业详细可执行。"

 print("测试案例：复杂规划任务")
 print("输入： {}".format(test_input[:50]))

 try:
 workflow_input = WorkflowInput(user_input=test_input)
 result = await workflow_service.execute(workflow_input)

 print("\n检查最终输出是否包含复杂度评估部分:")
 print("-" * 50)

 has_complexity = "复杂度评估与建议" in result.final_result

 if has_complexity:
 print("RESULT: 包含复杂度评估部分")
 idx = result.final_result.index("复杂度评估与建议")
 section = result.final_result[idx:idx+1500]
 print("\n复杂度评估内容:")
 print(section)
 else:
 print("RESULT: 未找到复杂度评估部分")
 print("\n输出长度:", len(result.final_result))
 print("\n输出末尾1000字符:")
 print(result.final_result[-1000:])

 print("\nJudge结果:")
 for step in result.steps:
 if step.agent_id == 'judge':
 print("Judge输出:", step.output)
 break

 except Exception as e:
 print("ERROR:", e)
 import traceback
 traceback.print_exc()

 await agent_registry.shutdown_all_agents()

if __name__ == "__main__":
 asyncio.run(test_complexity_section())
```

## backend\test\_config.py

```
from app.config import settings
from core.llm.client import get_llm_client

print("当前配置:")
```

```

print(f"ollama_host: {settings.ollama_host}")
print(f"ollama_model: {settings.ollama_model}")
print(f"deepseek_api_key: {'已设置' if settings.deepseek_api_key else '未设置'}")

llm_client = get_llm_client()
print(f"\nLLM Client 配置:")
print(f"ollama_host: {llm_client.ollama_host}")
print(f"ollama_model: {llm_client.ollama_model}")

```

## backend\test\_deepseek.py

```

import asyncio
import httpx

async def test_with_deepseek():
 print("=" * 70)
 print("测试 DeepSeek API Key 是否被正确使用")
 print("=" * 70)

 user_input = "帮我写一份详细的项目计划书，包含目标、时间线、预算和人员分工"
 print(f"\n发送请求: {user_input}")
 print("(这是一个复杂任务，应该会调用云端模型)\n")

 async with httpx.AsyncClient(timeout=180) as client:
 try:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": user_input}
)

 if response.status_code == 200:
 data = response.json()
 print(f"状态码: 200 OK")
 print(f"执行方式: {'本地执行' if data.get('executed_locally') else '云端执行 (使用 DeepSeek)'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f} 秒")

 print("\n" + "=" * 70)
 print("最终结果:")
 print("=" * 70)
 result = data.get('final_result', '')
 if len(result) > 1500:
 print(result[:1500] + "\n...")
 else:
 print(result)

 if not data.get('executed_locally'):
 print("\n" + "=" * 70)
 print("✅ 成功使用云端模型 (DeepSeek)")
 print("=" * 70)
 else:
 print("\n" + "=" * 70)
 print("⚠️ 使用了本地模型")
 print("=" * 70)
 else:
 print(f"请求失败: {response.status_code}")
 print(response.text)
 except Exception as e:
 print(f"发生错误: {str(e)}")

if __name__ == "__main__":
 asyncio.run(test_with_deepseek())

```

## backend\test\_deepseek\_chat.py

```

import asyncio
import httpx

```

```

async def test_deepseek_chat():
 print("=" * 80)
 print("测试 deepseek-chat 模型（付费模型）")
 print("=" * 80)

 # 首先清除缓存
 print("\n🧹 清除缓存...")
 async with httpx.AsyncClient(timeout=30) as client:
 await client.post("http://localhost:8000/api/v1/workflow/cache/clear")
 print("✅ 缓存已清除")

 # 执行复杂任务
 print("\n📋 执行复杂任务（应该使用 deepseek-chat）...")
 task = "帮我写一份详细的项目计划书，包含目标、时间线、预算和人员分工"

 async with httpx.AsyncClient(timeout=180) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": task}
)

 if response.status_code == 200:
 data = response.json()

 print("\n✅ 请求成功!")
 print(f"\n📋 执行结果:")
 print(f" 复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f" 执行方式: {'云端执行 (DeepSeek)' if not data.get('executed_locally', True) else '本地执行'}")
 print(f" 耗时: {data.get('total_duration_seconds', 0.0):.2f} 秒")

 # 检查步骤中的模型使用
 print(f"\n📋 各步骤详情:")
 for step in data.get('steps', []):
 model = step.get('metadata', {}).get('model_used', 'N/A')
 agent_name = step.get('agent_name', 'N/A')
 print(f" - {agent_name}: 模型={model}")

 # 检查 Result Agent 的模型
 result_step = [s for s in data.get('steps', []) if s.get('agent_id') == 'result']
 if result_step:
 model = result_step[0].get('metadata', {}).get('model_used', 'N/A')
 print(f"\n📋 Result Agent 使用模型: {model}")

 if 'deepseek-chat' in model:
 print("✅ 现在使用的是 deepseek-chat（付费模型）!")
 print("💡 请刷新 DeepSeek 平台查看消费记录!")
 else:
 print(f"⚠️ 当前模型: {model}")
 else:
 print(f"\n❌ 请求失败: {response.status_code}")

if __name__ == "__main__":
 asyncio.run(test_deepseek_chat())

```

## backend\test\_deepseek\_direct.py

```

import asyncio
import aiohttp

async def test_deepseek_direct():
 # 这里直接测试 DeepSeek API
 api_key = "sk-xxxxxxxxxxxxxxxxxxxxxxxxxxxx" # 用户需要替换为真实的 API Key
 url = "https://api.deepseek.com/v1/chat/completions"

 headers = {
 "Authorization": f"Bearer {api_key}",
 "Content-Type": "application/json"
 }

 payload = {
 "model": "deepseek-chat",
 "messages": [{"role": "user", "content": "Hello"}],
 "temperature": 0.7,
 "max_tokens": 50
 }

```

```

print(f"测试 URL: {url}")
print(f"API Key 长度: {len(api_key)}")

try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 status = response.status
 text = await response.text()

 print(f"\n状态码: {status}")
 print(f"\n响应内容: {text}")

 if status == 200:
 print("\n✅ API 调用成功!")
 else:
 print(f"\n❌ API 调用失败, 状态码: {status}")

except Exception as e:
 print(f"\n❌ 连接失败: {e}")

if __name__ == "__main__":
 asyncio.run(test_deepseek_direct())

```

## backend\test\_deepseek\_simple.py

```

#!/usr/bin/env python3
import asyncio
import aiohttp

async def main():
 print("-"*50)
 print("DeepSeek API 连接测试工具")
 print("-"*50)

 api_key = input("请输入你的 DeepSeek API Key: ").strip()

 if not api_key:
 print("❌ API Key 不能为空!")
 return

 print("\n正在测试 DeepSeek API...")

 # 测试多个可能的URL
 urls = [
 "https://api.deepseek.com/v1/chat/completions",
 "https://api.deepseek.com/chat/completions",
 "https://api.deepseek.com/openai/v1/chat/completions"
]

 payload = {
 "model": "deepseek-chat",
 "messages": [{"role": "user", "content": "Hello"}],
 "temperature": 0.7,
 "max_tokens": 50
 }

 headers = {
 "Authorization": f"Bearer {api_key}",
 "Content-Type": "application/json"
 }

 for url in urls:
 print(f"\n🔍 测试 URL: {url}")
 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=15)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 status = response.status
 text = await response.text()

 print(f" 状态码: {status}")

 if status == 200:
 print(" ✅ 成功!")

```

```

 try:
 import json
 data = json.loads(text)
 content = data.get("choices", [{}])[0].get("message", {}).get("content", "")
 print(f" 响应: {content[:50]}...")
 except:
 print(f" 响应: {text[:100]}...")
 return
 elif status == 401:
 print(" ✖ 认证失败 - API Key 无效")
 elif status == 404:
 print(" ✖ 路径不存在")
 elif status == 400:
 print(f" ✖ 请求错误: {text[:100]}")
 else:
 print(f" ✖ 未知错误: {text[:100]}")

except Exception as e:
 print(f" ✖ 连接失败: {e}")

print("\n✖ 所有URL测试失败, 请检查API Key和网络连接")

if __name__ == "__main__":
 asyncio.run(main())

```

## backend\test\_direct\_deepseek.py

```

import aiohttp
import asyncio
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def test_deepseek_directly():
 print("=" * 70)
 print("直接测试 DeepSeek API")
 print("=" * 70)

 api_key = "sk-6ccaead05a2f462d97417a268139b561"
 url = "https://api.deepseek.com/v1/chat/completions"

 messages = [
 {"role": "user", "content": "你好, 请简单介绍一下你自己"}
]

 payload = {
 "model": "deepseek-chat",
 "messages": messages,
 "temperature": 0.7,
 "max_tokens": 100
 }

 headers = {
 "Authorization": f"Bearer {api_key}",
 "Content-Type": "application/json"
 }

 print(f"\n发送请求到: {url}")
 print(f"模型: deepseek-chat")
 print(f"API Key: {api_key[:15]}...{api_key[-5:]}")
 print(f"消息: {messages[0]['content']}")
 print("=" * 70)

 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 print(f"\n响应状态码: {response.status}")

 if response.status == 200:
 data = await response.json()
 print("✅ API调用成功!")
 print(f"\n响应数据:")
 print(f" 模型: {data.get('model', 'N/A')}")

```

```

 print(f"Token使用: {data.get('usage', {})}")

 choices = data.get('choices', [])
 if choices:
 content = choices[0].get('message', {}).get('content', '')
 print(f"\n回复内容:")
 print(content)
 else:
 error_text = await response.text()
 print(f"❌ API调用失败")
 print(f"错误信息: {error_text}")

 except Exception as e:
 print(f"❌ 发生异常: {str(e)}")
 import traceback
 traceback.print_exc()

if __name__ == "__main__":
 asyncio.run(test_deepseek_directly())

```

## backend\test\_final.py

```

import asyncio
import httpx

async def test_love_letter():
 print("=" * 60)
 print("测试: 帮我给刘晓丹写一份情书")
 print("=" * 60)

 async with httpx.AsyncClient(timeout=120) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": "帮我给刘晓丹写一份情书"}
)

 if response.status_code == 200:
 data = response.json()
 print(f"\n执行方式: {'本地执行' if data.get('executed_locally') else '云端执行'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f}秒")

 print("\n" + "=" * 60)
 print("最终结果: ")
 print("=" * 60)
 print(data.get('final_result', ''))
 else:
 print(f"请求失败: {response.status_code}")
 print(response.text)

if __name__ == "__main__":
 asyncio.run(test_love_letter())

```

## backend\test\_full\_api.py

```

import asyncio
import aiohttp
import json
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

async def test_with_full_log():
 print("=" * 80)
 print("完整测试 DeepSeek API 调用")
 print("=" * 80)

```

```

api_key = "sk-6ccaead05a2f462d97417a268139b561"
model_name = "deepseek-v4-flash"
url = "https://api.deepseek.com/v1/chat/completions"

messages = [
 {"role": "user", "content": "用50字介绍你自己"}
]

payload = {
 "model": model_name,
 "messages": messages,
 "temperature": 0.7,
 "max_tokens": 100
}

headers = {
 "Authorization": f"Bearer {api_key}",
 "Content-Type": "application/json"
}

print(f"\n 📡 请求信息:")
print(f" URL: {url}")
print(f" 模型: {model_name}")
print(f" API Key: {api_key[:10]}...")
print(f" 消息: {messages[0]['content']}")
print(f" ~ * 80)

try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 print(f"\n 📡 正在发送请求...")

 async with session.post(url, json=payload, headers=headers) as response:
 print(f"\n 📡 响应状态码: {response.status}")

 if response.status == 200:
 print(f"\n ✅ API调用成功!")

 data = await response.json()
 print(f"\n 📄 响应详情:")
 print(f" 模型: {data.get('model', 'N/A')}")
 print(f" ID: {data.get('id', 'N/A')}")
 print(f" 对象类型: {data.get('object', 'N/A')}")

 usage = data.get('usage', {})
 prompt_tokens = usage.get('prompt_tokens', 0)
 completion_tokens = usage.get('completion_tokens', 0)
 total_tokens = usage.get('total_tokens', 0)

 print(f"\n 🧮 Token消耗:")
 print(f" 输入Token: {prompt_tokens}")
 print(f" 输出Token: {completion_tokens}")
 print(f" 总计Token: {total_tokens}")

 print(f"\n 💰 费用估算:")
 print(f" 约 ${total_tokens / 1000000 * 0.27:.6f} (假设 $0.27/1M tokens)")

 choices = data.get('choices', [])
 if choices:
 content = choices[0].get('message', {}).get('content', '')
 print(f"\n 📄 回复内容:")
 print(content)

 print(f"\n ✅ 确认: 已成功调用 {model_name} 模型!")
 print(f" ✅ Token已消耗: {total_tokens} tokens")

 else:
 error_text = await response.text()
 print(f"\n ❌ API调用失败")
 print(f" 状态码: {response.status}")
 print(f" 错误: {error_text}")

except Exception as e:
 print(f"\n ❌ 发生异常: {str(e)}")
 import traceback
 traceback.print_exc()

if __name__ == "__main__":
 asyncio.run(test_with_full_log())

```

## backend\test\_full\_workflow.py

```
import asyncio
import json
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.main import app
from core.workflow.service import WorkflowService
from agents.base.agent_registry import AgentRegistry
from models.workflow import WorkflowInput

async def test_full_workflow():
 agent_registry = AgentRegistry()
 await agent_registry.initialize_all_agents()

 workflow_service = WorkflowService(agent_registry)

 test_cases = [
 {
 "name": "简单问答",
 "input": "什么是人工智能?"
 },
 {
 "name": "复杂规划任务",
 "input": "帮我制定一份校园科技节活动策划方案，包括活动主题、流程安排、预算分配、时间线规划、人员分工、宣传方案、应急预案等内容，要求专业详细可执行。"
 }
]

 for test_case in test_cases:
 print("\n" + "-"*70)
 print("测试案例: {}".format(test_case['name']))
 print("输入: {}".format(test_case['input'][:50]))
 print("-"*70)

 try:
 workflow_input = WorkflowInput(user_input=test_case['input'])
 result = await workflow_service.execute(workflow_input)

 print("\n" + "-"*70)
 print("最终输出结果:")
 print("-"*70)

 # 只显示最后一部分，包含复杂度评估和建议
 final_output = result.final_result

 # 找到“复杂度评估与建议”部分
 if "复杂度评估与建议" in final_output:
 idx = final_output.index("复杂度评估与建议")
 summary_section = final_output[idx:]
 print(summary_section)
 else:
 print(final_output[-2000:])

 print("\n" + "-"*70)

 except Exception as e:
 print("错误: {}".format(e))
 import traceback
 traceback.print_exc()

 await agent_registry.shutdown_all_agents()

if __name__ == "__main__":
 asyncio.run(test_full_workflow())
```

## backend\test\_immediate.py

```

import asyncio
import httpx

async def test_immediate():
 user_input = "帮我给刘晓丹写一份情书"
 print(f"发送请求: {user_input}")

 async with httpx.AsyncClient(timeout=120) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": user_input}
)

 if response.status_code == 200:
 data = response.json()
 print(f"\n状态码: 200 OK")
 print(f"执行方式: {'本地执行' if data.get('executed_locally') else '云端执行'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f}秒")

 print("\n=== 最终结果 ===")
 print(data.get('final_result', ''))
 else:
 print(f"请求失败: {response.status_code}")
 print(response.text)

if __name__ == "__main__":
 asyncio.run(test_immediate())

```

## backend\test\_import.py

```

#!/usr/bin/env python3
"""
测试能否正常导入
"""

import sys
import os

print("=" * 60)
print("🔵 导入测试")
print("=" * 60)

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

print(f"\n工作目录: {os.getcwd()}")
print(f"Python路径: {sys.path[0]}")
print()

try:
 print("\n1. 测试导入 app.config...")
 from app.config import settings
 print(f"✅ 成功 - 应用名称: {settings.app_name}")
except Exception as e:
 print(f"❌ 失败: {e}")
 import traceback
 traceback.print_exc()
 sys.exit(1)

try:
 print("\n2. 测试导入 app.main...")
 from app.main import app
 print(f"✅ 成功")
except Exception as e:
 print(f"❌ 失败: {e}")
 import traceback
 traceback.print_exc()
 sys.exit(1)

try:
 print("\n3. 测试导入 agents...")
 from agents.knowledge.agent import KnowledgeAgent
 print(f"✅ 成功")
except Exception as e:

```

```
print(f" ❌ 失败: {e}")
import traceback
traceback.print_exc()
sys.exit(1)

print("\n" + "=" * 60)
print("✅ 所有导入测试通过!")
print("=" * 60)
print("\n🚀 现在可以启动服务了!")
print(" 运行: python app/main.py")
```

backend\test\_judge\_comprehensive.py

```
import asyncio
import json
from agents.judge.agent import JudgeAgent
from agents.base.agent import AgentInput

async def test_judge_comprehensive():
 judge_agent = JudgeAgent()

 test_cases = [
 {
 "name": "简单问答-什么是问题",
 "user_task": "什么是人工智能?",
 "writer_output": "人工智能是计算机科学的一个分支,致力于研究、开发用于模拟、延伸和扩展人的智能的理论、方法、技术及应用系统。",
 "review_score": 0.8
 },
 {
 "name": "简单问答-如何问题",
 "user_task": "如何学习编程?",
 "writer_output": "学习编程可以从选择一门编程语言开始,建议从Python入门,通过在线课程和实践项目来提升技能。",
 "review_score": 0.75
 },
 {
 "name": "中等复杂度-活动策划",
 "user_task": "帮我策划一个校园运动会活动方案,需要包含流程、预算、时间安排",
 "writer_output": "# 校园运动会活动方案\n\n## 一、活动概述\n本次运动会旨在增强学生体质,丰富校园文化生活...\n\n## 二、活动流程\n\n开幕式 -> 比赛项目 -> 闭幕式\n\n## 三、预算安排\n\n预计总预算5万元...",
 "review_score": 0.7
 },
 {
 "name": "中等复杂度-方案设计",
 "user_task": "设计一个企业团建活动方案,包含活动主题、流程安排、人员分工、预算规划",
 "writer_output": "# 企业团建活动方案\n\n## 活动主题\n团结协作,共创未来\n\n## 时间安排\n2024年12月15日\n\n## 人员分工\n活动策划组、后勤保障组、宣传组...",
 "review_score": 0.65
 },
 {
 "name": "高复杂度-AI系统设计",
 "user_task": "帮我设计一个AI智能客服系统的技术方案,包含架构设计、技术选型、实施步骤、风险评估和预算规划,要求专业详细",
 "writer_output": "# AI智能客服系统技术方案\n\n## 1. 需求分析\n\n分析用户需求和业务场景...\n\n## 2. 架构设计\n\n采用微服务架构,包含对话模块、知识库模块...\n\n## 3. 技术选型\n\n前端: React + TypeScript\n\n后端: Python + FastAPI\n\n## AI模型: DeepSeek R1...",
 "review_score": 0.7
 },
 {
 "name": "高复杂度-应急预案",
 "user_task": "请制定一份企业应急预案,包含风险评估、应急响应流程、人员分工、资源调配、演练计划等内容,要求符合行业标准",
 "writer_output": "# 企业应急预案\n\n## 一、风险评估\n\n识别潜在风险: 火灾、水灾、地震、网络安全...\n\n## 二、应急响应流程\n\n预警阶段 -> 响应阶段 -> 恢复阶段\n\n## 三、人员分工\n\n应急指挥中心、抢险救援组、后勤保障组...",
 "review_score": 0.75
 },
 {
 "name": "低质量-需要云端增强",
 "user_task": "帮我写一份详细的市场调研报告",
 "writer_output": "市场调研报告\n\n1. 市场现状\n2. 竞争分析\n3. 建议",
 "review_score": 0.4
 },
 {
 "name": "中等质量-本地重试",
```

```

 "user_task": "制定一个产品推广方案",
 "writer_output": "# 产品推广方案\n\n## 目标\n提高产品知名度\n\n## 渠道\n线上和线下",
 "review_score": 0.55
 },
 {
 "name": "长文本复杂任务",
 "user_task": "请为我撰写一份关于AI技术发展趋势的深度分析报告，要求涵盖2024-2025年的主要技术突破、行业应用案例、市场趋势预测、挑战与机遇分析，以及对未来5年发展的展望。报告需要数据支撑，引用至少5个权威机构的研究数据，包括Gartner、IDC、麦肯锡等，并提供具体的行业应用案例分析。",
 "writer_output": "# AI技术发展趋势深度分析报告\n\n## 一、引言\n人工智能技术近年来取得了飞速发展...\n\n## 二、2024-2025年主要技术突破\n1. 大语言模型能力提升\n2. 多模态AI融合\n3. Edge AI发展...",
 "review_score": 0.68
 },
 {
 "name": "简单问题但长描述",
 "user_task": "请问什么是云计算？请从定义、特点、服务类型、应用场景、优缺点等方面进行详细解释，最好能举一些实际的应用案例。",
 "writer_output": "云计算是一种基于互联网的计算方式...",
 "review_score": 0.72
 }
]

print("=" * 80)
print("Judge Agent 评审机制综合测试")
print("=" * 80)
print(f"测试用例总数: {len(test_cases)}")
print("=" * 80)

stats = {
 "local_output": 0,
 "local_retry": 0,
 "cloud_enhance": 0
}

for i, test_case in enumerate(test_cases, 1):
 print(f"\n【测试用例 {i}】{test_case['name']}")
 print("=" * 60)
 print(f"用户任务: {test_case['user_task'][:60]}..." if len(test_case['user_task']) > 60 else f"用户任务: {test_case['user_task']}")

 input_data = AgentInput(
 content=json.dumps({
 "user_task": test_case['user_task'],
 "summary_result": {"task": test_case['user_task']},
 "review_result": {"review_score": test_case['review_score']},
 "writer_output": test_case['writer_output']
 }),
 use_llm=False,
 use_cloud=False
)

 output = await judge_agent.execute(input_data)

 if output.success:
 result = json.loads(output.content)
 print(f"复杂度评分: {result['complexity_score']:.2f}")
 print(f"Review评分: {result['review_score']:.2f}")
 print(f"决策结果: {result['decision']}")
 print(f"Cloud模式: {result['cloud_mode']}")
 print(f"执行方式: {'本地执行' if result['decision'] == 'local_output' else '本地重试' if result['decision'] == 'local_retry' else '云端增强'}")
 print("决策理由:")
 for reason in result.get("reason", []):
 print(f" - {reason}")

 stats[result['decision']] += 1

 if result['decision'] == 'local_output':
 print(f"结论: 🟢 使用本地模型")
 elif result['decision'] == 'local_retry':
 print(f"结论: 🟡 本地重试优化")
 else:
 print(f"结论: ☁️ 调用云端服务")
 else:
 print(f"❌ 执行失败: {output.message}")

 print("=" * 60)

print("\n" + "=" * 80)
print("测试统计")

```

```

print("=" * 80)
print(f"本地执行: {stats['local_output']} 个")
print(f"本地重试: {stats['local_retry']} 个")
print(f"云端增强: {stats['cloud_enhance']} 个")
print(f"总测试: {sum(stats.values())} 个")
print("=" * 80)

if __name__ == "__main__":
 asyncio.run(test_judge_comprehensive())

```

## backend\test\_judge\_llm.py

```

import sys
import os
import asyncio

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import agents.knowledge.agent
import agents.summary.agent
import agents.writer.agent
import agents.review.agent
import agents.judge.agent
import agents.result.agent

from agents.base.agent_registry import AgentRegistry
from agents.base.agent import AgentInput

async def main():
 # 创建registry并注册agent
 registry = AgentRegistry()
 registry.register_agent(agents.knowledge.agent.KnowledgeAgent())
 registry.register_agent(agents.summary.agent.SummaryAgent())
 registry.register_agent(agents.writer.agent.WriterAgent())
 registry.register_agent(agents.review.agent.ReviewAgent())
 registry.register_agent(agents.judge.agent.JudgeAgent())
 registry.register_agent(agents.result.agent.ResultAgent())

 print("=== 测试 Judge Agent LLM 模式 ===")

 # 测试数据
 user_task = "帮我写一份校园活动策划, 包含时间线、人员安排、经费预算、风险分析和推广方案"
 summary_result = "用户需要一份完整的校园活动策划方案, 包含五个核心模块"
 writer_output = "这是一份校园活动策划方案... (内容略)"

 # 先获取 Review 结果
 import json
 review_input = json.dumps({
 "user_task": user_task,
 "summary": summary_result,
 "writer_output": writer_output
 })
 review_result = await registry.execute_agent("review", AgentInput(content=review_input, use_llm=True, use_cloud=False))
 print(f"Review 结果: {review_result.content}")

 # 测试 Judge Agent 使用 LLM 模式
 print("\n=== Judge Agent 使用 LLM 模式 ===")
 judge_input = json.dumps({
 "user_task": user_task,
 "summary_result": summary_result,
 "review_result": review_result.content,
 "writer_output": writer_output
 })
 # 使用 LLM 模式
 judge_result = await registry.execute_agent("judge", AgentInput(content=judge_input, use_llm=True, use_cloud=False))
 print(f"成功: {judge_result.success}")
 print(f"模型: {judge_result.model_used}")
 print(f"内容: {judge_result.content}")
 print(f"元数据: {judge_result.metadata}")

 # 验证结果
 try:
 judge_data = json.loads(judge_result.content)

```

```

 print(f"\n解析结果:")
 print(f" 复杂度评分: {judge_data.get('complexity_score')}")
 print(f" Review评分: {judge_data.get('review_score')}")
 print(f" 决策: {judge_data.get('decision')}")
 print(f" Cloud模式: {judge_data.get('cloud_mode')}")
 print(f" 理由: {judge_data.get('reason')}")
 except Exception as e:
 print(f"解析失败: {e}")

if __name__ == "__main__":
 asyncio.run(main())

```

## backend\test\_judge\_mechanism.py

```

import asyncio
import json
from agents.judge.agent import JudgeAgent
from agents.base.agent import AgentInput

async def test_judge_mechanism():
 judge_agent = JudgeAgent()

 test_cases = [
 {
 "name": "简单问题-什么是人工智能",
 "user_task": "什么是人工智能?",
 "writer_output": "人工智能是计算机科学的一个分支..."
 },
 {
 "name": "中等复杂度-活动策划",
 "user_task": "帮我策划一个校园运动会活动方案, 需要包含流程、预算、时间安排",
 "writer_output": "# 校园运动会活动方案\n\n## 一、活动概述\n..."
 },
 {
 "name": "高复杂度-项目设计",
 "user_task": "帮我设计一个AI智能客服系统的技术方案, 包含架构设计、技术选型、实施步骤、风险评估和预算规划, 要求专业详细",
 "writer_output": "# AI智能客服系统技术方案\n\n## 1. 需求分析\n..."
 },
 {
 "name": "复杂方案-应急预案",
 "user_task": "请制定一份企业应急预案, 包含风险评估、应急响应流程、人员分工、资源调配、演练计划等内容",
 "writer_output": "# 企业应急预案\n\n## 一、风险评估\n..."
 }
]

 print("=" * 80)
 print("Judge Agent 评审机制测试")
 print("=" * 80)

 for i, test_case in enumerate(test_cases, 1):
 print(f"\n【测试用例 {i}】{test_case['name']}")
 print("-" * 60)
 print(f"用户任务: {test_case['user_task'][:50]}..." if len(test_case['user_task']) > 50 else f"用户任务: {test_case['user_task']}")

 input_data = AgentInput(
 content=json.dumps({
 "user_task": test_case['user_task'],
 "summary_result": {"task": test_case['user_task']},
 "review_result": {"review_score": 0.7},
 "writer_output": test_case['writer_output']
 })
),
 use_llm=False,
 use_cloud=False
)

 output = await judge_agent.execute(input_data)

 if output.success:
 result = json.loads(output.content)
 print(f"复杂度评分: {result['complexity_score']:.2f}")
 print(f"Review评分: {result['review_score']:.2f}")
 print(f"决策结果: {result['decision']}")

```

```

 print(f"Cloud模式: {result['cloud_mode']}")
 print(f"执行方式: {'本地执行' if result['decision'] == 'local_output' else '云端增强'}")
 print("决策理由:")
 for reason in result.get("reason", []):
 print(f" - {reason}")

 executed_locally = result["decision"] == "local_output"
 print(f"\n结论: {'✅ 使用本地模型' if executed_locally else '☁️ 调用云端服务'}")
 else:
 print(f"❌ 执行失败: {output.message}")

 print("-" * 60)

 print("\n" + "=" * 80)
 print("测试完成!")
 print("=" * 80)

if __name__ == "__main__":
 asyncio.run(test_judge_mechanism())

```

## backend\test\_llm\_debug.py

```

import asyncio
from core.llm_client import get_llm_client

async def test_llm():
 print("=== 测试 LLM 调用 ===")

 llm_client = get_llm_client()

 # 测试本地模型
 print("\n--- 测试本地模型 (phi4-mini:3.8b) ---")
 try:
 response = await llm_client.generate_local("帮我写一段关于人工智能的介绍", model="phi4-mini:3.8b")
 print(f"响应长度: {len(response)}")
 print(f"响应内容: {response[:500]}")
 if len(response) < 50:
 print(f"警告: 响应过短! 完整响应: {response}")
 except Exception as e:
 print(f"本地模型调用失败: {e}")

 # 测试另一个本地模型
 print("\n--- 测试本地模型 (qwen2.5:1.5b) ---")
 try:
 response = await llm_client.generate_local("帮我写一段关于人工智能的介绍", model="qwen2.5:1.5b")
 print(f"响应长度: {len(response)}")
 print(f"响应内容: {response[:500]}")
 if len(response) < 50:
 print(f"警告: 响应过短! 完整响应: {response}")
 except Exception as e:
 print(f"qwen2.5模型调用失败: {e}")

if __name__ == "__main__":
 asyncio.run(test_llm())

```

## backend\test\_llm\_integration.py

```

import asyncio
import json
from agents.base.agent_registry import AgentRegistry
from agents.base.agent import AgentInput
from core.workflow.service import WorkflowService

async def test_llm_calls():
 print("=" * 80)
 print("测试 LLM 调用集成")
 print("=" * 80)

```

```

agent_registry = AgentRegistry()
await agent_registry.initialize_all_agents()

print("\n【1】检查所有 Agent 配置")
print("-" * 60)
for agent_id in ["knowledge", "summary", "writer", "review", "judge", "result"]:
 agent = agent_registry.get_agent(agent_id)
 if agent:
 print(f"Agent: {agent.agent_id} ({agent.name})")
 print(f" - 本地模型: {agent.local_model}")
 print(f" - 云端模型: {agent.cloud_model}")
 print()

print("\n【2】测试单个 Agent LLM 调用")
print("-" * 60)

测试 Summary Agent
summary_agent = agent_registry.get_agent("summary")
if summary_agent:
 print("测试 Summary Agent (使用本地LLM)...")
 input_data = AgentInput(content="帮我写一份活动策划", use_llm=True, use_cloud=False)
 output = await summary_agent.execute(input_data)
 print(f" 成功: {output.success}")
 print(f" 模型: {output.model_used}")
 print(f" 输出长度: {len(output.content)} 字符")
 print(f" 输出预览: {output.content[:100]}...")
 print()

测试 Writer Agent
writer_agent = agent_registry.get_agent("writer")
if writer_agent:
 print("测试 Writer Agent (使用本地LLM)...")
 summary_result = json.dumps({
 "task": "帮我写一份简单的活动策划",
 "keywords": ["活动", "策划"],
 "summary": "用户需要一份活动策划方案"
 })
 input_data = AgentInput(content=summary_result, use_llm=True, use_cloud=False)
 output = await writer_agent.execute(input_data)
 print(f" 成功: {output.success}")
 print(f" 模型: {output.model_used}")
 print(f" 输出长度: {len(output.content)} 字符")
 print(f" 输出预览: {output.content[:100]}...")
 print()

测试 Review Agent
review_agent = agent_registry.get_agent("review")
if review_agent:
 print("测试 Review Agent (使用本地LLM)...")
 review_input = json.dumps({
 "user_task": "帮我写一份活动策划",
 "summary": "用户需要一份活动策划方案",
 "writer_output": "## 活动策划方案\n\n## 活动概述\n\n这是一份活动策划方案。"
 })
 input_data = AgentInput(content=review_input, use_llm=True, use_cloud=False)
 output = await review_agent.execute(input_data)
 print(f" 成功: {output.success}")
 print(f" 模型: {output.model_used}")
 result = json.loads(output.content) if output.success else {}
 print(f" Review评分: {result.get('review_score', 'N/A')}")
 print()

print("\n【3】测试完整工作流")
print("-" * 60)

workflow_service = WorkflowService(agent_registry)

test_task = "帮我写一份简单的校园活动策划"

from models.workflow import WorkflowInput
workflow_input = WorkflowInput(user_input=test_task)

print(f"执行任务: {test_task}")
print("请稍候...")

try:
 result = await workflow_service.execute(workflow_input)
 print(f"\n工作流执行完成!")
 print(f" 执行方式: {'本地执行' if result.executed_locally else '云端执行'}")
 print(f" 复杂度评分: {result.complexity_score:.2f}")

```

```

print(f" 总耗时: {result.total_duration_seconds:.2f}秒")
print(f" 步骤数: {len(result.steps)}")

print("\n 各步骤详情:")
for step in result.steps:
 status = "✅" if step.success else "❌"
 model = step.metadata.get("model_used", "N/A")
 print(f" {status} {step.agent_name}: 模型={model}, 耗时={step.duration_seconds:.2f}s")

print(f"\n 最终结果长度: {len(result.final_result)} 字符")

except Exception as e:
 print(f"❌ workflow执行失败: {str(e)}")

print("\n" + "-" * 80)
print("测试完成!")
print("-" * 80)

if __name__ == "__main__":
 asyncio.run(test_llm_calls())

```

## backend\test\_love\_letter.py

```

import asyncio
import httpx
import json

async def test_love_letter():
 print("测试写情书功能...")
 print("-" * 60)

 async with httpx.AsyncClient(timeout=120) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": "帮我给刘晓丹写一份情书"}
)

 if response.status_code == 200:
 data = response.json()
 print(f"执行方式: {'本地执行' if data.get('executed_locally') else '云端执行'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f}秒")

 print("\n各步骤详情:")
 for step in data.get('steps', []):
 model = step.get('metadata', {}).get('model_used', 'N/A')
 status = "✅" if step.get('success') else "❌"
 print(f" {status} {step.get('agent_name')}: 模型={model}, 耗时={step.get('duration_seconds', 0.0):.2f}s")

 print("\nWriter Agent 输出:")
 writer_step = next((s for s in data.get('steps', []) if s.get('agent_id') == 'writer'), None)
 if writer_step:
 print(f" 输出长度: {len(writer_step.get('output', ''))}")
 print(f" 输出内容:")
 print(writer_step.get('output', '')[:800])

 print("\n最终结果:")
 final_result = data.get('final_result', '')
 print(final_result[:1200])
 else:
 print(f"❌ 请求失败: {response.status_code}")
 print(f"错误信息: {response.text}")

if __name__ == "__main__":
 asyncio.run(test_love_letter())

```

## backend\test\_model\_names.py

```

import aiohttp
import asyncio

async def test_model_comparison():
 print("=" * 70)
 print("测试不同的 DeepSeek 模型")
 print("=" * 70)

 api_key = "sk-6ccaead05a2f462d97417a268139b561"
 url = "https://api.deepseek.com/v1/chat/completions"

 messages = [
 {"role": "user", "content": "请用一句话介绍自己"}
]

 headers = {
 "Authorization": f"Bearer {api_key}",
 "Content-Type": "application/json"
 }

 models_to_test = [
 ("deepseek-chat", "标准对话模型"),
 ("deepseek-coder", "代码模型"),
 ("deepseek-r1-distill", "R1蒸馏模型（配置的模型）")
]

 for model_name, description in models_to_test:
 print(f"\n{'=' * 70}")
 print(f"测试模型: {model_name} ({description})")
 print(f"{'=' * 70}")

 payload = {
 "model": model_name,
 "messages": messages,
 "temperature": 0.7,
 "max_tokens": 50
 }

 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 if response.status == 200:
 data = await response.json()
 usage = data.get('usage', {})
 print(f"✅ 成功!")
 print(f"Token使用: {usage}")
 choices = data.get('choices', [])
 if choices:
 content = choices[0].get('message', {}).get('content', '')
 print(f"回复: {content[:100]}")
 else:
 error_text = await response.text()
 print(f"❌ 失败 (状态码: {response.status})")
 print(f"错误: {error_text[:200]}")

 except Exception as e:
 print(f"❌ 异常: {str(e)}")

if __name__ == "__main__":
 asyncio.run(test_model_comparison())

```

## backend\test\_model\_switch.py

```

import asyncio
import json
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from agents.base.agent_registry import AgentRegistry
from agents.base.agent import AgentInput

```

```

async def test_agent_model_switch():
 agent_registry = AgentRegistry()
 await agent_registry.initialize_all_agents()

 print("测试 Review Agent...")
 review_agent = agent_registry.get_agent("review")

 test_input = AgentInput(
 content=json.dumps({
 "user_task": "帮我写一份校园活动策划",
 "summary": "用户需要正式校园活动方案",
 "writer_output": "举办活动促进同学交流。"
 }),
 use_llm=True,
 use_cloud=True
)

 result = await review_agent.execute(test_input)
 print("Review Agent (云端):")
 print(" 成功: {}".format(result.success))
 print(" 模型: {}".format(result.model_used))
 print(" 内容预览: {}".format(result.content[:100]))

 test_input_local = AgentInput(
 content=json.dumps({
 "user_task": "帮我写一份校园活动策划",
 "summary": "用户需要正式校园活动方案",
 "writer_output": "举办活动促进同学交流。"
 }),
 use_llm=True,
 use_cloud=False
)

 result_local = await review_agent.execute(test_input_local)
 print("\nReview Agent (本地):")
 print(" 成功: {}".format(result_local.success))
 print(" 模型: {}".format(result_local.model_used))
 print(" 内容预览: {}".format(result_local.content[:100]))

 print("\n\n测试 Judge Agent...")
 judge_agent = agent_registry.get_agent("judge")

 judge_input_cloud = AgentInput(
 content=json.dumps({
 "user_task": "帮我制定一份校园科技节活动策划方案，包括活动主题、流程安排、预算分配、时间线规划、人员分工、宣传方案、应急预案等内容，要求专业详细可执行。",
 "summary_result": "用户需要详细的校园科技节策划方案",
 "review_result": '{"review_score": 0.7}',
 "writer_output": "这是一个详细的活动策划方案..."
 }),
 use_llm=True,
 use_cloud=True
)

 judge_result = await judge_agent.execute(judge_input_cloud)
 print("Judge Agent (云端):")
 print(" 成功: {}".format(judge_result.success))
 print(" 模型: {}".format(judge_result.model_used))
 print(" 内容: {}".format(judge_result.content))

 judge_input_local = AgentInput(
 content=json.dumps({
 "user_task": "帮我制定一份校园科技节活动策划方案，包括活动主题、流程安排、预算分配、时间线规划、人员分工、宣传方案、应急预案等内容，要求专业详细可执行。",
 "summary_result": "用户需要详细的校园科技节策划方案",
 "review_result": '{"review_score": 0.7}',
 "writer_output": "这是一个详细的活动策划方案..."
 }),
 use_llm=False,
 use_cloud=False
)

 judge_result_local = await judge_agent.execute(judge_input_local)
 print("\nJudge Agent (规则引擎):")
 print(" 成功: {}".format(judge_result_local.success))
 print(" 模型: {}".format(judge_result_local.model_used))
 print(" 内容: {}".format(judge_result_local.content))

 await agent_registry.shutdown_all_agents()

```

```
if __name__ == "__main__":
 asyncio.run(test_agent_model_switch())
```

## backend\test\_multiple\_tasks.py

```
import asyncio
import httpx

async def test_tasks():
 tasks = [
 "帮我给刘晓丹写一份情书",
 "写一首关于春天的诗",
 "帮我设计一个校园运动会活动方案",
 "什么是人工智能?",
 "写一封感谢信给老师"
]

 for task in tasks:
 print(f"\n{'='*60}")
 print(f"任务: {task}")
 print(f"{'='*60}")

 try:
 async with httpx.AsyncClient(timeout=120) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": task}
)

 if response.status_code == 200:
 data = response.json()
 print(f"执行方式: {'本地执行' if data.get('executed_locally') else '云端执行'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f}秒")

 final_result = data.get('final_result', '')
 print(f"结果长度: {len(final_result)} 字符")
 print("结果预览:")
 print(final_result[:800])
 else:
 print(f"❌ 请求失败: {response.status_code}")
 except Exception as e:
 print(f"❌ 异常: {str(e)}")

if __name__ == "__main__":
 asyncio.run(test_tasks())
```

## backend\test\_ollama\_connection.py

```
import asyncio
import httpx

async def test_ollama():
 print("测试 Ollama 连接...")

 # 测试默认端口
 ports = ["11434", "11435"]

 for port in ports:
 url = f"http://localhost:{port}/api/tags"
 print(f"\n测试: {url}")
 try:
 async with httpx.AsyncClient() as client:
 response = await client.get(url, timeout=5)
 if response.status_code == 200:
 data = response.json()
 print(f"✅ 连接成功!")
 print(f"模型列表: {[m['name'] for m in data.get('models', [])]}")
```

```

 else:
 print(f"❌ 状态码: {response.status_code}")
 except Exception as e:
 print(f"❌ 连接失败: {str(e)}")

测试生成
print("\n测试生成能力...")
url = "http://localhost:11435/api/generate"
payload = {
 "model": "qwen2.5:1.5b",
 "prompt": "你好，简单介绍一下你自己。",
 "stream": False
}

try:
 async with httpx.AsyncClient(timeout=60) as client:
 response = await client.post(url, json=payload)
 if response.status_code == 200:
 data = response.json()
 print(f"✅ 生成成功!")
 print(f" 响应: {data.get('response', '')[:100]}...")
 else:
 print(f"❌ 状态码: {response.status_code}")
 print(f" 错误: {await response.text()}")
 except Exception as e:
 print(f"❌ 生成失败: {str(e)}")

if __name__ == "__main__":
 asyncio.run(test_ollama())

```

## backend\test\_other\_tasks.py

```

import asyncio
import httpx

async def test_tasks():
 tasks = [
 "写一首关于春天的诗",
 "什么是人工智能?",
 "写一封感谢信给老师",
 "帮我设计一个校园运动会活动方案"
]

 for task in tasks:
 print(f"\n{'=' * 70}")
 print(f"任务: {task}")
 print(f"{'=' * 70}")

 try:
 async with httpx.AsyncClient(timeout=120) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": task}
)

 if response.status_code == 200:
 data = response.json()
 print(f"执行方式: {'本地执行' if data.get('executed_locally') else '云端执行'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f} 秒")

 print("\n结果:")
 print(f"{'-' * 70}")
 result = data.get('final_result', '')
 if len(result) > 1000:
 print(result[:1000] + "...")
 else:
 print(result)
 else:
 print(f"❌ 请求失败: {response.status_code}")
 except Exception as e:
 print(f"❌ 异常: {str(e)}")

```

```
if __name__ == "__main__":
 asyncio.run(test_tasks())
```

backend\test\_output.txt

```
📌 任务执行结果

🔄 执行流程
| 智能体 | 状态 | 模型 |
|-----|-----|-----|
| Knowledge Agent | ✅ 完成 | qwen2.5 |
| Summary Agent | ✅ 完成 | qwen2.5 |
| Writer Agent | ✅ 完成 | phi4-mini |
| Review Agent | ✅ 完成 | phi4-mini |
| Judge Agent | ✅ 完成 | rule-based |
| Result Agent | ✅ 完成 | qwen2.5 |

📋 执行详情
- 复杂度评分: 0.20
- Review评分: 0.70
- 执行方式: local_output
- Cloud模式: none

💡 决策原因
- 任务简单
- 本地结果可接受

📊 质量评审详情
| 维度 | 评分 |
|-----|-----|
| 结构完整性 | 0.90 |
| 需求相关性 | 0.70 |
| 内容丰富度 | 0.60 |
| 专业性 | 0.60 |
| 可执行性 | 0.70 |

💡 修改建议
- 内容质量良好，建议检查是否有遗漏细节

📄 最终输出

人工智能概述

一、任务概述
人工智能（AI）是一个广泛而深刻的话题。本文旨在提供对人类知识和理解中的“智能”这一概念的一般介绍，包括其定义、类型及实际应用。

二、核心需求
对于本项目的主要要求如下：

1. **准确性**: 确保所提供的信息准确且符合当前科学界关于人工智能的共识。
2. **全面性**: 涵盖人工智能领域内各个方面，给出一个全方位的理解。
3. **可操作性**: 内容需实用，可以作为教学材料或参考资料使用。

三、解决方案
为了有效地满足上述需求，本项目将按照以下步骤进行：

1. 从历史发展角度介绍人工智能概念。
2. 解释不同类型的人工智能，包括弱和强人工智能。
3. 概述当前的关键技术，如机器学习算法及其应用场景。
4. 分享一些实际案例，说明人类社会中人工智能在现实中的应用。

四、实施计划
详细分步骤实施方案如下：

1. **历史发展**:
 - 介绍第一代人工智能（1950年代-1960年代）。
 - 探讨第二代人工智能（1980年代-1990年代）的进展。
 - 阐述第三代人工智能（从2000年代起）如何随着技术的不断提升而演变。

2. **人类智能分类**:
 - 强人工智能: 可以完成与ature tasks demanding intellectual engagement, logical reasoning, creativity.
 - 认知人工智能: 模仿人类大脑功能，执行诸如理解自然语言、推理等任务。
```

```
- 弱人工智能 (Weak AI)：专注于解决特定问题，并不具备全面的智能能力。

3. **关键技术和算法**：
- 深度学习
 ...

 import tensorflow as tf

 model = tf.keras.models.Sequential([
 tf.keras.layers.Flatten(input_shape=(28, 28)),
 tf.keras.layers.Dense(128, activation='relu'),
 tf.keras.layers.Dropout(0.2),
 tf.keras.layers.Dense(10)
])
 ...

4. **实际案例与应用**：
- 自然语言处理 (NLP)：例如，谷歌翻译、微软的DALL-E。
- 决策支持系统：如IBM Watson在医疗和法律领域的应用。

结论
以上内容概括了人类智能及其分类、关键技术发展历程以及实际案例。这份方案文档应能为读者提供一个全面而深入的人脑理解，促进人工智能领域内知识的传播与深化。

=====
[Workflow] 复杂度评估与建议
=====

[Analysis] 复杂度分析
| 指标 | 数值 |
|-----|-----|
| 复杂度评分 | 0.20 |
| Review评分 | 0.70 |
| 执行方式 | local_output |
| Cloud模式 | none |
| 是否本地 | 是 |

[Level] 复杂度等级
```

复杂度等级：低 (\*\*\*)

评估: 任务简单, 适合本地处理 建议: 可直接使用本地模型回答, 响应速度快且成本低

```
[Path] 工作流路径
`用户输入 → Knowledge → Summary → Writer → Review → Judge → [直接输出]`

导出格式: Markdown | **生成时间**: 2026-05-15 12:33:21
```

backend\test\_output\_check.py

```
import asyncio
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from core.workflow.service import WorkflowService
from agents.base.agent_registry import AgentRegistry
from models.workflow import WorkflowInput

async def test_output():
 ar = AgentRegistry()
 await ar.initialize_all_agents()

 ws = WorkflowService(ar)

 test_input = "什么是人工智能?"
 workflow_input = WorkflowInput(user_input=test_input)
 result = await ws.execute(workflow_input)
```

```

print("输出长度:", len(result.final_result))
print("\n=== 检查是否包含复杂度评估 ===")
print("包含 '复杂度评估':", "复杂度评估" in result.final_result)
print("包含 '复杂度等级':", "复杂度等级" in result.final_result)
print("包含 '评估':", "评估:" in result.final_result)
print("包含 '建议':", "建议:" in result.final_result)

with open("test_output.txt", "w", encoding="utf-8") as f:
 f.write(result.final_result)
print("\n输出已保存到 test_output.txt")

await ar.shutdown_all_agents()

if __name__ == "__main__":
 asyncio.run(test_output())

```

## backend\test\_review\_debug.py

```

import sys
import os
import asyncio

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

import agents.knowledge.agent
import agents.summary.agent
import agents.writer.agent
import agents.review.agent

from agents.base.agent_registry import AgentRegistry
from agents.base.agent import AgentInput

async def main():
 # 创建registry并注册agent
 registry = AgentRegistry()
 registry.register_agent(agents.knowledge.agent.KnowledgeAgent())
 registry.register_agent(agents.summary.agent.SummaryAgent())
 registry.register_agent(agents.writer.agent.WriterAgent())
 registry.register_agent(agents.review.agent.ReviewAgent())

 print("=== 测试 Review Agent 调试 ===")

 # 测试数据 - 模拟用户输入
 user_task = "帮我写一个关于校园AI助手的年度规划包含: 1.时间线、2.人员安排、3.经费预算、4.风险分析、5.推广方案"

 print(f"\n1. 用户任务: {user_task}")
 print(f" 长度: {len(user_task)} 字符")

 # 测试 Knowledge Agent
 print("\n2. 测试 Knowledge Agent...")
 knowledge_result = await registry.execute_agent("knowledge", AgentInput(content=user_task, use_llm=False))
 print(f" 成功: {knowledge_result.success}")
 print(f" 长度: {len(knowledge_result.content)} 字符")

 # 测试 Summary Agent
 print("\n3. 测试 Summary Agent...")
 summary_result = await registry.execute_agent("summary", AgentInput(content=knowledge_result.content, use_llm=False))
 print(f" 成功: {summary_result.success}")
 print(f" 长度: {len(summary_result.content)} 字符")

 # 测试 Writer Agent
 print("\n4. 测试 Writer Agent...")
 writer_input = f"{knowledge_result.content}\n\n任务摘要: {summary_result.content}"
 writer_result = await registry.execute_agent("writer", AgentInput(content=writer_input, use_llm=False))
 print(f" 成功: {writer_result.success}")
 print(f" 长度: {len(writer_result.content)} 字符")
 print(f" 预览: {writer_result.content[:200]}...")

 # 测试 Review Agent (使用正确的JSON格式)
 print("\n5. 测试 Review Agent...")
 import json
 review_input = json.dumps({
 "user_task": user_task,
 "summary": summary_result.content,

```

```

 "writer_output": writer_result.content
 })
 print(f" 输入格式: JSON")
 print(f" 输入长度: {len(review_input)} 字符")

 try:
 review_result = await registry.execute_agent("review", AgentInput(content=review_input, use_llm=True, use_cloud=False))

 print(f" 成功: {review_result.success}")
 print(f" 长度: {len(review_result.content)} 字符")
 print(f" 内容: {review_result.content}")
 except Exception as e:
 print(f" 失败: {e}")
 import traceback
 traceback.print_exc()

if __name__ == "__main__":
 asyncio.run(main())

```

## backend\test\_settings.py

```

import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.config import settings

print("=" * 60)
print("查看当前的 DeepSeek 配置")
print("=" * 60)

print(f"\nDEEPSEEK_API_KEY: {settings.deepseek_api_key[:15]}..." if settings.deepseek_api_key else "DEEPSEEK_API_KEY: None")
print(f"DEEPSEEK_API_BASE: {settings.deepseek_api_base}")
print(f"DEEPSEEK_MODEL: {settings.deepseek_model}")
print(f"COMPLEXITY_THRESHOLD: {settings.complexity_threshold}")

print("\n✅ 配置加载完成!")

if settings.deepseek_model == "deepseek-v4-flash":
 print("\n🎯 模型名称正确! 当前使用的是: deepseek-v4-flash")
 print("这就是您平台上显示消费模型!")
else:
 print(f"\n❌ 当前模型: {settings.deepseek_model}, 期望: deepseek-v4-flash")

```

## backend\test\_setup.py

```

#!/usr/bin/env python3
"""
测试 AgentMatrix 环境是否配置正确
"""

import sys
import os

print("=" * 60)
print("🔍 AgentMatrix 环境检查")
print("=" * 60)

检查 Python 版本
print(f"\n1. Python 版本: {sys.version}")

检查必要的包
required_packages = [
 'uvicorn',
 'fastapi',
 'pydantic',
 'httpx',
 'aifiles'
]

```

```

]

print("\n2. 检查依赖包:")
all_ok = True
for package in required_packages:
 try:
 __import__(package)
 print(f"✅ {package}")
 except ImportError:
 print(f"❌ {package} - 未安装")
 all_ok = False

检查 .env 文件
print("\n3. 检查配置文件:")
if os.path.exists('.env'):
 print("✅ .env 文件存在")
 with open('.env', 'r', encoding='utf-8') as f:
 content = f.read()
 print("配置项:")
 for line in content.split('\n'):
 if line.strip() and not line.startswith('#'):
 key = line.split('=')[0]
 if 'KEY' in key or 'key' in key:
 print(f" {key}=***")
 else:
 print(f" {line}")
else:
 print("❌ .env 文件不存在")
 all_ok = False

检查 Ollama 配置
print("\n4. Ollama 配置:")
from app.config import settings
print(f"默认端口: {settings.ollama_host}")

print("\n" + "=" * 60)
if all_ok:
 print("✅ 环境检查通过! 可以启动服务")
 print("\n🚀 启动命令:")
 print("python -m uvicorn app.main:app --host 0.0.0.0 --port 8000")
else:
 print("❌ 环境检查失败, 请检查上述错误")
print("=" * 60)

```

## backend\test\_simple.py

```

import sys
import os
import asyncio

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

导入所有agent
import agents.knowledge.agent
import agents.summary.agent
import agents.writer.agent
import agents.review.agent
import agents.judge.agent
import agents.result.agent

from agents.base.agent_registry import AgentRegistry
from agents.base.agent import AgentInput

async def main():
 # 创建registry并注册agent
 registry = AgentRegistry()
 registry.register_agent(agents.knowledge.agent.KnowledgeAgent())
 registry.register_agent(agents.summary.agent.SummaryAgent())
 registry.register_agent(agents.writer.agent.WriterAgent())
 registry.register_agent(agents.review.agent.ReviewAgent())
 registry.register_agent(agents.judge.agent.JudgeAgent())
 registry.register_agent(agents.result.agent.ResultAgent())

 print("=== Agent注册完成 ===")

```

```

print(f"已注册的Agent: {list(registry.agents.keys())}")

测试单个agent（不使用LLM，只测试规则引擎）
print("\n=== 测试单个Agent（规则引擎模式） ===")

测试Knowledge Agent
print("\n1. 测试 Knowledge Agent...")
knowledge_result = await registry.execute_agent("knowledge", AgentInput(content="帮我写一份简单的校园活动策划", use_llm=False))
print(f" 成功: {knowledge_result.success}")
print(f" 内容长度: {len(knowledge_result.content)}")
print(f" 预览: {knowledge_result.content[:100]}...")

测试Summary Agent
print("\n2. 测试 Summary Agent...")
summary_result = await registry.execute_agent("summary", AgentInput(content=knowledge_result.content, use_llm=False))
print(f" 成功: {summary_result.success}")
print(f" 内容长度: {len(summary_result.content)}")

测试Writer Agent
print("\n3. 测试 Writer Agent...")
writer_input = f"{knowledge_result.content}\n\n任务摘要: {summary_result.content}"
writer_result = await registry.execute_agent("writer", AgentInput(content=writer_input, use_llm=False))
print(f" 成功: {writer_result.success}")
print(f" 内容长度: {len(writer_result.content)}")

测试Review Agent（使用正确的JSON格式）
print("\n4. 测试 Review Agent...")
import json
review_input = json.dumps({
 "user_task": "帮我写一份简单的校园活动策划",
 "summary": summary_result.content,
 "writer_output": writer_result.content
})
review_result = await registry.execute_agent("review", AgentInput(content=review_input, use_llm=False))
print(f" 成功: {review_result.success}")
print(f" 内容长度: {len(review_result.content)}")
print(f" 内容: {review_result.content}")

测试Judge Agent（使用正确的JSON格式）
print("\n5. 测试 Judge Agent...")
judge_input = json.dumps({
 "user_task": "帮我写一份简单的校园活动策划",
 "summary_result": summary_result.content,
 "review_result": review_result.content,
 "writer_output": writer_result.content
})
judge_result = await registry.execute_agent("judge", AgentInput(content=judge_input, use_llm=False))
print(f" 成功: {judge_result.success}")
print(f" 内容长度: {len(judge_result.content)}")
print(f" 内容: {judge_result.content}")

测试结果 Agent（使用正确的JSON格式）
print("\n6. 测试 Result Agent...")
try:
 judge_data = json.loads(judge_result.content)
 result_input = json.dumps({
 "user_task": "帮我写一份简单的校园活动策划",
 "summary_result": summary_result.content,
 "review_result": review_result.content,
 "judge_result": judge_result.content,
 "writer_output": writer_result.content,
 "executed_locally": True,
 "complexity_score": judge_data.get("complexity_score", 0.0),
 "judge_decision": judge_data.get("decision", "local_output"),
 "cloud_mode": judge_data.get("cloud_mode", "none")
 })
 result_result = await registry.execute_agent("result", AgentInput(content=result_input, use_llm=False))
 print(f" 成功: {result_result.success}")
 print(f" 内容长度: {len(result_result.content)}")
 print(f" 预览: {result_result.content[:200]}...")
except Exception as e:
 print(f" 失败: {e}")

print("\n=== 所有单个Agent测试完成 ===")

if __name__ == "__main__":
 asyncio.run(main())

```

## backend\test\_threshold.py

```
import asyncio
import httpx

async def test_threshold():
 print("=" * 70)
 print("测试复杂度阈值决策逻辑")
 print("阈值: 0.65")
 print("=" * 70)

 test_cases = [
 ("帮我写一份情书", "简单任务, 应该本地执行"),
 ("帮我设计一个校园运动会活动方案, 包含流程、预算、时间线、人员分工", "复杂任务, 应该云端执行"),
 ("写一份详细的项目计划书, 包含目标、时间线、预算和人员分工", "复杂任务, 应该云端执行"),
 ("什么是人工智能?", "简单任务, 应该本地执行"),
]

 for task, expected in test_cases:
 print(f"\n{'=' * 70}")
 print(f"任务: {task}")
 print(f"预期: {expected}")
 print(f"{'=' * 70}")

 async with httpx.AsyncClient(timeout=180) as client:
 try:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": task}
)

 if response.status_code == 200:
 data = response.json()
 complexity = data.get('complexity_score', 0.0)
 is_local = data.get('executed_locally', True)

 print(f"复杂度评分: {complexity:.2f}")
 print(f"执行方式: {'本地执行' if is_local else '云端执行 (DeepSeek)'}")

 if complexity >= 0.65:
 if not is_local:
 print("✅ 正确: 复杂度 >= 0.65, 使用云端模型")
 else:
 print("❌ 错误: 复杂度 >= 0.65, 但使用了本地模型")
 else:
 if is_local:
 print("✅ 正确: 复杂度 < 0.65, 使用本地模型")
 else:
 print("⚠️ 注意: 复杂度 < 0.65, 但使用了云端模型")
 else:
 print(f"请求失败: {response.status_code}")
 except Exception as e:
 print(f"发生错误: {str(e)}")

if __name__ == "__main__":
 asyncio.run(test_threshold())
```

## backend\test\_v4\_flash.py

```
import asyncio
import aiohttp
import json

async def test_deepseek_v4_flash():
 print("=" * 80)
 print("测试 deepseek-v4-flash 模型")
 print("=" * 80)

 api_key = "sk-6ccaead05a2f462d97417a268139b561"
```

```

url = "https://api.deepseek.com/v1/chat/completions"

messages = [
 {"role": "user", "content": "用100字介绍你自己"}
]

payload = {
 "model": "deepseek-v4-flash",
 "messages": messages,
 "temperature": 0.7,
 "max_tokens": 200
}

headers = {
 "Authorization": f"Bearer {api_key}",
 "Content-Type": "application/json"
}

print(f"\n发送请求到: {url}")
print(f"使用模型: deepseek-v4-flash")
print(f"API Key: {api_key[:15]}...")
print("-" * 80)

try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 print(f"\n响应状态码: {response.status}")

 if response.status == 200:
 data = await response.json()
 print(f"\n✅ API调用成功!")

 print("\n响应数据:")
 print(f" 模型: {data.get('model', 'N/A')}")

 usage = data.get('usage', {})
 print(f"\n🔥 Token消耗:")
 print(f" prompt: {usage.get('prompt_tokens', 0)}")
 print(f" completion: {usage.get('completion_tokens', 0)}")
 print(f" total: {usage.get('total_tokens', 0)}")

 choices = data.get('choices', [])
 if choices:
 content = choices[0].get('message', {}).get('content', '')
 print(f"\n回复内容:")
 print(content)
 else:
 error_text = await response.text()
 print(f"\n❌ 错误: {response.status}")
 print(error_text)
 except Exception as e:
 print(f"\n❌ 异常: {str(e)}")
 import traceback
 traceback.print_exc()

if __name__ == "__main__":
 asyncio.run(test_deepseek_v4_flash())

```

## backend\test\_workflow.py

```

import asyncio
from agents.base.agent_registry import AgentRegistry
from agents.base.agent import AgentInput

async def test_workflow():
 print('Initializing agent registry...')
 registry = AgentRegistry()
 await registry.initialize_all_agents()

 print('\nAgent statuses:')
 statuses = registry.get_all_agent_statuses()
 for agent_id, status in statuses.items():
 print(f' {agent_id}: {status["status"]}')

```

```

print('\nTesting simple workflow...')
user_input = '生成一个校园AI助手方案'

try:
 # Knowledge Agent
 print('1. Knowledge Agent...')
 knowledge_output = await registry.execute_agent('knowledge', AgentInput(content=user_input))
 print(f' Success: {knowledge_output.success}')

 # Summary Agent
 print('2. Summary Agent...')
 summary_output = await registry.execute_agent('summary', AgentInput(content=knowledge_output.content))
 print(f' Success: {summary_output.success}')

 # Writer Agent
 print('3. Writer Agent...')
 writer_output = await registry.execute_agent('writer', AgentInput(content=summary_output.content))
 print(f' Success: {writer_output.success}')

 # Review Agent
 print('4. Review Agent...')
 review_output = await registry.execute_agent('review', AgentInput(content=writer_output.content))
 print(f' Success: {review_output.success}')

 # Judge Agent
 print('5. Judge Agent...')
 judge_output = await registry.execute_agent('judge', AgentInput(content=review_output.content))
 print(f' Success: {judge_output.success}')
 print(f' Decision: {judge_output.message}')

 # Result Agent
 print('6. Result Agent...')
 result_output = await registry.execute_agent('result', AgentInput(content=judge_output.content, context={'writer': writer_output.content}))
 print(f' Success: {result_output.success}')

 print('\n=== Final Result ===')
 print(result_output.content[:500] + '...' if len(result_output.content) > 500 else result_output.content)

 return True
except Exception as e:
 print(f'Error: {e}')
 import traceback
 traceback.print_exc()
 return False

if __name__ == '__main__':
 asyncio.run(test_workflow())

```

## backend\test\_workflow\_api.py

```

import asyncio
import httpx

async def test_workflow_api():
 print("测试 workflow API...")
 print("=" * 60)

 tasks = [
 "帮我写一份简单的活动策划",
 "什么是人工智能?",
 "帮我设计一个校园运动会方案"
]

 for task in tasks:
 print(f"\n任务: {task}")
 print("-" * 40)

 try:
 async with httpx.AsyncClient(timeout=120.0) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",

```

```

 json={"user_input": task}
)

 if response.status_code == 200:
 data = response.json()
 print(f"执行方式: {'本地执行' if data.get('executed_locally') else '云端执行'}")
 print(f"复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f"总耗时: {data.get('total_duration_seconds', 0.0):.2f}秒")
 print(f"步骤数: {len(data.get('steps', []))}")

 print("\n各步骤详情:")
 for step in data.get('steps', []):
 model = step.get('metadata', {}).get('model_used', 'N/A')
 status = "✅" if step.get('success') else "❌"
 print(f" {status} {step.get('agent_name')}: 模型={model}, 耗时={step.get('duration_seconds', 0.0):.2f}秒")

 final_result = data.get('final_result', '')
 print(f"\n最终结果预览 (前{len(final_result)} 字符):")
 print(f" {final_result[:500]}..." if len(final_result) > 500 else final_result)
 else:
 print(f"❌ 请求失败: {response.status_code}")
 print(f"错误信息: {response.text}")

 except Exception as e:
 print(f"❌ 请求异常: {str(e)}")

if __name__ == "__main__":
 asyncio.run(test_workflow_api())

```

## backend\test\_workflow\_consume.py

```

import asyncio
import httpx

async def test_workflow_with_clear_cache():
 print("=" * 80)
 print("测试工作流 API - 确保产生真实消耗")
 print("=" * 80)

 # 首先清除缓存
 print("\n🧹 第一步: 清除缓存")
 async with httpx.AsyncClient(timeout=30) as client:
 response = await client.post("http://localhost:8000/api/v1/workflow/cache/clear")
 print(f" 缓存清除: {response.status_code}")

 # 然后执行复杂任务
 print("\n🧠 第二步: 执行复杂任务 (复杂度 >= 0.65)")
 task = "帮我设计一个详细的校园运动会活动方案，包含完整的流程安排、预算规划、人员分工和时间线"

 print(f" 任务: {task[:50]}...")

 async with httpx.AsyncClient(timeout=180) as client:
 response = await client.post(
 "http://localhost:8000/api/v1/workflow/execute",
 json={"user_input": task}
)

 if response.status_code == 200:
 data = response.json()

 print("\n✅ 请求成功!")
 print(f"\n📋 执行结果:")
 print(f" 复杂度评分: {data.get('complexity_score', 0.0):.2f}")
 print(f" 执行方式: {'云端执行' if not data.get('executed_locally', True) else '本地执行'}")
 print(f" 耗时: {data.get('total_duration_seconds', 0.0):.2f}秒")

 # 检查步骤中的模型使用
 print(f"\n📄 各步骤详情:")
 for step in data.get('steps', []):
 model = step.get('metadata', {}).get('model_used', 'N/A')
 agent_name = step.get('agent_name', 'N/A')
 print(f" - {agent_name}: 模型={model}")

```

```

 print(f"\n👉 请刷新您的 DeepSeek 平台页面查看最新消耗！")
 print(f" 此请求应该会产生约 2000+ tokens 的消耗")
 else:
 print(f"\n❌ 请求失败: {response.status_code}")
 print(f" 错误: {response.text}")

if __name__ == "__main__":
 asyncio.run(test_workflow_with_clear_cache())

```

## backend\test\_workflow\_debug.py

```

import asyncio
import json
from agents.knowledge.agent import KnowledgeAgent
from agents.summary.agent import SummaryAgent
from agents.writer.agent import WriterAgent
from agents.review.agent import ReviewAgent
from agents.base.agent import AgentInput

async def test_full_workflow():
 print("=== 测试完整工作流 ===")

 user_input = "帮我写一个关于校园AI助手的年度规划包含：1. 时间线、2. 目标分析、3. 实施步骤"

 print(f"用户输入: {user_input}")
 print(f"输入长度: {len(user_input)}")

 # 执行 Knowledge Agent
 print("\n--- 1. 执行 Knowledge Agent ---")
 knowledge_agent = KnowledgeAgent()
 knowledge_input = AgentInput(content=user_input, context={}, use_llm=False, use_cloud=False)
 knowledge_result = await knowledge_agent.execute(knowledge_input)
 print(f"输出状态: {'成功' if knowledge_result.success else '失败'}")
 print(f"输出长度: {len(knowledge_result.content)}")
 print(f"输出预览: {knowledge_result.content[:300]}...")

 # 执行 Summary Agent
 print("\n--- 2. 执行 Summary Agent ---")
 summary_agent = SummaryAgent()
 summary_input = AgentInput(content=knowledge_result.content, context={}, use_llm=False, use_cloud=False)
 summary_result = await summary_agent.execute(summary_input)
 print(f"输出状态: {'成功' if summary_result.success else '失败'}")
 print(f"输出长度: {len(summary_result.content)}")
 print(f"输出预览: {summary_result.content[:500]}")

 # 解析 Summary 输出
 try:
 summary_data = json.loads(summary_result.content)
 print(f"\n解析的任务: {summary_data.get('task')}")
 print(f"关键词: {summary_data.get('keywords')}")
 print(f"需求: {summary_data.get('requirements')}")
 print(f"大纲: {summary_data.get('outline')}")
 except json.JSONDecodeError as e:
 print(f"Summary 输出 JSON 解析失败: {e}")
 print(f"原始输出: {summary_result.content}")

 # 执行 Writer Agent
 print("\n--- 3. 执行 Writer Agent ---")
 writer_agent = WriterAgent()
 writer_input = AgentInput(content=summary_result.content, context={}, use_llm=True, use_cloud=False)
 writer_result = await writer_agent.execute(writer_input)
 print(f"输出状态: {'成功' if writer_result.success else '失败'}")
 print(f"输出长度: {len(writer_result.content)}")
 print(f"输出预览: {writer_result.content[:500]}")

 if len(writer_result.content) < 100:
 print(f"\n警告: Writer Agent 输出过短!")
 print(f"完整输出: {writer_result.content}")
 print(f"错误信息: {writer_result.message}")

 # 执行 Review Agent
 print("\n--- 4. 执行 Review Agent ---")
 review_agent = ReviewAgent()

```

```

review_input_data = {
 "user_task": user_input,
 "summary": summary_data.get("summary", "") if 'summary_data' in dir() else "",
 "writer_output": writer_result.content
}
print(f"Review 输入数据: {json.dumps(review_input_data)[:300]}...")

review_input = AgentInput(content=json.dumps(review_input_data), context={}, use_llm=True, use_cloud=False)
review_result = await review_agent.execute(review_input)
print(f"输出状态: {'成功' if review_result.success else '失败'}")
print(f"输出长度: {len(review_result.content)}")
print(f"输出内容: {review_result.content}")

if __name__ == "__main__":
 asyncio.run(test_full_workflow())

```

## backend\test\_workflow\_fix.py

```

import asyncio
import sys
import os

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))

from app.main import app
from app.dependencies import get_agent_registry
from models.workflow import WorkflowInput

async def test_workflow():
 """测试 workflow 执行"""
 try:
 # 获取 agent registry
 registry = get_agent_registry()
 # 初始化所有 agent
 registry.initialize_all_agents_sync()

 # 创建测试输入
 test_input = WorkflowInput(
 user_input="帮我写一份简单的校园活动策划",
 context={}
)

 # 测试单个 agent
 print("=== 测试单个 Agent ===")

 # 测试 Knowledge Agent
 print("\n1. 测试 Knowledge Agent...")
 from agents.base.agent import AgentInput
 knowledge_result = await registry.execute_agent("knowledge", AgentInput(content="帮我写一份简单的校园活动策划"))
 print(f" 成功: {knowledge_result.success}")
 print(f" 内容长度: {len(knowledge_result.content)}")
 print(f" 预览: {knowledge_result.content[:100]}...")

 # 测试 Summary Agent
 print("\n2. 测试 Summary Agent...")
 summary_result = await registry.execute_agent("summary", AgentInput(content=knowledge_result.content))
 print(f" 成功: {summary_result.success}")
 print(f" 内容长度: {len(summary_result.content)}")

 # 测试 Writer Agent
 print("\n3. 测试 Writer Agent...")
 writer_input = f"{knowledge_result.content}\n\n任务摘要: {summary_result.content}"
 writer_result = await registry.execute_agent("writer", AgentInput(content=writer_input))
 print(f" 成功: {writer_result.success}")
 print(f" 内容长度: {len(writer_result.content)}")

 # 测试 Review Agent (使用正确的 JSON 格式)
 print("\n4. 测试 Review Agent...")
 import json
 review_input = json.dumps({
 "user_task": "帮我写一份简单的校园活动策划",
 "summary": summary_result.content,
 "writer_output": writer_result.content
 })

```

```

review_result = await registry.execute_agent("review", AgentInput(content=review_input))
print(f" 成功: {review_result.success}")
print(f" 内容长度: {len(review_result.content)}")
print(f" 内容: {review_result.content}")

测试Judge Agent (使用正确的JSON格式)
print("\n5. 测试 Judge Agent...")
judge_input = json.dumps({
 "user_task": "帮我写一份简单的校园活动策划",
 "summary_result": summary_result.content,
 "review_result": review_result.content,
 "writer_output": writer_result.content
})
judge_result = await registry.execute_agent("judge", AgentInput(content=judge_input))
print(f" 成功: {judge_result.success}")
print(f" 内容长度: {len(judge_result.content)}")
print(f" 内容: {judge_result.content}")

测试结果 Agent (使用正确的JSON格式)
print("\n6. 测试 Result Agent...")
try:
 judge_data = json.loads(judge_result.content)
 result_input = json.dumps({
 "user_task": "帮我写一份简单的校园活动策划",
 "summary_result": summary_result.content,
 "review_result": review_result.content,
 "judge_result": judge_result.content,
 "writer_output": writer_result.content,
 "executed_locally": True,
 "complexity_score": judge_data.get("complexity_score", 0.0),
 "judge_decision": judge_data.get("decision", "local_output"),
 "cloud_mode": judge_data.get("cloud_mode", "none")
 })
 result_result = await registry.execute_agent("result", AgentInput(content=result_input))
 print(f" 成功: {result_result.success}")
 print(f" 内容长度: {len(result_result.content)}")
 print(f" 预览: {result_result.content[:200]}...")
except Exception as e:
 print(f" 失败: {e}")

print("\n=== 所有单个Agent测试完成 ===")

except Exception as e:
 print(f"测试失败: {e}")
 import traceback
 traceback.print_exc()

if __name__ == "__main__":
 asyncio.run(test_workflow())

```

## backend\tests\_\_init\_\_.py

## backend\tests\test\_agents\_\_init\_\_.py

## backend\tests\test\_agents\test\_judge\_agent.py

```

import pytest
from agents.judge.agent import JudgeAgent

```

```

from agents.base.agent import AgentInput

class TestJudgeAgent:
 @pytest.mark.asyncio
 async def test_execute(self):
 agent = JudgeAgent()
 input_data = AgentInput(content="测试内容")

 result = await agent.execute(input_data)

 assert result.success is True
 assert "complexity_score" in result.content

 @pytest.mark.asyncio
 async def test_complexity_threshold(self):
 agent = JudgeAgent()

 input_data_high = AgentInput(content="复杂任务：深度分析AI模型架构，涉及算法研究和系统设计")
 result_high = await agent.execute(input_data_high)

 input_data_low = AgentInput(content="简单任务：生成一个简单的问候语")
 result_low = await agent.execute(input_data_low)

 assert result_high.metadata["complexity_score"] >= result_low.metadata["complexity_score"]

```

## backend\tests\test\_agents\test\_knowledge\_agent.py

```

import pytest
from agents.knowledge.agent import KnowledgeAgent
from agents.base.agent import AgentInput

class TestKnowledgeAgent:
 @pytest.mark.asyncio
 async def test_execute(self):
 agent = KnowledgeAgent()
 input_data = AgentInput(content="生成校园AI助手方案")

 result = await agent.execute(input_data)

 assert result.success is True
 assert "校园AI助手方案" in result.content
 assert "知识增强" in result.content

 @pytest.mark.asyncio
 async def test_extract_keywords(self):
 agent = KnowledgeAgent()
 keywords = agent._extract_keywords("生成校园AI助手方案")

 assert isinstance(keywords, list)
 assert "校园" in keywords
 assert "AI" in keywords

 @pytest.mark.asyncio
 async def test_empty_input(self):
 agent = KnowledgeAgent()
 input_data = AgentInput(content="")

 result = await agent.execute(input_data)

 assert result.success is True

```

## backend\tests\test\_api\_\_init\_\_.py

## backend\tests\test\_api\test\_knowledge\_api.py

```
import pytest
from knowledge.service import KnowledgeService

class TestKnowledgeService:
 def test_search(self):
 service = KnowledgeService()
 results = service.search("AI")

 assert isinstance(results, dict)
 assert "AI" in results

 def test_add_knowledge(self):
 service = KnowledgeService()
 service.add_knowledge("test_keyword", ["test content"])

 result = service.get_knowledge_by_keyword("test_keyword")
 assert result is not None
 assert "test content" in result

 service.delete_knowledge("test_keyword")

 def test_enhance_content(self):
 service = KnowledgeService()
 enhanced = service.enhance_content("测试内容", ["AI"])

 assert "知识增强" in enhanced
 assert "测试内容" in enhanced

 def test_get_stats(self):
 service = KnowledgeService()
 stats = service.get_knowledge_stats()

 assert "total_keywords" in stats
 assert "total_items" in stats
 assert stats["total_keywords"] > 0
```

## backend\tests\test\_api\_performance.py

```
"""API Performance Test for AgentMatrix backend."""

import asyncio
import aiohttp
import time
import json
from typing import Dict, Any, List, Optional
from datetime import datetime

BASE_URL = "http://localhost:8000/api/v1"

class APITester:
 """API tester class for performance testing."""

 def __init__(self) -> None:
 self.results: List[Dict[str, Any]] = []

 async def test_endpoint(
 self,
 session: aiohttp.ClientSession,
 method: str,
 endpoint: str,
 data: Optional[Dict[str, Any]] = None,
 description: str = ""
) -> Dict[str, Any]:
 """Test a single API endpoint."""
 url = f"{BASE_URL}{endpoint}"
```

```

start_time = time.time()

try:
 response: aiohttp.ClientResponse
 if method.upper() == "GET":
 async with session.get(url) as response:
 status = response.status
 try:
 response_data = await response.json()
 except Exception:
 response_data = await response.text()
 elif method.upper() == "POST":
 async with session.post(url, json=data) as response:
 status = response.status
 try:
 response_data = await response.json()
 except Exception:
 response_data = await response.text()
 else:
 return {"error": "Unsupported method: " + method}

 duration = time.time() - start_time

 result: Dict[str, Any] = {
 "endpoint": endpoint,
 "method": method,
 "description": description,
 "status": status,
 "duration_ms": round(duration * 1000, 2),
 "success": status in [200, 201],
 "response_size": len(str(response_data)) if isinstance(response_data, (dict, list)) else len(response_data)
 }

 if not result["success"]:
 result["error"] = response_data

except Exception as e:
 duration = time.time() - start_time
 result = {
 "endpoint": endpoint,
 "method": method,
 "description": description,
 "status": -1,
 "duration_ms": round(duration * 1000, 2),
 "success": False,
 "error": str(e)
 }

self.results.append(result)
return result

async def run_performance_test(self, iterations: int = 3) -> None:
 """Run comprehensive performance tests."""
 print("\n" + "=" * 60)
 print("[TEST] AgentMatrix API Performance Test")
 print("=" * 60 + "\n")

 async with aiohttp.ClientSession() as session:
 print("[PHASE 1] Basic Health Check")
 print("-" * 40)

 await self.test_endpoint(session, "GET", "/metrics/", "System Metrics")
 await self.test_endpoint(session, "GET", "/chat/health", "Chat Service Health")
 await self.test_endpoint(session, "GET", "/knowledge/", "Knowledge Base")

 print("\n[PHASE 2] Workflow Execution (Cold Start)")
 print("-" * 40)

 workflow_result = await self.test_endpoint(
 session, "POST", "/workflow/execute",
 {"user_input": "test workflow execution"},
 "Workflow Cold Start"
)
 print("First execution: " + str(workflow_result["duration_ms"]) + "ms")

 print("\n[PHASE 3] Workflow Execution (Cached)")
 print("-" * 40)

 for i in range(iterations):
 cached_result = await self.test_endpoint(

```

```

 session, "POST", "/workflow/execute",
 {"user_input": "test workflow execution"},
 "Workflow Cache Test #" + str(i + 1)
)
 print(" Cached execution #" + str(i + 1) + ": " + str(cached_result["duration_ms"]) + "ms")

print("\n[PHASE 4] Chat API Performance")
print("-" * 40)

chat_cold = await self.test_endpoint(
 session, "POST", "/chat/send",
 {"content": "hello, this is a test"},
 "Chat Cold Start"
)
print(" First chat: " + str(chat_cold["duration_ms"]) + "ms")

for i in range(iterations):
 chat_cached = await self.test_endpoint(
 session, "POST", "/chat/send",
 {"content": "hello, this is a test"},
 "Chat Cache Test #" + str(i + 1)
)
 print(" Cached chat #" + str(i + 1) + ": " + str(chat_cached["duration_ms"]) + "ms")

print("\n[PHASE 5] Knowledge Base Operations")
print("-" * 40)

await self.test_endpoint(session, "GET", "/knowledge/search?query=AI", "Knowledge Search")
await self.test_endpoint(session, "GET", "/knowledge/keyword/AI", "Get Keyword")

print("\n[PHASE 6] Agent API")
print("-" * 40)

await self.test_endpoint(session, "GET", "/agents/", "Get All Agents")
await self.test_endpoint(session, "GET", "/agents/knowledge/status", "Knowledge Agent Status")

print("\n[PHASE 7] Batch Requests")
print("-" * 40)

batch_data = [{"content": "test message " + str(i)} for i in range(3)]
batch_result = await self.test_endpoint(
 session, "POST", "/chat/send/batch",
 batch_data,
 "Batch Messages"
)
print(" Batch request: " + str(batch_result["duration_ms"]) + "ms")

self.print_summary()

async def run_stress_test(self, concurrent_requests: int = 10) -> None:
 """Run stress test with concurrent requests."""
 print("\n" + "=" * 60)
 print("[STRESS TEST] Concurrent Requests")
 print("=" * 60 + "\n")

 async with aiohttp.ClientSession() as session:
 tasks: List[asyncio.Task[Dict[str, Any]]] = []
 start_time = time.time()

 for i in range(concurrent_requests):
 task = asyncio.create_task(
 self.test_endpoint(
 session, "POST", "/chat/send",
 {"content": "stress test request " + str(i)},
 "Concurrent Request #" + str(i)
)
)
 tasks.append(task)

 await asyncio.gather(*tasks)
 total_duration = time.time() - start_time

 print("Completed " + str(concurrent_requests) + " concurrent requests")
 print("Total time: " + str(total_duration * 1000) + "ms")
 print("Throughput: " + str(concurrent_requests / total_duration) + " req/s")

def print_summary(self) -> None:
 """Print test summary and save report."""
 print("\n" + "=" * 60)
 print("[TEST SUMMARY]")

```

```

print("=" * 60)

successes = [r for r in self.results if r["success"]]
failures = [r for r in self.results if not r["success"]]

avg_duration = sum(r["duration_ms"] for r in successes) / len(successes) if successes else 0
max_duration = max(r["duration_ms"] for r in successes) if successes else 0
min_duration = min(r["duration_ms"] for r in successes) if successes else 0

print("\n[STATS]")
print(" Total tests: " + str(len(self.results)))
print(" Success: " + str(len(successes)))
print(" Failed: " + str(len(failures)))
print(" Success rate: " + str(len(successes) / len(self.results) * 100) + "%")

print("\n[RESPONSE TIME]")
print(" Average: " + str(avg_duration) + "ms")
print(" Min: " + str(min_duration) + "ms")
print(" Max: " + str(max_duration) + "ms")

cached_requests = [r for r in self.results if "Cache" in r["description"]]
if cached_requests:
 avg_cached = sum(r["duration_ms"] for r in cached_requests) / len(cached_requests)
 print("\n[CACHE EFFECTIVENESS]")
 print(" Cached requests: " + str(len(cached_requests)))
 print(" Average time: " + str(avg_cached) + "ms")

if failures:
 print("\n[FAILED REQUESTS]")
 for r in failures:
 print(" - " + r["method"] + " " + r["endpoint"] + ": " + str(r["status"]) + " - " + str(r.get("error", "Unknown")))

print("\n" + "=" * 60)

timestamp = datetime.now().isoformat()
report = {
 "timestamp": timestamp,
 "total_tests": len(self.results),
 "successes": len(successes),
 "failures": len(failures),
 "success_rate": len(successes) / len(self.results) * 100,
 "avg_duration_ms": avg_duration,
 "min_duration_ms": min_duration,
 "max_duration_ms": max_duration,
 "results": self.results
}

with open("tests/test_report.json", "w", encoding="utf-8") as f:
 json.dump(report, f, ensure_ascii=False, indent=2)

print("Test report saved to: tests/test_report.json")

if __name__ == "__main__":
 tester = APITester()
 asyncio.run(tester.run_performance_test(iterations=3))
 asyncio.run(tester.run_stress_test(concurrent_requests=10))

```

## backend\tests\test\_report.json

```

{
 "timestamp": "2026-05-13T09:30:08.945350",
 "total_tests": 16,
 "successes": 16,
 "failures": 0,
 "success_rate": 100.0,
 "avg_duration_ms": 3.328125,
 "min_duration_ms": 1.03,
 "max_duration_ms": 28.93,
 "results": [
 {
 "endpoint": "/metrics/",
 "method": "GET",

```

```

 "description": "",
 "status": 200,
 "duration_ms": 28.93,
 "success": true,
 "response_size": 1057
 },
 {
 "endpoint": "/chat/health",
 "method": "GET",
 "description": "",
 "status": 200,
 "duration_ms": 2.19,
 "success": true,
 "response_size": 52
 },
 {
 "endpoint": "/knowledge/",
 "method": "GET",
 "description": "",
 "status": 200,
 "duration_ms": 2.09,
 "success": true,
 "response_size": 770
 },
 {
 "endpoint": "/workflow/execute",
 "method": "POST",
 "description": "Workflow Cold Start",
 "status": 200,
 "duration_ms": 3.43,
 "success": true,
 "response_size": 4164
 },
 {
 "endpoint": "/workflow/execute",
 "method": "POST",
 "description": "Workflow Cache Test #1",
 "status": 200,
 "duration_ms": 1.31,
 "success": true,
 "response_size": 4164
 },
 {
 "endpoint": "/workflow/execute",
 "method": "POST",
 "description": "Workflow Cache Test #2",
 "status": 200,
 "duration_ms": 1.23,
 "success": true,
 "response_size": 4164
 },
 {
 "endpoint": "/workflow/execute",
 "method": "POST",
 "description": "Workflow Cache Test #3",
 "status": 200,
 "duration_ms": 1.2,
 "success": true,
 "response_size": 4164
 },
 {
 "endpoint": "/chat/send",
 "method": "POST",
 "description": "Chat Cold Start",
 "status": 200,
 "duration_ms": 1.71,
 "success": true,
 "response_size": 561
 },
 {
 "endpoint": "/chat/send",
 "method": "POST",
 "description": "Chat Cache Test #1",
 "status": 200,
 "duration_ms": 1.46,
 "success": true,
 "response_size": 561
 },
 {
 "endpoint": "/chat/send",

```

```

 "method": "POST",
 "description": "Chat Cache Test #2",
 "status": 200,
 "duration_ms": 1.33,
 "success": true,
 "response_size": 561
 },
 {
 "endpoint": "/chat/send",
 "method": "POST",
 "description": "Chat Cache Test #3",
 "status": 200,
 "duration_ms": 1.36,
 "success": true,
 "response_size": 561
 },
 {
 "endpoint": "/knowledge/search?query=AI",
 "method": "GET",
 "description": "",
 "status": 200,
 "duration_ms": 1.25,
 "success": true,
 "response_size": 158
 },
 {
 "endpoint": "/knowledge/keyword/AI",
 "method": "GET",
 "description": "",
 "status": 200,
 "duration_ms": 1.03,
 "success": true,
 "response_size": 116
 },
 {
 "endpoint": "/agents/",
 "method": "GET",
 "description": "",
 "status": 200,
 "duration_ms": 1.67,
 "success": true,
 "response_size": 741
 },
 {
 "endpoint": "/agents/knowledge/status",
 "method": "GET",
 "description": "",
 "status": 200,
 "duration_ms": 1.09,
 "success": true,
 "response_size": 112
 },
 {
 "endpoint": "/chat/send/batch",
 "method": "POST",
 "description": "Batch Messages",
 "status": 200,
 "duration_ms": 1.97,
 "success": true,
 "response_size": 1726
 }
]
}

```

---

[backend\tests\test\\_workflow\\_\\_init\\_\\_.py](#)

---



---

[backend\tests\test\\_workflow\test\\_workflow\\_service.py](#)

---

```

import pytest
from models.workflow import WorkflowInput
from core.workflow.service import WorkflowService
from agents.base.agent_registry import AgentRegistry

class TestWorkflowService:
 @pytest.mark.asyncio
 async def test_execute_workflow(self):
 registry = AgentRegistry()
 await registry.initialize_all_agents()

 service = WorkflowService(registry)
 input_data = WorkflowInput(user_input="生成校园AI助手方案")

 result = await service.execute(input_data)

 assert result is not None
 assert result.final_result is not None
 assert len(result.steps) == 6
 assert result.executed_locally is True

 await registry.shutdown_all_agents()

 @pytest.mark.asyncio
 async def test_workflow_steps(self):
 registry = AgentRegistry()
 await registry.initialize_all_agents()

 service = WorkflowService(registry)
 input_data = WorkflowInput(user_input="测试")

 result = await service.execute(input_data)

 agent_ids = [step.agent_id for step in result.steps]
 assert agent_ids == ["knowledge", "summary", "writer", "review", "judge", "result"]

 await registry.shutdown_all_agents()

```

## backend\utils\_\_init\_\_.py

```

from .logger import StructuredLogger, log_function, setup_logging

__all__ = ["StructuredLogger", "log_function", "setup_logging"]

```

## backend\utils\logger.py

```

import logging
import json
from datetime import datetime
from typing import Dict, Any, Optional
from functools import wraps

class StructuredLogger:
 def __init__(self, name: str):
 self.logger = logging.getLogger(name)

 def log(self, level: int, message: str, **kwargs):
 extra = {
 "timestamp": datetime.utcnow().isoformat(),
 "message": message,
 **kwargs
 }
 self.logger.log(level, json.dumps(extra, ensure_ascii=False))

 def debug(self, message: str, **kwargs):

```

```

 self.log(logging.DEBUG, message, **kwargs)

 def info(self, message: str, **kwargs):
 self.log(logging.INFO, message, **kwargs)

 def warning(self, message: str, **kwargs):
 self.log(logging.WARNING, message, **kwargs)

 def error(self, message: str, **kwargs):
 self.log(logging.ERROR, message, **kwargs)

 def critical(self, message: str, **kwargs):
 self.log(logging.CRITICAL, message, **kwargs)

def log_function(func):
 @wraps(func)
 async def async_wrapper(*args, **kwargs):
 logger = logging.getLogger(func.__module__)
 start_time = datetime.now()
 logger.info(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")

 try:
 result = await func(*args, **kwargs)
 duration = (datetime.now() - start_time).total_seconds()
 logger.info(f"Completed {func.__name__} in {duration:.2f}s")
 return result
 except Exception as e:
 duration = (datetime.now() - start_time).total_seconds()
 logger.error(f"Failed {func.__name__} in {duration:.2f}s: {e}")
 raise

 @wraps(func)
 def sync_wrapper(*args, **kwargs):
 logger = logging.getLogger(func.__module__)
 start_time = datetime.now()
 logger.info(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")

 try:
 result = func(*args, **kwargs)
 duration = (datetime.now() - start_time).total_seconds()
 logger.info(f"Completed {func.__name__} in {duration:.2f}s")
 return result
 except Exception as e:
 duration = (datetime.now() - start_time).total_seconds()
 logger.error(f"Failed {func.__name__} in {duration:.2f}s: {e}")
 raise

 import asyncio
 if asyncio.iscoroutinefunction(func):
 return async_wrapper
 return sync_wrapper

def setup_logging(level: str = "INFO", log_file: str = "logs/system.log"):
 log_level = getattr(logging, level.upper(), logging.INFO)

 handlers = [
 logging.FileHandler(log_file, encoding="utf-8"),
 logging.StreamHandler()
]

 logging.basicConfig(
 level=log_level,
 format="%asctime)s - %(name)s - %(levelname)s - %(message)s",
 handlers=handlers
)

 logging.info(f"Logging configured at level {level}, output to {log_file}")

```

## backend\verify\_deepseek\_key.py

```

import asyncio
import aiohttp

```

```

async def verify_api_key():
 print("=" * 80)
 print("验证 DeepSeek API Key 是否正确使用")
 print("=" * 80)

 api_key = "sk-6ccaead05a2f462d97417a268139b561"
 url = "https://api.deepseek.com/v1/chat/completions"

 print(f"\n🔑 API Key: {api_key}")
 print(f"🌐 API URL: {url}")
 print("\n" + "=" * 80)

 # 测试1: 使用 deepseek-chat 模型
 print("\n📄 测试1: deepseek-chat 模型")
 payload = {
 "model": "deepseek-chat",
 "messages": [{"role": "user", "content": "你好"}],
 "max_tokens": 10
 }
 headers = {"Authorization": f"Bearer {api_key}", "Content-Type": "application/json"}

 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 if response.status == 200:
 data = await response.json()
 print(f"✅ 成功")
 print(f"模型: {data.get('model', 'N/A')}")
 print(f"Token消耗: {data.get('usage', {}).get('total_tokens', 0)}")
 else:
 error = await response.text()
 print(f"❌ 失败: {response.status} - {error}")
 except Exception as e:
 print(f"❌ 异常: {str(e)}")

 # 测试2: 使用 deepseek-v4-flash 模型
 print("\n📄 测试2: deepseek-v4-flash 模型")
 payload = {
 "model": "deepseek-v4-flash",
 "messages": [{"role": "user", "content": "你好"}],
 "max_tokens": 10
 }

 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.post(url, json=payload, headers=headers) as response:
 if response.status == 200:
 data = await response.json()
 print(f"✅ 成功")
 print(f"模型: {data.get('model', 'N/A')}")
 print(f"Token消耗: {data.get('usage', {}).get('total_tokens', 0)}")
 else:
 error = await response.text()
 print(f"❌ 失败: {response.status} - {error}")
 except Exception as e:
 print(f"❌ 异常: {str(e)}")

 # 测试3: 查询账户余额
 print("\n📄 测试3: 查询账户余额")
 balance_url = "https://api.deepseek.com/v1/wallet/balance"

 try:
 async with aiohttp.ClientSession(timeout=aiohttp.ClientTimeout(total=30)) as session:
 async with session.get(balance_url, headers=headers) as response:
 if response.status == 200:
 data = await response.json()
 print(f"✅ 查询成功")
 print(f"余额信息: {data}")
 else:
 error = await response.text()
 print(f"❌ 查询失败: {response.status} - {error}")
 print(f"可能原因: API Key无效或无权访问")
 except Exception as e:
 print(f"❌ 异常: {str(e)}")

 print("\n" + "=" * 80)
 print("💡 建议:")
 print("1. 检查API Key是否正确")
 print("2. 确认API Key是否属于您的账户")

```

```
print(" 3. 检查DeepSeek平台的消费记录")
print("=" * 80)

if __name__ == "__main__":
 asyncio.run(verify_api_key())
```

backend\使用说明.md

```
AgentMatrix 使用说明

🚀 快速启动

方式一：使用启动脚本（推荐）

```bash
python start_service.py
```

方式二：直接运行

```
python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

📄 功能说明

1. 本地模型 (Ollama) 配置

支持的本地模型：

- **qwen2.5:1.5b** - 用于 Knowledge, Summary, Judge, Result Agent
- **phi4-mini:3.8b** - 用于 Writer, Review Agent

自动端口检测

系统会自动检测 Ollama 服务端点： - 默认检测 11434, 11435, 8080 - 自动选择可用端口

手动配置端口：

1. 点击页面右上角 "⚙️ 设置" 按钮
2. 在 "Ollama 本地模型配置" 区域
3. 点击 "自动检测端口" 按钮
4. 或手动输入 Ollama 服务地址 (如 http://localhost:11435)
5. 点击 "测试连接" 验证

2. 云服务 (DeepSeek) 配置

1. 点击页面右上角 "⚙️ 设置" 按钮
2. 在 "DeepSeek 云服务配置" 区域
3. 输入你的 DeepSeek API Key
4. 点击 "保存"
5. 点击 "测试连接" 验证

🤖 智能体工作流程

完整的 6 智能体协同工作流程：

智能体	模型	职责
Knowledge Agent	qwen2.5:1.5b	知识检索、关键词匹配
Summary Agent	qwen2.5:1.5b	需求理解、大纲生成
Writer Agent	phi4-mini:3.8b	内容生成、文档撰写
Review Agent	phi4-mini:3.8b	质量评审、优化建议
Judge Agent	qwen2.5:1.5b	复杂度判断、执行决策
Result Agent	qwen2.5:1.5b	结果整合、导出格式

 使用方法

基本使用流程

- 1. 打开应用
- 2. 在浏览器中访问 `http://localhost:8000`
- 3. **配置服务（首次使用）
- 4. 点击 "  设置"
- 5. 配置 Ollama 端口（本地模型）
- 6. 配置 DeepSeek API Key（可选）
- 7. **选择执行模式
- 8. 点击 "  本地模型" - 使用本地 Ollama
- 9. 点击 "  云服务" - 使用 DeepSeek
- 10. 输入问题
- 11. 在输入框中输入你的需求
- 12. 例如："帮我写一个校园马拉松活动策划"
- 13. **发送执行
- 14. 点击 "发送" 按钮
- 15. 观看 6 个智能体协同工作

 常见问题

Q: 本地模型连接失败怎么办？

A: 请检查以下几点： **

- 1. 确认 Ollama 服务已安装并运行： `bash ollama serve`
- 2. 确认模型已下载： `bash ollama pull qwen2.5:1.5b ollama pull phi4-mini:3.8b`
- 3. 使用自动端口检测功能

Q: 云服务 API Key 哪里获取？

A: 访问 DeepSeek 官网注册并获取 API Key

Q: 如何切换本地/云端模式？

A: 点击页面顶部按钮： -  本地模型 - 使用本地 Ollama -  云服务 - 使用 DeepSeek

 API 端点

配置管理 API

端点	方法	说明
/api/v1/config	GET	获取当前配置
/api/v1/config	POST	更新配置
/api/v1/config/detect-ollama	POST	自动检测 Ollama 端口
/api/v1/config/test-ollama	POST	测试 Ollama 连接
/api/v1/config/test-deepseek	POST	测试 DeepSeek 连接

智能体 API

端点	方法	说明
/api/v1/agents/{agent_id}/execute	POST	执行智能体

工作流 API

端点	方法	说明
/api/v1/workflow/start	POST	启动工作流

```
---

## backend\启动.bat

```batch
@echo off
chcp 65001 >nul
echo =====
echo AgentMatrix 启动器
echo =====
echo.
echo [1/3] 检查环境...
python --version
if errorlevel 1 (
 echo ❌ Python 未安装或不在PATH中
 pause
 exit /b 1
)
echo ✅ Python 检查通过
echo.

echo [2/3] 启动服务...
echo.
echo =====
echo 服务地址: http://localhost:8000
echo API文档: http://localhost:8000/docs
echo 按 Ctrl+C 停止服务
echo =====
echo.

python app/main.py

if errorlevel 1 (
 echo.
 echo ❌ 启动失败!
 echo.
 echo 尝试其他方式:
 echo python -m uvicorn app.main:app --host 0.0.0.0 --port 8000
 echo.
)
pause
```

## configs\agents\judge\_config.yaml

---

```
agent_id: judge
name: Judge Agent
description: 复杂度判断与路由决策
enabled: true
priority: 5

complexity_threshold: 0.65

routing_rules:
 - condition: complexity > 0.8
 action: cloud
 model: gemini-pro
 - condition: complexity > 0.65
 action: cloud
 model: gemini-pro
 - condition: complexity <= 0.65
 action: local
 model: qwen2.5:3b

confidence_threshold: 0.7

logging:
 enabled: true
 level: INFO
```

## configs\agents\knowledge\_config.yaml

---

```
agent_id: knowledge
name: Knowledge Agent
description: 知识检索与增强
enabled: true
priority: 1

retrieval:
 top_k: 5
 similarity_threshold: 0.7
 max_context_length: 2000

knowledge_sources:
 - type: json
 path: knowledge/json/domain_knowledge.json
 - type: json
 path: knowledge/json/system_knowledge.json

cache:
 enabled: true
 ttl_seconds: 3600
```

## configs\models\local\_models.yaml

---

```
default_model: qwen2.5:1.5b

models:
 - name: qwen2.5:1.5b
 provider: ollama
 host: http://localhost:11434
 parameters:
 temperature: 0.7
 max_tokens: 2048
 top_p: 0.9
 capabilities:
 - summarization
 - classification
 - reasoning
```

```
- knowledge_retrieval

- name: phi4-mini:3.8b
 provider: ollama
 host: http://localhost:11434
 parameters:
 temperature: 0.2
 max_tokens: 4096
 top_p: 0.95
 capabilities:
 - review
 - reasoning
 - quality_assessment

- name: internlm2:1.8b
 provider: ollama
 host: http://localhost:11434
 parameters:
 temperature: 0.7
 max_tokens: 2048
 top_p: 0.9
 capabilities:
 - summarization
 - general_purpose

agent_mappings:
- agent_id: knowledge
 local_model: qwen2.5:1.5b
 cloud_model: deepseek-rl-distill
 description: 知识检索与增强

- agent_id: summary
 local_model: qwen2.5:1.5b
 cloud_model: deepseek-rl-distill
 description: 需求摘要提取

- agent_id: writer
 local_model: qwen2.5:1.5b
 cloud_model: deepseek-rl-distill
 description: 内容生成与文档撰写

- agent_id: review
 local_model: phi4-mini:3.8b
 cloud_model: deepseek-rl-distill
 description: 质量评审与检查

- agent_id: judge
 local_model: qwen2.5:1.5b
 cloud_model: deepseek-rl-distill
 description: 复杂度判断与路由决策

- agent_id: result
 local_model: qwen2.5:1.5b
 cloud_model: deepseek-rl-distill
 description: 结果导出与格式转换
```

deepseek密钥.txt

```
sk-6ccaead05a2f462d97417a268139b561
```

docs\AgentMatrix 安装说明.md

```
AgentMatrix 安装说明

> 多智能体动态协同与国产算力优化平台\
> 版本: v0.1.0 | 日期: 2026-05-17

```

```
目录

1. [环境准备] (#一环境准备)
2. [快速安装 (Windows)] (#二快速安装windows)
3. [手动安装 (Windows)] (#三手动安装windows)
4. [手动安装 (Linux/macOS)] (#四手动安装linuxmacos)
5. [安装验证] (#五安装验证)
6. [常见问题] (#六常见问题)

一、环境准备

1.1 系统要求

| 组件 | 最低版本 | 检查命令 |
| ----- | ----- | ----- |
| 操作系统 | Windows 10+ / macOS 12+ / Ubuntu 22.04+ | - |
| Python | 3.12.0+ | `python --version` |
| Node.js | 20.0.0+ | `node --version` |
| npm | 9.0.0+ | `npm --version` |
| Git | 2.40.0+ | `git --version` |

1.2 安装 Python 3.12+

Windows:

1. 访问 <https://www.python.org/downloads/>
2. 下载 Python 3.12 安装包
3. 安装时勾选 "Add Python to PATH"
4. 验证: `python --version`

Linux:

```bash
sudo add-apt-repository ppa:deadsnakes/ppa
sudo apt update
sudo apt install python3.12 python3.12-venv
```

macOS:

```
brew install python@3.12
```

1.3 安装 Node.js 20+

Windows/macOS:

- 1. 访问 <https://nodejs.org/>
- 2. 下载 Node.js 20 LTS 版本
- 3. 安装完成后验证: `node --version` 和 `npm --version`

Linux:

```
curl -fsSL https://deb.nodesource.com/setup_20.x | sudo -E bash -
sudo apt install nodejs
```

1.4 安装 Ollama

Windows:

- 1. 访问 <https://ollama.com/download/windows>
- 2. 下载并安装
- 3. 验证: `ollama --version`

Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

macOS:

```
brew install ollama
```

二、快速安装（Windows）

2.1 获取项目代码

```
# 克隆仓库
git clone <repository-url>
cd AgentMatrix
```

2.2 一键安装

```
# 以管理员身份打开 PowerShell
# cd 到项目目录后运行:

# 1. 启动 Ollama 服务（如果尚未运行）
Start-Process ollama -ArgumentList "serve" -WindowStyle Normal

# 2. 下载所需的本地模型（各约需 5-15 分钟）
ollama pull qwen2.5:1.5b
ollama pull phi4-mini:3.8b

# 3. 安装后端依赖
cd backend
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru websockets psutil
cd ..

# 4. 安装前端依赖
cd frontend
npm install
cd ..
```

2.3 配置环境变量

```
# 设置 Ollama 端口（如果默认 11434 被占用）
$env:OLLAMA_HOST = "0.0.0.0:11435"

# 可选：设置 DeepSeek API Key
# 创建 backend/.env 文件并添加：
# DEEPSEEK_API_KEY=sk-your-api-key-here
```

2.4 启动服务

```
# 方式一：使用一键脚本
.\start.ps1

# 方式二：手动启动
# 终端 1 - 后端
cd backend
.\venv\Scripts\Activate.ps1
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# 终端 2 - 前端
cd frontend
npm run dev
```

2.5 访问

- 前端: <http://localhost:3000>
- 后端 API 文档: <http://localhost:8000/docs>
- 健康检查: <http://localhost:8000/health>

三、手动安装（Windows）

3.1 克隆项目

```
git clone <repository-url>
cd AgentMatrix
```

3.2 后端安装

```
# 进入后端目录
cd backend

# 创建虚拟环境
python -m venv venv

# 激活虚拟环境
.\venv\Scripts\Activate.ps1

# 注意: 如果遇到执行策略限制, 先运行:
# Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser

# 安装核心依赖
pip install fastapi==0.110.0 uvicorn==0.29.0 pydantic==2.6.0
pip install python-dotenv==1.0.0 aiohttp==3.9.0 sqlalchemy==2.0.0
pip install loguru==0.7.0 websockets==12.0 psutil==5.9.0
pip install python-pptx==0.6.23 python-docx==1.1.0 pymdown-extensions==10.0.0

# 可选: 安装开发依赖
pip install pytest pytest-asyncio httpx ruff black pre-commit

# 返回项目根目录
cd ..
```

3.3 前端安装

```
# 进入前端目录
cd frontend

# 安装依赖
npm install

# 返回项目根目录
cd ..
```

3.4 配置

创建后端配置文件 backend/.env :

```
# 在 backend 目录下创建 .env 文件
New-Item -Path backend\.env -ItemType File -Force

# 内容如下:
@"
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=true
LOG_LEVEL=INFO
DATABASE_URL=sqlite:///./agentmatrix.db
OLLAMA_HOST=http://localhost:11435
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b
```

```
DEEPSEEK_API_KEY=  
DEEPSEEK_API_BASE=https://api.deepseek.com/v1  
DEEPSEEK_MODEL=deepseek-r1-distill  
COMPLEXITY_THRESHOLD=0.65  
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000  
"@ | Out-File -FilePath backend\.env -Encoding UTF8
```

创建前端配置文件 frontend/.env.local :

```
@  
NEXT_PUBLIC_API_URL=http://localhost:8000  
NEXT_PUBLIC_WS_URL=ws://localhost:8000  
"@ | Out-File -FilePath frontend\.env.local -Encoding UTF8
```

3.5 Ollama 模型安装

```
# 确保 Ollama 正在运行  
ollama serve  
  
# 新建终端，下载模型  
ollama pull qwen2.5:1.5b # ~986MB, 约5-10分钟  
ollama pull phi4-mini:3.8b # ~2.5GB, 约10-20分钟  
  
# 验证  
ollama list
```

3.6 启动

启动三个终端窗口：

终端 1 - Ollama:

```
# 如果尚未运行  
ollama serve
```

终端 2 - 后端:

```
cd backend  
.\venv\Scripts\Activate.ps1  
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

终端 3 - 前端:

```
cd frontend  
npm run dev
```

四、手动安装（Linux/macOS）

4.1 克隆项目

```
git clone <repository-url>  
cd AgentMatrix
```

4.2 后端安装

```
cd backend  
  
# 创建虚拟环境  
python3 -m venv venv
```

```
source venv/bin/activate

# 安装依赖
pip install fastapi uvicorn pydantic python-dotenv aiohttp sqlalchemy loguru websockets psutil
pip install python-pptx python-docx pymdown-extensions

cd ..
```

4.3 前端安装

```
cd frontend
npm install
cd ..
```

4.4 配置

```
# 后端配置
cat > backend/.env << 'EOF'
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b
DEEPSEEK_API_KEY=
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-rl-distill
COMPLEXITY_THRESHOLD=0.65
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
EOF

# 前端配置
cat > frontend/.env.local << 'EOF'
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000
EOF
```

4.5 Ollama 模型

```
# Linux: 注意 Ollama 默认端口是 11434
ollama pull qwen2.5:1.5b
ollama pull phi4-mini:3.8b
```

4.6 启动

```
# 终端 1 - Ollama
ollama serve

# 终端 2 - 后端
cd backend
source venv/bin/activate
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# 终端 3 - 前端
cd frontend
npm run dev
```

五、安装验证

5.1 验证 Ollama

```
# 检查服务状态
ollama list
```

```
# 预期输出:
# NAME          ID          SIZE      MODIFIED
# phi4-mini:3.8b 78fad5d182a7 2.5 GB    ...
# qwen2.5:1.5b  65ec06548149 986 MB    ...
```

5.2 验证后端

```
# 健康检查 API
curl http://localhost:8000/health

# 预期输出包含:
# {"status":"healthy","agents":{"knowledge":{"status":"ready"},...}}
```

浏览器访问 <http://localhost:8000/docs> 查看 API 文档。

5.3 验证前端

浏览器访问 <http://localhost:3000>, 预期看到 AgentMatrix 主界面:

- ☒ 顶部任务输入框
- ☒ 左侧 Agent 舰队列表
- ☒ 中间流水线可视化
- ☒ 右侧输出面板

5.4 端到端验证

在前端输入框中输入一个简单测试任务:

```
"你好, 请介绍一下你自己"
```

预期结果:

1. 6 个 Agent 依次执行 (流水线动画)
2. 因为复杂度低, 全部由本地模型完成
3. 左侧 "成本节省" 指标增加
4. 右侧显示最终结果

六、常见问题

Q1: PowerShell 无法激活虚拟环境?

```
错误: "无法加载文件 .\venv\Scripts\Activate.ps1, 因为在此系统上禁止运行脚本。"

解决:
以管理员身份运行 PowerShell, 执行:
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Q2: pip install 报错?

```
错误: "Could not find a version that satisfies the requirement..."

解决:
1. 确保 Python >= 3.12: python --version
2. 更新 pip: python -m pip install --upgrade pip
3. 如果在公司网络, 配置代理: pip install --proxy=http://proxy:port ...
```

Q3: npm install 报错?

错误: "npm ERR! code ERESOLVE"

解决:

1. 删除 node_modules 和 package-lock.json
2. 重新安装: npm install
3. 或使用: npm install --legacy-peer-deps

Q4: Ollama 连接失败?

错误: "Failed to call Ollama: Connection refused"

解决:

1. 确认 Ollama 正在运行: ollama list
2. 检查端口号:
 - Windows (本项目配置): 11435
 - Linux/macOS 默认: 11434
3. 测试连接: curl http://localhost:11435/api/tags
4. 如果端口不对, 修改 backend/.env 中的 OLLAMA_HOST

Q5: 前端页面空白?

可能原因:

1. 后端服务未启动
2. 端口号不匹配
3. CORS 配置问题

解决:

1. 确认后端正在运行: curl http://localhost:8000/health
2. 检查 frontend/.env.local 配置
3. 打开浏览器开发者工具 (F12) 查看错误信息

Q6: 模型下载速度慢?

解决:

1. 使用代理加速下载
2. 仅下载需要的模型 (可以只安装一个)
3. 最低要求: 至少安装 qwen2.5:1.5b

Q7: 如何获取 DeepSeek API Key?

1. 访问 <https://platform.deepseek.com/>
2. 注册账号并登录
3. 进入 "API Keys" 页面
4. 创建新的 API Key
5. 将 Key 添加到 backend/.env 中的 DEEPSEEK_API_KEY

Q8: 不想使用云端 API, 可以吗?

可以。不设置 DEEPSEEK_API_KEY 即可。
所有任务将由本地 Ollama 模型处理。

七、开发工具配置 (可选)

7.1 代码质量工具

```
# 后端代码检查
cd backend
pip install ruff black pre-commit
ruff check .
```

```
black .

# 前端代码检查
cd frontend
npm run lint
npm run format
```

7.2 VS Code 配置

项目 `.vscode/` 目录已包含推荐配置：

- `launch.json` - 调试配置
- `settings.json` - 编辑器设置
- `tasks.json` - 任务配置

7.3 运行测试

```
cd backend
pytest tests/ -v
```

八、目录结构说明

AgentMatrix/	
├── backend/	# 后端服务 (FastAPI + Python)
│ ├── agents/	# 6 个 Agent 实现
│ │ ├── base/	# Agent 基类和注册中心
│ │ ├── knowledge/	# Knowledge Agent (知识检索)
│ │ ├── summary/	# Summary Agent (需求摘要)
│ │ ├── writer/	# Writer Agent (内容生成)
│ │ ├── review/	# Review Agent (质量评审)
│ │ ├── judge/	# Judge Agent (复杂度判断)
│ │ └── result/	# Result Agent (成果导出)
│ ├── api/	# API 路由
│ │ ├── v1/	# v1 版本 API
│ │ └── websocket/	# WebSocket 管理
│ ├── core/	# 核心业务逻辑
│ │ ├── workflow/	# 工作流编排
│ │ ├── llm/	# LLM 客户端
│ │ └── dynamic_router/	# 动态算力路由
│ ├── models/	# Pydantic 数据模型
│ ├── prompts/	# Prompt 模板
│ ├── knowledge/	# 知识库服务
│ ├── config/	# 配置管理
│ ├── services/	# 聚合服务层
│ ├── app/	# 应用入口
│ │ ├── main.py	# FastAPI 入口
│ │ ├── config.py	# 全局配置
│ │ └── database.py	# 数据库初始化
│ └── tests/	# 测试
├── frontend/	# 前端服务 (Next.js + React)
│ ├── src/	
│ │ ├── app/	# Next.js App Router
│ │ ├── components/	# React 组件
│ │ │ └── layout/	# 主界面布局
│ │ ├── stores/	# Zustand 状态管理
│ │ ├── services/api/	# API 调用封装
│ │ └── types/	# TypeScript 类型定义
│ ├── package.json	# 依赖清单
│ └── next.config.js	# Next.js 配置
├── configs/	# 配置文件
├── docs/	# 文档
│ ├── TECH_REPORT.md	# 技术报告
│ ├── DEPLOYMENT.md	# 部署文档
│ └── INSTALL.md	# 安装说明 (本文档)
└── start.ps1	# Windows 一键启动脚本

```
└── stop.ps1          # Windows 停止脚本
└── README.md         # 项目说明

---

## docs\AgentMatrix 技术报告.md

```markdown
AgentMatrix 技术报告

> 多智能体动态协同与国产算力优化平台
> 版本: v0.1.0 | 日期: 2026-05-17

一、项目概述

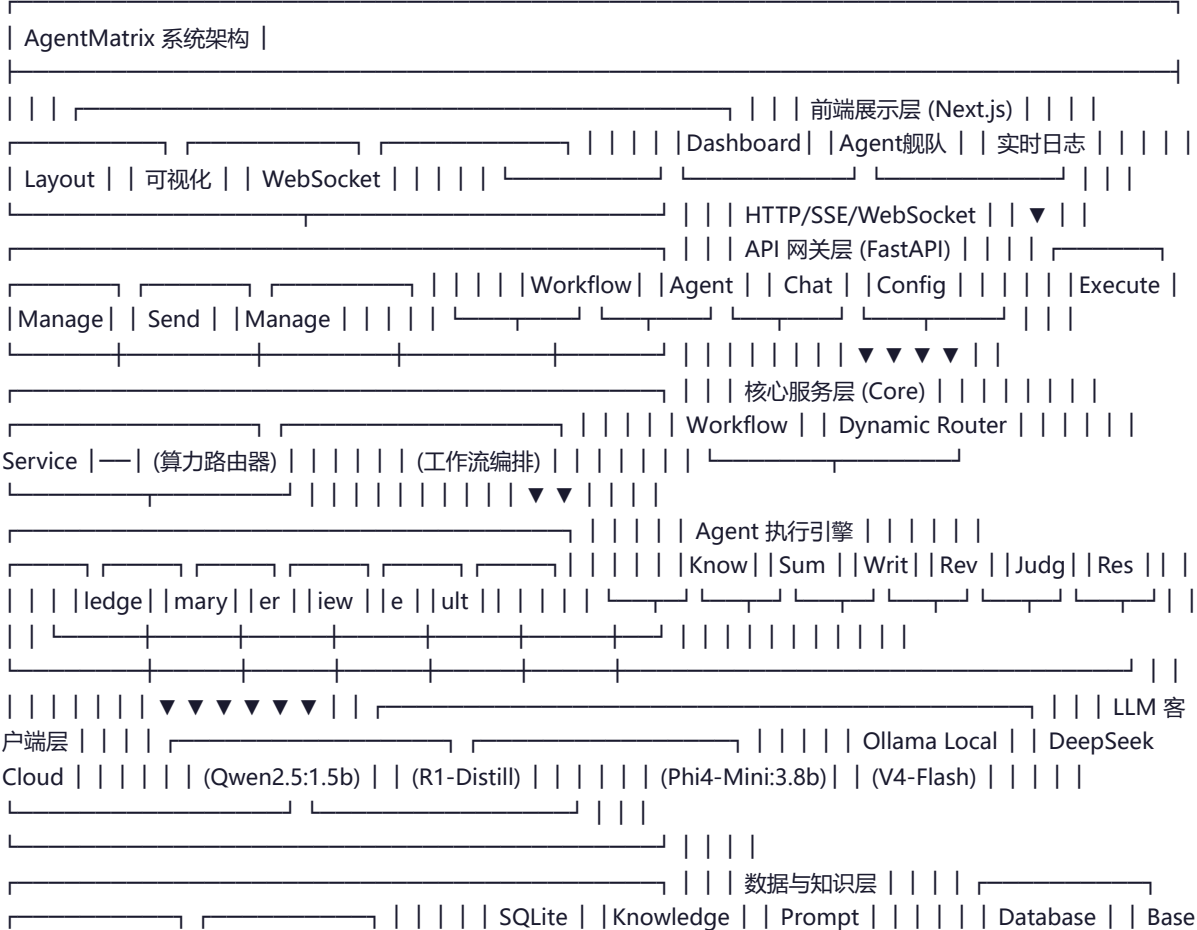
AgentMatrix 是一个基于多智能体协同架构与动态算力路由的 AI 应用平台。系统通过六个专业化 Agent 的流水线协作，结合 Judge Agent 的复杂度感知机制，实现任务的智能分流：简单任务由本地 Ollama 模型（Qwen2.5、Phi4-mini）高效处理，复杂任务自动升级至云端 DeepSeek 大模型进行增强推理，从而在保障输出质量的同时显著降低 API 调用成本。

1.1 核心设计目标

| 目标 | 实现策略 |
|-----|-----|
| **成本优化** | 80%+ 简单任务本地执行，仅复杂任务调用云端 API |
| **响应速度** | 本地模型毫秒级响应，避免网络延迟 |
| **质量保障** | 云端大模型兜底复杂场景，Review Agent 质量评审 |
| **国产算力** | 优先使用 Qwen2.5、DeepSeek 等国产模型 |
| **可扩展性** | 插件化 Agent 架构，支持模型热切换 |

二、系统架构

2.1 总体架构图
```



层级	组件	技术栈	职责
**展示层**   DashboardLayout   Next.js 14, React 18, TailwindCSS, Zustand, Framer Motion   用户交互界面、多Agent可视化、实时状态推送			
**网关层**   API Routes   FastAPI, Uvicorn, WebSocket, SSE   请求路由、参数校验、流式响应、WebSocket广播			
**服务层**   WorkflowService, DynamicRouter   Python asyncio   工作流编排、算力路由决策、Agent调度			
**执行层**   Six Agents   规则引擎 + LLM推理   知识检索→摘要→生成→评审→判断→导出			
**模型层**   LLMClient   Ollama API, DeepSeek API   本地模型推理、云端API调用、多服务商适配			
**数据层**   SQLite + JSON + Cache   SQLAlchemy, SimpleCache   持久化存储、知识库管理、Prompt模板、结果缓存			

### ### 3.2 意图识别流水线

```

graph LR
 subgraph Stage1 [第一阶段: 知识检索与上下文增强 Knowledge Agent]
 S1_1[关键词提取: "校园运动会", "策划方案", "预算", "风险防控"]
 S1_2[知识库匹配: 命中 "校园"、"规划"、"方案" 类别]
 S1_3[输出: 检索到的相关知识片段 + 知识命中标记]
 S1_4[模型: qwen2.5:1.5b Ollama 本地]
 end

 subgraph Stage2 [第二阶段: 需求摘要与结构提取 Summary Agent]
 S2_1[提取任务类型: "策划方案生成"]
 S2_2[识别关键需求: "运动会", "预算", "风险防控"]
 S2_3[结构化摘要: task_type: "planning", entities: [...], ...]
 S2_4[模型: qwen2.5:1.5b Ollama 本地]
 end

 subgraph Stage3 [第三阶段: 内容生成 Writer Agent]
 S3_1[基于摘要和知识库内容生成初稿]
 S3_2[输出长度: ~1500 字符的结构化方案]
 S3_3[模型: qwen2.5:1.5b Ollama 本地]
 end

 subgraph Stage4 [第四阶段: 质量评审 Review Agent]
 S4_1[对Writer输出进行多维度评分 0-1]
 S4_2[评估维度: 完整性、准确性、结构性、实用性]
 S4_3[输出: review_score 0.55 中等质量, 需增强]
 S4_4[模型: phi4-mini:3.8b Ollama 本地, 更高推理能力]
 end

 subgraph Stage5 [第五阶段: 复杂度判断 Judge Agent]
 S5_1[★核心★]
 S5_2[融合前四个Agent的所有输出进行综合判定]
 S5_3[详见下方 "Judge Agent 决策机制"]
 end

 Stage1 --> Stage2
 Stage2 --> Stage3
 Stage3 --> Stage4
 Stage4 --> Stage5

```

该图展示了生成校园运动会策划方案的五个阶段流程：

- 第一阶段: 知识检索与上下文增强 (Knowledge Agent)**
  - 关键词提取: ["校园运动会", "策划方案", "预算", "风险防控"]
  - 知识库匹配: 命中 "校园"、"规划"、"方案" 类别
  - 输出: 检索到的相关知识片段 + 知识命中标记
  - 模型: qwen2.5:1.5b (Ollama 本地)
- 第二阶段: 需求摘要与结构提取 (Summary Agent)**
  - 提取任务类型: "策划方案生成"
  - 识别关键需求: ["运动会", "预算", "风险防控"]
  - 结构化摘要: {task\_type: "planning", entities: [...], ...}
  - 模型: qwen2.5:1.5b (Ollama 本地)
- 第三阶段: 内容生成 (Writer Agent)**
  - 基于摘要和知识库内容生成初稿
  - 输出长度: ~1500 字符的结构化方案
  - 模型: qwen2.5:1.5b (Ollama 本地)
- 第四阶段: 质量评审 (Review Agent)**
  - 对Writer输出进行多维度评分 (0-1)
  - 评估维度: 完整性、准确性、结构性、实用性
  - 输出: review\_score 0.55 (中等质量, 需增强)
  - 模型: phi4-mini:3.8b (Ollama 本地, 更高推理能力)
- 第五阶段: 复杂度判断 (Judge Agent) ★核心★**
  - 融合前四个Agent的所有输出进行综合判定
  - 详见下方 "Judge Agent 决策机制"

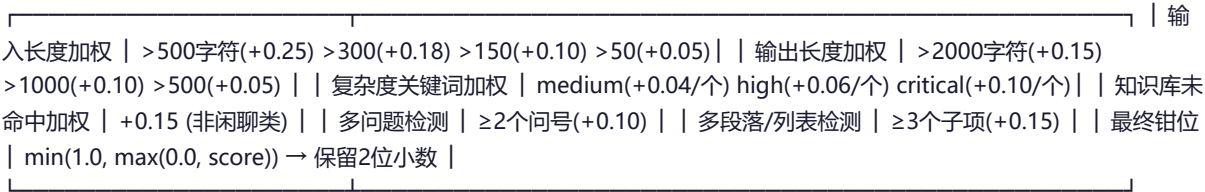
类别	基础复杂度	强制决策	典型示例
**greeting**	(问候)	0.10	`local_output`   "你好", "hi", "早上好"
**identity**	(身份询问)	0.12	`local_output`   "你是谁", "你叫什么名字"
**chitchat**	(闲聊)	0.15	`local_output`   "今天天气不错", "谢谢"
**simple_fact**	(简单事实)	0.25	无   "什么是AI", "苹果多少钱"
**knowledge_qa**	(知识问答)	0.45	无   "AI和ML有什么区别"
**howto**	(操作指南)	0.55	无   "怎么安装Python"
**creation**	(内容创作)	0.65	无   "帮我写一封情书"
**planning**	(策划规划)	0.75	无   "完整AI项目答辩方案与PPT"
**complex_task**	(复杂任务)	0.85	无   "完整AI项目答辩方案与PPT"

```
3.3.2 多维加权复杂度计算

Judge Agent 在基础复杂度之上，融合以下维度进行加权调节（见 [_calculate_complexity](file:///c:/Users/王森/Desktop/AgentMatrix/backend/agents/judge/agent.py#L165-L209)）：
```

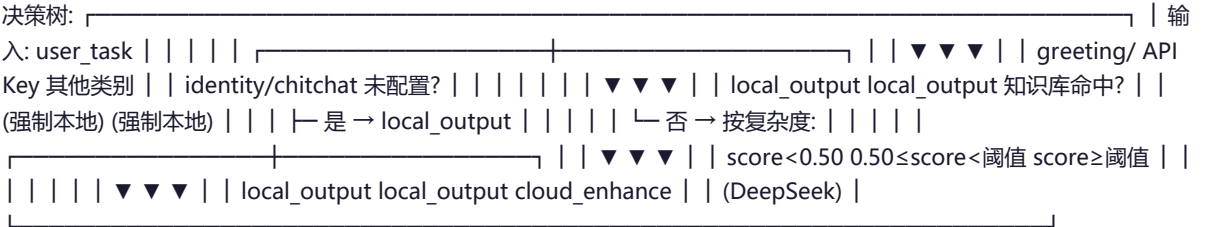
复杂度评分 = 基础复杂度 + 输入长度加权 + 输出长度加权 + 关键词加权 + 知识库加权 + 结构复杂度加权

详细加权规则:



```
3.3.3 决策矩阵（融合知识库命中状态）

最终决策融合三个关键因素（见 [_make_decision](file:///c:/Users/王森/Desktop/AgentMatrix/backend/agents/judge/agent.py#L245-L278)）：
```



关键阈值: complexity\_threshold = 0.65

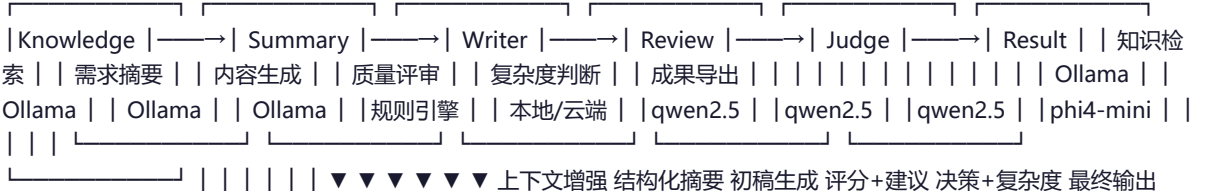
```
3.3.4 LLM增强判断（可选模式）

当规则引擎判断不确定时，系统支持调用本地 LLM（phi4-mini:3.8b）进行语义级复杂度判断（见 [_judge_complexity_with_llm](file:///c:/Users/王森/Desktop/AgentMatrix/backend/agents/judge/agent.py#L280-L345)），LLM 被要求严格遵循与规则引擎相同的决策矩阵，确保一致性。如果 LLM 输出解析失败，自动回退到规则引擎结果。

四、智能体动态编排与任务调度机制

4.1 六智能体协同流水线

AgentMatrix 采用固定顺序的流水线架构，六个 Agent 依次执行，每个 Agent 的输出成为下一个 Agent 的部分输入：
```



```
各 Agent 的输入构造（见 [WorkflowService.execute](file:///c:/Users/王森/Desktop/AgentMatrix/backend/core/workflow/service.py#L52-L91)）：
```

Agent	输入构造方式	特殊处理
**Knowledge**	原始用户输入	返回知识命中标记 `knowledge_found`
**Summary**	原始用户输入	提取任务类型和关键实体
**Writer**	原始用户输入 + 知识上下文	生成文本初稿
**Review**	用户任务 + Summary摘要 + Writer输出	多维度质量评分

```
| **Judge** | 用户任务 + Summary + Review + Writer + knowledge_found | **不使用LLM**，纯规则引擎决策 |
| **Result** | 用户任务 + 所有前序输出 + Judge决策 | 根据 `judge_decision` 决定是否调用云端 |

4.2 动态算力路由

DynamicRouter 是系统的算力调度核心（见 [core/dynamic_router/router.py](file:///c:/Users/王森/Desktop/AgentMatrix/backend/core/dynamic_router/router.py)），实现基于复杂度的智能路由：
```

```
 | | should_use_cloud(complexity_score): | | return complexity_score > settings.complexity_threshold | | # 阈值: 0.65
 | | | | route(prompt, complexity_score, agent_id): | | if use_cloud: | | → DeepSeekClient.call(prompt) | |
 | | → 返回云端结果 + "cloud"标记 | | else: | | → 返回空 + "local"标记 | | → 由Agent自身调用Ollama | | | |
 | | _select_local_model(agent_id): | | 不同Agent使用不同本地模型: | | • review → phi4-mini:3.8b (推理能力强) | | •
 | | writer/summary/knowledge/judge/result → qwen2.5:1.5b |
```

```
4.3 本地模型差异化分配

系统根据各 Agent 的任务特性，分配不同能力的本地模型：

| Agent | 本地模型 | 选型理由 |
|-----|-----|-----|
| Knowledge | qwen2.5:1.5b | 轻量文本检索，无需强推理 |
| Summary | qwen2.5:1.5b | 摘要提取，中等难度 |
| Writer | qwen2.5:1.5b | 内容生成需要流畅输出 |
| **Review** | **phi4-mini:3.8b** | 需要较强逻辑推理和评分能力 |
| Judge | qwen2.5:1.5b | 使用规则引擎，模型仅兜底 |
| Result | qwen2.5:1.5b | 格式化整合，低难度 |

4.4 工作流缓存策略

为实现高效的任务调度，系统内置了三层缓存机制：

| 缓存层 | 位置 | 容量 | TTL | 策略 |
|-----|-----|-----|-----|-----|
| **Workflow缓存** | WorkflowService | 100条 | 300秒 | 仅缓存 `executed_locally=true` 且 `final_result<5000字符` |
| **搜索缓存** | KnowledgeService | 500条 | 300秒 | 基于关键词的搜索结果缓存 |
| **聊天缓存** | ChatService | 200条 | 300秒 | 聊天结果缓存 |

4.5 实时状态推送（WebSocket）

系统通过 WebSocket 实现前后端实时通信（见 [api/websocket/manager.py](file:///c:/Users/王森/Desktop/AgentMatrix/backend/api/websocket/manager.py)），支持三种消息类型：

| 消息类型 | 方向 | 触发时机 | 数据结构 |
|-----|-----|-----|-----|
| `agent_status` | 后端→前端 | Agent状态变更 | `Dict[agent_id, AgentStatus]` |
| `workflow_step` | 后端→前端 | 每个Agent步骤完成 | `WorkflowStep` |
| `final_result` | 后端→前端 | 工作流全部完成 | `WorkflowOutput` |

前端通过 Socket.IO 客户端（[socketService.ts](file:///c:/Users/王森/Desktop/AgentMatrix/frontend/src/services/api/socketService.ts)）监听 7 种事件：
- `workflow:step_start` - Agent 开始执行
- `workflow:step_complete` - Agent 执行完成
- `workflow:step_error` - Agent 执行出错
- `workflow:complete` - 整个工作流完成
- `agent:status_update` - Agent 状态更新
- `metrics:update` - 指标数据更新
- `log:new` - 新日志产生

五、关键技术细节

5.1 分布式错误处理与容错

 | | Agent级错误隔离 | | | try: | |
 | | output = agent.execute(input) | | except Exception: | | agent.status = "error" | | 记录错误但不阻断流水线 | |
 | | continue # 下一个Agent继续执行 | | | 全局异常处理: | | • WorkflowService层面统一捕获 | | • 返回部分结果 + 错
```

### 5.2 成本优化模型

每次本地执行节省成本 ≈ 0.01 元 (vs 云端API调用)

月成本估算 (假设 10000次请求/月, 80%本地执行): 本地执行: 8000次 × 0.001元 = 8元 云端执行: 2000次 × 0.01元 = 20元 总成本: 28元/月

对比全云端方案: 10000次 × 0.01元 = 100元/月 成本节省率: 72%

### 5.3 前后端通信架构

通信方式对比									
					方式	HTTP/REST	SSE (流式)	WebSocket	
					用途	CRUD操作	实时输出流	状态推送	示例
					Agent状态查询	聊天流式响应	Agent状态	端点	/api/v1/*
					方向	请求-响应	单向推送	双向	协议
					stream	/chat/send/	/ws	stream	HTTP/1.1
					单向推送	双向	WS	HTTP/1.1	WS

### 5.4 Prompt 模板系统

每个 Agent 拥有专属的 Prompt 模板文件, 由 `PromptManager` 统一管理:

prompts/templates/ |—— knowledge/enhance.txt # 知识增强Prompt |—— summary/extract.txt # 需求提取Prompt |—— writer/generate.txt # 内容生成Prompt |—— review/review.txt # 质量评审Prompt |—— judge/complexity.txt # 复杂度判断Prompt (LLM模式) |—— result/format.txt # 结果格式化Prompt

模板支持 `{variable}` 占位符动态替换, 实现上下文注入。

### 5.5 安全与合规

安全措施	实现方式
**API密钥保护**	仅存储在后端 `.env` 文件, 不暴露给前端
**CORS限制**	开发环境限制为 `localhost:3000` 和 `localhost:8000`
**输入校验**	Pydantic 模型校验所有输入, 禁止空字符串
**SQL注入防护**	SQLAlchemy ORM 参数化查询
**日志脱敏**	API Key 在日志中截断显示 (仅前10字符)

---

## 六、技术栈总览

层次	技术	版本
**前端框架**	Next.js	14.1.0
**前端语言**	TypeScript	5.3.3
**UI框架**	React	18.2.0
**样式**	TailwindCSS	3.4.1
**状态管理**	Zustand	4.5.0
**动画**	Framer Motion	10.18.0
**图表**	Chart.js + react-chartjs-2	4.4.1
**流程图**	ReactFlow	11.10.0
**后端框架**	FastAPI	0.110+
**服务器**	Uvicorn	0.29+
**数据校验**	Pydantic	2.6+
**数据库ORM**	SQLAlchemy	2.0+
**HTTP客户端**	aiohttp	3.9+
**本地模型**	Ollama (Qwen2.5:1.5b, Phi4-mini:3.8b)	0.23.4

```
| **云端API** | DeepSeek (R1-Distill, V4-Flash) | API v1 |
| **数据库** | SQLite | - |
| **实时通信** | WebSocket (websockets) | 12.0+ |

七、总结与展望

AgentMatrix 通过六智能体协同流水线和 Judge Agent 的九分类规则引擎，实现了任务的智能分流和动态算力路由。系统在实际运行中表现出以下优势：

- **成本效益**：约 80% 的请求由本地模型处理，API 成本节省超 70%
- **响应速度**：简单任务本地执行，延迟 < 2 秒（vs 云端 5-10 秒）
- **质量保障**：复杂任务自动升级云端大模型，Review Agent 质量把关
- **可观测性**：WebSocket 实时推送 + 完整日志体系

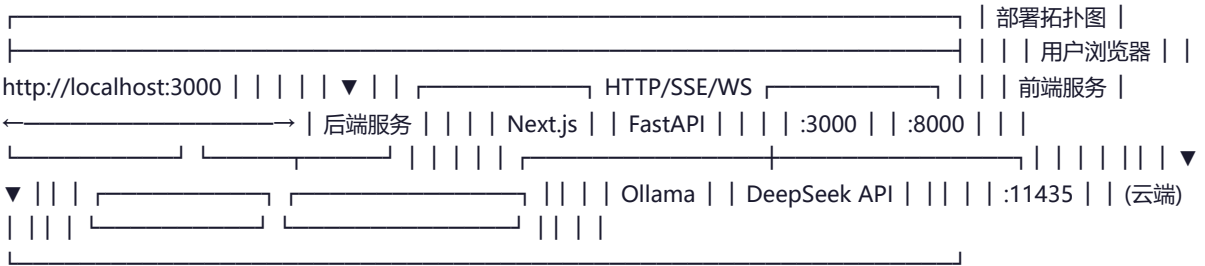
未来演进方向：
1. 知识库升级为向量数据库（FAISS/ChromaDB），支持语义检索
2. 引入更多国产模型（ChatGLM、Baichuan 等）
3. Agent 间引入并行执行和动态 DAG 编排
4. 支持多轮对话的上下文记忆管理
5. 生产级容器化部署（Docker + Kubernetes）
```

docs\AgentMatrix 部署文档.md

```
AgentMatrix 部署文档

> 多智能体动态协同与国产算力优化平台
> 版本: v0.1.0 | 日期: 2026-05-17

一、部署架构概览
```



```

二、环境要求

2.1 硬件要求

| 环境 | 最低配置 | 推荐配置 |
|-----|-----|-----|
| **开发环境** | CPU 4核, 内存 8GB, 磁盘 20GB | CPU 8核, 内存 16GB, 磁盘 50GB |
| **生产环境** | CPU 4核, 内存 16GB, 磁盘 50GB | CPU 8核, 内存 32GB, 磁盘 100GB |
| **GPU (可选)** | - | NVIDIA GPU 8GB+ VRAM (加速本地模型推理) |

2.2 软件要求

| 软件 | 最低版本 | 用途 |
|-----|-----|-----|
| **Python** | 3.12+ | 后端运行环境 |
| **Node.js** | 20.0+ | 前端运行环境 |
| **Ollama** | 0.23.0+ | 本地模型推理服务 |
| **Git** | 2.40+ | 代码版本管理 |

2.3 操作系统支持

| 操作系统 | 支持状态 | 备注 |
```

```
|-----|-----|-----|
| Windows 10/11 | |  完全支持 | PowerShell 5.1+ |
| macOS 12+ | |  完全支持 | 原生终端 |
| Ubuntu 22.04+ | |  完全支持 | Bash |
|-----|-----|-----|

三、本地模型部署 (Ollama)

3.1 安装 Ollama

Windows:
```powershell
# 1. 下载安装包
# https://ollama.com/download/windows

# 2. 安装完成后验证
ollama --version
```

Linux:

```
curl -fsSL https://ollama.com/install.sh | sh
```

macOS:

```
brew install ollama
```

3.2 配置 Ollama 端口

默认情况下，Ollama 监听 11434 端口。如果该端口被占用，可以通过环境变量修改：

```
# Linux/macOS
export OLLAMA_HOST="0.0.0.0:11435"
ollama serve

# Windows PowerShell
$env:OLLAMA_HOST="0.0.0.0:11435"
ollama serve
```

本项目的默认配置使用 11435 端口：

```
# backend/.env
OLLAMA_HOST=http://localhost:11435
```

3.3 下载本地模型

```
# 下载 Qwen2.5 1.5B 模型 (~986MB)
ollama pull qwen2.5:1.5b

# 下载 Phi-4 Mini 3.8B 模型 (~2.5GB)
ollama pull phi4-mini:3.8b

# 验证模型已安装
ollama list
```

预期输出：

NAME	ID	SIZE	MODIFIED
phi4-mini:3.8b	78fad5d182a7	2.5 GB	...
qwen2.5:1.5b	65ec06548149	986 MB	...

3.4 验证 Ollama 服务

```
# 测试 Ollama 是否正常运行
curl http://localhost:11435/api/tags

# 预期返回:
# {"models":[{"name":"qwen2.5:1.5b",...}, {"name":"phi4-mini:3.8b",...}]}
```

四、后端服务部署

4.1 获取代码

```
git clone <repository-url>
cd AgentMatrix
```

4.2 安装后端依赖

方式一：使用 pip 直接安装

```
cd backend

# 创建虚拟环境（推荐）
python -m venv venv

# 激活虚拟环境
# Windows:
venv\Scripts\activate
# Linux/macOS:
source venv/bin/activate

# 安装依赖
pip install -e .
```

方式二：使用 requirements 手动安装

```
cd backend
pip install fastapi>=0.110.0 uvicorn>=0.29.0 pydantic>=2.6.0 \
python-dotenv>=1.0.0 requests>=2.31.0 aiohttp>=3.9.0 \
sqlalchemy>=2.0.0 python-pptx>=0.6.23 python-docx>=1.1.0 \
pymdown-extensions>=10.0.0 ollama>=0.1.0 \
google-generativeai>=0.5.0 numpy>=1.26.0 scipy>=1.12.0 \
loguru>=0.7.0 websockets>=12.0 psutil>=5.9.0
```

4.3 配置环境变量

创建 `backend/.env` 文件：

```
# 服务配置
SERVER_HOST=0.0.0.0
SERVER_PORT=8000
SERVER_RELOAD=true

# 日志配置
LOG_LEVEL=INFO
LOG_FILE=logs/system.log

# 数据库配置
DATABASE_URL=sqlite:///./agentmatrix.db

# Ollama 配置
OLLAMA_HOST=http://localhost:11435
OLLAMA_MODEL=qwen2.5:1.5b
OLLAMA_REVIEW_MODEL=phi4-mini:3.8b

# DeepSeek API 配置（可选，不配置则仅使用本地模型）
DEEPSEEK_API_KEY=your_deepseek_api_key_here
```

```
DEEPSEEK_API_BASE=https://api.deepseek.com/v1
DEEPSEEK_MODEL=deepseek-r1-distill

# 复杂度阈值（0-1，超过此值调用云端）
COMPLEXITY_THRESHOLD=0.65

# CORS 配置
ALLOWED_ORIGINS=http://localhost:3000,http://localhost:8000
```

4.4 初始化数据库

```
cd backend

# 首次启动会自动创建数据库表
python -c "from app.database import init_db; import asyncio; asyncio.run(init_db())"
```

4.5 启动后端服务

```
cd backend

# 开发模式（支持热重载）
uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload

# 生产模式（多 worker）
uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
```

启动成功日志：

```
INFO: Started reloader process [xxxxx] using StatReload
INFO: Database tables created successfully
INFO: Database initialized successfully
INFO: Loaded knowledge base with 20 keywords
INFO: All agents initialized successfully
INFO: WebSocket manager initialized
INFO: Application startup complete.
```

4.6 验证后端服务

```
# 健康检查
curl http://localhost:8000/health

# 预期返回:
# {"status": "healthy", "agents": {...}, "version": "0.1.0"}

# 查看 API 文档
# 浏览器访问: http://localhost:8000/docs
```

五、前端服务部署

5.1 安装前端依赖

```
cd frontend
npm install
```

5.2 配置环境变量

创建 frontend/.env.local 文件：

```
# API 地址
NEXT_PUBLIC_API_URL=http://localhost:8000
```

```
# WebSocket 地址
NEXT_PUBLIC_WS_URL=ws://localhost:8000
```

5.3 启动前端服务

```
cd frontend

# 开发模式
npm run dev

# 生产构建
npm run build
npm start
```

启动成功日志:

```
▲ Next.js 14.1.0
- Local:      http://localhost:3000
- Environments: .env.local, .env
✓ Ready in 5.4s
```

5.4 验证前端服务

浏览器访问 `http://localhost:3000` , 应能看到 AgentMatrix 主界面。

六、一键启动脚本

6.1 Windows PowerShell 脚本

项目提供了 `start.ps1` 一键启动脚本:

```
# 在项目根目录运行
.\start.ps1
```

脚本会自动完成: 1. 检查 Python 和 Node.js 环境 2. 检查后端依赖安装情况 3. 检查前端依赖安装情况 4. 启动后端服务 (:8000) 5. 启动前端服务 (:3000)

6.2 Windows 批处理脚本

```
# 在项目根目录运行
.\start.bat
```

七、生产环境部署

7.1 使用 systemd (Linux)

后端服务:

```
# /etc/systemd/system/agentmatrix-backend.service
[Unit]
Description=AgentMatrix Backend Service
After=network.target ollama.service

[Service]
Type=simple
User=agentmatrix
```

```

WorkingDirectory=/opt/agentmatrix/backend
Environment=PATH=/opt/agentmatrix/backend/venv/bin:/usr/bin
ExecStart=/opt/agentmatrix/backend/venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --workers 4
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

```

前端服务:

```

# /etc/systemd/system/agentmatrix-frontend.service
[Unit]
Description=AgentMatrix Frontend Service
After=network.target agentmatrix-backend.service

[Service]
Type=simple
User=agentmatrix
WorkingDirectory=/opt/agentmatrix/frontend
ExecStart=/usr/bin/npm start
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target

```

```

# 启动服务
sudo systemctl daemon-reload
sudo systemctl enable agentmatrix-backend agentmatrix-frontend
sudo systemctl start agentmatrix-backend agentmatrix-frontend

# 查看状态
sudo systemctl status agentmatrix-backend
sudo systemctl status agentmatrix-frontend

```

7.2 使用 Docker (推荐)

Dockerfile - 后端:

```

# backend/Dockerfile
FROM python:3.12-slim

WORKDIR /app
COPY . .
RUN pip install -e .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

```

Dockerfile - 前端:

```

# frontend/Dockerfile
FROM node:20-alpine

WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
COPY . .
RUN npm run build

EXPOSE 3000
CMD ["npm", "start"]

```

docker-compose.yml:

```

version: '3.8'

services:

```

```

backend:
  build: ./backend
  ports:
    - "8000:8000"
  environment:
    - OLLAMA_HOST=http://ollama:11434
    - DEEPSEEK_API_KEY=${DEEPSEEK_API_KEY}
  volumes:
    - ./backend/data:/app/data
  depends_on:
    - ollama
  restart: unless-stopped

frontend:
  build: ./frontend
  ports:
    - "3000:3000"
  environment:
    - NEXT_PUBLIC_API_URL=http://backend:8000
    - NEXT_PUBLIC_WS_URL=ws://backend:8000
  depends_on:
    - backend
  restart: unless-stopped

ollama:
  image: ollama/ollama:latest
  ports:
    - "11435:11434"
  volumes:
    - ollama_data:/root/.ollama
  restart: unless-stopped

volumes:
  ollama_data:

```

```

# 启动全部服务
docker-compose up -d

# 下载模型到容器
docker-compose exec ollama ollama pull qwen2.5:1.5b
docker-compose exec ollama ollama pull phi4-mini:3.8b

```

八、Nginx 反向代理配置（生产环境）

```

server {
    listen 80;
    server_name agentmatrix.example.com;

    # 前端
    location / {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # 后端 API
    location /api/ {
        proxy_pass http://localhost:8000/api/;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # WebSocket
    location /ws {
        proxy_pass http://localhost:8000/ws;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
    }
}

```

```
    proxy_set_header Connection "upgrade";
    proxy_set_header Host $host;
}
}
```

九、监控与健康检查

9.1 健康检查端点

端点	方法	用途
/health	GET	系统整体健康状态 + Agent 状态
/api/v1/metrics	GET	系统运行指标
/api/v1/chat/health	GET	聊天服务状态
/api/v1/workflow/cache/stats	GET	工作流缓存统计

9.2 日志配置

```
# backend/.env
LOG_LEVEL=INFO           # 开发: DEBUG, 生产: INFO 或 WARNING
LOG_FILE=logs/system.log
```

日志文件位置: backend/logs/system.log

9.3 性能监控

系统指标 (GET /api/v1/metrics) 包含: - total_requests - 总请求数 - local_executions - 本地执行次数 - cloud_executions - 云端执行次数 - api_calls - API 调用次数 - cost_saved - 节省费用估算

十、常见问题排查

10.1 Ollama 连接失败

```
现象: 后端日志 "Failed to call Ollama: Connection refused"
解决:
1. 检查 Ollama 是否在运行: ollama list
2. 检查端口是否正确: 默认 11434, 本项目配置 11435
3. 检查环境变量: echo $env:OLLAMA_HOST (Windows)
4. 手动测试连接: curl http://localhost:11435/api/tags
```

10.2 端口被占用

```
现象: uvicorn 启动报 "Address already in use"
解决:
1. 查找占用进程: netstat -ano | findstr :8000
2. 修改端口: 在 .env 中设置 SERVER_PORT=8001
```

10.3 前端 API 请求 404

```
现象: 浏览器控制台显示 /api/v1/... 返回 404
解决:
1. 检查 .env.local 中 NEXT_PUBLIC_API_URL 是否正确
2. 确认后端服务正在运行
3. 检查后端是否在正确的端口上运行
```

10.4 DeepSeek API 调用失败

现象：返回 “Error: DeepSeek API Key 未设置”

解决：

1. 检查 backend/.env 中 DEEPSEEK_API_KEY 是否已设置

2. 确认 API Key 是否有效（未过期）

3. 可以在前端系统设置中配置 API Key

十一、安全注意事项

1. **API 密钥管理:** 不要将 `.env` 文件提交到版本控制。已加入 `.gitignore`。

2. **生产环境 CORS:** 修改 `ALLOWED_ORIGINS` 为实际前端域名，不要使用 `*`。

3. **防火墙:** 生产环境应配置防火墙，仅开放必要端口（80/443）。

4. **HTTPS:** 生产环境应使用 SSL/TLS 加密通信。

5. **定期更新:** 定期更新 Ollama 模型和后端依赖，获取安全补丁。

```
---

## frontend\.env.example

```text
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000
NEXT_PUBLIC_APP_NAME=AgentMatrix
NEXT_PUBLIC_APP_VERSION=0.1.0
```

## frontend\FE\_DEVELOPMENT\_SPEC.md

```
AgentMatrix 前端开发规范文档

一、项目概述

本文档定义了 AgentMatrix 前端项目的开发规范，确保代码质量、可维护性和与后端的正确对接。

二、技术栈规范

| 分类 | 技术 | 版本 | 说明 |
|-----|-----|-----|-----|
| 框架 | Next.js | 14.1.0 | App Router 模式 |
| 语言 | TypeScript | 5.3.3 | 严格类型检查 |
| 样式 | TailwindCSS | 3.4.1 | 原子化CSS |
| 状态管理 | Zustand | ^4.5.0 | 轻量级状态管理 |
| HTTP请求 | Axios | ^1.6.5 | HTTP客户端 |
| WebSocket | Socket.IO Client | ^4.6.1 | 实时通信 |
| 图标 | Lucide React | ^0.323.0 | 图标库 |
| 图表 | Chart.js + react-chartjs-2 | ^4.4.1 | 数据可视化 |
| 动画 | Framer Motion | ^10.18.0 | 动画库 |
| 流程图 | React Flow | ^11.10.0 | 工作流可视化 |

三、API配置规范

3.1 基础地址

```typescript
// REST API基础地址
const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000';

// WebSocket地址
const WS_URL = process.env.NEXT_PUBLIC_WS_URL || 'ws://localhost:8000';
```

```
// API版本前缀
const API_VERSION = '/api/v1';
```

3.2 完整API端点清单

API路径	HTTP方法	功能描述	请求体	响应体
/health	GET	健康检查	无	{status, agents, version}
/workflow/execute	POST	执行工作流（串行）	WorkflowInput	WorkflowOutput
/workflow/execute/parallel	POST	执行工作流（并行）	WorkflowInput	WorkflowOutput
/agents	GET	获取所有Agent状态	无	Agent状态列表
/agents/{id}	GET	获取单个Agent状态	无	Agent状态对象
/metrics	GET	获取系统指标	无	指标数据
/knowledge	GET	查询知识库	无	知识库列表
/knowledge	POST	添加知识	{content, tags}	知识对象
/export	POST	导出结果	{content, format}	导出文件

四、数据模型规范

4.1 WorkflowInput（工作流输入）

```
interface WorkflowInput {
  user_input: string;           // 用户输入内容（必填）
  context?: Record<string, unknown>; // 上下文信息（可选）
}
```

4.2 WorkflowStep（工作流步骤）

```
interface WorkflowStep {
  agent_id: string;           // Agent ID
  agent_name: string;         // Agent名称
  input: string;              // 输入内容
  output: string;             // 输出内容
  success: boolean;           // 是否成功
  duration_seconds: number;    // 执行耗时（秒）
  timestamp: string;          // 时间戳（ISO格式）
  metadata?: Record<string, unknown>; // 元数据
}
```

4.3 WorkflowOutput（工作流输出）

```
interface WorkflowOutput {
  final_result: string;       // 最终结果
  steps: WorkflowStep[];      // 执行步骤列表
  executed_locally: boolean;   // 是否本地执行
  total_duration_seconds: number; // 总耗时（秒）
  start_time: string;         // 开始时间
  end_time: string;           // 结束时间
  complexity_score?: number;  // 复杂度评分（0-1）
}
```

4.4 AgentStatus（Agent状态）

```
interface AgentStatus {
  agent_id: string;           // Agent ID
  name: string;               // Agent名称
  status: 'idle' | 'ready' | 'running' | 'shutdown'; // 状态
}
```

```
current_task?: string;           // 当前任务
last_error?: string;            // 最后错误
}
```

4.5 MetricsData (指标数据)

```
interface MetricsData {
  total_requests: number;        // 总请求数
  local_executions: number;      // 本地执行次数
  cloud_executions: number;      // 云端执行次数
  api_calls: number;            // API调用次数
  cost_saved: number;           // 节省成本
  avg_response_time: number;     // 平均响应时间
}
```

4.6 ChatMessage (聊天消息)

```
interface ChatMessage {
  id?: string;                   // 消息ID
  role: 'user' | 'assistant' | 'system'; // 角色
  content: string;               // 消息内容
  timestamp?: number;            // 时间戳
  metadata?: Record<string, unknown>; // 元数据
}
```

五、六大Agent定义

```
const AGENTS: Record<string, string> = {
  knowledge: 'Knowledge Agent', // 知识检索 - 从知识库检索相关信息
  summary: 'Summary Agent',     // 需求摘要 - 提取用户核心需求
  writer: 'Writer Agent',       // 内容生成 - 根据需求生成内容
  review: 'Review Agent',       // 质量评审 - 审核内容质量
  judge: 'Judge Agent',        // 复杂度判断 - 判断任务复杂度
  result: 'Result Agent'       // 成果导出 - 输出最终结果
};

// Agent执行顺序
const AGENT_EXECUTION_ORDER = ['knowledge', 'summary', 'writer', 'review', 'judge', 'result'];
```

六、目录结构规范

```
src/
├── app/                                # Next.js App Router
│   ├── layout.tsx                     # 根布局组件
│   ├── page.tsx                       # 首页
│   └── globals.css                    # 全局样式
├── components/                         # UI组件
│   ├── layout/                        # 布局组件
│   │   └── DashboardLayout/           # 仪表盘布局
│   ├── workflow/                     # 工作流组件
│   │   └── WorkflowCanvas/            # 工作流画布
│   ├── logs/                         # 日志组件
│   │   └── LogViewer/                 # 日志查看器
│   ├── result/                       # 结果组件
│   │   └── ResultPreview/             # 结果预览
│   └── common/                       # 通用组件
├── services/                          # 服务层
│   └── api/                           # API服务封装
│       ├── agentService.ts           # Agent相关API
│       ├── workflowService.ts        # 工作流相关API
│       ├── metricsService.ts         # 指标相关API
│       └── knowledgeService.ts       # 知识库相关API
└── stores/                            # Zustand状态管理
```

		agentStore.ts	# Agent状态
		workflowStore.ts	# workflow状态
		metricsStore.ts	# 指标数据
		types/	# TypeScript类型定义
		index.ts	# 全局类型导出
		utils/	# 工具函数
		apiClient.ts	# Axios配置
		formatters.ts	# 格式化工具

七、开发规范

7.1 代码规范

```
# 代码检查
npm run lint

# 自动修复
npm run lint:fix

# 代码格式化
npm run format

# TypeScript类型检查
npm run typecheck
```

7.2 API调用规范

HTTP请求 - 使用封装的Axios客户端:

```
import apiClient from '@utils/apiClient';

// POST请求示例
const response = await apiClient.post('/workflow/execute', {
  user_input: '用户输入内容',
  context: {}
});

// GET请求示例
const response = await apiClient.get('/agents');
```

WebSocket连接 - 使用Socket.IO:

```
import { io, Socket } from 'socket.io-client';

const socket = io(WS_URL);

socket.on('workflow_update', (data) => {
  // 处理实时更新
});
```

7.3 状态管理规范

使用Zustand定义store:

```
import { create } from 'zustand';

interface WorkflowStore {
  steps: WorkflowStep[];
  isRunning: boolean;
  setSteps: (steps: WorkflowStep[]) => void;
  setIsRunning: (running: boolean) => void;
}

const useWorkflowStore = create<WorkflowStore>((set) => ({
  steps: [],
```

```
isRunning: false,
setSteps: (steps) => set({ steps }),
setIsRunning: (running) => set({ isRunning: running })
}));
```

7.4 错误处理规范

```
try {
  const response = await apiClient.post('/workflow/execute', payload);
  return response.data;
} catch (error) {
  if (axios.isAxiosError(error)) {
    // 处理HTTP错误
    console.error('API Error:', error.response?.data || error.message);
    throw new Error(error.response?.data?.detail || '请求失败');
  }
  // 处理其他错误
  console.error('Unexpected error:', error);
  throw new Error('发生未知错误');
}
```

八、Judge Agent特殊说明

Judge Agent的metadata包含关键决策信息：

```
// Judge步骤的metadata示例
{
  executed_locally: true,    // true=本地执行, false=云端执行
  complexity_score: 0.72    // 复杂度评分(0-1)
}

// 复杂度阈值配置
const COMPLEXITY_THRESHOLD = 0.65;

// 判断逻辑
const shouldUseCloud = complexity_score >= COMPLEXITY_THRESHOLD;
```

九、环境配置规范

环境变量

```
# .env文件配置
NEXT_PUBLIC_API_URL=http://localhost:8000
NEXT_PUBLIC_WS_URL=ws://localhost:8000
```

开发服务器

```
# 启动开发服务器（端口3000）
npm run dev

# 指定端口
npm run dev -- -p 3001

# 生产构建
npm run build

# 生产运行
npm run start
```

十、对接要点总结

- 1. 严格遵循数据模型定义，确保请求和响应格式与后端一致
- 2. 使用封装的API服务，统一错误处理和请求配置
- 3. Agent ID必须为：knowledge, summary, writer, review, judge, result
- 4. 处理长耗时请求：显示加载状态、实现超时处理
- 5. Judge步骤的metadata包含执行方式和复杂度评分
- 6. WebSocket用于实时更新： workflow状态、日志等

附录： 示例API调用

执行工作流

```
import apiClient from '@utils/apiClient';
import type { WorkflowInput, WorkflowOutput } from '@types';

async function executeWorkflow(userInput: string): Promise<WorkflowOutput> {
  const payload: WorkflowInput = {
    user_input: userInput,
    context: {}
  };

  const response = await apiClient.post<WorkflowOutput>('workflow/execute', payload);

  return response.data;
}
```

获取Agent状态

```
import apiClient from '@utils/apiClient';
import type { AgentStatus } from '@types';

async function getAgentStatuses(): Promise<AgentStatus[]> {
  const response = await apiClient.get('/agents');
  return response.data;
}
```

文档版本: v1.0
创建日期: 2026-05-13
适用项目: AgentMatrix Frontend

```
---
## frontend\next-env.d.ts

```typescript
/// <reference types="next" />
/// <reference types="next/image-types/global" />

// NOTE: This file should not be edited
// see https://nextjs.org/docs/basic-features/typescript for more information.
```

## frontend\next.config.js

```

/** @type {import('next').NextConfig} */
const nextConfig = {
 reactStrictMode: true,
 swcMinify: true,
 env: {
 NEXT_PUBLIC_API_URL: process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000',
 NEXT_PUBLIC_WS_URL: process.env.NEXT_PUBLIC_WS_URL || 'ws://localhost:8000',
 },
 images: {
 domains: ['localhost', '127.0.0.1'],
 deviceSizes: [640, 750, 828, 1080, 1200, 1920, 2048, 3840],
 imageSizes: [16, 32, 48, 64, 96, 128, 256, 384],
 },
 async headers() {
 return [
 {
 source: '/api/:path*',
 headers: [
 { key: 'Access-Control-Allow-Credentials', value: 'true' },
 { key: 'Access-Control-Allow-Origin', value: '*' },
 { key: 'Access-Control-Allow-Methods', value: 'GET, OPTIONS, PATCH, DELETE, POST, PUT' },
 { key: 'Access-Control-Allow-Headers', value: 'X-CSRF-Token, X-Requested-With, Accept, Accept-Version, Content-Length, Content-MD5, Content-Type, Date, X-API-Version' },
],
 },
];
 },
};

module.exports = nextConfig;

```

## frontend\package.json

```

{
 "name": "agentmatrix-frontend",
 "version": "0.1.0",
 "description": "多智能体动态协同与国产算力优化平台 - 前端界面",
 "private": true,
 "scripts": {
 "dev": "next dev",
 "build": "next build",
 "start": "next start",
 "lint": "next lint",
 "lint:fix": "next lint --fix",
 "format": "prettier --write .",
 "typecheck": "tsc --noEmit"
 },
 "dependencies": {
 "@types/node": "20.11.0",
 "@types/react": "18.2.48",
 "@types/react-dom": "18.2.18",
 "next": "14.1.0",
 "react": "18.2.0",
 "react-dom": "18.2.0",
 "typescript": "5.3.3",
 "tailwindcss": "3.4.1",
 "postcss": "8.4.33",
 "autoprefixer": "10.4.17",
 "lucide-react": "^0.323.0",
 "zustand": "^4.5.0",
 "axios": "^1.6.5",
 "socket.io-client": "^4.6.1",
 "chart.js": "^4.4.1",
 "react-chartjs-2": "^5.2.0",
 "framer-motion": "^10.18.0",
 "reactflow": "^11.10.0"
 },
 "devDependencies": {
 "@typescript-eslint/eslint-plugin": "^6.19.0",
 "@typescript-eslint/parser": "^6.19.0",
 "eslint": "^8.56.0",
 "eslint-config-next": "14.1.0",
 "eslint-config-prettier": "^9.1.0",
 }
}

```

```

 "eslint-plugin-import": "^2.29.1",
 "eslint-plugin-jsx-ally": "^6.8.0",
 "eslint-plugin-react": "^7.33.2",
 "eslint-plugin-react-hooks": "^4.6.0",
 "prettier": "^3.2.4"
 }
}

```

## frontend\src\app\globals.css

```

@tailwind base;
@tailwind components;
@tailwind utilities;

@import url('https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700&display=swap');

:root {
 --bg-primary: #f1f5f9;
 --bg-secondary: #ffffff;
 --bg-card: #ffffff;
 --bg-input: #f1f5f9;
 --bg-hover: #f8fafa;
 --blue: #3b82f6;
 --green: #10b981;
 --orange: #f59e0b;
 --purple: #8b5cf6;
 --red: #ef4444;
 --text-primary: #0f172a;
 --text-secondary: #475569;
 --text-muted: #94a3b8;
 --border-color: #e2e8f0;
 --shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.08);
 --node-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
 --card-shadow: 0 2px 8px rgba(0, 0, 0, 0.06);
 --scrollbar-track: #f1f5f9;
 --scrollbar-thumb: #cbd5e1;
 --scrollbar-thumb-hover: #94a3b8;
 --log-bg: #f8fafa;
 --log-bg-hover: #f1f5f9;
 --answer-bg: #f1f5f9;
 --answer-border: #e2e8f0;
 --answer-text: #334155;
 --answer-heading: #0f172a;
 --stat-bg: #f1f5f9;
 --stat-border: #e2e8f0;
 --stat-label: #64748b;
 --stat-value: #0f172a;
 --branch-bg: rgba(255, 255, 255, 0.9);
 --branch-border: #e2e8f0;
 --branch-text: #475569;
 --task-bar-bg: #f1f5f9;
 --task-bar-border: #e2e8f0;
 --btn-secondary-hover: #f8fafa;
 --btn-icon-hover: #f8fafa;
 --agent-item-hover: rgba(241, 245, 249, 0.5);
 --node-text: #0f172a;
 --node-subtitle: #475569;
 --node-status-completed-bg: rgba(16, 185, 129, 0.1);
 --node-status-completed-text: #059669;
 --node-status-working-bg: rgba(59, 130, 246, 0.1);
 --node-status-working-text: #2563eb;
 --node-status-pending-bg: rgba(100, 116, 139, 0.1);
 --node-status-pending-text: #64748b;
 --grid-line: rgba(59, 130, 246, 0.04);
 --arrow-color: #94a3b8;
 --difficulty-bg: rgba(245, 158, 11, 0.08);
 --difficulty-border: rgba(245, 158, 11, 0.2);
 --modal-overlay: rgba(0, 0, 0, 0.3);
 --modal-shadow: 0 16px 48px rgba(0, 0, 0, 0.1);
}

.dark {
 --bg-primary: #0f172a;
 --bg-secondary: #1e293b;

```

```

--bg-card: #1e293b;
--bg-input: #0f172a;
--bg-hover: #1a2332;
--blue: #3b82f6;
--green: #10b981;
--orange: #f59e0b;
--purple: #8b5cf6;
--red: #ef4444;
--text-primary: #ffffff;
--text-secondary: #94a3b8;
--text-muted: #64748b;
--border-color: #334155;
--shadow: 0 4px 6px -1px rgba(0, 0, 0, 0.3);
--node-shadow: 0 8px 16px rgba(0, 0, 0, 0.3);
--card-shadow: 0 4px 12px rgba(0, 0, 0, 0.2);
--scrollbar-track: #1e293b;
--scrollbar-thumb: #475569;
--scrollbar-thumb-hover: #64748b;
--log-bg: rgba(30, 41, 59, 0.5);
--log-bg-hover: rgba(30, 41, 59, 0.8);
--answer-bg: rgba(30, 41, 59, 0.9);
--answer-border: #334155;
--answer-text: #cbd5e1;
--answer-heading: #ffffff;
--stat-bg: rgba(15, 23, 42, 0.8);
--stat-border: #334155;
--stat-label: #94a3b8;
--stat-value: #ffffff;
--branch-bg: rgba(15, 23, 42, 0.9);
--branch-border: #334155;
--branch-text: #cbd5e1;
--task-bar-bg: #000000;
--task-bar-border: #334155;
--btn-secondary-hover: #1a2332;
--btn-icon-hover: #1a2332;
--agent-item-hover: rgba(30, 41, 59, 0.5);
--node-text: #ffffff;
--node-subtitle: #cbd5e1;
--node-status-completed-bg: rgba(16, 185, 129, 0.2);
--node-status-completed-text: #4ade80;
--node-status-working-bg: rgba(59, 130, 246, 0.2);
--node-status-working-text: #60a5fa;
--node-status-pending-bg: rgba(100, 116, 139, 0.2);
--node-status-pending-text: #94a3b8;
--grid-line: rgba(59, 130, 246, 0.03);
--arrow-color: #64748b;
--difficulty-bg: rgba(245, 158, 11, 0.15);
--difficulty-border: rgba(245, 158, 11, 0.3);
--modal-overlay: rgba(0, 0, 0, 0.6);
--modal-shadow: 0 16px 48px rgba(0, 0, 0, 0.4);
}

* {
 margin: 0;
 padding: 0;
 box-sizing: border-box;
}

body {
 font-family: 'Inter', sans-serif;
 background-color: var(--bg-primary);
 color: var(--text-primary);
 min-height: 100vh;
 overflow-x: hidden;
 transition:
 background-color 0.3s ease,
 color 0.3s ease;
 word-break: normal;
 overflow-wrap: break-word;
}

body::before {
 content: '';
 position: fixed;
 top: 0;
 left: 0;
 width: 100%;
 height: 100%;
 background-image:
 linear-gradient(var(--grid-line) 1px, transparent 1px),

```

```

 linear-gradient(90deg, var(--grid-line) 1px, transparent 1px);
background-size: 40px 40px;
pointer-events: none;
z-index: 0;
}

.header {
background: var(--bg-secondary);
border-bottom: 1px solid var(--border-color);
padding: 12px 24px;
display: flex;
align-items: center;
justify-content: space-between;
position: sticky;
top: 0;
z-index: 100;
transition:
 background-color 0.3s ease,
 border-color 0.3s ease;
}

.header-left {
display: flex;
align-items: center;
gap: 16px;
flex: 1;
}

.logo {
display: flex;
align-items: center;
gap: 8px;
flex-shrink: 0;
}

.logo-hexagon {
width: 32px;
height: 32px;
background: var(--blue);
clip-path: polygon(50% 0%, 100% 25%, 100% 75%, 50% 100%, 0% 75%, 0% 25%);
display: flex;
align-items: center;
justify-content: center;
}

.logo-hexagon::after {
content: 'N';
color: white;
font-weight: 700;
font-size: 16px;
}

.logo-text {
font-weight: 700;
font-size: 18px;
color: var(--text-primary);
}

.logo-subtitle {
font-size: 11px;
color: var(--text-secondary);
margin-left: 4px;
}

.task-bar {
background: var(--task-bar-bg);
padding: 8px 16px;
border-radius: 8px;
display: flex;
align-items: center;
gap: 12px;
flex: 1;
max-width: 500px;
border: 1px solid var(--task-bar-border);
}

.task-label {
color: var(--text-secondary);
font-size: 13px;
white-space: nowrap;
}

```

```

}

.task-text {
 color: var(--text-primary);
 font-size: 13px;
 overflow: hidden;
 text-overflow: ellipsis;
 white-space: nowrap;
}

.task-input {
 flex: 1;
 background: transparent;
 border: none;
 outline: none;
 color: var(--text-primary);
 font-size: 13px;
 font-family: 'Inter', sans-serif;
}

.task-input::placeholder {
 color: var(--text-muted);
}

.task-status {
 display: flex;
 align-items: center;
 gap: 8px;
 flex-shrink: 0;
}

.status-dot {
 width: 8px;
 height: 8px;
 background: var(--blue);
 border-radius: 50%;
 animation: pulse 2s infinite;
}

@keyframes pulse {
 0%,
 100% {
 opacity: 1;
 }
 50% {
 opacity: 0.5;
 }
}

.status-text {
 color: var(--blue);
 font-size: 13px;
 font-weight: 500;
}

.timer {
 color: var(--blue);
 font-size: 14px;
 font-weight: 600;
 font-variant-numeric: tabular-nums;
}

.header-right {
 display: flex;
 align-items: center;
 gap: 8px;
}

.btn {
 padding: 8px 16px;
 border-radius: 8px;
 border: none;
 font-size: 13px;
 font-weight: 500;
 cursor: pointer;
 display: flex;
 align-items: center;
 gap: 6px;
 transition: all 0.2s ease;
 user-select: none;
}

```

```

}

.btn-primary {
 background: var(--blue);
 color: white;
 transition: all 0.2s ease;
}

.btn-primary:hover {
 background: #2563eb;
 filter: brightness(1.1);
 transform: translateY(-1px);
 box-shadow: 0 4px 12px rgba(59, 130, 246, 0.3);
}

.btn-primary:active {
 transform: translateY(0);
 filter: brightness(0.95);
 box-shadow: none;
}

.btn-primary:disabled {
 opacity: 0.5;
 cursor: not-allowed;
 transform: none;
 filter: none;
 box-shadow: none;
}

.btn-primary.running {
 background: var(--red);
}

.btn-primary.running:hover {
 background: #dc2626;
 box-shadow: 0 4px 12px rgba(239, 68, 68, 0.3);
}

.btn-secondary {
 background: var(--bg-primary);
 color: var(--text-primary);
 border: 1px solid var(--border-color);
 transition: all 0.2s ease;
}

.btn-secondary:hover {
 background: var(--btn-secondary-hover);
 border-color: var(--text-muted);
 transform: translateY(-1px);
}

.btn-secondary:active {
 transform: translateY(0);
}

.btn-secondary:disabled {
 opacity: 0.5;
 cursor: not-allowed;
 transform: none;
}

.btn-icon {
 width: 36px;
 height: 36px;
 padding: 0;
 display: flex;
 align-items: center;
 justify-content: center;
 background: var(--bg-primary);
 border: 1px solid var(--border-color);
 border-radius: 8px;
 color: var(--text-primary);
 cursor: pointer;
 transition: all 0.2s ease;
}

.btn-icon:hover {
 background: var(--btn-icon-hover);
 border-color: var(--text-muted);
 transform: translateY(-1px);
}

```

```

}

.btn-icon:active {
 transform: translateY(0);
}

.main-container {
 display: flex;
 height: calc(100vh - 57px);
 position: relative;
 z-index: 1;
}

.sidebar-left {
 width: 280px;
 min-width: 280px;
 background: var(--bg-secondary);
 border-right: 1px solid var(--border-color);
 overflow-y: auto;
 padding: 16px;
 transition:
 background-color 0.3s ease,
 border-color 0.3s ease;
}

.sidebar-left::-webkit-scrollbar {
 width: 4px;
}

.sidebar-left::-webkit-scrollbar-track {
 background: transparent;
}

.sidebar-left::-webkit-scrollbar-thumb {
 background: var(--border-color);
 border-radius: 2px;
}

.card {
 background: var(--bg-card);
 border: 1px solid var(--border-color);
 border-radius: 12px;
 padding: 16px;
 margin-bottom: 16px;
 transition: all 0.25s ease;
 box-shadow: var(--card-shadow);
}

.card:hover {
 transform: translateY(-2px);
 box-shadow: var(--node-shadow);
 border-color: var(--text-muted);
}

.card-title {
 font-size: 12px;
 color: var(--text-secondary);
 font-weight: 500;
 text-transform: uppercase;
 letter-spacing: 0.5px;
 margin-bottom: 8px;
}

.card-value {
 font-size: 24px;
 font-weight: 700;
 color: var(--text-primary);
 margin-bottom: 4px;
}

.card-value.green {
 color: var(--green);
}

.card-info {
 display: flex;
 align-items: center;
 justify-content: space-between;
 font-size: 11px;
 color: var(--text-muted);

```

```

}

.card-info.highlight {
 color: var(--blue);
 font-weight: 500;
}

.card-info.highlight.green {
 color: var(--green);
}

.progress-ring-container {
 display: flex;
 align-items: center;
 gap: 12px;
 margin-top: 10px;
}

.progress-ring {
 width: 48px;
 height: 48px;
 position: relative;
}

.progress-ring svg {
 transform: rotate(-90deg);
}

.progress-ring-bg {
 fill: none;
 stroke: var(--bg-primary);
 stroke-width: 4;
}

.progress-ring-fill {
 fill: none;
 stroke-width: 4;
 stroke-linecap: round;
 transition: stroke-dashoffset 0.5s ease;
}

.progress-ring-text {
 position: absolute;
 top: 50%;
 left: 50%;
 transform: translate(-50%, -50%);
 font-size: 11px;
 font-weight: 600;
}

.progress-bar {
 flex: 1;
 height: 6px;
 background: var(--bg-primary);
 border-radius: 9999px;
 overflow: hidden;
}

.progress-bar-fill {
 height: 100%;
 border-radius: 9999px;
 transition: width 0.5s ease;
 position: relative;
}

.progress-bar-fill::after {
 content: '';
 position: absolute;
 top: 0;
 left: 0;
 right: 0;
 bottom: 0;
 background: linear-gradient(90deg, transparent, rgba(255, 255, 255, 0.2), transparent);
 animation: shimmer 2s infinite;
}

@keyframes shimmer {
 0% {
 transform: translateX(-100%);
 }
}

```

```

 100% {
 transform: translateX(100%);
 }
 }

 .stats-row {
 display: flex;
 gap: 8px;
 margin-top: 8px;
 }

 .stat-item {
 flex: 1;
 background: var(--bg-primary);
 padding: 10px;
 border-radius: 8px;
 text-align: left;
 }

 .stat-label {
 font-size: 10px;
 color: var(--text-muted);
 margin-bottom: 4px;
 line-height: 1.25;
 }

 .stat-value {
 font-size: 14px;
 font-weight: 600;
 color: var(--text-primary);
 line-height: 1.25;
 }

 .agent-fleet-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 margin-bottom: 12px;
 }

 .agent-fleet-title {
 font-size: 12px;
 color: var(--text-secondary);
 font-weight: 500;
 text-transform: uppercase;
 letter-spacing: 0.5px;
 }

 .agent-fleet-status {
 font-size: 11px;
 color: var(--blue);
 font-weight: 500;
 }

 .agent-item {
 display: flex;
 align-items: center;
 gap: 10px;
 padding: 16px;
 background: var(--bg-primary);
 border-radius: 12px;
 margin-bottom: 8px;
 border: 1px solid transparent;
 transition: all 0.25s ease;
 cursor: default;
 min-height: 112px;
 }

 .agent-item:hover {
 border-color: var(--border-color);
 background: var(--agent-item-hover);
 }

 .agent-item.disabled-agent {
 opacity: 0.5;
 }

 .agent-icon {
 width: 36px;
 height: 36px;

```

```

border-radius: 8px;
display: flex;
align-items: center;
justify-content: center;
font-size: 16px;
flex-shrink: 0;
}

.agent-icon.blue {
background: rgba(59, 130, 246, 0.15);
color: var(--blue);
}

.agent-icon.gold {
background: rgba(245, 158, 11, 0.15);
color: var(--orange);
}

.agent-icon.purple {
background: rgba(139, 92, 246, 0.15);
color: var(--purple);
}

.agent-icon.emerald {
background: rgba(16, 185, 129, 0.15);
color: #10b981;
}

.agent-icon.orange {
background: rgba(249, 115, 22, 0.15);
color: #f97316;
}

.agent-icon.violet {
background: rgba(139, 92, 246, 0.15);
color: #8b5cf6;
}

.agent-info {
flex: 1;
min-width: 0;
}

.agent-name {
font-size: 14px;
font-weight: 600;
color: var(--text-primary);
margin-bottom: 2px;
white-space: nowrap;
overflow: hidden;
text-overflow: ellipsis;
line-height: 1.25;
}

.agent-model {
font-size: 11px;
color: var(--text-muted);
margin-bottom: 2px;
white-space: nowrap;
}

.agent-desc {
font-size: 11px;
color: var(--text-secondary);
display: -webkit-box;
-webkit-line-clamp: 2;
-webkit-box-orient: vertical;
overflow: hidden;
line-height: 1.625;
}

.agent-status {
display: flex;
align-items: center;
gap: 8px;
margin-left: auto;
flex-shrink: 0;
}

.status-badge {

```

```
padding: 3px 8px;
border-radius: 6px;
font-size: 10px;
font-weight: 500;
display: inline-flex;
align-items: center;
gap: 4px;
}

.status-badge.completed {
 background: rgba(16, 185, 129, 0.15);
 color: var(--green);
}

.status-badge.working {
 background: rgba(59, 130, 246, 0.15);
 color: var(--blue);
 animation: pulse 2s infinite;
}

.status-badge.loading {
 background: rgba(245, 158, 11, 0.15);
 color: var(--orange);
}

.status-badge.idle {
 background: rgba(100, 116, 139, 0.15);
 color: var(--text-muted);
}

.toggle-switch {
 width: 32px;
 height: 18px;
 background: var(--blue);
 border-radius: 9px;
 position: relative;
 cursor: pointer;
 transition: background-color 0.25s ease;
}

.toggle-switch.off {
 background: var(--text-muted);
}

.toggle-switch::after {
 content: '';
 position: absolute;
 width: 14px;
 height: 14px;
 background: white;
 border-radius: 50%;
 top: 2px;
 right: 2px;
 transition: all 0.25s ease;
}

.toggle-switch.off::after {
 right: auto;
 left: 2px;
}

.resource-item {
 margin-bottom: 12px;
}

.resource-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 margin-bottom: 6px;
}

.resource-label {
 font-size: 12px;
 color: var(--text-secondary);
}

.resource-value {
 font-size: 12px;
 font-weight: 600;
}
```

```

}

.resource-chart {
 height: 32px;
 background: var(--bg-primary);
 border-radius: 4px;
 overflow: hidden;
 position: relative;
}

.setting-item {
 margin-bottom: 12px;
}

.setting-label {
 font-size: 12px;
 color: var(--text-secondary);
 margin-bottom: 6px;
}

.setting-input {
 width: 100%;
 padding: 8px 12px;
 background: var(--bg-primary);
 border: 1px solid var(--border-color);
 border-radius: 8px;
 color: var(--text-primary);
 font-size: 12px;
 font-family: 'Inter', monospace;
 transition:
 border-color 0.25s ease,
 box-shadow 0.25s ease;
}

.setting-input:focus {
 outline: none;
 border-color: var(--blue);
 box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.15);
}

.setting-input:hover {
 border-color: var(--text-muted);
}

.slider-container {
 position: relative;
 padding-top: 4px;
}

.slider {
 width: 100%;
 height: 6px;
 -webkit-appearance: none;
 appearance: none;
 background: var(--bg-primary);
 border-radius: 3px;
 outline: none;
 cursor: pointer;
}

.slider::-webkit-slider-thumb {
 -webkit-appearance: none;
 appearance: none;
 width: 16px;
 height: 16px;
 background: var(--blue);
 border-radius: 50%;
 cursor: pointer;
 transition:
 transform 0.2s ease,
 box-shadow 0.2s ease;
}

.slider::-webkit-slider-thumb:hover {
 transform: scale(1.2);
 box-shadow: 0 0 0 4px rgba(59, 130, 246, 0.2);
}

.slider::-webkit-slider-thumb:active {
 transform: scale(1.1);
}

```

```
}

.slider-labels {
 display: flex;
 justify-content: space-between;
 margin-top: 4px;
}

.slider-label {
 font-size: 10px;
 color: var(--text-muted);
}

.slider-value {
 text-align: center;
 margin-top: 4px;
 font-size: 14px;
 font-weight: 600;
 color: var(--blue);
}

.content-center {
 flex: 1;
 overflow-y: auto;
 padding: 16px;
 background: var(--bg-primary);
 transition: background-color 0.3s ease;
}

.content-center::-webkit-scrollbar {
 width: 6px;
}

.content-center::-webkit-scrollbar-track {
 background: transparent;
}

.content-center::-webkit-scrollbar-thumb {
 background: var(--border-color);
 border-radius: 3px;
}

.pipeline-section {
 margin-bottom: 20px;
}

.section-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 margin-bottom: 16px;
}

.section-title {
 font-size: 14px;
 font-weight: 600;
 color: var(--text-primary);
}

.legend {
 display: flex;
 gap: 16px;
}

.legend-item {
 display: flex;
 align-items: center;
 gap: 6px;
 font-size: 11px;
 color: var(--text-secondary);
}

.legend-line {
 width: 24px;
 height: 2px;
}

.legend-line.solid {
 background: var(--blue);
}
```

```

.legend-line.dashed {
 background: repeating-linear-gradient(
 90deg,
 var(--text-muted),
 var(--text-muted) 4px,
 transparent 4px,
 transparent 8px
);
}

.pipeline-container {
 display: flex;
 flex-direction: column;
 align-items: center;
 gap: 10px;
}

.pipeline-row {
 display: flex;
 align-items: center;
 gap: 16px;
 width: 100%;
 justify-content: center;
}

.pipeline-node {
 background: var(--bg-card);
 border: 2px solid var(--border-color);
 border-radius: 12px;
 padding: 16px 12px;
 min-width: 260px;
 width: 260px;
 height: 110px;
 text-align: center;
 position: relative;
 transition: all 0.25s ease;
 cursor: pointer;
 box-shadow: var(--node-shadow);
 display: flex;
 flex-direction: column;
 align-items: center;
 justify-content: center;
}

.pipeline-node:hover {
 border-color: var(--blue);
 transform: translateY(-2px) scale(1.02);
 box-shadow:
 0 0 20px rgba(59, 130, 246, 0.15),
 var(--node-shadow);
}

.pipeline-node.node-knowledge {
 background: rgba(126, 34, 206, 0.8);
 border-color: #a78bfa;
}

.pipeline-node.node-summary {
 background: rgba(4, 120, 87, 0.8);
 border-color: #34d399;
}

.pipeline-node.node-writer {
 background: rgba(29, 78, 216, 0.8);
 border-color: #60a5fa;
}

.pipeline-node.node-review {
 background: rgba(194, 65, 12, 0.8);
 border-color: #fb923c;
}

.pipeline-node.node-judge {
 background: rgba(109, 40, 217, 0.8);
 border-color: #a78bfa;
}

.pipeline-node.node-result {
 background: rgba(51, 65, 85, 0.8);
}

```

```

border-color: #94a3b8;
}

.pipeline-node.node-input {
background: var(--bg-card);
border-color: var(--border-color);
}

.pipeline-node.node-pk {
background: rgba(13, 148, 136, 0.8);
border-color: #2dd4bf;
}

.pipeline-node.node-api {
background: rgba(161, 98, 7, 0.8);
border-color: #facc15;
}

.pipeline-node.node-output {
background: var(--bg-card);
border-color: var(--border-color);
}

:not(.dark) .pipeline-node.node-knowledge {
background: rgba(126, 34, 206, 0.1);
border-color: #c4b5fd;
}

:not(.dark) .pipeline-node.node-knowledge .node-title,
:not(.dark) .pipeline-node.node-knowledge .node-subtitle {
color: #6b21a8;
}

:not(.dark) .pipeline-node.node-summary {
background: rgba(4, 120, 87, 0.1);
border-color: #6ee7b7;
}

:not(.dark) .pipeline-node.node-summary .node-title,
:not(.dark) .pipeline-node.node-summary .node-subtitle {
color: #065f46;
}

:not(.dark) .pipeline-node.node-writer {
background: rgba(29, 78, 216, 0.1);
border-color: #93c5fd;
}

:not(.dark) .pipeline-node.node-writer .node-title,
:not(.dark) .pipeline-node.node-writer .node-subtitle {
color: #1e40af;
}

:not(.dark) .pipeline-node.node-review {
background: rgba(194, 65, 12, 0.1);
border-color: #fdb74d;
}

:not(.dark) .pipeline-node.node-review .node-title,
:not(.dark) .pipeline-node.node-review .node-subtitle {
color: #9a3412;
}

:not(.dark) .pipeline-node.node-judge {
background: rgba(109, 40, 217, 0.1);
border-color: #c4b5fd;
}

:not(.dark) .pipeline-node.node-judge .node-title,
:not(.dark) .pipeline-node.node-judge .node-subtitle {
color: #5b21b6;
}

:not(.dark) .pipeline-node.node-result {
background: rgba(51, 65, 85, 0.1);
border-color: #cbd5e1;
}

:not(.dark) .pipeline-node.node-result .node-title,
:not(.dark) .pipeline-node.node-result .node-subtitle {

```

```

 color: #334155;
 }

 :not(.dark) .pipeline-node.node-pk {
 background: rgba(13, 148, 136, 0.1);
 border-color: #5eead4;
 }

 :not(.dark) .pipeline-node.node-pk .node-title,
 :not(.dark) .pipeline-node.node-pk .node-subtitle {
 color: #115e59;
 }

 :not(.dark) .pipeline-node.node-api {
 background: rgba(161, 98, 7, 0.1);
 border-color: #fde047;
 }

 :not(.dark) .pipeline-node.node-api .node-title,
 :not(.dark) .pipeline-node.node-api .node-subtitle {
 color: #854d0e;
 }

 .pipeline-node.node-completed {
 box-shadow:
 0 4px 12px rgba(16, 185, 129, 0.1),
 var(--node-shadow);
 }

 .pipeline-node.node-working {
 animation: nodePulse 2s ease-in-out infinite;
 box-shadow:
 0 4px 16px rgba(59, 130, 246, 0.2),
 var(--node-shadow);
 }

 @keyframes nodePulse {
 0%,
 100% {
 box-shadow:
 0 0 12px rgba(59, 130, 246, 0.2),
 var(--node-shadow);
 }
 50% {
 box-shadow:
 0 0 24px rgba(59, 130, 246, 0.35),
 var(--node-shadow);
 }
 }

 .node-icon {
 font-size: 18px;
 margin-bottom: 4px;
 }

 .node-title {
 font-size: 16px;
 font-weight: 600;
 color: var(--node-text);
 margin-bottom: 4px;
 text-align: center;
 line-height: 1.25;
 }

 .node-subtitle {
 font-size: 12px;
 color: var(--node-subtitle);
 opacity: 0.8;
 text-align: center;
 line-height: 1.25;
 }

 .node-status {
 position: absolute;
 top: 6px;
 right: 6px;
 display: inline-flex;
 align-items: center;
 gap: 3px;
 padding: 2px 8px;

```

```

border-radius: 9999px;
font-size: 10px;
font-weight: 500;
z-index: 2;
}

.node-status.completed {
 background: var(--node-status-completed-bg);
 color: var(--node-status-completed-text);
}

.node-status.working {
 background: var(--node-status-working-bg);
 color: var(--node-status-working-text);
 animation: pulse 2s infinite;
}

.node-status.loading {
 background: rgba(245, 158, 11, 0.2);
 color: #fbbf24;
}

.node-status.pending {
 background: var(--node-status-pending-bg);
 color: var(--node-status-pending-text);
}

.spinner {
 display: inline-block;
 width: 10px;
 height: 10px;
 border: 2px solid currentColor;
 border-right-color: transparent;
 border-radius: 50%;
 animation: spin 1s linear infinite;
}

@keyframes spin {
 to {
 transform: rotate(360deg);
 }
}

.arrow-down {
 width: 1.5px;
 height: 40px;
 background: var(--arrow-color);
 position: relative;
 transition: background-color 0.3s ease;
}

.arrow-down::after {
 content: '';
 position: absolute;
 bottom: -5px;
 left: 50%;
 transform: translateX(-50%);
 width: 0;
 height: 0;
 border-left: 3px solid transparent;
 border-right: 3px solid transparent;
 border-top: 6px solid var(--arrow-color);
 transition: border-top-color 0.3s ease;
}

.arrow-down.flow-active {
 background: #3b82f6;
}

.arrow-down.flow-active::after {
 border-top-color: #3b82f6;
}

.arrow-down.flow-success {
 background: #10b981;
}

.arrow-down.flow-success::after {
 border-top-color: #10b981;
}

```

```

.arrow-down.flow-error {
 background: var(--red);
}

.arrow-down.flow-error::after {
 border-top-color: var(--red);
}

.arrow-down.data-flow {
 background: #3b82f6;
}

.arrow-down.data-flow::after {
 border-top-color: #3b82f6;
}

.arrow-right {
 width: 40px;
 height: 1.5px;
 background: var(--border-color);
 position: relative;
}

.arrow-right::after {
 content: '';
 position: absolute;
 right: -3px;
 top: 50%;
 transform: translateY(-50%);
 width: 0;
 height: 0;
 border-top: 3px solid transparent;
 border-bottom: 3px solid transparent;
 border-left: 6px solid var(--border-color);
}

.branch-arrows {
 display: flex;
 justify-content: center;
 gap: 100px;
 margin-top: 8px;
}

.branch-arrow {
 display: flex;
 flex-direction: column;
 align-items: center;
 gap: 4px;
}

.branch-label {
 font-size: 11px;
 font-weight: 500;
 padding: 4px 10px;
 border-radius: 8px;
 background: rgba(255, 255, 255, 0.5);
 border: 1px solid rgba(255, 255, 255, 0.3);
 color: var(--branch-text);
 backdrop-filter: blur(12px);
 -webkit-backdrop-filter: blur(12px);
}

.branch-label.green {
 color: #4ade80;
 background: rgba(74, 222, 128, 0.15);
 border: 1px solid rgba(74, 222, 128, 0.3);
 backdrop-filter: blur(12px);
 -webkit-backdrop-filter: blur(12px);
}

:not(.dark) .branch-label.green {
 color: #15803d;
 background: rgba(22, 163, 74, 0.1);
 border: 1px solid rgba(22, 163, 74, 0.2);
}

.branch-label.orange {
 color: #fbbf24;
 background: rgba(251, 191, 36, 0.15);
}

```

```

border: 1px solid rgba(251, 191, 36, 0.3);
backdrop-filter: blur(12px);
-webkit-backdrop-filter: blur(12px);
}

:not(.dark) .branch-label.orange {
 color: #b45309;
 background: rgba(217, 119, 6, 0.1);
 border: 1px solid rgba(217, 119, 6, 0.2);
}

.difficulty-box {
 position: absolute;
 right: -100px;
 top: 50%;
 transform: translateY(-50%);
 background: var(--difficulty-bg);
 border: 1px solid var(--difficulty-border);
 border-radius: 12px;
 padding: 8px 12px;
 text-align: center;
 box-shadow: var(--node-shadow);
}

.difficulty-score {
 font-size: 20px;
 font-weight: 700;
 color: var(--orange);
}

.difficulty-threshold {
 font-size: 10px;
 color: var(--text-muted);
}

.final-answer-section {
 margin-top: 20px;
}

.answer-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 margin-bottom: 12px;
}

.answer-title {
 font-size: 14px;
 font-weight: 600;
 color: var(--text-primary);
 display: flex;
 align-items: center;
 gap: 8px;
}

.answer-progress-badge {
 background: rgba(16, 185, 129, 0.15);
 color: var(--green);
 padding: 3px 10px;
 border-radius: 6px;
 font-size: 11px;
 font-weight: 500;
 animation: breathe 2s ease-in-out infinite;
}

@keyframes breathe {
 0%,
 100% {
 opacity: 1;
 }
 50% {
 opacity: 0.7;
 }
}

.answer-container {
 display: flex;
 gap: 16px;
}

```

```
.answer-content {
 flex: 1;
 background: var(--answer-bg);
 border: 1px solid var(--answer-border);
 border-radius: 16px;
 padding: 28px 32px;
 color: var(--answer-text);
 font-size: 14px;
 line-height: 1.75;
 max-height: 400px;
 overflow-y: auto;
 box-shadow: var(--node-shadow);
 transition: all 0.3s ease;
}

.answer-content h1 {
 font-size: 18px;
 font-weight: 700;
 margin-bottom: 16px;
 color: var(--answer-heading);
}

.answer-content h2 {
 font-size: 15px;
 font-weight: 600;
 margin-top: 16px;
 margin-bottom: 8px;
 color: var(--answer-heading);
}

.answer-content h3 {
 font-size: 13px;
 font-weight: 600;
 margin-top: 12px;
 margin-bottom: 6px;
 color: var(--answer-heading);
}

.answer-content ul {
 padding-left: 20px;
 margin-bottom: 16px;
}

.answer-content li {
 margin-bottom: 4px;
 color: var(--answer-text);
}

.answer-content p {
 margin-bottom: 16px;
 color: var(--answer-text);
}

.answer-content strong {
 color: var(--answer-heading);
}

.custom-scrollbar::-webkit-scrollbar {
 width: 6px;
}

.custom-scrollbar::-webkit-scrollbar-track {
 background: var(--scrollbar-track);
 border-radius: 3px;
}

.custom-scrollbar::-webkit-scrollbar-thumb {
 background: var(--scrollbar-thumb);
 border-radius: 3px;
}

.custom-scrollbar::-webkit-scrollbar-thumb:hover {
 background: var(--scrollbar-thumb-hover);
}

.answer-sidebar {
 width: 160px;
 display: flex;
 flex-direction: column;
 gap: 12px;
```

```

}

.answer-stat {
 background: var(--stat-bg);
 border: 1px solid var(--stat-border);
 border-radius: 12px;
 padding: 14px;
 text-align: center;
 transition: all 0.25s ease;
}

.answer-stat:hover {
 transform: translateY(-2px);
 box-shadow: var(--card-shadow);
}

.answer-stat-label {
 font-size: 10px;
 color: var(--stat-label);
 margin-bottom: 6px;
 line-height: 1.25;
}

.answer-stat-value {
 font-size: 20px;
 font-weight: 700;
 color: var(--stat-value);
 transition: all 0.3s ease;
 line-height: 1.25;
}

.stop-generation {
 color: var(--red);
 font-size: 12px;
 cursor: pointer;
 text-align: center;
 margin-top: 12px;
 transition: opacity 0.2s;
}

.stop-generation:hover {
 opacity: 0.8;
}

.sidebar-right {
 width: 320px;
 min-width: 320px;
 background: var(--bg-secondary);
 border-left: 1px solid var(--border-color);
 display: flex;
 flex-direction: column;
 transition:
 background-color 0.3s ease,
 border-color 0.3s ease;
}

.tabs {
 display: flex;
 border-bottom: 1px solid var(--border-color);
}

.tab {
 flex: 1;
 padding: 12px 16px;
 text-align: center;
 font-size: 12px;
 font-weight: 500;
 color: var(--text-secondary);
 cursor: pointer;
 border-bottom: 2px solid transparent;
 transition: all 0.25s ease;
 user-select: none;
 position: relative;
}

.tab:hover {
 color: var(--text-primary);
 background: rgba(59, 130, 246, 0.05);
}

```

```

.tab.active {
 color: var(--blue);
 border-bottom-color: var(--blue);
}

.tab.active::after {
 content: '';
 position: absolute;
 bottom: -1px;
 left: 16px;
 right: 16px;
 height: 2px;
 background: var(--blue);
 border-radius: 1px;
 animation: tabUnderline 0.3s ease forwards;
}

@keyframes tabUnderline {
 from {
 transform: scaleX(0);
 }
 to {
 transform: scaleX(1);
 }
}

.tab-content {
 flex: 1;
 overflow-y: auto;
 padding: 12px;
}

.tab-content::-webkit-scrollbar {
 width: 4px;
}

.tab-content::-webkit-scrollbar-track {
 background: transparent;
}

.tab-content::-webkit-scrollbar-thumb {
 background: var(--border-color);
 border-radius: 2px;
}

.timeline-item {
 display: flex;
 gap: 12px;
 padding: 12px;
 background: var(--log-bg);
 border: 1px solid var(--border-color);
 border-radius: 12px;
 margin-bottom: 10px;
 transition: all 0.25s ease;
 cursor: default;
}

.timeline-item:hover {
 border-color: var(--blue);
 background: var(--log-bg-hover);
}

.timeline-time {
 font-size: 10px;
 color: var(--text-muted);
 white-space: nowrap;
 padding-top: 2px;
}

.timeline-icon {
 width: 28px;
 height: 28px;
 border-radius: 8px;
 display: flex;
 align-items: center;
 justify-content: center;
 font-size: 12px;
 flex-shrink: 0;
}

```

```
.timeline-icon.blue {
 background: rgba(59, 130, 246, 0.15);
 color: var(--blue);
}

.timeline-icon.green {
 background: rgba(16, 185, 129, 0.15);
 color: var(--green);
}

.timeline-icon.gold {
 background: rgba(245, 158, 11, 0.15);
 color: var(--orange);
}

.timeline-icon.purple {
 background: rgba(139, 92, 246, 0.15);
 color: var(--purple);
}

.timeline-body {
 flex: 1;
 min-width: 0;
}

.timeline-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 margin-bottom: 6px;
}

.timeline-title {
 font-size: 12px;
 font-weight: 600;
 color: var(--text-primary);
}

.timeline-link {
 font-size: 10px;
 color: var(--blue);
 cursor: pointer;
 transition: opacity 0.2s;
}

.timeline-link:hover {
 opacity: 0.8;
 text-decoration: underline;
}

.timeline-label {
 font-size: 10px;
 color: var(--text-muted);
 margin-bottom: 4px;
}

.timeline-content {
 font-size: 11px;
 color: var(--text-secondary);
 line-height: 1.625;
 display: -webkit-box;
 -webkit-line-clamp: 3;
 -webkit-box-orient: vertical;
 overflow: hidden;
 word-break: normal;
 overflow-wrap: break-word;
}

.timeline-content code {
 background: var(--bg-primary);
 padding: 2px 4px;
 border-radius: 3px;
 font-family: 'Inter', monospace;
 font-size: 10px;
}

.timeline-list {
 list-style: none;
 padding: 0;
 margin: 4px 0;
}
```

```

}

.timeline-list li {
 font-size: 11px;
 color: var(--text-secondary);
 padding-left: 12px;
 position: relative;
 margin-bottom: 2px;
}

.timeline-list li::before {
 content: '\2022';
 position: absolute;
 left: 0;
 color: var(--text-muted);
}

@keyframes fadeIn {
 from {
 opacity: 0;
 transform: translateY(10px);
 }
 to {
 opacity: 1;
 transform: translateY(0);
 }
}

.animate-in {
 animation: fadeIn 0.4s ease forwards;
}

.delay-1 {
 animation-delay: 0.1s;
 opacity: 0;
}
.delay-2 {
 animation-delay: 0.2s;
 opacity: 0;
}
.delay-3 {
 animation-delay: 0.3s;
 opacity: 0;
}
.delay-4 {
 animation-delay: 0.4s;
 opacity: 0;
}
.delay-5 {
 animation-delay: 0.5s;
 opacity: 0;
}

@keyframes flowDash {
 to {
 stroke-dashoffset: -20;
 }
}

.flow-arrow-animated {
 animation: flowDash 1s linear infinite;
}

@keyframes checkmarkPop {
 0% {
 transform: scale(0);
 opacity: 0;
 }
 50% {
 transform: scale(1.3);
 }
 100% {
 transform: scale(1);
 opacity: 1;
 }
}

.checkmark-anim {
 display: inline-block;
 animation: checkmarkPop 0.4s ease forwards;
}

```

```

}

.context-menu {
 position: fixed;
 background: var(--bg-secondary);
 border: 1px solid var(--border-color);
 border-radius: 12px;
 padding: 4px;
 min-width: 160px;
 z-index: 1000;
 box-shadow: var(--node-shadow);
}

.context-menu-item {
 display: flex;
 align-items: center;
 gap: 8px;
 padding: 8px 12px;
 font-size: 12px;
 color: var(--text-primary);
 cursor: pointer;
 border-radius: 8px;
 transition: background-color 0.15s ease;
}

.context-menu-item:hover {
 background: rgba(59, 130, 246, 0.15);
}

.context-menu-item.danger {
 color: var(--red);
}

.context-menu-item.danger:hover {
 background: rgba(239, 68, 68, 0.15);
}

.node-detail-modal {
 position: fixed;
 top: 0;
 left: 0;
 right: 0;
 bottom: 0;
 background: var(--modal-overlay);
 display: flex;
 align-items: center;
 justify-content: center;
 z-index: 200;
 animation: fadeIn 0.2s ease;
}

.node-detail-panel {
 background: var(--bg-secondary);
 border: 1px solid var(--border-color);
 border-radius: 12px;
 padding: 24px;
 min-width: 420px;
 max-width: 560px;
 max-height: 80vh;
 overflow-y: auto;
 box-shadow: var(--modal-shadow);
}

.node-detail-panel h3 {
 font-size: 16px;
 font-weight: 700;
 color: var(--text-primary);
 margin-bottom: 16px;
 display: flex;
 align-items: center;
 gap: 8px;
}

.node-detail-row {
 display: flex;
 justify-content: space-between;
 align-items: flex-start;
 padding: 8px 0;
 border-bottom: 1px solid var(--border-color);
}

```

```
.node-detail-row:last-child {
 border-bottom: none;
}

.node-detail-label {
 font-size: 12px;
 color: var(--text-muted);
 flex-shrink: 0;
}

.node-detail-value {
 font-size: 12px;
 color: var(--text-primary);
 text-align: right;
 max-width: 300px;
 word-break: normal;
 overflow-wrap: break-word;
 line-height: 1.625;
}

.confirm-dialog {
 position: fixed;
 top: 0;
 left: 0;
 right: 0;
 bottom: 0;
 background: var(--modal-overlay);
 display: flex;
 align-items: center;
 justify-content: center;
 z-index: 300;
 animation: fadeIn 0.2s ease;
}

.confirm-dialog-content {
 background: var(--bg-secondary);
 border: 1px solid var(--border-color);
 border-radius: 12px;
 padding: 24px;
 min-width: 320px;
 box-shadow: var(--modal-shadow);
}

.confirm-dialog-content h4 {
 font-size: 14px;
 font-weight: 600;
 color: var(--text-primary);
 margin-bottom: 12px;
}

.confirm-dialog-content p {
 font-size: 13px;
 color: var(--text-secondary);
 margin-bottom: 20px;
}

.confirm-dialog-actions {
 display: flex;
 justify-content: flex-end;
 gap: 8px;
}

.console-log-section {
 margin-top: 12px;
}

.console-log-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 cursor: pointer;
 padding: 8px 0;
 user-select: none;
}

.console-log-header:hover {
 opacity: 0.8;
}
```

```
.console-log-title {
 font-size: 12px;
 color: var(--text-secondary);
 font-weight: 500;
 text-transform: uppercase;
 letter-spacing: 0.5px;
}

.console-log-toggle {
 font-size: 10px;
 color: var(--text-muted);
 transition: transform 0.25s ease;
}

.console-log-toggle.expanded {
 transform: rotate(180deg);
}

.console-log-entries {
 max-height: 0;
 overflow: hidden;
 transition: max-height 0.3s ease;
}

.console-log-entries.expanded {
 max-height: 200px;
 overflow-y: auto;
}

.console-log-entry {
 display: flex;
 align-items: flex-start;
 gap: 8px;
 padding: 4px 0;
 font-size: 11px;
 font-family: 'Inter', monospace;
}

.console-log-time {
 color: var(--text-muted);
 flex-shrink: 0;
}

.console-log-type {
 padding: 1px 4px;
 border-radius: 2px;
 font-size: 9px;
 font-weight: 600;
 flex-shrink: 0;
}

.console-log-type.info {
 background: rgba(59, 130, 246, 0.15);
 color: var(--blue);
}

.console-log-type.success {
 background: rgba(16, 185, 129, 0.15);
 color: var(--green);
}

.console-log-type.warning {
 background: rgba(245, 158, 11, 0.15);
 color: var(--orange);
}

.console-log-type.error {
 background: rgba(239, 68, 68, 0.15);
 color: var(--red);
}

.console-log-msg {
 color: var(--text-secondary);
 word-break: normal;
 overflow-wrap: break-word;
 line-height: 1.625;
}

.api-key-wrapper {
 position: relative;
}

.api-key-toggle {
```

```

position: absolute;
right: 8px;
top: 50%;
transform: translateY(-50%);
background: none;
border: none;
color: var(--text-muted);
cursor: pointer;
padding: 4px;
font-size: 12px;
transition: color 0.2s;
}

.api-key-toggle:hover {
 color: var(--text-primary);
}

.setting-input.with-toggle {
 padding-right: 32px;
}

@media (max-width: 1200px) {
 .sidebar-left {
 width: 240px;
 min-width: 240px;
 }
 .sidebar-right {
 width: 280px;
 min-width: 280px;
 }
}

@media (max-width: 900px) {
 .main-container {
 flex-direction: column;
 height: auto;
 }
 .sidebar-left,
 .sidebar-right {
 width: 100%;
 min-width: 100%;
 max-height: 400px;
 }
 .pipeline-row {
 flex-wrap: wrap;
 }
 .difficulty-box {
 position: static;
 transform: none;
 margin-top: 8px;
 }
}

.modal-overlay {
 position: fixed;
 top: 0;
 left: 0;
 right: 0;
 bottom: 0;
 background: var(--modal-overlay);
 display: flex;
 align-items: center;
 justify-content: center;
 z-index: 500;
 animation: fadeIn 0.2s ease;
}

.modal-content {
 background: var(--bg-secondary);
 border: 1px solid var(--border-color);
 border-radius: 16px;
 padding: 0;
 min-width: 480px;
 max-width: 600px;
 max-height: 85vh;
 overflow: hidden;
 box-shadow: var(--modal-shadow);
 display: flex;
 flex-direction: column;
}

```

```
.modal-header {
 display: flex;
 align-items: center;
 justify-content: space-between;
 padding: 20px 24px;
 border-bottom: 1px solid var(--border-color);
}

.modal-header h3 {
 font-size: 16px;
 font-weight: 600;
 color: var(--text-primary);
 margin: 0;
}

.modal-close {
 background: none;
 border: none;
 color: var(--text-muted);
 cursor: pointer;
 padding: 4px;
 border-radius: 6px;
 transition: all 0.2s ease;
}

.modal-close:hover {
 background: var(--bg-hover);
 color: var(--text-primary);
}

.modal-body {
 flex: 1;
 padding: 24px;
 overflow-y: auto;
}

.modal-footer {
 display: flex;
 justify-content: flex-end;
 gap: 12px;
 padding: 16px 24px;
 border-top: 1px solid var(--border-color);
}

.modal-tabs {
 display: flex;
 border-bottom: 1px solid var(--border-color);
}

.modal-tab {
 flex: 1;
 padding: 14px 24px;
 text-align: center;
 font-size: 13px;
 font-weight: 500;
 color: var(--text-secondary);
 cursor: pointer;
 border-bottom: 2px solid transparent;
 transition: all 0.2s ease;
 background: transparent;
 border: none;
}

.modal-tab:hover {
 color: var(--text-primary);
}

.modal-tab.active {
 color: var(--blue);
 border-bottom-color: var(--blue);
}

.form-group {
 margin-bottom: 20px;
}

.form-label {
 display: block;
 font-size: 13px;
```

```
font-weight: 500;
color: var(--text-secondary);
margin-bottom: 8px;
}

.form-label.required {
 color: var(--red);
 margin-right: 4px;
}

.form-input {
 width: 100%;
 padding: 10px 14px;
 background: var(--bg-primary);
 border: 1px solid var(--border-color);
 border-radius: 10px;
 color: var(--text-primary);
 font-size: 14px;
 font-family: 'Inter', sans-serif;
 transition: all 0.25s ease;
 outline: none;
}

.form-input:focus {
 border-color: var(--blue);
 box-shadow: 0 0 0 3px rgba(59, 130, 246, 0.15);
}

.form-input:hover {
 border-color: var(--text-muted);
}

.form-input:disabled {
 opacity: 0.5;
 cursor: not-allowed;
}

.form-actions {
 display: flex;
 gap: 10px;
}

.result-message {
 display: flex;
 align-items: center;
 gap: 10px;
 padding: 12px 14px;
 border-radius: 10px;
 margin-top: 12px;
 font-size: 13px;
}

.result-message.success {
 background: rgba(16, 185, 129, 0.15);
 color: var(--green);
}

.result-message.error {
 background: rgba(239, 68, 68, 0.15);
 color: var(--red);
}

.api-key-toggle-btn {
 margin-top: 8px;
 padding: 6px 12px;
 font-size: 12px;
 background: var(--bg-primary);
 border: 1px solid var(--border-color);
 border-radius: 8px;
 color: var(--text-secondary);
 cursor: pointer;
 transition: all 0.2s ease;
}

.api-key-toggle-btn:hover:not(:disabled) {
 background: var(--bg-hover);
 border-color: var(--blue);
 color: var(--blue);
}
```

```
.api-key-toggle-btn:disabled {
 opacity: 0.5;
 cursor: not-allowed;
}

.advanced-toggle {
 display: flex;
 align-items: center;
 gap: 10px;
 padding: 12px 16px;
 background: var(--bg-primary);
 border: 1px solid var(--border-color);
 border-radius: 12px;
 color: var(--text-secondary);
 cursor: pointer;
 transition: all 0.3s ease;
 font-size: 13px;
 width: 100%;
 text-align: left;
}

.advanced-toggle:hover {
 background: var(--bg-hover);
 border-color: var(--blue);
 color: var(--blue);
}

.advanced-toggle svg {
 transition: transform 0.3s ease;
 flex-shrink: 0;
}

.advanced-toggle svg.rotated {
 transform: rotate(180deg);
}

.advanced-desc {
 flex: 1;
 font-size: 12px;
 color: var(--text-muted);
 text-align: right;
}

.advanced-config-container {
 overflow: hidden;
 animation: expandDown 0.3s ease-out;
}

@keyframes expandDown {
 from {
 max-height: 0;
 opacity: 0;
 transform: translateY(-10px);
 }
 to {
 max-height: 500px;
 opacity: 1;
 transform: translateY(0);
 }
}

.advanced-config {
 background: var(--bg-primary);
 border: 1px solid var(--border-color);
 border-radius: 12px;
 padding: 20px;
 margin-top: 16px;
 box-shadow: 0 4px 12px rgba(0, 0, 0, 0.1);
}

.spinner.small {
 width: 14px;
 height: 14px;
 border-width: 2px;
}
```

## frontend\src\app\layout.tsx

```
import './globals.css';
import type { Metadata } from 'next';

export const metadata: Metadata = {
 title: 'AgentMatrix - 多智能体动态协同与国产算力优化平台',
 description: '基于多Agent协同 + 动态算力路由的AI系统',
};

export default function RootLayout({
 children,
}): {
 children: React.ReactNode;
}) {
 return (
 <html lang="zh-CN" suppressHydrationWarning>
 <head>
 <script
 dangerouslySetInnerHTML={{
 __html: `
 (function() {
 try {
 var theme = localStorage.getItem('neuroflow-theme');
 if (theme === 'light') {
 document.documentElement.classList.remove('dark');
 } else {
 document.documentElement.classList.add('dark');
 }
 } catch(e) {
 document.documentElement.classList.add('dark');
 }
 })();
 `,
 }}
 />
 </head>
 <body>{children}</body>
 </html>
);
}
```

## frontend\src\app\page.tsx

```
import Dashboard from '@/components/layout/DashboardLayout';

export default function Home() {
 return <Dashboard />;
}
```

## frontend\src\components\layout\DashboardLayout\index.tsx

```
'use client';

import { useState, useCallback, useEffect, useRef } from 'react';
import { useWorkflowStore } from '@/stores/workflowStore';
import { useAgentStore } from '@/stores/agentStore';
import { workflowService, chatService, configService } from '@/services/api/agentService';
import { AGENT_ORDER, AGENT_NAMES, AGENT_MODELS, AGENT_EMOJIS, AGENT_DESCRIPTIONS, AGENT_ICON_CLASSES } from '@/types';
import type { AgentId, WorkflowOutput, LogEntry } from '@/types';

const COMPLEXITY_THRESHOLD = 0.65;

const AGENT_DISPLAY_NAMES: Record<AgentId, string> = {
 knowledge: 'Knowledge Agent',
 summary: 'A摘要Agent',
```

```

 writer: 'B撰写Agent',
 review: 'Review Agent',
 judge: 'Judge Agent',
 result: 'Result Agent',
 };

const AGENT_DISPLAY_COLORS: Record<AgentId, string> = {
 knowledge: 'purple',
 summary: 'emerald',
 writer: 'blue',
 review: 'orange',
 judge: 'violet',
 result: 'green',
};

const AGENT_TIMELINE_COLORS: Record<AgentId, string> = {
 knowledge: 'purple',
 summary: 'emerald',
 writer: 'blue',
 review: 'orange',
 judge: 'violet',
 result: 'green',
};

const AGENT_NODE_CLASSES: Record<AgentId, string> = {
 knowledge: 'node-knowledge',
 summary: 'node-summary',
 writer: 'node-writer',
 review: 'node-review',
 judge: 'node-judge',
 result: 'node-result',
};

const AGENT_SVG_ICONS: Record<AgentId, React.ReactNode> = {
 knowledge: (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <path d="M4 19.5A2.5 2.5 0 0 1 6.5 17H20" />
 <path d="M6.5 2H20v20H6.5A2.5 2.5 0 0 1 4 19.5v-15A2.5 2.5 0 0 1 6.5 2z" />
 <line x1="12" y1="6" x2="12" y2="10" />
 <line x1="10" y1="8" x2="14" y2="8" />
 </svg>
),
 summary: (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 2 2h12a2 2 0 0 0 2-2V8z" />
 <polyline points="14 2 14 8 20 8" />
 <line x1="16" y1="13" x2="8" y2="13" />
 <line x1="16" y1="17" x2="8" y2="17" />
 </svg>
),
 writer: (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <path d="M12 20h9" />
 <path d="M6.5 3.5a2.121 2.121 0 0 1 3 3L7 19l-4 1 1-4L16.5 3.5z" />
 </svg>
),
 review: (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <circle cx="11" cy="11" r="8" />
 <line x1="21" y1="21" x2="16.65" y2="16.65" />
 </svg>
),
 judge: (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <line x1="12" y1="2" x2="12" y2="22" />
 <path d="M5 7l7-5 7 5" />
 <line x1="5" y1="7" x2="19" y2="7" />
 <line x1="5" y1="12" x2="19" y2="12" />
 <path d="M3 17l3 3" />
 <path d="M21 17l-3 3" />
 <line x1="6" y1="20" x2="18" y2="20" />
 </svg>
),
 result: (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <path d="M14 2H6a2 2 0 0 0-2 2v16a2 2 0 0 2 2h12a2 2 0 0 0 2-2V8z" />
 <polyline points="14 2 14 8 20 8" />
 <line x1="16" y1="13" x2="8" y2="13" />
 <line x1="16" y1="17" x2="8" y2="17" />
 </svg>
),
};

```

```

),
 };

const AGENT_OUTPUT_SUMMARIES: Record<AgentId, string> = {
 knowledge: '检索到相关知识库内容: 智能体协作、技术研究、产品开发...',
 summary: '构建高效的多智能体协同生态, 提升任务处理效率...',
 writer: '基于知识库上下文生成初稿, 包含多维度内容规划...',
 review: '质量评估完成: 内容完整性0.85, 逻辑清晰度0.78...',
 judge: '最终决策: 调用API网关增强, 难度评分0.72 > 阈值0.65...',
 result: '最终结果已整合生成',
};

const AGENT_DETAIL_OUTPUTS: Record<AgentId, string> = {
 knowledge: '检索到相关知识库内容: 智能体协作、技术研究、产品开发...',
 summary: '构建高效的多智能体协同生态, 提升任务处理效率...',
 writer: '基于知识库上下文生成初稿, 包含多维度内容规划...',
 review: '质量评估完成: 内容完整性0.85, 逻辑清晰度0.78...',
 judge: '最终决策: 调用API网关增强, 难度评分0.72 > 阈值0.65...',
 result: '最终结果已整合生成',
};

const NODE_DETAIL_INFO: Record<string, { input: string; output: string; duration: string; model: string }> = {
 '用户输入': { input: '用户自然语言输入', output: '结构化任务描述', duration: '-', model: '-' },
 'Knowledge Agent': { input: '用户原始输入', output: '知识库检索结果 + 用户原始输入', duration: '0.8s', model: 'Qwen2.5-3B' },
},
 'A摘要Agent': { input: '知识库增强后的输入', output: '关键信息摘要与提取', duration: '1.2s', model: 'Qwen2.5-3B' },
 'B撰写Agent': { input: '知识库增强后的输入', output: '内容初稿生成', duration: '1.8s', model: 'Qwen2.5-7B' },
 'Review Agent': { input: 'B撰写Agent输出', output: '质量评分与修改建议', duration: '0.9s', model: 'Qwen2.5-3B' },
 'Judge Agent': { input: 'Review Agent输出 + 阈值0.65', output: '最终决策与路径选择', duration: '1.1s', model: 'Qwen2.5-3B' },
},
 '内部PK胜出': { input: '本地模型输出', output: '优化后的本地结果', duration: '0.8s', model: 'Local' },
 'API网关': { input: '高难度任务', output: '云端增强生成结果', duration: '4.4s', model: 'DeepSeek-V4' },
 '最终答案输出': { input: '整合后内容', output: '格式化最终回答', duration: '0.5s', model: 'Local' },
};

function ProgressRing({ percentage, color }: { percentage: number; color: string }) {
 const radius = 20;
 const circumference = 2 * Math.PI * radius;
 const offset = circumference - (percentage / 100) * circumference;

 return (
 <div className="progress-ring">
 <svg width="48" height="48" viewBox="0 0 48 48">
 <circle className="progress-ring-bg" cx="24" cy="24" r={radius} />
 <circle
 className="progress-ring-fill"
 cx="24" cy="24" r={radius}
 stroke={color}
 strokeDasharray={circumference}
 strokeDashoffset={offset}
 />
 </svg>
 {percentage}%
 </div>
);
}

interface ContextMenuState {
 visible: boolean;
 x: number;
 y: number;
 target: string;
 type: 'node' | 'agent';
}

export default function DashboardLayout() {
 const [inputValue, setInputValue] = useState('');
 const [activeTab, setActiveTab] = useState<'output' | 'decision' | 'api'>('output');
 const [threshold, setThreshold] = useState(0.65);
 const [isDarkTheme, setIsDarkTheme] = useState(() => {
 if (typeof window !== 'undefined') {
 const saved = localStorage.getItem('neuroflow-theme');
 return saved !== 'light';
 }
 return true;
 });
 const [consoleExpanded, setConsoleExpanded] = useState(false);
 const [showApiKey, setShowApiKey] = useState(false);
 const [modelPath, setModelPath] = useState('/models');
 const [showModelDropdown, setShowModelDropdown] = useState(false);

```

```

const [nodeDetail, setNodeDetail] = useState<string | null>(null);
const [confirmDialog, setConfirmDialog] = useState<{ title: string; message: string; onConfirm: () => void } | null>(null);
const [contextMenu, setContextMenu] = useState<ContextMenuState>({ visible: false, x: 0, y: 0, target: '', type: 'node' });
const [expandedAgentDetails, setExpandedAgentDetails] = useState<Set<string>>(new Set());
const [taskListOpen, setTaskListOpen] = useState(false);
const [cpuData, setCpuData] = useState<number[]>([20, 22, 18, 24, 16, 20, 14, 18, 22, 20]);
const [memData, setMemData] = useState<number[]>([16, 14, 18, 12, 16, 14, 18, 16, 14, 16]);
const [gpuData, setGpuData] = useState<number[]>([24, 20, 24, 18, 22, 24, 20, 22, 18, 24]);
const [cpuValue, setCpuValue] = useState(34);
const [memValue, setMemValue] = useState(4.2);
const [gpuValue, setGpuValue] = useState(1.2);

const [showLocalModelDialog, setShowLocalModelDialog] = useState(false);
const [localModelPathInput, setLocalModelPathInput] = useState('localhost');
const [localModelPort, setLocalModelPort] = useState('11435');
const [detectingLocalModel, setDetectingLocalModel] = useState(false);
const [localModelDetectResult, setLocalModelDetectResult] = useState<{ success: boolean; message: string; models?: any[] } | null>(null);

const [showCloudModelDialog, setShowCloudModelDialog] = useState(false);
const [cloudModelTab, setCloudModelTab] = useState<'provider' | 'custom'>('provider');
const [selectedProvider, setSelectedProvider] = useState('');
const [selectedModel, setSelectedModel] = useState('');
const [apiKey, setApiKey] = useState('');
const [showAdvancedConfig, setShowAdvancedConfig] = useState(false);
const [displayName, setDisplayName] = useState('');
const [maxTokens, setMaxTokens] = useState(4096);
const [temperature, setTemperature] = useState(0.7);

const [testConnectionResult, setTestConnectionResult] = useState<{ success: boolean; message: string } | null>(null);
const [testingConnection, setTestingConnection] = useState(false);

const [validatingApiKey, setValidatingApiKey] = useState(false);
const [apiKeyValidationResult, setApiKeyValidationResult] = useState<{ success: boolean; message: string } | null>(null);

const contextMenuRef = useRef<HTMLDivElement>(null);

const {
 isRunning, currentTask, currentStep, elapsedSeconds, useMock,
 completedSteps, result, judgeDecision, complexityScore, logs, chatHistory,
 setCurrentTask, setIsRunning, setCurrentStep,
 setResult, setJudgeDecision, setComplexityScore, addLog, addCompletedStep,
 addWorkflowStep, resetWorkflow, addChatHistory, clearChatHistory, getContext,
} = useWorkflowStore();
const { agents, updateAgentStatus, resetAllAgents, toggleAgentEnabled } = useAgentStore();

useEffect(() => {
 if (isDarkTheme) {
 document.documentElement.classList.add('dark');
 localStorage.setItem('neuroflow-theme', 'dark');
 } else {
 document.documentElement.classList.remove('dark');
 localStorage.setItem('neuroflow-theme', 'light');
 }
}, [isDarkTheme]);

useEffect(() => {
 const interval = setInterval(() => {
 const newCpu = 30 + Math.random() * 15;
 const newMem = 3.8 + Math.random() * 1.2;
 const newGpu = 1.0 + Math.random() * 0.6;
 setCpuValue(Math.round(newCpu));
 setMemValue(parseFloat(newMem.toFixed(1)));
 setGpuValue(parseFloat(newGpu.toFixed(1)));
 setCpuData(prev => [...prev.slice(1), 32 - (newCpu / 100) * 28]);
 setMemData(prev => [...prev.slice(1), 32 - (newMem / 16) * 28]);
 setGpuData(prev => [...prev.slice(1), 32 - (newGpu / 8) * 28]);
 }, 2000);
 return () => clearInterval(interval);
}, []);

useEffect(() => {
 const handleClick = () => {
 setContextMenu(prev => ({ ...prev, visible: false }));
 setShowModelDropdown(false);
 };
 document.addEventListener('click', handleClick);
 return () => document.removeEventListener('click', handleClick);
}, []);

```

```

const formatTime = (seconds: number) => {
 const h = Math.floor(seconds / 3600).toString().padStart(2, '0');
 const m = Math.floor((seconds % 3600) / 60).toString().padStart(2, '0');
 const s = (seconds % 60).toString().padStart(2, '0');
 return `${h}:${m}:${s}`;
};

useEffect(() => {
 if (showLocalModelDialog) {
 configService.get().then(config => {
 if (config.ollama_host) {
 try {
 const url = new URL(config.ollama_host);
 setLocalModelPathInput(url.hostname);
 setLocalModelPort(url.port || '11435');
 } catch (e) {
 console.log('解析 ollama_host 失败, 使用默认值');
 }
 }
 });
 }
}, [showLocalModelDialog]);

const generateSvgPath = (data: number[]) => {
 const points = data.map((y, i) => {
 const x = (i / (data.length - 1)) * 240;
 return `${i === 0 ? 'M' : 'T'}${x},${y}`;
 });
 const firstX = 0;
 const firstY = data[0];
 return `M${firstX},${firstY} ${data.slice(1).map((y, i) => {
 const x = ((i + 1) / (data.length - 1)) * 240;
 return `Q${x - 12},${y < data[i] ? y - 2 : y + 2} ${x},${y}`;
 })).join(' ')`;
};

const executeWithMock = useCallback(async (taskText: string) => {
 setCurrentTask(taskText);
 setIsRunning(true);

 const startTime = new Date().toISOString();

 for (const agentId of AGENT_ORDER) {
 if (!agents[agentId].enabled) continue;
 setCurrentStep(agentId);
 updateAgentStatus(agentId, 'processing');
 addLog(agentId, 'info', `${AGENT_NAMES[agentId]} 开始处理...`);

 const stepStart = Date.now();
 await new Promise((resolve) => setTimeout(resolve, 800 + Math.random() * 1200));
 const duration = (Date.now() - stepStart) / 1000;

 addCompletedStep(agentId);
 updateAgentStatus(agentId, 'completed');

 if (agentId === 'judge') {
 setComplexityScore(0.72);
 const isComplex = 0.72 >= COMPLEXITY_THRESHOLD;
 setJudgeDecision(isComplex ? 'cloud' : 'local');
 addLog(agentId, 'info', `难度评估: 0.72 (困难) - 超过阈值 0.65`);
 addLog(agentId, isComplex ? 'warning' : 'success',
 `决策: ${isComplex ? '调用API网关 - 难度评分0.72 > 阈值0.65' : '本地处理 - 难度评分0.38 < 阈值0.65'}`);
 }

 addLog(agentId, 'success', `${AGENT_NAMES[agentId]} 处理完成 (${duration.toFixed(1)}s)`);
 }

 const mockResult: WorkflowOutput = {
 final_result: `# 智能体协作年度计划 (2024) \n\n## 一、目标概述\n\n本年度计划旨在搭建更加智能、高效、可扩展的多智能体协作系统, 提升任务处理效率和用户体验, 实现技术创新与商业价值的双重突破。 \n\n## 二、主要工作计划\n\n### 1. 技术研发 (Q1-Q2) \n\n- 优化多智能体协同算法 \n- 提升任务分配效率 \n- 增强模型推理能力 \n\n### 2. 产品迭代 (Q2-Q3) \n\n- 用户界面优化升级 \n- 新增知识库管理功能 \n- 支持自定义Agent配置 \n\n### 3. 生态建设 (Q3-Q4) \n\n- 开放API接口 \n- 建设开发者社区 \n- 推动行业应用落地 \n\n## 三、预算规划\n\n| 项目 | Q1 | Q2 | Q3 | Q4 | 合计 | \n|-----|-----|-----|-----|-----|-----| \n| 研发投入 | 50万 | 45万 | 35万 | 30万 | 160万 | \n| 云服务 | 10万 | 12万 | 15万 | 15万 | 52万 | \n| 人力成本 | 80万 | 80万 | 80万 | 80万 | 320万 |`,
 steps: [],
 executed_locally: false,
 total_duration_seconds: 12.6,
 start_time: startTime,
 end_time: new Date().toISOString(),
 };

```

```

 complexity_score: 0.72,
 };

 addChatHistory(taskText, mockResult.final_result);
 setResult(mockResult);
 addLog('result', 'success', '所有步骤已完成, 最终结果已生成');
 setIsRunning(false);
 setCurrentStep(null);
 }, [setCurrentTask, setIsRunning, setCurrentStep, setResult, setJudgeDecision, setComplexityScore, addLog, addCompletedStep,
 addWorkflowStep, updateAgentStatus, agents, addChatHistory]);

const executeWithAPI = useCallback(async (taskText: string) => {
 setCurrentTask(taskText);
 setIsRunning(true);
 setResult(null);
 setJudgeDecision(null);
 setComplexityScore(0);

 try {
 await chatService.sendStream(
 { content: taskText },
 (data) => {
 switch (data.type) {
 case 'start':
 addLog('system', 'info', ' workflow开始执行');
 break;

 case 'agent_start':
 if (data.agent_id) {
 updateAgentStatus(data.agent_id as AgentId, 'processing');
 setCurrentStep(data.agent_id as AgentId);
 addLog(data.agent_id, 'info', `${data.agent_name} 开始处理...`);
 }
 break;

 case 'agent_complete':
 if (data.agent_id) {
 const agentId = data.agent_id as AgentId;
 addCompletedStep(agentId);
 updateAgentStatus(agentId, data.success ? 'completed' : 'error');
 setCurrentStep(null);
 addLog(agentId, data.success ? 'success' : 'error',
 `${data.agent_name} ${data.success ? '完成' : '失败'} (${data.duration}s)`);
 }

 if (agentId === 'judge' && data.complexity_score !== undefined) {
 setComplexityScore(data.complexity_score);
 setJudgeDecision(data.executed_locally ? 'local' : 'cloud');
 addLog(agentId, 'info', `复杂度评分: ${data.complexity_score.toFixed(2)}`);
 addLog(agentId, data.executed_locally ? 'success' : 'warning',
 `决策: ${data.executed_locally ? '本地处理' : '云端处理'}`);
 }
 }
 }
 break;

 case 'judge_decision':
 if (data.complexity_score !== undefined) {
 setComplexityScore(data.complexity_score);
 setJudgeDecision(data.executed_locally ? 'local' : 'cloud');
 const categoryLabel = data.category || 'unknown';
 addLog('judge', 'info', `分类: ${categoryLabel} | 复杂度: ${data.complexity_score.toFixed(2)}`);
 addLog('judge', data.executed_locally ? 'success' : 'warning',
 `决策: ${data.executed_locally ? '本地处理 (内部PK胜出)' : '云端处理 (API网关PK胜出)'}`);
 }
 if (data.reason && data.reason.length > 0) {
 addLog('judge', 'info', `理由: ${data.reason.join('; ')}`);
 }
 }
 break;

 case 'agent_error':
 if (data.agent_id) {
 const agentId = data.agent_id as AgentId;
 updateAgentStatus(agentId, 'error');
 setCurrentStep(null);
 addLog(agentId, 'error', `${data.agent_name} 执行出错: ${data.error}`);
 }
 break;
 }
 }

```

```

 case 'complete':
 console.log(' [DEBUG] Received complete event:', data);
 if (data.complexity_score !== undefined) {
 setComplexityScore(data.complexity_score);
 setJudgeDecision(data.executed_locally ? 'local' : 'cloud');
 }
 if (data.final_result) {
 console.log(' [DEBUG] final_result length:', data.final_result.length);
 console.log(' [DEBUG] final_result preview:', data.final_result.substring(0, 100));

 addChatHistory(taskText, data.final_result);

 setResult({
 final_result: data.final_result,
 steps: [],
 executed_locally: data.executed_locally,
 total_duration_seconds: data.total_duration,
 start_time: new Date().toISOString(),
 end_time: new Date().toISOString(),
 complexity_score: data.complexity_score,
 });
 }
 addLog('result', 'success', '所有步骤已完成，最终结果已生成');
 break;

 case 'error':
 addLog('system', 'error', ` workflow执行失败: ${data.error}`);
 break;
 }
}
);
} catch (error) {
 console.error(' [Workflow API Error]', error);
 addLog(undefined, 'error', `后端API调用失败: ${error instanceof Error ? error.message : String(error)}`);
 setResult(null);
 setIsRunning(false);
 setCurrentStep(null);
} finally {
 setIsRunning(false);
 setCurrentStep(null);
}
}, [setCurrentTask, setIsRunning, setResult, setCurrentStep, setComplexityScore, setJudgeDecision, addLog, addCompletedStep,
addWorkflowStep, updateAgentStatus, addChatHistory]);

const handleSubmit = async () => {
 const taskText = inputValue.trim();
 if (!taskText || isRunning) return;

 if (useMock) {
 await executeWithMock(taskText);
 } else {
 await executeWithAPI(taskText);
 }

 setInputValue('');
};

const handleStop = () => {
 setConfirmDialog({
 title: '停止任务',
 message: '确定要停止当前正在运行的任务吗？已完成的步骤将保留。',
 onConfirm: () => {
 setIsRunning(false);
 setCurrentStep(null);
 setConfirmDialog(null);
 },
 });
};

const handleClear = () => {
 setConfirmDialog({
 title: '清空对话',
 message: '确定要清空所有对话历史和任务数据吗？此操作将清除所有记忆，开始新的对话。',
 onConfirm: () => {
 resetWorkflow();
 resetAllAgents();
 clearChatHistory();
 setInputValue('');
 setConfirmDialog(null);
 },
 });
};

```

```

 });
};

const handleDetectLocalModel = async () => {
 console.log('[检测本地模型] 开始自动检测 Ollama 端口...');
 setDetectingLocalModel(true);
 setLocalModelDetectResult(null);
 try {
 // 不传递 host 和 port, 让后端自动检测本地常见端口
 const response = await configService.detectOllama();
 console.log('[检测本地模型] 检测结果:', response);

 setLocalModelDetectResult({
 success: true,
 message: response.message,
 });

 if (response.ollama_host) {
 try {
 const url = new URL(response.ollama_host);
 setLocalModelPathInput(url.hostname);
 setLocalModelPort(url.port || '11435');
 } catch (e) {
 console.error('[检测本地模型] 解析 URL 失败:', e);
 }
 }
 } catch (error) {
 console.error('[检测本地模型] 错误:', error);
 const errMsg = error instanceof Error ? error.message : '检测失败';
 setLocalModelDetectResult({
 success: false,
 message: errMsg,
 });
 }
 setDetectingLocalModel(false);
};

const handleTestLocalConnection = async () => {
 console.log('[测试连接] 测试当前端口...');
 setTestingConnection(true);
 setTestConnectionResult(null);
 try {
 // 构建测试用的 host 和 port
 const host = localModelPathInput;
 const port = localModelPort;

 console.log('[测试连接] 测试:', { host, port });
 const response = await configService.testOllama(host, port);
 console.log('[测试连接] 结果:', response);

 setTestConnectionResult({
 success: response.success,
 message: response.message,
 });
 } catch (error) {
 console.error('[测试连接] 错误:', error);
 const errMsg = error instanceof Error ? error.message : '测试失败';
 setTestConnectionResult({
 success: false,
 message: errMsg,
 });
 }
 setTestingConnection(false);
};

const handleSaveLocalModel = async () => {
 console.log('[保存本地模型] 开始保存...');
 try {
 let ollamaHost;
 if (localModelPathInput.indexOf('http') === 0) {
 ollamaHost = localModelPathInput;
 } else {
 ollamaHost = 'http://' + localModelPathInput + ':' + localModelPort;
 }

 console.log('[保存本地模型] ollamaHost:', ollamaHost);
 await configService.update({ ollama_host: ollamaHost });

 console.log('[保存本地模型] 保存成功');
 setShowLocalModelDialog(false);
 }
};

```

```

 addLog('system', 'info', '本地模型配置已保存: ' + ollamaHost);
 } catch (error) {
 console.error('[保存本地模型] 错误:', error);
 const errMsg = error instanceof Error ? error.message : '未知错误';
 addLog('system', 'error', '配置保存失败: ' + errMsg);
 }
};

const handleAddCloudModel = async () => {
 try {
 await configService.createModel({
 name: selectedModel,
 provider: selectedProvider,
 model: selectedModel,
 api_key: apiKey,
 display_name: displayName || selectedModel,
 max_tokens: maxTokens,
 temperature: temperature,
 });
 addLog('system', 'success', '云端模型 ' + selectedModel + ' 已添加');
 setShowCloudModelDialog(false);
 setSelectedProvider('');
 setSelectedModel('');
 setApiKey('');
 setDisplayName('');
 setMaxTokens(4096);
 setTemperature(0.7);
 } catch (error) {
 addLog('system', 'error', '添加云端模型失败: ' + (error instanceof Error ? error.message : '未知错误'));
 }
};

const handleValidateApiKey = async () => {
 console.log('[验证密钥] 开始验证...');
 if (!selectedProvider || !apiKey) {
 console.log('[验证密钥] 缺少服务商或API密钥');
 return;
 }
 setValidatingApiKey(true);
 setApiKeyValidationResult(null);
 try {
 console.log('[验证密钥] 参数:', { provider: selectedProvider, api_key: apiKey.length > 0 ? '***' : '', model: selectedModel });
 const response = await configService.validateApiKey({
 provider: selectedProvider,
 api_key: apiKey,
 model: selectedModel,
 });
 console.log('[验证密钥] 响应:', response);
 setApiKeyValidationResult({
 success: response.success,
 message: response.message,
 });
 } catch (error) {
 console.error('[验证密钥] 错误:', error);
 setApiKeyValidationResult({
 success: false,
 message: error instanceof Error ? error.message : '验证失败',
 });
 }
 setValidatingApiKey(false);
};

const modelProviders = [
 { value: 'deepseek', label: 'DeepSeek' },
 { value: 'gemini', label: 'Gemini' },
 { value: 'openai', label: 'OpenAI' },
 { value: 'anthropic', label: 'Anthropic' },
];

const modelsByProvider: Record<string, { value: string; label: string }[]> = {
 deepseek: [
 { value: 'deepseek-chat', label: 'DeepSeek Chat' },
 { value: 'deepseek-r1-distill', label: 'DeepSeek R1 Distill' },
 { value: 'deepseek-r1', label: 'DeepSeek R1' },
 { value: 'deepseek-v4-flash', label: 'DeepSeek V4 Flash' },
],
 gemini: [
 { value: 'gemini-pro', label: 'Gemini Pro' },
 { value: 'gemini-1.5-pro', label: 'Gemini 1.5 Pro' },
],
};

```

```

 { value: 'gemini-1.5-flash', label: 'Gemini 1.5 Flash' },
],
 openai: [
 { value: 'gpt-4o', label: 'GPT-4o' },
 { value: 'gpt-4-turbo', label: 'GPT-4 Turbo' },
 { value: 'gpt-3.5-turbo', label: 'GPT-3.5 Turbo' },
],
 anthropic: [
 { value: 'claude-3-5-sonnet', label: 'Claude 3.5 Sonnet' },
 { value: 'claude-3-opus', label: 'Claude 3 Opus' },
 { value: 'claude-3-sonnet', label: 'Claude 3 Sonnet' },
],
];

const handleContextMenu = (e: React.MouseEvent, target: string, type: 'node' | 'agent') => {
 e.preventDefault();
 e.stopPropagation();
 setContextMenu({
 visible: true,
 x: e.clientX,
 y: e.clientY,
 target,
 type,
 });
};

const handleContextAction = (action: string) => {
 if (action === 'copy') {
 navigator.clipboard?.writeText(contextMenu.target);
 } else if (action === 'rerun') {
 setInputValue(contextMenu.target);
 } else if (action === 'detail') {
 setNodeDetail(contextMenu.target);
 }
 setContextMenu(prev => ({ ...prev, visible: false }));
};

const toggleAgentDetail = (agentId: string) => {
 setExpandedAgentDetails(prev => {
 const next = new Set(prev);
 if (next.has(agentId)) next.delete(agentId);
 else next.add(agentId);
 return next;
 });
};

const getNodeStatus = (agentId: AgentId) => {
 if (completedSteps.includes(agentId)) return 'completed';
 if (currentStep === agentId) return 'working';
 return 'pending';
};

const getArrowClass = (fromAgent: AgentId | null, toAgent: AgentId | null) => {
 if (!fromAgent && toAgent) {
 if (completedSteps.includes(toAgent)) return 'flow-success';
 if (currentStep === toAgent) return 'flow-active';
 return '';
 }
 if (fromAgent && completedSteps.includes(fromAgent)) {
 if (toAgent && (completedSteps.includes(toAgent) || currentStep === toAgent)) return 'flow-success';
 return '';
 }
 if (fromAgent && currentStep === fromAgent) return 'flow-active';
 return '';
};

const apiCallCount = completedSteps.includes('judge') ? 1 : 0;
const apiCallTotal = 1;
const apiCallPercent = apiCallTotal > 0 ? Math.round((apiCallCount / apiCallTotal) * 100) : 0;

const costSaved = completedSteps.length > 0 ? (completedSteps.length * 0.02).toFixed(3) : '0.000';
const costSavedPercent = completedSteps.length > 0 ? Math.min(62, completedSteps.length * 10) : 0;

const progressPercent = completedSteps.length > 0 ? Math.round((completedSteps.length / 6) * 100) : 0;

const now = new Date();
const getTimeStr = (offset: number) => {
 const d = new Date(now.getTime() - offset * 1000);
 return d.toLocaleTimeString('zh-CN', { hour12: false });
};

```

```

const enabledCount = Object.values(agents).filter(a => a.enabled).length;

const renderMarkdown = (text: string) => {
 return text
 .replace(/~# (.*)/gm, '<h1>$1</h1>')
 .replace(/~## (.*)/gm, '<h2>$1</h2>')
 .replace(/~### (.*)/gm, '<h3>$1</h3>')
 .replace(/~*(.*)~*/gm, '$1')
 .replace(/~ (.*)/gm, '$1')
 .replace(/~[\s\S]*?/g, '$1')
 .replace(/~\n~/g, '
');
};

const renderResourceChart = (data: number[], gradId: string, strokeColor: string, fillColor: string) => {
 const pathD = generateSvgPath(data);
 const fillD = `${pathD} V32 H0 Z`;
 return (
 <svg width="100%" height="32" viewBox="0 0 240 32" preserveAspectRatio="none">
 <defs>
 <linearGradient id={gradId} x1="0" y1="0" x2="0" y2="1">
 <stop offset="0%" stopColor={fillColor} />
 <stop offset="100%" stopColor={fillColor.replace(/[\d.]+\$/, '0')} />
 </linearGradient>
 </defs>
 <path d={fillD} fill={`url(#${gradId})`} />
 <path d={pathD} fill="none" stroke={strokeColor} strokeWidth="1.5" />
 </svg>
);
};

return (
 <>
 <header className="header">
 <div className="header-left">
 <div className="logo">
 <div className="logo-hexagon"></div>
 NeuroFlow
 多智能体协同平台
 </div>
 <div className="task-bar" style={{ cursor: 'pointer' }} onClick={() => setTaskListOpen(!taskListOpen)}>
 任务: {chatHistory.length + 1}
 <input
 type="text"
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
 onKeyDown={(e) => {
 if (e.key === 'Enter') {
 e.preventDefault();
 handleSubmit();
 }
 }}
 onClick={(e) => e.stopPropagation()}
 placeholder={currentTask ? `继续对话... 上次任务: ${currentTask.slice(0, 30)}${currentTask.length > 30 ? '...'` : ''}` : `输入您的任务需求... 例如: 帮我写一份关于智能体协作的年度计划`}
 className="task-input"
 disabled={isRunning}
 />
 {taskListOpen && chatHistory.length > 0 && (
 <div style={{
 position: 'absolute', top: '100%', left: 0, right: 0,
 background: 'var(--bg-card)', border: '1px solid var(--border-color)',
 borderRadius: 8, padding: 8, zIndex: 50, marginTop: 4,
 boxShadow: '0 8px 24px rgba(0,0,0,0.3)',
 maxHeight: '300px',
 overflowY: 'auto',
 }}>
 <div style={{ fontSize: 12, color: 'var(--text-muted)', padding: '4px 8px', marginBottom: 8, borderBottom: '1px solid var(--border-color)' }}>
 对话历史 ({chatHistory.length} 条)
 </div>
 {[...chatHistory].reverse().map((item, index) => (
 <div key={index} style={{
 padding: '8px',
 marginBottom: 8,
 background: 'var(--bg-secondary)',
 borderRadius: 4,
 cursor: 'pointer',
 transition: 'background 0.2s',
 }}

```

```

 onClick={(e) => {
 e.stopPropagation();
 setInputValue(item.user_input);
 setTaskListOpen(false);
 }}
 onMouseEnter={(e) => (e.currentTarget as HTMLElement).style.background = 'var(--bg-hover)'}
 onMouseLeave={(e) => (e.currentTarget as HTMLElement).style.background = 'var(--bg-secondary)'}>
 <div style={{ fontSize: 13, color: 'var(--text-primary)', marginBottom: 4 }}>
 用户: {item.user_input}
 </div>
 <div style={{ fontSize: 12, color: 'var(--text-secondary)', maxHeight: '60px', overflow: 'hidden' }}>
 助手: {item.response.slice(0, 100)}{item.response.length > 100 ? '...' : ''}
 </div>
 </div>
))}
 </div>
)}
</div>
{isRunning && (
 <div className="task-status">
 <div className="status-dot"></div>
 运行中
 {formatTime(elapsedSeconds)}
 </div>
)}
{!isRunning && currentTask && completedSteps.length > 0 && (
 <div className="task-status">
 <div style={{ width: 8, height: 8, background: 'var(--green)', borderRadius: '50%', boxShadow: '0 0 6px rgba(16, 185, 129, 0.5)' }}></div>
 已完成
 </div>
)}
</div>
<div className="header-right">
 <button
 className={`btn btn-primary${isRunning ? ' running' : ''}`}
 onClick={isRunning ? handleStop : handleSubmit}
 disabled={!isRunning && (!inputValue.trim() && !currentTask)}
 >
 {isRunning ? (
 <
 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <rect x="6" y="4" width="4" height="16" />
 <rect x="14" y="4" width="4" height="16" />
 </svg>
 停止
 </
) : (
 <
 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <polyline points="5 3 19 12 5 21 5 3" />
 </svg>
 运行任务
 </
)
)}
</button>
<button className="btn btn-secondary" onClick={handleStop} disabled={!isRunning}>
 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <rect x="6" y="4" width="4" height="16" />
 <rect x="14" y="4" width="4" height="16" />
 </svg>
 停止
</button>
<button className="btn btn-secondary" onClick={handleClear}>
 <svg width="14" height="14" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <polyline points="3 6 5 6 21 6" />
 <path d="M19 6v14a2 2 0 0 1-2 2H7a2 2 0 0 1-2 2V6m3 0V4a2 2 0 0 1 2-2h4a2 2 0 0 1 2-2v2" />
 </svg>
 清空
</button>
<button className="btn btn-icon" onClick={() => setIsDarkTheme(!isDarkTheme)} title={isDarkTheme ? '切换浅色主题' : '切换深色主题'}>
 {isDarkTheme ? (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <circle cx="12" cy="12" r="5" />
 <line x1="12" y1="1" x2="12" y2="3" />
 <line x1="12" y1="21" x2="12" y2="23" />
 <line x1="4.22" y1="4.22" x2="5.64" y2="5.64" />
 <line x1="18.36" y1="18.36" x2="19.78" y2="19.78" />
 <line x1="1" y1="12" x2="3" y2="12" />
 </svg>
) : (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <circle cx="12" cy="12" r="5" />
 <line x1="12" y1="1" x2="12" y2="3" />
 <line x1="12" y1="21" x2="12" y2="23" />
 <line x1="4.22" y1="4.22" x2="5.64" y2="5.64" />
 <line x1="18.36" y1="18.36" x2="19.78" y2="19.78" />
 <line x1="1" y1="12" x2="3" y2="12" />
 </svg>
)}
</button>

```

```

 <line x1="21" y1="12" x2="23" y2="12" />
 <line x1="4.22" y1="19.78" x2="5.64" y2="18.36" />
 <line x1="18.36" y1="5.64" x2="19.78" y2="4.22" />
 </svg>
) : (
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <path d="M21 12.79A9 9 0 1 1 11.21 3 7 7 0 0 0 21 12.79z" />
 </svg>
)
 </button>
</div>
</header>

<div className="main-container">
 <aside className="sidebar-left">
 <div className="card animate-in delay-1">
 <div className="card-title">API调用次数</div>
 <div className="card-value">{apiCallCount} / {apiCallTotal} 次</div>
 <div className="progress-ring-container">
 <ProgressRing percentage={apiCallPercent} color="var(--blue)" />
 <div style={{ flex: 1 }}>
 <div className="progress-bar">
 <div className="progress-bar-fill" style={{ width: `${apiCallPercent}%`, background: 'var(--blue)' }}></div>
 </div>
 </div>
 </div>
 <div className="card-info" style={{ marginTop: 8 }}>
 纯API模式基准: 1次
 </div>
 </div>

 <div className="card animate-in delay-2">
 <div className="card-title">预估节省成本</div>
 <div className="card-value green">¥{costSaved}</div>
 <div className="progress-ring-container">
 <ProgressRing percentage={costSavedPercent} color="var(--green)" />
 <div style={{ flex: 1 }}>
 <div className="progress-bar">
 <div className="progress-bar-fill" style={{ width: `${costSavedPercent}%`, background: 'var(--green)' }}></div>
 </div>
 </div>
 </div>
 <div className="card-info" style={{ marginTop: 8 }}>
 纯API成本: ¥0.18
 节省 {costSavedPercent}%
 </div>
 </div>

 <div className="card animate-in delay-3">
 <div className="card-title">本地算力负载</div>
 <div className="stats-row">
 <div className="stat-item">
 <div className="stat-label">CPU</div>
 <div className="stat-value">{cpuValue}%</div>
 </div>
 <div className="stat-item">
 <div className="stat-label">显存</div>
 <div className="stat-value">{gpuValue}GB</div>
 </div>
 </div>
 <div className="progress-ring-container">
 <ProgressRing percentage={cpuValue} color="var(--green)" />
 <div style={{ flex: 1 }}>
 <div className="progress-bar">
 <div className="progress-bar-fill" style={{ width: `${cpuValue}%`, background: 'var(--green)' }}></div>
 </div>
 </div>
 </div>
 </div>

 <div className="card animate-in delay-4">
 <div className="card-title">响应时间</div>
 <div className="card-value">2.3s</div>
 <div className="progress-ring-container">
 <ProgressRing percentage={56} color="var(--blue)" />
 <div style={{ flex: 1 }}>
 <div className="progress-bar">
 <div className="progress-bar-fill" style={{ width: '56%', background: 'var(--blue)' }}></div>
 </div>
 </div>
 </div>
 </div>
 </aside>

```

```

 </div>
 </div>
 <div className="card-info" style={{ marginTop: 8 }}>
 纯API模式: 4.1s
 更快
 </div>
</div>

<div className="card animate-in delay-5">
 <div className="agent-fleet-header">
 Agent舰队
 {enabledCount}/6 启用
 </div>

 {AGENT_ORDER.map((agentId) => {
 const agent = agents[agentId];
 const iconClass = AGENT_ICON_CLASSES[agentId];
 const isActive = agent.status === 'processing';
 const isCompleted = agent.status === 'completed';

 return (
 <div
 key={agentId}
 className={`agent-item${!agent.enabled ? ' disabled-agent' : ''}`}
 onContextMenu={(e) => handleContextMenu(e, AGENT_DISPLAY_NAMES[agentId], 'agent')}
 >
 <div className={`agent-icon ${iconClass}`}>
 {AGENT_SVG_ICONS[agentId]}
 </div>
 <div className="agent-info">
 <div className="agent-name">{AGENT_DISPLAY_NAMES[agentId]}</div>
 <div className="agent-model">{AGENT_MODELS[agentId]}</div>
 <div className="agent-desc">{AGENT_DESCRIPTIONS[agentId]}</div>
 </div>
 <div className="agent-status">
 {isCompleted && <
span className="checkmark-anim">✓ 已完成}
 {isActive && 工作中}
 {!isActive && !isCompleted && 空闲}
 <div
 className={`toggle-switch${!agent.enabled ? ' off' : ''}`}
 onClick={() => toggleAgentEnabled(agentId)}
 title={agent.enabled ? '点击禁用' : '点击启用'}
 >
 />
 </div>
 </div>
);
 })}
</div>

<div className="card animate-in delay-5">
 <div className="card-title">资源监控</div>
 <div className="resource-item">
 <div className="resource-header">
 CPU使用率
 {cpuValue}%
 </div>
 <div className="resource-chart">
 {renderResourceChart(cpuData, 'cpuGrad', 'var(--blue)', 'rgba(59, 130, 246, 0.3)')}
 </div>
 </div>
 <div className="resource-item">
 <div className="resource-header">
 内存占用
 {memValue}GB / 16GB
 </div>
 <div className="resource-chart">
 {renderResourceChart(memData, 'memGrad', 'var(--green)', 'rgba(16, 185, 129, 0.3)')}
 </div>
 </div>
 <div className="resource-item">
 <div className="resource-header">
 显存占用
 {gpuValue}GB / 8GB
 </div>
 <div className="resource-chart">
 {renderResourceChart(gpuData, 'gpuGrad', 'var(--purple)', 'rgba(139, 92, 246, 0.3)')}
 </div>
 </div>
</div>

```

```

<div className="card animate-in delay-5">
 <div className="card-title">系统设置</div>
 <div className="setting-item">
 <button
 className="btn btn-secondary"
 onClick={() => setShowLocalModelDialog(true)}
 style={{ width: '100%', justifyContent: 'flex-start' }}
 >
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <path d="M12 20h9" />
 <path d="M16.5 3.5a2.121 2.121 0 0 1 3.317 19.1-4 1 1-4L16.5 3.5z" />
 </svg>
 本地模型设置
 </button>
 </div>
 <div className="setting-item">
 <button
 className="btn btn-secondary"
 onClick={() => setShowCloudModelDialog(true)}
 style={{ width: '100%', justifyContent: 'flex-start' }}
 >
 <svg width="16" height="16" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2">
 <path d="M19 18h6a4 4 0 0 1-1.172-7.836 5.5 5.5 0 0 1 10.63-1.636 3.5 3.5 0 0 1 4.54 5.46z" />
 <path d="M12 12m-2 0a2 2 0 1 0 4 0a2 2 0 1 0 -4 0" />
 <path d="M21 12v4a2 2 0 0 1-2 2H5a2 2 0 0 1-2-2v-4" />
 </svg>
 添加云端模型
 </button>
 </div>
 <div className="setting-item">
 <div className="setting-label">难度阈值（置信度）</div>
 <div className="slider-container">
 <input
 type="range"
 className="slider"
 min={30}
 max={90}
 value={Math.round(threshold * 100)}
 onChange={(e) => setThreshold(Number(e.target.value) / 100)}
 />
 <div className="slider-labels">
 0.3
 0.9
 </div>
 <div className="slider-value">{threshold.toFixed(2)}</div>
 </div>
 </div>
</div>

<div className="console-log-section">
 <div className="console-log-header" onClick={() => setConsoleExpanded(!consoleExpanded)}>
 控制台日志
 ▼
 </div>
 <div className={`console-log-entries${consoleExpanded ? 'expanded' : ''}`>
 {logs.length === 0 ? (
 <div style={{ fontSize: 11, color: 'var(--text-muted)', padding: '8px 0' }}>暂无日志</div>
) : (
 [...logs].reverse().slice(0, 50).map((log: LogEntry) => (
 <div key={log.id} className="console-log-entry">

 {new Date(log.timestamp).toLocaleTimeString('zh-CN', { hour12: false })}

 {log.type.toUpperCase()}
 {log.message}
 </div>
))
)}
 </div>
</div>
</aside>

<main className="content-center">
 <div className="pipeline-section">
 <div className="section-header">
 多智能体协同流水线
 </div>
 <div className="legend">
 <div className="legend-item">
 <div className="legend-line solid"></div>

```

```

 数据流
 </div>
 <div className="legend-item">
 <div className="legend-line dashed"></div>
 控制流
 </div>
 </div>
 </div>

 <div className="pipeline-container">
 <div
 className="pipeline-node node-input animate-in delay-1"
 onClick={() => setNodeDetail('用户输入')}
 onContextMenu={(e) => handleContextMenu(e, '用户输入', 'node')}
 >
 <div className="node-icon">👤</div>
 <div className="node-title">用户输入</div>
 <div className="node-subtitle">{currentTask || '等待输入任务...'}</div>
 </div>

 <div className={`arrow-down data-flow ${getArrowClass(null, 'knowledge')}`}></div>

 <div
 className={`pipeline-node ${AGENT_NODE_CLASSES.knowledge} ${getNodeStatus('knowledge') === 'completed' ? 'node-completed' : 'node-working' ? 'node-working' : ''} animate-in delay-2`}
 onClick={() => setNodeDetail(AGENT_DISPLAY_NAMES.knowledge)}
 onContextMenu={(e) => handleContextMenu(e, AGENT_DISPLAY_NAMES.knowledge, 'node')}
 >
 <div className="node-icon">{AGENT_EMOJIS.knowledge}</div>
 <div className="node-title">{AGENT_DISPLAY_NAMES.knowledge}</div>
 <div className="node-subtitle">知识库检索与上下文增强</div>
 <div className={`node-status ${getNodeStatus('knowledge')}`}>
 {getNodeStatus('knowledge') === 'working' ? (
 <> 检索中...</>
) : getNodeStatus('knowledge') === 'completed' ? (
 <> 已完成</>
) : (
 '等待中'
)}
 </div>
 </div>

 <div className={`arrow-down data-flow ${getArrowClass('knowledge', 'summary')}`}></div>

 <div className="pipeline-row animate-in delay-3">
 {[('summary', 'writer') as AgentId[]].map((agentId) => {
 const status = getNodeStatus(agentId);
 const subtitles: Record<AgentId, string> = {
 knowledge: '知识库检索与上下文增强',
 summary: '关键信息摘要与提取',
 writer: '内容初稿生成与规划',
 review: '质量评估与修改建议',
 judge: '最终决策与路径选择',
 result: '生成最终完整回答',
 };
 });
 return (
 <div
 key={agentId}
 className={`pipeline-node ${AGENT_NODE_CLASSES[agentId]} ${status === 'completed' ? 'node-completed' : ''} ${status === 'working' ? 'node-working' : ''}`}
 onClick={() => setNodeDetail(AGENT_DISPLAY_NAMES[agentId])}
 onContextMenu={(e) => handleContextMenu(e, AGENT_DISPLAY_NAMES[agentId], 'node')}
 >
 <div className="node-icon">{AGENT_EMOJIS[agentId]}</div>
 <div className="node-title">{AGENT_DISPLAY_NAMES[agentId]}</div>
 <div className="node-subtitle">{subtitles[agentId]}</div>
 <div className={`node-status ${status}`}>
 {status === 'working' ? (
 <> 处理中...</>
) : status === 'completed' ? (
 <> 已完成</>
) : (
 '等待中'
)}
 </div>
 </div>
);
)}
 </div>

```

```

<div className={`arrow-down data-flow ${getArrowClass('writer', 'review')}}></div>

<div
 className={`pipeline-node node-review ${getNodeStatus('review') === 'completed' ? 'node-completed' : ''} ${get
NodeStatus('review') === 'working' ? 'node-working' : ''} animate-in delay-4`}
 onClick={() => setNodeDetail(AGENT_DISPLAY_NAMES.review)}
 onContextMenu={(e) => handleContextMenu(e, AGENT_DISPLAY_NAMES.review, 'node')}
>
 <div className="node-icon">{AG
... (内容过长, 已截断) ...

```

## frontend\src\services\api\agentService.ts

```

import axios from 'axios';
import type {
 WorkflowInput,
 WorkflowOutput,
 Metrics,
 AgentId,
 AgentStatusResponse,
 KnowledgeEntry,
 KnowledgeInput,
 ExportInput,
 HealthResponse,
 ChatRequest,
 ChatResponse,
 ExportRequest,
 ExportResponse,
} from '@/types';

const API_BASE_URL = process.env.NEXT_PUBLIC_API_URL || 'http://localhost:8000';

const api = axios.create({
 baseURL: `${API_BASE_URL}/api/v1`,
 headers: {
 'Content-Type': 'application/json',
 },
 timeout: 120000,
});

api.interceptors.request.use(
 (config) => {
 return config;
 },
 (error) => Promise.reject(error)
);

api.interceptors.response.use(
 (response) => response,
 (error) => {
 if (axios.isAxiosError(error)) {
 const message = error.response?.data?.detail || error.message || '请求失败';
 console.error('[API Error] ${error.config?.url}: ', message);
 } else {
 console.error('[API Error] Unexpected error:', error);
 }
 return Promise.reject(error);
 }
);

// ===== Workflow Service =====
export const workflowService = {
 async execute(input: WorkflowInput): Promise<WorkflowOutput> {
 const response = await api.post<WorkflowOutput>('/workflow/execute', input);
 return response.data;
 },

 async executeParallel(input: WorkflowInput): Promise<WorkflowOutput> {
 const response = await api.post<WorkflowOutput>('/workflow/execute/parallel', input);
 return response.data;
 },

 async getCacheStats(): Promise<{ cache_size: number; max_size: number; ttl: number }> {

```

```

 const response = await api.get('/workflow/cache/stats');
 return response.data;
 },

 async clearCache(): Promise<{ status: string; message: string }> {
 const response = await api.post('/workflow/cache/clear');
 return response.data;
 },
};

// ===== Agent Service =====
export const agentService = {
 async getAll(): Promise<{ agents: AgentStatusResponse[]; count: number }> {
 const response = await api.get('/agents');
 return response.data;
 },

 async get(agentId: AgentId): Promise<AgentStatusResponse> {
 const response = await api.get<AgentStatusResponse>(`/agents/${agentId}`);
 return response.data;
 },

 async getStatus(agentId: AgentId): Promise<AgentStatusResponse> {
 const response = await api.get<AgentStatusResponse>(`/agents/${agentId}/status`);
 return response.data;
 },

 async executeAgent(agentId: AgentId, input: { content: string; context?: Record<string, unknown> }): Promise<any> {
 const response = await api.post(`/agents/${agentId}/execute`, input);
 return response.data;
 },
};

// ===== Metrics Service =====
export const metricsService = {
 async getDashboard(): Promise<Metrics> {
 const response = await api.get<Metrics>('/metrics');
 return response.data;
 },

 async getSystem(): Promise<{ cpu_usage: number; memory_usage: number; disk_usage: number; process_count: number }> {
 const response = await api.get('/metrics/system');
 return response.data;
 },

 async incrementMetric(metricType: string, value: number = 1.0): Promise<{ status: string; metric: string; value: number }> {
 const response = await api.post(`/metrics/increment/${metricType}`, { value });
 return response.data;
 },
};

// ===== Knowledge Service =====
export const knowledgeService = {
 async list(): Promise<{ knowledge_base: Record<string, string[]>; keywords: string[]; stats: any }> {
 const response = await api.get('/knowledge');
 return response.data;
 },

 async getStats(): Promise<{ total_entries: number; total_keywords: number; total_size: number }> {
 const response = await api.get('/knowledge/stats');
 return response.data;
 },

 async add(entry: KnowledgeInput): Promise<{ status: string; keyword: string }> {
 const response = await api.post('/knowledge', entry);
 return response.data;
 },

 async search(query: string, limit: number = 5): Promise<{ query: string; results: any[]; count: number }> {
 const response = await api.get('/knowledge/search', { params: { query, limit } });
 return response.data;
 },

 async getKeyword(keyword: string): Promise<{ keyword: string; content: string[] }> {
 const response = await api.get(`/knowledge/keyword/${keyword}`);
 return response.data;
 },

 async updateKeyword(keyword: string, content: string[]): Promise<{ status: string; keyword: string }> {
 const response = await api.put(`/knowledge/keyword/${keyword}`, content);

```

```

 return response.data;
 },

 async deleteKeyword(keyword: string): Promise<{ status: string; keyword: string }> {
 const response = await api.delete(`/knowledge/keyword/${keyword}`);
 return response.data;
 },

 async enhance(content: string, keywords: string[]): Promise<{ original: string; enhanced: string; keywords: string[] }> {
 const response = await api.post(`/knowledge/enhance`, { content, keywords });
 return response.data;
 },
};

// ===== Export Service =====
export const exportService = {
 async exportMarkdown(request: ExportRequest): Promise<ExportResponse> {
 const response = await api.post<ExportResponse>(`/export/markdown`, request);
 return response.data;
 },

 async exportDocx(request: ExportRequest): Promise<ExportResponse> {
 const response = await api.post<ExportResponse>(`/export/docx`, request);
 return response.data;
 },

 async exportPptx(request: ExportRequest): Promise<ExportResponse> {
 const response = await api.post<ExportResponse>(`/export/pptx`, request);
 return response.data;
 },

 async download(filename: string): Promise<Blob> {
 const response = await api.get(`/export/download/${filename}`, { responseType: 'blob' });
 return response.data;
 },

 async list(): Promise<{ exports: any[]; count: number }> {
 const response = await api.get(`/export/list`);
 return response.data;
 },
};

// ===== Chat Service =====
export const chatService = {
 async send(request: ChatRequest): Promise<ChatResponse> {
 const response = await api.post<ChatResponse>(`/chat/send`, request);
 return response.data;
 },

 async sendStream(
 request: ChatRequest,
 onMessage: (data: { type: string; agent_id?: string; agent_name?: string; message?: string; final_result?: string; execute_d_locally?: boolean; complexity_score?: number; total_duration?: number; steps_count?: number; duration?: number; success?: boolean; error?: string }) => void
): Promise<void> {
 const response = await fetch(`${API_BASE_URL}/api/v1/chat/send/stream`, {
 method: 'POST',
 headers: {
 'Content-Type': 'application/json',
 Accept: 'text/event-stream',
 },
 body: JSON.stringify(request),
 });

 if (!response.ok) {
 throw new Error(`HTTP error! status: ${response.status}`);
 }

 const reader = response.body?.getReader();
 if (!reader) {
 throw new Error('Failed to get response reader');
 }

 const decoder = new TextDecoder('utf-8');
 let buffer = '';

 try {
 while (true) {
 const { done, value } = await reader.read();
 if (done) break;

```

```

 buffer += decoder.decode(value, { stream: true });

 while (true) {
 const newlineIndex = buffer.indexOf('\n');
 if (newlineIndex === -1) break;

 const line = buffer.substring(0, newlineIndex);
 buffer = buffer.substring(newlineIndex + 1);

 if (line.startsWith('data: ')) {
 try {
 const parsedData = JSON.parse(line.substring(6));
 onMessage(parsedData);
 } catch (e) {
 console.error('Error parsing stream data:', e);
 }
 }
 }
 } finally {
 reader.releaseLock();
 }
},

async sendBatch(requests: ChatRequest[]): Promise<{ results: ChatResponse[] }> {
 const response = await api.post('/chat/send/batch', requests);
 return response.data;
},

async health(): Promise<{ status: string; service: string; cache_size: number }> {
 const response = await api.get('/chat/health');
 return response.data;
},

async getCacheStats(): Promise<{
 chat_cache_size: number;
 chat_cache_max_size: number;
 chat_cache_ttl: number;
 workflow_cache_size: number;
 workflow_cache_max_size: number;
 workflow_cache_ttl: number;
}> {
 const response = await api.get('/chat/cache/stats');
 return response.data;
},

async clearCache(): Promise<{ status: string; message: string }> {
 const response = await api.post('/chat/cache/clear');
 return response.data;
},
};

// ===== Config Service =====
export const configService = {
 async get(): Promise<{ ollama_host: string; ollama_model: string; deepseek_api_key_set: boolean; deepseek_model: string; models: any[] }> {
 const response = await api.get('/config');
 return response.data;
 },

 async update(config: { deepseek_api_key?: string; ollama_host?: string }): Promise<{ message: string; saved: boolean }> {
 const response = await api.post('/config', config);
 return response.data;
 },

 async listModels(): Promise<{ models: any[] }> {
 const response = await api.get('/config/models');
 return response.data;
 },

 async createModel(model: { name: string; provider: string; model: string; api_key?: string; display_name?: string; max_tokens?: number; temperature?: number }): Promise<{ message: string; model: any }> {
 const response = await api.post('/config/models', model);
 return response.data;
 },

 async deleteModel(modelName: string): Promise<{ message: string }> {
 const response = await api.delete('/config/models/${modelName}');
 return response.data;
 }
};

```

```

 },

 async validateApiKey(request: { provider: string; api_key: string; model?: string }): Promise<{ success: boolean; message: string }> {
 const response = await api.post('/config/validate-key', request);
 return response.data;
 },

 async detectOllama(host?: string, port?: string): Promise<{ ollama_host: string; message: string }> {
 const response = await api.post('/config/detect-ollama', { host, port });
 return response.data;
 },

 async testOllama(host?: string, port?: string): Promise<{ success: boolean; message: string; details?: Record<string, string> }> {
 const response = await api.post('/config/test-ollama', { host, port });
 return response.data;
 },

 async testDeepseek(): Promise<{ success: boolean; message: string; details?: Record<string, string> }> {
 const response = await api.post('/config/test-deepseek');
 return response.data;
 },
};

// ===== Health Service =====
export const healthService = {
 async check(): Promise<HealthResponse> {
 const response = await api.get<HealthResponse>('/health', { baseUrl: API_BASE_URL });
 return response.data;
 },
};

export default api;

```

## frontend\src\services\api\socketService.ts

```

import { io, Socket } from 'socket.io-client';
import type { AgentId, AgentStatus, Metrics, WorkflowStep, WorkflowOutput, LogEntry } from '@types';

const WS_URL = process.env.NEXT_PUBLIC_WS_URL || 'ws://localhost:8000';

type EventHandler<T = unknown> = (data: T) => void;

interface SocketService {
 connect: () => void;
 disconnect: () => void;
 on: {
 stepStart: (handler: EventHandler<{ agent_id: AgentId; agent_name: string }>) => void;
 stepComplete: (handler: EventHandler<WorkflowStep>) => void;
 stepError: (handler: EventHandler<{ agent_id: AgentId; error: string }>) => void;
 workflowComplete: (handler: EventHandler<WorkflowOutput>) => void;
 agentStatusUpdate: (handler: EventHandler<{ agent_id: AgentId; status: AgentStatus; task?: string }>) => void;
 metricsUpdate: (handler: EventHandler<Metrics>) => void;
 newLog: (handler: EventHandler<LogEntry>) => void;
 connect: (handler: EventHandler) => void;
 disconnect: (handler: EventHandler) => void;
 };
 off: {
 stepStart: (handler: EventHandler) => void;
 stepComplete: (handler: EventHandler) => void;
 stepError: (handler: EventHandler) => void;
 workflowComplete: (handler: EventHandler) => void;
 agentStatusUpdate: (handler: EventHandler) => void;
 metricsUpdate: (handler: EventHandler) => void;
 newLog: (handler: EventHandler) => void;
 connect: (handler: EventHandler) => void;
 disconnect: (handler: EventHandler) => void;
 };
 isConnected: () => boolean;
}

let socket: Socket | null = null;

```

```

const eventMap = {
 stepStart: 'workflow:step_start',
 stepComplete: 'workflow:step_complete',
 stepError: 'workflow:step_error',
 workflowComplete: 'workflow:complete',
 agentStatusUpdate: 'agent:status_update',
 metricsUpdate: 'metrics:update',
 newLog: 'log:new',
 connect: 'connect',
 disconnect: 'disconnect',
} as const;

export const socketService: SocketService = {
 connect: () => {
 if (socket?.connected) return;

 socket = io(WS_URL, {
 transports: ['websocket', 'polling'],
 reconnection: true,
 reconnectionAttempts: 10,
 reconnectionDelay: 1000,
 reconnectionDelayMax: 5000,
 timeout: 10000,
 });

 socket.on('connect', () => {
 console.log('[Socket] Connected to server');
 });

 socket.on('disconnect', (reason) => {
 console.log('[Socket] Disconnected:', reason);
 });

 socket.on('connect_error', (error) => {
 console.warn('[Socket] Connection error:', error.message);
 });
 },

 disconnect: () => {
 if (socket) {
 socket.disconnect();
 socket = null;
 }
 },

 on: {
 stepStart: (handler) => socket?.on(eventMap.stepStart, handler as (...args: unknown[]) => void),
 stepComplete: (handler) => socket?.on(eventMap.stepComplete, handler as (...args: unknown[]) => void),
 stepError: (handler) => socket?.on(eventMap.stepError, handler as (...args: unknown[]) => void),
 workflowComplete: (handler) => socket?.on(eventMap.workflowComplete, handler as (...args: unknown[]) => void),
 agentStatusUpdate: (handler) => socket?.on(eventMap.agentStatusUpdate, handler as (...args: unknown[]) => void),
 metricsUpdate: (handler) => socket?.on(eventMap.metricsUpdate, handler as (...args: unknown[]) => void),
 newLog: (handler) => socket?.on(eventMap.newLog, handler as (...args: unknown[]) => void),
 connect: (handler) => socket?.on(eventMap.connect, handler as (...args: unknown[]) => void),
 disconnect: (handler) => socket?.on(eventMap.disconnect, handler as (...args: unknown[]) => void),
 },

 off: {
 stepStart: (handler) => socket?.off(eventMap.stepStart, handler as (...args: unknown[]) => void),
 stepComplete: (handler) => socket?.off(eventMap.stepComplete, handler as (...args: unknown[]) => void),
 stepError: (handler) => socket?.off(eventMap.stepError, handler as (...args: unknown[]) => void),
 workflowComplete: (handler) => socket?.off(eventMap.workflowComplete, handler as (...args: unknown[]) => void),
 agentStatusUpdate: (handler) => socket?.off(eventMap.agentStatusUpdate, handler as (...args: unknown[]) => void),
 metricsUpdate: (handler) => socket?.off(eventMap.metricsUpdate, handler as (...args: unknown[]) => void),
 newLog: (handler) => socket?.off(eventMap.newLog, handler as (...args: unknown[]) => void),
 connect: (handler) => socket?.off(eventMap.connect, handler as (...args: unknown[]) => void),
 disconnect: (handler) => socket?.off(eventMap.disconnect, handler as (...args: unknown[]) => void),
 },

 isConnected: () => socket?.connected ?? false,
};

```

```

import { create } from 'zustand';
import type { AgentId, AgentStatus } from '@types';
import { AGENT_NAMES, AGENT_MODELS, AGENT_DESCRIPTIONS, AGENT_ICON_CLASSES } from '@types';

interface AgentState {
 agent_id: AgentId;
 name: string;
 status: AgentStatus;
 current_task: string | null;
 last_error: string | null;
 model: string;
 description: string;
 icon_class: string;
 enabled: boolean;
}

interface AgentStore {
 agents: Record<AgentId, AgentState>;
 selectedAgent: AgentId;
 setSelectedAgent: (agentId: AgentId) => void;
 updateAgentStatus: (agentId: AgentId, status: AgentStatus) => void;
 updateAgentTask: (agentId: AgentId, task: string | null) => void;
 toggleAgentEnabled: (agentId: AgentId) => void;
 resetAllAgents: () => void;
}

const createInitialAgent = (id: AgentId): AgentState => ({
 agent_id: id,
 name: AGENT_NAMES[id],
 status: 'idle' as AgentStatus,
 current_task: null,
 last_error: null,
 model: AGENT_MODELS[id],
 description: AGENT_DESCRIPTIONS[id],
 icon_class: AGENT_ICON_CLASSES[id],
 enabled: true,
});

const createInitialAgents = (): Record<AgentId, AgentState> => {
 const agents = {} as Record<AgentId, AgentState>;
 const ids: AgentId[] = ['knowledge', 'summary', 'writer', 'review', 'judge', 'result'];
 for (const id of ids) {
 agents[id] = createInitialAgent(id);
 }
 return agents;
};

const initialAgents = createInitialAgents();

export const useAgentStore = create<AgentStore>((set) => ({
 agents: initialAgents,
 selectedAgent: 'knowledge',

 setSelectedAgent: (agentId) => set({ selectedAgent: agentId }),

 updateAgentStatus: (agentId, status) =>
 set((state) => ({
 agents: {
 ...state.agents,
 [agentId]: { ...state.agents[agentId], status },
 },
 })),

 updateAgentTask: (agentId, task) =>
 set((state) => ({
 agents: {
 ...state.agents,
 [agentId]: { ...state.agents[agentId], current_task: task },
 },
 })),

 toggleAgentEnabled: (agentId) =>
 set((state) => ({
 agents: {
 ...state.agents,
 [agentId]: {
 ...state.agents[agentId],
 enabled: !state.agents[agentId].enabled,
 status: !state.agents[agentId].enabled ? 'idle' : state.agents[agentId].status,
 },
 },
 })),

```

```

 },
 },
)))

 resetAllAgents: () => set({ agents: createInitialAgents() }),
}));

export const AGENT_CONFIGS = {
 knowledge: { name: AGENT_NAMES.knowledge, model: AGENT_MODELS.knowledge, description: AGENT_DESCRIPTIONS.knowledge, icon_class: AGENT_ICON_CLASSES.knowledge },
 summary: { name: AGENT_NAMES.summary, model: AGENT_MODELS.summary, description: AGENT_DESCRIPTIONS.summary, icon_class: AGENT_ICON_CLASSES.summary },
 writer: { name: AGENT_NAMES.writer, model: AGENT_MODELS.writer, description: AGENT_DESCRIPTIONS.writer, icon_class: AGENT_ICON_CLASSES.writer },
 review: { name: AGENT_NAMES.review, model: AGENT_MODELS.review, description: AGENT_DESCRIPTIONS.review, icon_class: AGENT_ICON_CLASSES.review },
 judge: { name: AGENT_NAMES.judge, model: AGENT_MODELS.judge, description: AGENT_DESCRIPTIONS.judge, icon_class: AGENT_ICON_CLASSES.judge },
 result: { name: AGENT_NAMES.result, model: AGENT_MODELS.result, description: AGENT_DESCRIPTIONS.result, icon_class: AGENT_ICON_CLASSES.result },
};

```

## frontend\src\stores\workflowStore.ts

```

import { create } from 'zustand';
import { agentService, workflowService } from '@/services/api/agentService';
import type { AgentId, WorkflowStep, WorkflowOutput } from '@/types';

interface LogEntry {
 id: string;
 timestamp: Date;
 agent: string;
 type: 'info' | 'success' | 'warning' | 'error';
 message: string;
}

interface ChatHistory {
 user_input: string;
 response: string;
 timestamp: Date;
}

interface WorkflowStore {
 isRunning: boolean;
 currentTask: string;
 currentStep: AgentId | null;
 elapsedSeconds: number;
 useMock: boolean;
 completedSteps: AgentId[];
 workflowSteps: WorkflowStep[];
 result: WorkflowOutput | null;
 judgeDecision: 'local' | 'cloud' | null;
 complexityScore: number;
 logs: LogEntry[];
 chatHistory: ChatHistory[];

 // Actions
 executeWorkflow: (input: string) => Promise<void>;
 addLog: (agent: string | undefined, type: LogEntry['type'], message: string) => void;
 clearLogs: () => void;
 setCurrentTask: (task: string) => void;
 setIsRunning: (running: boolean) => void;
 setCurrentStep: (step: AgentId | null) => void;
 setResult: (result: WorkflowOutput | null) => void;
 setJudgeDecision: (decision: 'local' | 'cloud' | null) => void;
 setComplexityScore: (score: number) => void;
 addCompletedStep: (agentId: AgentId) => void;
 addWorkflowStep: (step: WorkflowStep) => void;
 resetWorkflow: () => void;
 setUseMock: (useMock: boolean) => void;
 addChatHistory: (input: string, response: string) => void;
 clearChatHistory: () => void;
 getContext: () => string;
}

```

```

export const useWorkflowStore = create<WorkflowStore>((set, get) => ({
 isRunning: false,
 currentTask: '',
 currentStep: null,
 elapsedSeconds: 0,
 useMock: false,
 completedSteps: [],
 workflowSteps: [],
 result: null,
 judgeDecision: null,
 complexityScore: 0,
 logs: [],
 chatHistory: [],

 setCurrentTask: (task) => set({ currentTask: task }),
 setIsRunning: (running) => set({ isRunning: running }),
 setCurrentStep: (step) => set({ currentStep: step }),
 setResult: (result) => set({ result }),
 setJudgeDecision: (decision) => set({ judgeDecision: decision }),
 setComplexityScore: (score) => set({ complexityScore: score }),
 setUseMock: (useMock) => set({ useMock }),

 addLog: (agent, type, message) =>
 set((state) => ({
 logs: [
 ...state.logs,
 {
 id: Date.now().toString(),
 timestamp: new Date(),
 agent: agent || 'system',
 type,
 message,
 },
],
 })),

 clearLogs: () => set({ logs: [] }),

 addCompletedStep: (agentId) =>
 set((state) => ({
 completedSteps: state.completedSteps.includes(agentId)
 ? state.completedSteps
 : [...state.completedSteps, agentId],
 })),

 addWorkflowStep: (step) =>
 set((state) => ({
 workflowSteps: [...state.workflowSteps, step],
 })),

 addChatHistory: (input, response) =>
 set((state) => ({
 chatHistory: [...state.chatHistory, { user_input: input, response, timestamp: new Date() }],
 })),

 clearChatHistory: () => set({ chatHistory: [] }),

 getContext: () => {
 const { chatHistory } = get();
 if (chatHistory.length === 0) return '';
 return chatHistory
 .map((item) => `用户: ${item.user_input}\n助手: ${item.response}`)
 .join('\n\n');
 },

 resetWorkflow: () =>
 set({
 isRunning: false,
 currentTask: '',
 currentStep: null,
 elapsedSeconds: 0,
 completedSteps: [],
 workflowSteps: [],
 result: null,
 judgeDecision: null,
 complexityScore: 0,
 logs: [],
 chatHistory: [],
 })),

```

```

executeWorkflow: async (input) => {
 const { addLog } = get();
 set({ isRunning: true, result: null, logs: [], completedSteps: [], workflowSteps: [] });

 try {
 addLog('system', 'info', `开始执行工作流: ${input.slice(0, 50)}...`);

 addLog('knowledge', 'info', '知识检索开始');
 const knowledgeResult = await agentService.executeAgent('knowledge', { content: input });
 addLog('knowledge', 'success', '知识检索完成');

 addLog('summary', 'info', '需求摘要开始');
 const summaryResult = await agentService.executeAgent('summary', {
 content: knowledgeResult.content,
 });
 addLog('summary', 'success', '需求摘要完成');

 addLog('writer', 'info', '内容生成开始');
 const writerResult = await agentService.executeAgent('writer', {
 content: summaryResult.content,
 });
 addLog('writer', 'success', '内容生成完成');

 addLog('review', 'info', '质量评审开始');
 const reviewResult = await agentService.executeAgent('review', {
 content: writerResult.content,
 });
 addLog('review', 'success', `质量评审完成, 评分: ${reviewResult.metadata?.score || 'N/A'}`);

 addLog('judge', 'info', '复杂度判断开始');
 const judgeResult = await agentService.executeAgent('judge', {
 content: reviewResult.content,
 });
 const executedLocally = judgeResult.metadata?.executed_locally ?? true;
 addLog(
 'judge',
 executedLocally ? 'success' : 'warning',
 `复杂度判断完成, ${executedLocally ? '本地执行' : '调用云端API'}`
);

 addLog('result', 'info', '结果生成开始');
 const resultResult = await agentService.executeAgent('result', {
 content: judgeResult.content,
 context: { writer: writerResult.content },
 });
 addLog('result', 'success', '结果生成完成');

 const finalResult = resultResult.content;

 get().addChatHistory(input, finalResult);

 set({
 result: {
 final_result: finalResult,
 steps: [],
 executed_locally: executedLocally,
 total_duration_seconds: 0,
 start_time: new Date().toISOString(),
 end_time: new Date().toISOString(),
 complexity_score: 0,
 },
 isRunning: false,
 });

 addLog('system', 'success', '工作流执行完成');
 } catch (error) {
 addLog(
 'system',
 'error',
 `工作流执行失败: ${error instanceof Error ? error.message : '未知错误'}`
);
 set({ isRunning: false });
 }
},
));

```

## frontend\src\types\index.ts

```

export interface WorkflowInput {
 user_input: string;
 context?: Record<string, unknown>;
}

export interface WorkflowOutput {
 final_result: string;
 steps: WorkflowStep[];
 executed_locally: boolean;
 total_duration_seconds: number;
 start_time: string;
 end_time: string;
 complexity_score?: number;
}

export interface WorkflowStep {
 agent_id: string;
 agent_name: string;
 input: string;
 output: string;
 success: boolean;
 duration_seconds: number;
 timestamp?: string;
 metadata?: Record<string, unknown>;
}

export type AgentId = 'knowledge' | 'summary' | 'writer' | 'review' | 'judge' | 'result';

export const AGENT_ORDER: AgentId[] = ['knowledge', 'summary', 'writer', 'review', 'judge', 'result'];

export const AGENT_NAMES: Record<AgentId, string> = {
 knowledge: 'Knowledge Agent',
 summary: 'A摘要Agent',
 writer: 'B撰写Agent',
 review: 'Review Agent',
 judge: 'Judge Agent',
 result: 'Result Agent',
};

export const AGENT_MODELS: Record<AgentId, string> = {
 knowledge: 'Qwen2.5-3B',
 summary: 'Qwen2.5-3B',
 writer: 'Qwen2.5-7B',
 review: 'Qwen2.5-3B',
 judge: 'Qwen2.5-3B',
 result: 'Local',
};

export const AGENT_EMOJIS: Record<AgentId, string> = {
 knowledge: '🧠',
 summary: '📄',
 writer: '🖋️',
 review: '🔍',
 judge: '⚖️',
 result: '📋',
};

export const AGENT_COLORS: Record<AgentId, string> = {
 knowledge: 'purple',
 summary: 'emerald',
 writer: 'blue',
 review: 'orange',
 judge: 'violet',
 result: 'green',
};

export const AGENT_ICON_CLASSES: Record<AgentId, string> = {
 knowledge: 'purple',
 summary: 'emerald',
 writer: 'blue',
 review: 'orange',
 judge: 'violet',
 result: 'blue',
};

```

```

export const AGENT_DESCRIPTIONS: Record<AgentId, string> = {
 knowledge: '知识库检索与上下文增强',
 summary: '需求摘要与关键信息提取',
 writer: '内容生成与初稿撰写',
 review: '质量评估与修改建议生成',
 judge: '最终决策与路径选择',
 result: '成果导出与格式化',
};

export type AgentStatus = 'idle' | 'ready' | 'processing' | 'error' | 'completed' | 'shutdown';

export interface AgentStatusResponse {
 agent_id: string;
 name: string;
 status: 'idle' | 'ready' | 'running' | 'shutdown';
 current_task?: string;
 last_error?: string;
}

export interface Metrics {
 total_requests: number;
 local_executions: number;
 cloud_executions: number;
 api_calls: number;
 cost_saved: number;
 avg_response_time: number;
 cpu_usage: number;
 gpu_usage: number;
 total_tasks: number;
 uptime_seconds: number;
}

export type LogType = 'info' | 'success' | 'warning' | 'error' | 'system' | 'judge_decision';

export interface LogEntry {
 id: string;
 timestamp: Date;
 agent_id?: AgentId;
 type: LogType;
 message: string;
 metadata?: Record<string, unknown>;
}

export interface KnowledgeEntry {
 id: string;
 content: string;
 tags: string[];
 score?: number;
}

export interface KnowledgeInput {
 content: string;
 tags: string[];
}

export interface JudgeDecision {
 complexity_score: number;
 threshold: number;
 route: 'local' | 'cloud';
 reason: string;
}

export interface ExportInput {
 content: string;
 format: 'markdown' | 'json' | 'pdf' | 'docx';
}

export interface ExportRequest {
 content: string;
 format: string;
 filename?: string;
}

export interface ExportResponse {
 status: string;
 format: string;
 filename: string;
 filepath?: string;
}

```

```

export interface HealthResponse {
 status: string;
 agents: number;
 version: string;
}

// ===== Chat Types =====
export interface ChatRequest {
 content: string;
 use_cloud?: boolean;
 model_name?: string;
}

export interface ChatResponse {
 response: string;
 executed_locally: boolean;
 complexity_score: number;
 total_duration: number;
 steps_count: number;
 mode: string;
 model_used?: string;
}

// ===== Config Types =====
export interface ConfigUpdate {
 deepseek_api_key?: string;
 ollama_host?: string;
}

export interface ConnectionTestResult {
 success: boolean;
 message: string;
 details?: Record<string, string>;
}

export interface ModelConfig {
 name: string;
 provider: string;
 model: string;
 api_key?: string;
 display_name?: string;
 max_tokens: number;
 temperature: number;
}

export interface ValidateKeyRequest {
 provider: string;
 api_key: string;
 model?: string;
}

export interface ValidateKeyResponse {
 success: boolean;
 message: string;
}

export interface ConfigResponse {
 ollama_host: string;
 ollama_model: string;
 deepseek_api_key_set: boolean;
 deepseek_model: string;
 models: ModelConfig[];
}

export interface OllamaDetectResponse {
 ollama_host: string;
 message: string;
}

export interface CacheStats {
 cache_size: number;
 max_size: number;
 ttl: number;
}

export interface ChatCacheStats {
 chat_cache_size: number;
 chat_cache_max_size: number;
 chat_cache_ttl: number;
 workflow_cache_size: number;
}

```

```

 workflow_cache_max_size: number;
 workflow_cache_ttl: number;
 }

 export interface KnowledgeStats {
 total_entries: number;
 total_keywords: number;
 total_size: number;
 }

 export interface SocketEvents {
 'workflow:step_start': (data: { agent_id: AgentId; agent_name: string }) => void;
 'workflow:step_complete': (data: WorkflowStep) => void;
 'workflow:step_error': (data: { agent_id: AgentId; error: string }) => void;
 'workflow:complete': (data: WorkflowOutput) => void;
 'agent:status update': (data: { agent_id: AgentId; status: AgentStatus; task?: string }) => void;
 'metrics:update': (data: Metrics) => void;
 'log:new': (data: LogEntry) => void;
 }

 export const COMPLEXITY_THRESHOLD = 0.65;

```

## frontend\tailwind.config.ts

```

import type { Config } from 'tailwindcss';

const config: Config = {
 darkMode: 'class',
 content: [
 './src/pages/**/*. {js,ts,jsx,tsx,mdx}',
 './src/components/**/*. {js,ts,jsx,tsx,mdx}',
 './src/app/**/*. {js,ts,jsx,tsx,mdx}',
],
 theme: {
 extend: {
 colors: {
 dark: {
 primary: '#0a0e1a',
 secondary: '#0d1220',
 tertiary: '#131a2b',
 card: '#161d2f',
 'card-hover': '#1a2338',
 elevated: '#1e2842',
 },
 border: {
 default: '#1e2942',
 light: '#253352',
 glow: 'rgba(56, 189, 248, 0.15)',
 },
 text: {
 primary: '#e8edf5',
 secondary: '#94a3b8',
 tertiary: '#64748b',
 muted: '#475569',
 },
 accent: {
 cyan: '#38bdf8',
 blue: '#3b82f6',
 green: '#10b981',
 emerald: '#34d399',
 amber: '#f59e0b',
 orange: '#f97316',
 red: '#ef4444',
 purple: '#a78bfa',
 },
 },
 fontFamily: {
 sans: ['Noto Sans SC', '-apple-system', 'BlinkMacSystemFont', 'sans-serif'],
 mono: ['JetBrains Mono', 'Fira Code', 'monospace'],
 },
 borderRadius: {
 sm: '8px',
 md: '12px',
 lg: '16px',
 },
 },
 },
};

```

```

 },
 boxShadow: {
 card: '0 1px 3px rgba(0, 0, 0, 0.3)',
 elevated: '0 4px 12px rgba(0, 0, 0, 0.4)',
 glow: '0 0 20px rgba(56, 189, 248, 0.15)',
 'glow-lg': '0 0 40px rgba(56, 189, 248, 0.2)',
 'glow-green': '0 0 20px rgba(16, 185, 129, 0.2)',
 'glow-amber': '0 0 20px rgba(245, 158, 11, 0.2)',
 },
 },
 plugins: [],
};

export default config;

```

## frontend\tsconfig.json

```

{
 "compilerOptions": {
 "target": "ES2017",
 "lib": ["dom", "dom.iterable", "esnext"],
 "allowJs": true,
 "skipLibCheck": true,
 "strict": true,
 "forceConsistentCasingInFileNames": true,
 "noEmit": true,
 "esModuleInterop": true,
 "module": "esnext",
 "moduleResolution": "bundler",
 "resolveJsonModule": true,
 "isolatedModules": true,
 "jsx": "preserve",
 "incremental": true,
 "plugins": [
 {
 "name": "next"
 }
],
 "baseUrl": ".",
 "paths": {
 "@/*": ["src/*"]
 }
 },
 "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
 "exclude": ["node_modules"]
}

```

## pytest.ini

```

[pytest]
testpaths = [~backend/tests~]
pythonpath = backend
addopts = -v --tb=short
filterwarnings =
 ignore::DeprecationWarning
 ignore::PendingDeprecationWarning
log_cli = true
log_cli_level = INFO
log_format = %(asctime)s %(levelname)s %(message)s
log_date_format = %Y-%m-%d %H:%M:%S

```

## ruff.toml

```

line-length = 100

select = [
 "E",
 "F",
 "W",
 "I",
 "N",
 "Q",
 "RUF",
 "S",
 "UP",
 "B",
 "A",
 "ARG",
 "BLE",
 "DTZ",
 "EM",
 "ERA",
 "EXE",
 "G",
 "ICN",
 "INP",
 "ISC",
 "NPY",
 "PTH",
 "PYI",
 "RET",
 "RSE",
 "SIM",
 "TID",
 "TRY",
 "YTT",
]

ignore = [
 "E501",
 "F401",
 "RUF100",
]

per-file-ignores = {
 "**/__init__.py" = ["F401"],
 "**/tests/**" = ["S101", "B101"],
}

target-version = "py312"

[format]
quote-style = "double"
indent-width = 4
line-ending = "lf"

```

## scripts\convert\_docs\_to\_pdf.py

```

import markdown
import os
import subprocess

PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DOCS_DIR = os.path.join(PROJECT_ROOT, "docs")

docs = [
 ("AgentMatrix 技术报告.md", "AgentMatrix 技术报告.pdf"),
 ("AgentMatrix 部署文档.md", "AgentMatrix 部署文档.pdf"),
 ("AgentMatrix 安装说明.md", "AgentMatrix 安装说明.pdf"),
]

CSS_STYLE = """
* { box-sizing: border-box; }
body {
 font-family: "Microsoft YaHei", "SimHei", sans-serif;
 font-size: 13px; line-height: 1.8; color: #1a1a2e;
}

```

```

max-width: 900px; margin: 0 auto; padding: 40px 50px;
}
h1 { font-size: 28px; border-bottom: 3px solid #2563eb; padding-bottom: 12px; color: #1e3a5f; margin-top: 40px; }
h2 { font-size: 22px; border-bottom: 2px solid #93c5fd; padding-bottom: 8px; color: #1e40af; margin-top: 35px; }
h3 { font-size: 17px; color: #1d4ed8; margin-top: 28px; }
h4 { font-size: 15px; color: #2563eb; margin-top: 22px; }
p { margin: 10px 0; }
code { background: #f1f5f9; padding: 2px 6px; border-radius: 4px; font-family: "Consolas", "Courier New", monospace; font-size: 12px; }
pre { background: #0f172a; color: #e2e8f0; padding: 16px 20px; border-radius: 8px; overflow-x: auto; font-size: 11px; line-height: 1.5; white-space: pre-wrap; word-break: break-all; }
pre code { background: transparent; padding: 0; color: #e2e8f0; }
table { border-collapse: collapse; width: 100%; margin: 16px 0; font-size: 12px; }
th { background: #1e40af; color: white; padding: 10px 12px; text-align: left; }
td { border: 1px solid #d1d5db; padding: 8px 12px; }
tr:nth-child(even) { background: #f8fafc; }
blockquote { border-left: 4px solid #93c5fd; padding: 8px 16px; margin: 16px 0; background: #eff6ff; color: #1e40af; }
a { color: #2563eb; }
hr { border: none; border-top: 1px solid #e2e8f0; margin: 30px 0; }
strong { color: #1e3a5f; }
@page { size: A4; margin: 2cm; }
"""

def find_browser():
 paths = [
 r"C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe",
 r"C:\Program Files\Microsoft\Edge\Application\msedge.exe",
 r"C:\Program Files\Google\Chrome\Application\chrome.exe",
 r"C:\Program Files (x86)\Google\Chrome\Application\chrome.exe",
]
 for path in paths:
 if os.path.exists(path):
 return path
 return None

def convert_md_to_pdf(md_filename, pdf_filename):
 md_path = os.path.join(DOCS_DIR, md_filename)
 pdf_path = os.path.join(DOCS_DIR, pdf_filename)

 if not os.path.exists(md_path):
 print(f"[ERROR] File not found: {md_path}")
 # Try to find any matching file
 for f in os.listdir(DOCS_DIR):
 if f.endswith('.md') and md_filename.replace('.md', '') in f:
 md_path = os.path.join(DOCS_DIR, f)
 print(f"[INFO] Found: {md_path}")
 break
 else:
 return False

 with open(md_path, "r", encoding="utf-8") as f:
 md_content = f.read()

 html_body = markdown.markdown(md_content, extensions=["fenced_code", "tables", "toc"])

 full_html = f"""<!DOCTYPE html>
<html lang="zh-CN">
<head><meta charset="utf-8"><style>{CSS_STYLE}</style></head>
<body>{html_body}</body>
</html>"""

 browser = find_browser()
 if not browser:
 print(f"[WARN] No browser found for {md_filename}")
 with open(md_path.replace('.md', '.html'), "w", encoding="utf-8") as f:
 f.write(full_html)
 return False

 temp_html = os.path.join(DOCS_DIR, f"_temp_print.html")
 with open(temp_html, "w", encoding="utf-8") as f:
 f.write(full_html)

 cmd = [browser, "--headless=new", "--disable-gpu", "--no-sandbox",
 f"--print-to-pdf={pdf_path}", temp_html]

 try:
 result = subprocess.run(cmd, capture_output=True, text=True, timeout=60)
 if result.returncode == 0 and os.path.exists(pdf_path):
 sz = os.path.getsize(pdf_path)
 print(f"[OK] {pdf_filename} ({sz//1024} KB)")

```

```

 else:
 print(f"[WARN] Browser returned {result.returncode}: {result.stderr[:200]}")
 return False
 except Exception as e:
 print(f"[ERROR] {e}")
 return False
 finally:
 if os.path.exists(temp_html):
 os.unlink(temp_html)
 return True

if __name__ == "__main__":
 browser_path = find_browser()
 print(f"Browser: {browser_path or 'NOT FOUND'}")

 for md_file, pdf_file in docs:
 print(f"Converting: {md_file} -> {pdf_file}")
 convert_md_to_pdf(md_file, pdf_file)

 print("\n[DONE] PDF generation complete!")

```

## scripts\extract\_source\_code.py

```

import os
import sys

PROJECT_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
OUTPUT_MD = os.path.join(PROJECT_ROOT, "docs", "AgentMatrix 源代码.md")

Directories/files to exclude
EXCLUDE_DIRS = {
 "node_modules", ".git", "__pycache__", ".next", ".venv", ".venv",
 "dist", "build", ".turbo", ".trae", "logs", "data",
 "frontend\\frontend", # Duplicate nested frontend
}

EXCLUDE_FILES = {
 ".DS_Store", "Thumbs.db", "*.pyc", "*.pyo",
 "package-lock.json", "pnpm-lock.yaml", "yarn.lock",
 "ollama_models.json",
}

Only include these extensions
INCLUDE_EXTENSIONS = {
 ".py", ".ts", ".tsx", ".js", ".jsx", ".json", ".yaml", ".yml",
 ".toml", ".cfg", ".ini", ".css", ".html", ".md", ".bat", ".ps1",
 ".sh", ".txt", ".env.example", ".editorconfig", ".prettierrc",
 ".eslintrc.js", "Dockerfile", ".gitignore",
}

MAX_FILE_SIZE = 200 * 1024

LANG_MAP = {
 ".py": "python", ".ts": "typescript", ".tsx": "tsx", ".js": "javascript",
 ".jsx": "jsx", ".json": "json", ".yaml": "yaml", ".yml": "yaml",
 ".toml": "toml", ".cfg": "ini", ".ini": "ini", ".css": "css",
 ".html": "html", ".md": "markdown", ".bat": "batch", ".ps1": "powershell",
 ".sh": "bash", ".txt": "text",
}

def should_exclude_dir(dirname):
 for ex in EXCLUDE_DIRS:
 if ex in dirname:
 return True
 return False

def should_include_file(filename):
 name = os.path.basename(filename)
 # Exact match
 if name in EXCLUDE_FILES:
 return False

```

```

Extension match
_, ext = os.path.splitext(name)
if ext.lower() in INCLUDE_EXTENSIONS:
 return True

Special files without extensions
special_names = {"Dockerfile", ".gitignore", ".editorconfig", ".prettierrc", ".env.example", ".eslintrc.js"}
if name in special_names:
 return True
return False

def get_language(filename):
 _, ext = os.path.splitext(filename)
 if ext.lower() in LANG_MAP:
 return LANG_MAP[ext.lower()]
 name = os.path.basename(filename)
 if name == "Dockerfile":
 return "dockerfile"
 if name in {".gitignore", ".editorconfig", ".prettierrc"}:
 return "text"
 return "text"

def collect_files(root_dir):
 files = []
 for dirpath, dirnames, filenames in os.walk(root_dir):
 # Filter dirs in-place
 dirnames[:] = [d for d in dirnames if not should_exclude_dir(os.path.join(dirpath, d))]

 for fname in filenames:
 fullpath = os.path.join(dirpath, fname)
 if should_include_file(fullpath):
 files.append(fullpath)
 return sorted(files)

def generate_source_md():
 all_files = collect_files(PROJECT_ROOT)

 with open(OUTPUT_MD, "w", encoding="utf-8") as out:
 out.write("# AgentMatrix 完整源代码\n\n")
 out.write(f"> 项目根目录: {PROJECT_ROOT}\n")
 out.write(f"> 包含文件数: {len(all_files)}\n")
 out.write(f"> 生成日期: 2026-05-17\n\n")
 out.write("---\n\n")

 # Table of contents
 out.write("## 文件索引\n\n")
 for fpath in all_files:
 rel = os.path.relpath(fpath, PROJECT_ROOT)
 out.write(f"- [{rel}]\n")
 out.write("\n---\n\n")

 for fpath in all_files:
 rel = os.path.relpath(fpath, PROJECT_ROOT)
 fsize = os.path.getsize(fpath)

 if fsize > MAX_FILE_SIZE:
 out.write(f"## [{rel}]\n\n")
 out.write(f"> ⚠ 文件过大 ({fsize//1024} KB), 已跳过内容\n\n")
 out.write("---\n\n")
 continue

 lang = get_language(fpath)

 try:
 with open(fpath, "r", encoding="utf-8") as f:
 content = f.read()
 except UnicodeDecodeError:
 out.write(f"## [{rel}]\n\n")
 out.write(f"> ⚠ 二进制文件, 无法读取\n\n")
 out.write("---\n\n")
 continue

 out.write(f"## [{rel}]\n\n")

 # Add code block
 if len(content) > 50000:
 # Truncate very long files
 content = content[:50000] + "\n\n... (内容过长, 已截断) ..."

```

```

 out.write(f"```` {lang}\n")
 out.write(content)
 if not content.endswith("\n"):
 out.write("\n")
 out.write("````\n\n")
 out.write("----\n\n")

 md_size = os.path.getsize(OUTPUT_MD)
 print(f"[OK] Generated: {OUTPUT_MD}")
 print(f" Files: {len(all_files)}")
 print(f" Size: {md_size//1024} KB")

if __name__ == "__main__":
 generate_source_md()

```

## shared\_\_init\_\_.py

```

shared__init__.py

```

## shared\constants\_\_init\_\_.py

```

shared\constants__init__.py

```

## shared\types\_\_init\_\_.py

```

shared\types__init__.py

```

## shared\utils\_\_init\_\_.py

```

shared\utils__init__.py

```

## start.bat

```

@echo off
chcp 65001 >nul
title AgentMatrix - 一键启动

echo =====
echo AgentMatrix - 多智能体动态协同系统
echo =====
echo.

REM 检查是否安装了Python
python --version >nul 2>&1
if errorlevel 1 (
 echo [错误] 未检测到 Python, 请先安装 Python 3.10+
 pause
 exit /b 1
)

```

```

)

REM 检查是否安装了Node.js
node --version >nul 2>&1
if errorlevel 1 (
 echo [错误] 未检测到 Node.js, 请先安装 Node.js 20+
 pause
 exit /b 1echo
)

echo [检查通过] Python 和 Node.js 环境已就绪
echo.

REM 切换到项目根目录
cd /d "%~dp0"

REM 检查并安装后端依赖
echo [步骤 1/4] 检查后端依赖...
cd backend
if not exist "venv" (
 echo 正在安装后端依赖...
 pip install pydantic pydantic-settings fastapi uvicorn httpx python-dotenv >nul 2>&1
) else (
 echo 后端依赖已存在
)
cd ..

REM 检查并安装前端依赖
echo [步骤 2/4] 检查前端依赖...
cd frontend
if not exist "node_modules" (
 echo 正在安装前端依赖...
 call npm install >nul 2>&1
) else (
 echo 前端依赖已存在
)
cd ..

echo.
echo [步骤 3/4] 启动后端服务...
cd backend
start "AgentMatrix Backend" cmd /k "python -m uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload"
cd ..

timeout /t 2 /nobreak >nul

echo [步骤 4/4] 启动前端服务...
cd frontend
start "AgentMatrix Frontend" cmd /k "npm run dev"
cd ..

echo.
echo =====
echo 系统启动完成!
echo =====
echo.
echo 访问地址:
echo - 前端界面: http://localhost:3000
echo - 后端API: http://localhost:8000
echo - API文档: http://localhost:8000/docs
echo.
echo 按任意键关闭此窗口 (服务会继续运行)...
pause >nul

```

## start.ps1

```

$OutputEncoding = [console]::InputEncoding = [console]::OutputEncoding = New-Object System.Text.UTF8Encoding

Write-Host "===== " -ForegroundColor Cyan
Write-Host " AgentMatrix - Multi-Agent Collaboration System" -ForegroundColor Cyan
Write-Host "===== " -ForegroundColor Cyan
Write-Host ""

try {

```

```

$pythonVersion = python --version 2>&1
if ($LASTEXITCODE -ne 0) {
 Write-Host "[ERROR] Python not found. Please install Python 3.10+" -ForegroundColor Red
 Read-Host "Press any key to exit..."
 exit 1
}
} catch {
 Write-Host "[ERROR] Python not found. Please install Python 3.10+" -ForegroundColor Red
 Read-Host "Press any key to exit..."
 exit 1
}

try {
 $nodeVersion = node --version 2>&1
 if ($LASTEXITCODE -ne 0) {
 Write-Host "[ERROR] Node.js not found. Please install Node.js 20+" -ForegroundColor Red
 Read-Host "Press any key to exit..."
 exit 1
 }
} catch {
 Write-Host "[ERROR] Node.js not found. Please install Node.js 20+" -ForegroundColor Red
 Read-Host "Press any key to exit..."
 exit 1
}

Write-Host "[OK] Python and Node.js environment ready" -ForegroundColor Green
Write-Host ""

$projectPath = Get-Location

Write-Host "[Step 1/4] Checking backend dependencies..." -ForegroundColor Yellow
Set-Location (Join-Path $projectPath "backend")
if (-not (Test-Path "venv")) {
 Write-Host "Installing backend dependencies..." -ForegroundColor Yellow
 pip install pydantic pydantic-settings fastapi uvicorn httpx python-dotenv | Out-Null
} else {
 Write-Host "Backend dependencies already installed" -ForegroundColor Green
}
Set-Location $projectPath

Write-Host "[Step 2/4] Checking frontend dependencies..." -ForegroundColor Yellow
Set-Location (Join-Path $projectPath "frontend")
if (-not (Test-Path "node_modules")) {
 Write-Host "Installing frontend dependencies..." -ForegroundColor Yellow
 npm install | Out-Null
} else {
 Write-Host "Frontend dependencies already installed" -ForegroundColor Green
}
Set-Location $projectPath

Write-Host ""
Write-Host "[Step 3/4] Starting backend server..." -ForegroundColor Yellow
Start-Process -FilePath "powershell" -ArgumentList "-Command", "cd '$(Join-Path $projectPath 'backend')'; python -m uvicorn app.p.main:app --host 0.0.0.0 --port 8000 --reload" -WindowStyle Normal -WorkingDirectory (Join-Path $projectPath "backend")

Start-Sleep -Seconds 2

Write-Host "[Step 4/4] Starting frontend server..." -ForegroundColor Yellow
Start-Process -FilePath "powershell" -ArgumentList "-Command", "cd '$(Join-Path $projectPath 'frontend')'; npm run dev" -WindowStyle Normal -WorkingDirectory (Join-Path $projectPath "frontend")

Write-Host ""
Write-Host "===== " -ForegroundColor Cyan
Write-Host "System started successfully!" -ForegroundColor Green
Write-Host "===== " -ForegroundColor Cyan
Write-Host ""
Write-Host "Access URLs:" -ForegroundColor White
Write-Host " - Frontend: http://localhost:3000" -ForegroundColor Green
Write-Host " - Backend API: http://localhost:8000" -ForegroundColor Green
Write-Host " - API Docs: http://localhost:8000/docs" -ForegroundColor Green
Write-Host ""
Read-Host "Press any key to close this window (services will continue running)..."

```

```

@echo off
chcp 65001 >nul
title AgentMatrix - 停止服务

echo =====
echo 正在停止 AgentMatrix 服务...
echo =====

REM 停止后端进程
taskkill /F /IM python.exe /FI "WINDOWTITLE eq AgentMatrix Backend*" >nul 2>&1
if errorlevel 1 (
 echo [提示] 未找到后端进程或已停止
) else (
 echo [完成] 后端服务已停止
)

REM 停止前端进程
taskkill /F /IM node.exe /FI "WINDOWTITLE eq AgentMatrix Frontend*" >nul 2>&1
if errorlevel 1 (
 echo [提示] 未找到前端进程或已停止
) else (
 echo [完成] 前端服务已停止
)

taskkill /F /IM cmd.exe /FI "WINDOWTITLE eq AgentMatrix Backend*" >nul 2>&1
taskkill /F /IM cmd.exe /FI "WINDOWTITLE eq AgentMatrix Frontend*" >nul 2>&1

echo.
echo 所有服务已停止
echo.
pause

```

## stop.ps1

```

$OutputEncoding = [console]::InputEncoding = [console]::OutputEncoding = New-Object System.Text.UTF8Encoding

Write-Host "===== " -ForegroundColor Cyan
Write-Host " Stopping AgentMatrix Services..." -ForegroundColor Cyan
Write-Host "===== " -ForegroundColor Cyan

$backendProcesses = Get-Process python -ErrorAction SilentlyContinue | Where-Object { $_.MainWindowTitle -like "*AgentMatrix*" }
if ($backendProcesses) {
 $backendProcesses | ForEach-Object {
 $_.Kill()
 Write-Host "[OK] Backend server stopped" -ForegroundColor Green
 }
} else {
 Write-Host "[INFO] Backend process not found or already stopped" -ForegroundColor Yellow
}

$frontendProcesses = Get-Process node -ErrorAction SilentlyContinue | Where-Object { $_.MainWindowTitle -like "*AgentMatrix*" }
if ($frontendProcesses) {
 $frontendProcesses | ForEach-Object {
 $_.Kill()
 Write-Host "[OK] Frontend server stopped" -ForegroundColor Green
 }
} else {
 Write-Host "[INFO] Frontend process not found or already stopped" -ForegroundColor Yellow
}

Write-Host ""
Write-Host "All services stopped" -ForegroundColor Green
Write-Host ""
Read-Host "Press any key to exit..."

```

## temp.py

⚠️ 二进制文件，无法读取

## temp2.py

⚠️ 二进制文件，无法读取

## test.bat

```
@echo off
chcp 65001 >nul
title AgentMatrix - workflow测试

echo =====
echo 测试 AgentMatrix workflow
echo =====
echo.

cd /d "%~dp0backend"
python test_workflow.py

echo.
echo.
echo 测试完成!
pause
```

## test.ps1

```
$OutputEncoding = [console]::InputEncoding = [console]::OutputEncoding = New-Object System.Text.UTF8Encoding

Write-Host "===== " -ForegroundColor Cyan
Write-Host " Testing AgentMatrix Workflow" -ForegroundColor Cyan
Write-Host "===== " -ForegroundColor Cyan
Write-Host ""

$projectPath = Get-Location
Set-Location (Join-Path $projectPath "backend")

python test_workflow.py

Write-Host ""
Write-Host ""
Write-Host "Test completed!" -ForegroundColor Green
Read-Host "Press any key to exit..."
```